

基于精确松弛与滚动优化的微能源网经济调度研究

摘要

本文研究非孤岛式微网在负载、光伏出力及外网电价变化条件下的购电与储能调度问题，以降低微网运行过程中的总购电费用为目标，针对各问信息条件的差异分别建立优化模型并求解。针对储能充放电互补约束，分析其精确松弛条件，并根据不同问题的模型规模采用混合整数线性规划（MILP）与连续线性规划（LP）进行分层求解。

针对问题一，在电价、负载和光伏出力均已知的条件下，建立以全天购电费用最小为目标的混合整数线性规划模型，联合优化各时段购电量与储能充放电量，并考虑储能容量、充放电功率及首末储电量一致等约束。采用 HiGHS 求解得到全天购电量为 59482.6990 kWh，购电费为 35126.8486 元，0:00 与 24:00 储电量均为 6000 kWh。

针对问题二，负载和光伏出力在制定日前计划时尚未知。比较前一日同时段值、近 7 日同时段均值和 LightGBM 三种预测方法，选取历史预测误差最小的方法进行点预测，并利用历史整日预测残差构造 10 个典型场景；进一步建立两阶段随机规划模型，第一阶段确定日前购电计划，第二阶段根据不同场景优化储能充放电与紧急购电，并考虑期末储能价值。采用精确松弛后的连续线性规划求解，334 天评价期内总费用为 1424.42 万元，日均费用为 42647.32 元。

针对问题三，光伏预测在 0:00、6:00、12:00 和 18:00 分四次发布，以 0:00 确定的计划为初始方案，根据预测来滚动调整计划。并考虑增购按 1.5 倍电价结算、减购按 0.5 倍电价退款及紧急购电机制。通过比较 8 种更新组合，得到 6:00、12:00 和 18:00 全部更新时总费用最低，为 1385.86 万元，较三个时刻均不更新的基准方案日均降低 1897.42 元；在单次更新方案中，12:00 更新的降费效果最明显。

针对问题四，外网电价随日期变化，且当天真实电价在 0:00 时尚未知。采用近 7 日同时段均值预测各时段电价，以预测价格制定购电与储能计划，并按实际电价结算，在问题二和问题三的随机与滚动调度框架下分别求解。问题 4-2 中，两阶段随机规划方案总费用为 1499.55 万元，较点预测方案降低 55.47 万元；问题 4-3 引入日内更新后，部署方案总费用为 1571.61 万元，较相应不更新方案日均降低 2365.50 元。

综合上述结果，本文针对微网运行中信息逐步揭示的特点，分别采用确定性优化、随机规划和滚动优化处理不同阶段的调度决策，并通过储能充放电互补约束的精确松弛实现连续线性化求解，为非孤岛式微网的日前与日内经济调度提供决策依据。

关键词：微网调度；精确松弛；两阶段随机规划；滚动优化；电价预测；线性规划

一、问题重述

1.1 问题背景

微网将光伏、储能和本地负载统一调度，可利用本地光伏供电，并通过储能调节供需差额。光伏出力随天气和时刻变化，负载高峰也未必与发电高峰重合。因此，仅依靠光伏难以持续满足负载，还需要协调储能与外网购电。

非孤岛式微网可在低电价或光伏富余时充电，在高电价时放电，以降低购电费用。但负载波动和光伏预测误差会使实际供需偏离计划。供电不足时，微网还需按惩罚电价紧急购电。因此，购电计划既要利用峰谷价差，也要考虑预测误差。若日内获得新的光伏预报，还需比较调整计划的收益与费用。本文据此研究固定电价和波动电价下的购电及储能调度策略。

1.2 问题要求

问题一：根据已知的日内电价、小区负载和光伏预测数据，制定当天 0:00 的购电与储能充放电计划。计划需满足负载需求，并保证 0:00 和 24:00 储电量相同。按指定格式给出各时段购电量、全日购电量、购电费和 0:00 储电量。

问题二：电价仍按日重复，但小区负载和光伏出力在一天内随时间变化。微网向小区提供的电能不得低于负载；若出现负载缺口，需按缺口时刻电价的 5 倍从外网紧急购电，产生紧急购电费用。计划购电量按对应时刻电价结算。要求给出每天 0:00 形成的当天计划购电策略。

问题三：在问题二基础上，从每天 0:00 开始每隔 6 个小时获得一次光伏预报，每次预报覆盖未来 24 小时整点。微网可先按 0:00 预报制定购电计划，也可根据后续预报调整购电量。若计划购电量高于最终调整购电量，差额部分按对应时刻电价的 50% 结算；若调整购电量高于计划购电量，差额部分按对应时刻电价的 1.5 倍结算。建立当天购电策略模型，并分析引入其他时刻预报是否必要。

问题四：外部电网电价改为实时波动。针对该电价环境建立微网当天购电策略模型，并利用附件 2 的负载与光伏数据、附件 3 的预报数据和附件 4 的实时电价，重新求解问题二和问题三。

二、问题分析

问题一的分析

问题一中，电价与小区负载按日重复，光伏预测曲线已知，要求在当天 0:00 确定当日购电计划。首先，以计划购电量、储能充放电量等作为决策变量，以计划购电费用最小为目标函数；接着，根据微网的实际运行过程，建立能量平衡、储能状态递推、储能容量、充放电功率及充放电互斥等约束条件，同时满足每日 0:00 和 24:00 储电量均为 6000kWh。由此建立以计划购电费用最小为目标的混合整数线性规划模型。该问仅涉及单日单场景调度，模型规模较小，故保留充放电互斥的二进制变量直接求解，所得结果同时作为后续各问精确松弛处理的对照基准。

问题二的分析

问题二要求在当天 0:00 制定购电计划，此时当日负载和光伏出力尚未知，不能直接按实际曲线安排购电。为获得当天的供需基准，先利用历史数据预测负载和光伏的日

内变化；预测值仍会与实际运行产生偏差，因此进一步利用历史预测误差构造若干可能的运行场景。在此基础上，将 0:00 确定的计划购电量作为第一阶段决策，将各场景下的储能调度和紧急购电量作为第二阶段决策，建立两阶段随机规划模型。实际运行时，计划购电量保持不变，储能根据实时负载和光伏出力调整，24:00 的实际储电量传递至下一日。由于场景数使模型规模明显扩大，储能充放电互补约束采用 5.2 节的精确松弛处理，不引入二进制变量，模型仍为线性规划。

问题三的分析

问题三允许利用日内新预报调整购电计划。光伏预报每天在 0:00、6:00、12:00 和 18:00 发布，每次覆盖未来 24 小时。增购部分按交易时刻电价的 1.5 倍结算，减购部分按 0.5 倍退款。因此，需要比较紧急购电费的减少量与调整带来的额外费用。给定更新时刻后，对剩余时段的购电和储能调度进行滚动优化，储能充放电互补约束沿用 5.2 节的精确松弛处理，更新子问题仍为线性规划。再对不同更新时刻组合开展历史回测，比较费用，判断哪些预报值得采用。

问题四的分析

问题四要求在电价逐日变化的条件下制定购电策略。由于当天 0:00 只知道历史实际电价，故需要在前两问的基础上预测当日电价。因此先用历史电价预测当日的时段价格，按预测电价决策购电计划与储能调度，实际运行后再按真实电价结算，同时按当日预测电价计算日末储电量价值。随后沿用问题二和问题三的构建相应的线性规划决策模型求解。

三、模型假设

时段离散：全天划分为 144 个十分钟时段，各时段内电价、负载、光伏功率及储能充放电功率取常值。

储能简化：储能充放电效率均为 90%，不考虑自放电、温升、老化及循环寿命等因素，储电量按时段递推。

微网聚合：微网按单节点处理，负载与光伏均采用小区聚合功率，忽略线路损耗、无功功率、电压约束以及设备启停和爬坡限制。

状态连续：储能状态在相邻日期间连续传递，每日 0:00 的储电量取前一日末值，仅问题一要求日初、日末储电量相同。

购售电限制：微网仅从外部电网购电，不向电网售电，光伏富余电量由储能吸收或弃去。

四、符号说明

表 1 主要符号说明

符号	符号含义及说明
t	时段序号
d	日期序号
L_t 、 V_t 、 $L_t^{(s)}$ 、 $V_t^{(s)}$	第 t 个时段正常日/场景 s 下的负载功率与光伏功率 (kW)
p_t 、 $\tilde{p}_t$	第 t 个时段的真实电价与预测电价 (元/kWh)
a_t 、 $a_t^{(0)}$ 、 a_t^{old} 、 a_t^{new}	第 t 个时段的计划购电量 (kWh)
C_t 、 D_t 、 $C_t^{(s)}$ 、 $D_t^{(s)}$	第 t 个时段正常日/场景 s 下的储能充电量与放电量 (kWh)
E_t 、 $E_t^{(s)}$	第 t 个时段末正常日/场景 s 下的储电量 (kWh)
E_0	窗口初态储电量 (kWh)
$e_t^{(s)}$	场景 s 下第 t 个时段的紧急购电量 (kWh)
η	储能充放电效率
ρ 、 ρ_d	日末储电量价值系数 (元/kWh)
S	场景总数
s	场景编号
r	更新时刻编号
t_r	第 r 次更新前已执行的时段数
$\Delta_{r,t}^+$ 、 $\Delta_{r,t}^-$	第 r 次更新中第 t 个时段的增购量与减购量 (kWh)
a_t^{ch} 、 a_t^{dc}	第 t 个时段充电与放电的边际成本 (元/kWh)
κ_t^0	第 t 个时段的边际循环成本 (空转代价, 元/kWh)
Q 、 Q^R	原混合整数模型与松弛模型的最优目标值 (元)
R	全年各日各时段同时充放电幅度的最大值 (核验指标, kWh)

五、模型的建立与求解

5.1 数据预处理与分析

附件包含负载功率、光伏实际功率、光伏预测功率和外网电价。数据分为单日、全年逐日记录及日内不同发布时刻的光伏预报，分别用于后续各问的建模与求解。

缺失值与异常值处理

对负载、光伏与电价数据进行完整性检查，未发现需要插补或修正的缺失、异常数据。

光伏预报按发布日分块记录，每日占连续四行。日期仅填在首行，其余三行留空。按块向下填充即可恢复日期，这类空白不属于缺失值。

光伏实际功率受时间变化影响，按昼夜分别检查。夜间核查是否出现异常发电，白天核查功率是否非负、取值范围是否合理。结果未发现需要删除或修正的样本。

实际电价既有接近零的低值，也有超过 1.5 元/kWh 的高值。这些数值反映价格的真实波动，不作截尾或缩尾处理。

检查后保留全部原始观测值，用于后续调度计算。

量纲转换

调度以十分钟为基本时段，全天分为 144 个时段，单个时段的长度为

$$\Delta t = \frac{10}{60} = \frac{1}{6} \text{ h.} \quad (1)$$

记第 t 个时段的负载功率、光伏功率和外网电价分别为 L_t 、 V_t 和 p_t 。前两者单位为

kW，电价单位为元/kWh。为统一购电和储能计算，将功率换算为时段电量：

$$\ell_t = L_t \Delta t, \quad v_t = V_t \Delta t, \quad (2)$$

其中 ℓ_t 与 v_t 分别表示该时段的负载电量与光伏电量，单位均为 kWh。后文中负载与光伏出力采用功率单位 kW 表示，购电量、充放电量与储能状态采用电量单位 kWh 表示。

时间尺度统一

附件中的数据包含时间类型和文本类型，需要统一数据格式。由于各问均以每日 0:00 为调度起点，首先将时间换算为距当日 0:00 的分钟数，随后以 10 min 为一个调度时段，全天共 144 个，故约定次日 0:00 记为 1440 min。原始数记第 t 个时段的起始时刻为

$$\tau_t = 10(t - 1), \quad t = 1, 2, \dots, 144, \quad (3)$$

则第 t 个时段覆盖区间

$$I_t = [\tau_t, \tau_t + 10 \text{ min}), \quad (4)$$

逐日记录由日期与日内时段共同定位，日期记为 $d = 1, 2, \dots, 365$ ，每条记录对应唯一的 (d, t) 。

数据分析

总体分布特征

附件 1 的日曲线与全年同时段均值基本一致，可代表平均日内运行水平，记为典型日。统计得到的负载、光伏、净负荷和电价的描述统计见表 2，其中净负荷为负载功率减去光伏功率。

表 2 全年 52560 个时段的变量描述统计

	均值	中位数	5% 分位	95% 分位	最小值	最大值	标准差
负载功率 (kW)	4626.03	4270.86	2649.87	6918.46	1995.72	7978.88	1411.75
光伏功率 (kW)	2311.79	18.86	0	7596.01	0	10216.20	2818.87
净负荷 (kW)	2314.25	3126.76	-2582.22	5277.60	-6601.99	6687.75	2432.80
电价 (元/kWh)	0.77	0.74	0.34	1.34	0.0076	1.7936	0.34

由表 2 可知，负载功率均值为 4626.03 kW，标准差为 1411.75 kW，存在较明显的波动；光伏功率中位数仅为 18.86 kW，最大值达到 10216.20 kW，出力变化范围较大；电价在 0.0076—1.7936 元/kWh 之间变化，峰谷差异明显。

变量相关性

负载、光伏和电价的日内变化存在部分同步，线性相关程度用 Pearson 相关系数衡量，见图 1。

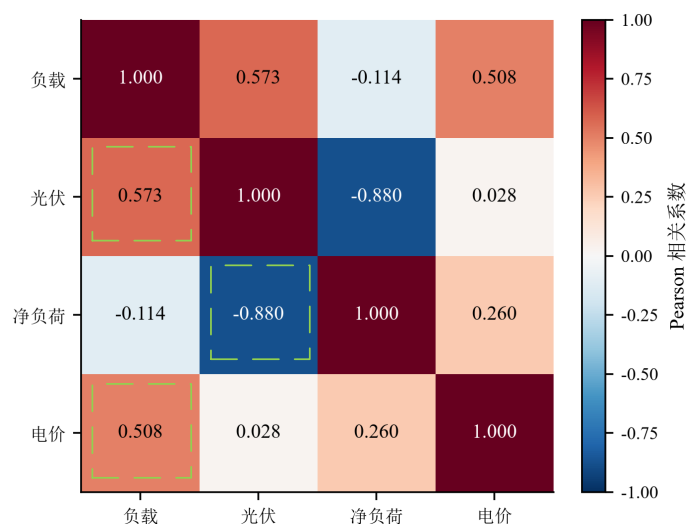


图 1 全年运行变量的 Pearson 相关系数热力图

从图中观察得到，负载功率与光伏功率的相关系数为 0.573，呈中等正相关；电价与负载功率的相关系数为 0.508，而电价与光伏功率的相关系数仅为 0.028，二者线性相关性很弱。对于负载、光伏与电价，日间波动主要与时间有关，故相关性主要来源于时间相关。净负荷与光伏功率的相关系数为 -0.880 ，呈较强负相关，表示光伏出力增加会相应降低净负荷。

日内与季节特征

光伏和负载的年内变化先从全年时空分布看起。图 2 给出全年光伏功率的逐日逐时段分布，出力集中在每日午间，冬季整体偏低；图 3 给出全年负载功率的对应分布，每日白天形成高位平台，年内波动明显小于光伏。

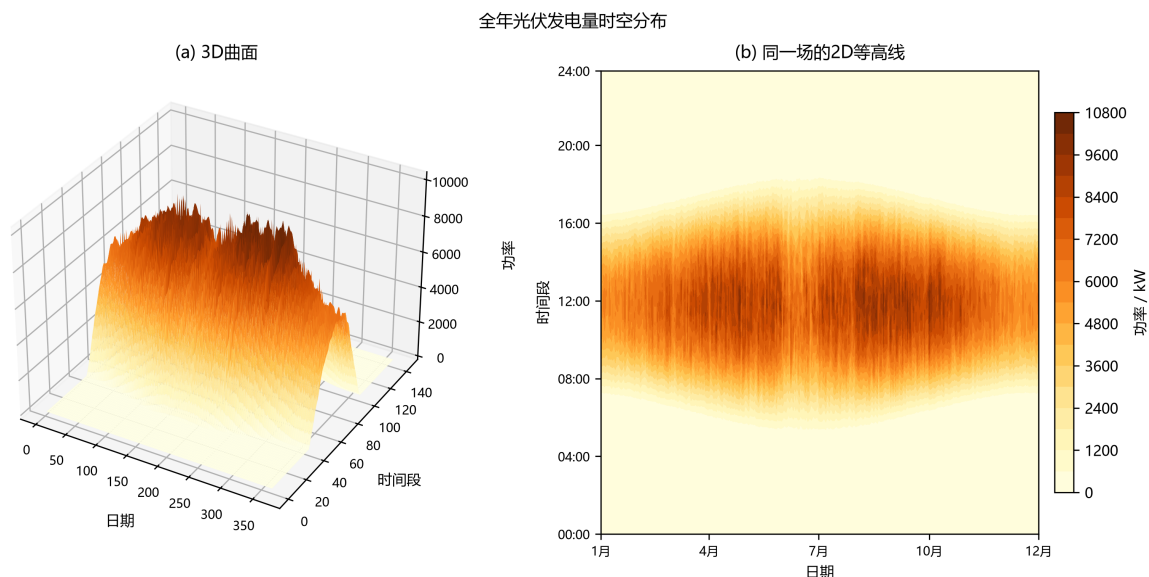


图 2 全年光伏功率时空分布

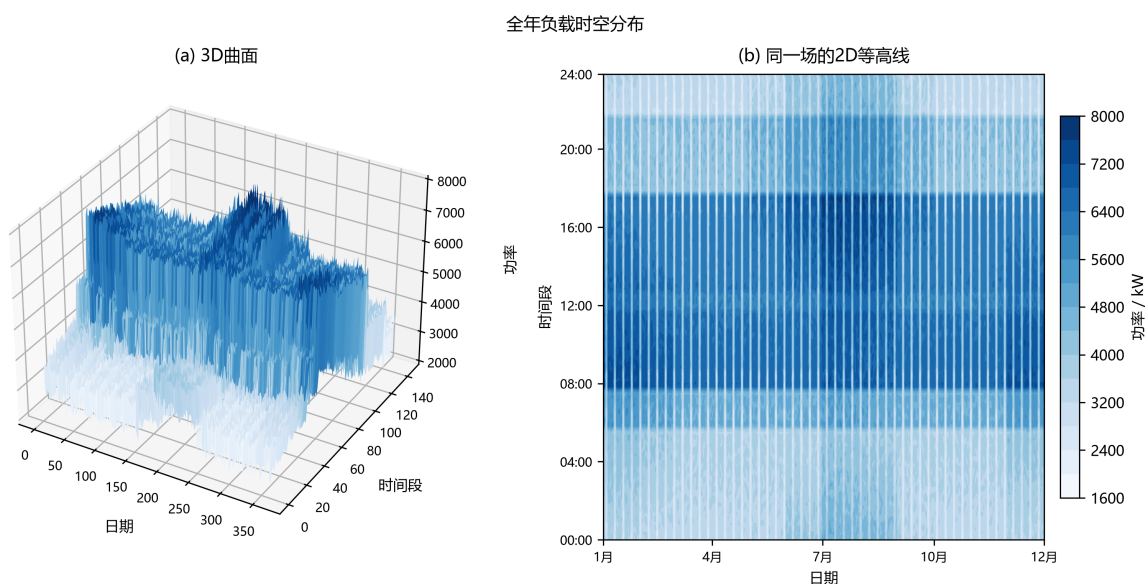


图3 全年负载功率时空分布

按季节汇总，图4给出四个季度光伏与负载每日总能量的箱线分布。四个季度的负载中位数接近，均在 27×10^4 — 40×10^4 kWh 之间；光伏的季度间差异明显，第三季度日总发电量最高，第一季度最低。日内尺度上，图5给出四个典型日的光伏、负载与净负荷曲线，午间光伏推低净负荷，夏季典型日午间净负荷为负；图??单独比较四季典型日的负载曲线，冬季与过渡季白天负载明显高于夏季。典型日电价在 5:40 最低，为 0.3713 元/kWh；在 20:40 最高，为 1.3952 元/kWh。峰谷比为 3.76。这一价差使低价充电、高价放电能够明显摊薄购电费用，是各问储能调度的共同费用来源。

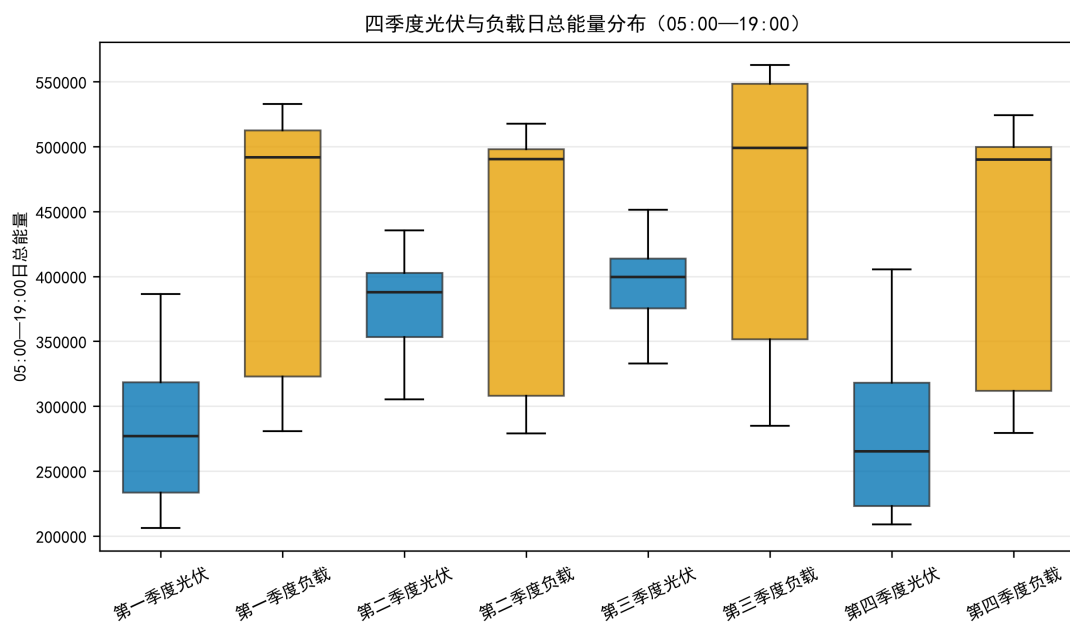


图4 四季度光伏与负载日总能量分布箱线图

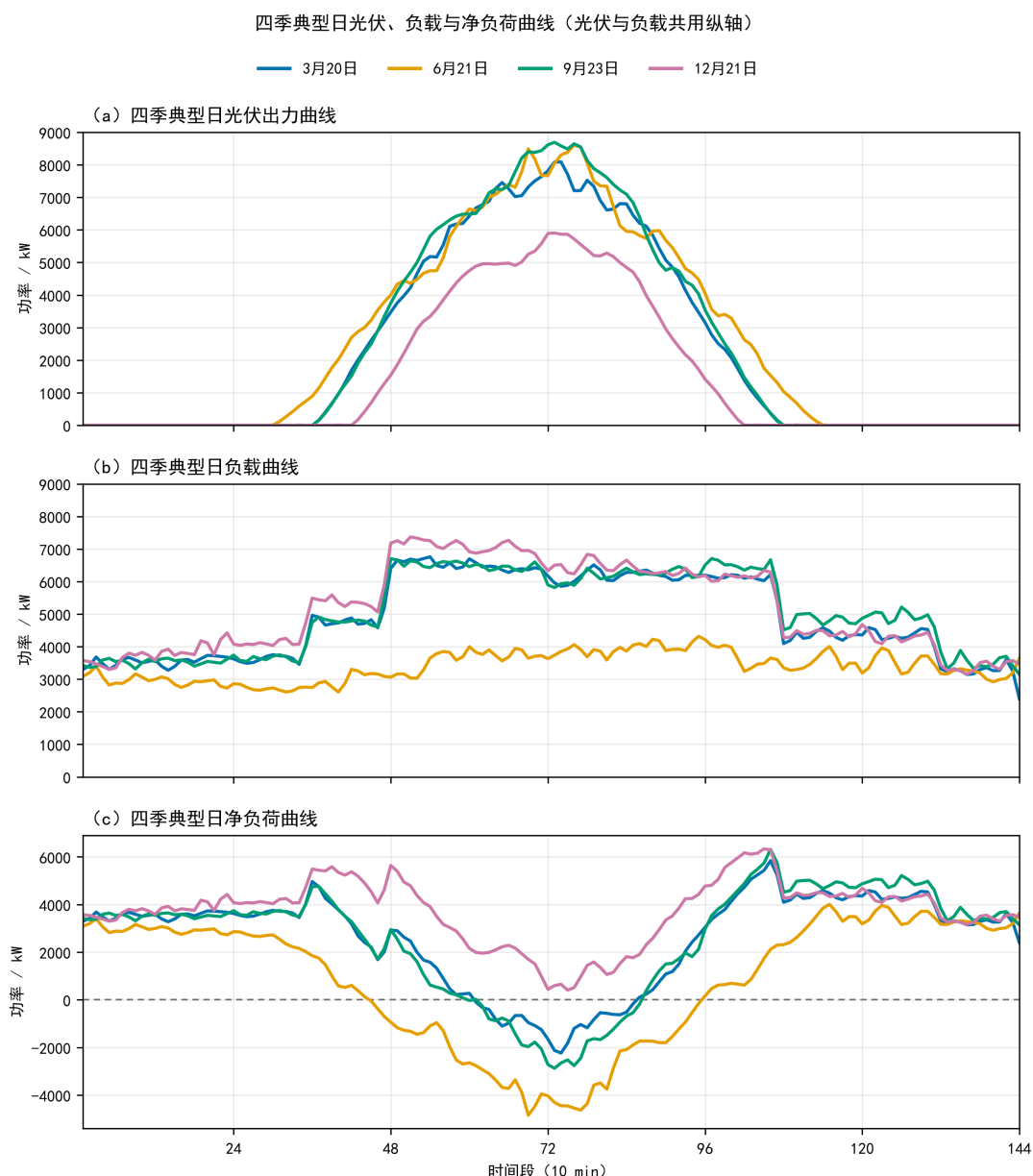


图 5 四季典型日光伏、负载与净负荷曲线

按周统计，工作日平均负载高于周末，光伏和电价的周际差异较小。按月统计，光伏夏季较高、冬季较低，负载相对平稳。全年 19.27% 的时段光伏超过负载，其余 80.73% 的时段存在供电缺额。部分时段的缺额功率超过储能的 5000 kW 功率上限，仍需外网供电。全年光伏电量与负载电量之比为 0.500，各日比值也均低于 1。因此，光伏在总量上不足，但日内仍有部分时段出力过剩。

净负荷与能量供需

净负荷的全年时空分布见图 6。午间负值带与夜间正值带交替出现，夜间缺额幅度大于午间盈余幅度。

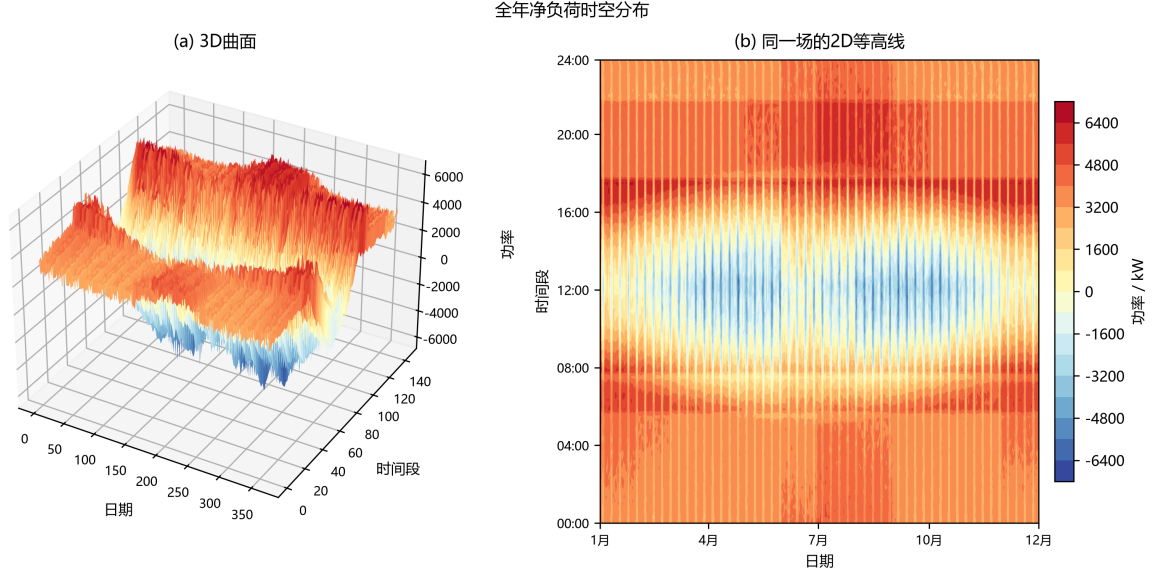


图 6 全年净负荷时空分布

5.2 储能充放电互补约束的精确松弛

储能同一时段不能同时充、放电，其物理约束为

$$0 \leq C_t \perp D_t \geq 0 \iff C_t D_t = 0, \quad (5)$$

其中单时段充放电电量上界为 $M = 5000\Delta t = 833.3333 \text{ kWh}$ 。传统线性化方法需引入二进制变量

$$0 \leq C_t \leq M y_t, \quad 0 \leq D_t \leq M(1 - y_t), \quad y_t \in \{0, 1\}. \quad (6)$$

若去除 y_t ，仅保留 $0 \leq C_t, D_t \leq M$ ，则得到原混合整数模型的线性松弛，其最优值满足 $Q^R \leq Q$ 。若进一步有 $Q^R = Q$ ，则称该松弛为精确松弛。

设 $\Delta E_t = E_t - E_{t-1}$ ，相同储电量变化下的最小充放组合为

$$C_t^0 = \frac{[\Delta E_t]^+}{\eta_c}, \quad D_t^0 = \eta_d [-\Delta E_t]^+. \quad (7)$$

任一松弛解均可表示为

$$C_t = C_t^0 + z_t, \quad D_t = D_t^0 + \eta_c \eta_d z_t, \quad z_t \geq 0, \quad (8)$$

其中 z_t 表示不改变储电量的同时充放电空转量。若充、放电边际成本分别为 a_t^{ch} 和 a_t^{dc} ，则增加 z_t 所产生的目标函数变化为

$$\kappa_t^0 z_t, \quad \kappa_t^0 = a_t^{\text{ch}} + \eta_c \eta_d a_t^{\text{dc}}. \quad (9)$$

因此，当 $\kappa_t^0 \geq 0$ 时，空转不会改善目标值；若 $\kappa_t^0 > 0$ 对所有 t 成立，则松弛最优解必不存在同时充放电，故线性松弛为精确松弛。

本文微网仅从电网购电且不向电网售电，因此

$$a_t^{\text{ch}} = p_t, \quad a_t^{\text{dc}} = -p_t. \quad (10)$$

由 $\eta_c = \eta_d = 0.9$ 得

$$\kappa_t^0 = p_t - 0.81p_t = 0.19p_t > 0, \quad (11)$$

其中附件 1、附件 4 的电价均为正值，故所有调度时段均满足精确松弛条件。该条件仅由电价和储能效率决定，与负载、光伏出力及随机场景无关。因此，问题二至问题四均

可去除充放电二进制变量而不改变最优目标值。

基于上述结论，本文采用差异化求解策略：问题一为单日单场景确定性调度，规模较小，保留 144 个二进制互斥变量，采用 MILP 求解并作为基准；问题二至问题四随着随机场景、滚动优化及全年时序数据的引入，整数变量将被反复处理，因此利用精确松弛转化为连续 LP，以降低计算负担。具体处理方式见表 3。

表 3 四个问题的储能互补约束处理方式

问题	处理	模型	原因
一	保留互斥	MILP	单日单场景，144 个二进制变量，规模可接受
二	精确松弛	LP	10 个场景重复求解，整数变量数量显著增加
三	精确松弛	LP	滚动优化，多次重复处理互斥变量
四	精确松弛	LP	全年逐日优化，叠加时变电价，计算规模最大

为避免线性规划的数值退化，在问题二至问题四的目标函数中加入极小吞吐量正则项

$$\tau \sum_t (C_t + D_t), \quad \tau = 10^{-6} \text{ } \square/\text{kWh}, \quad (12)$$

此时 $\kappa_t^{0,\text{eff}} = 0.19p_t + 1.81\tau > 0$ ，故正则项仅用于数值稳定，并非精确松弛成立的必要条件。最终对求解结果进行逐时段核验，定义

$$R = \max_{d,t} \min(C_{d,t}, D_{d,t}). \quad (13)$$

问题二、三共核验 78681 个时段，问题四核验 75566 个时段，均得到 $R = 0 \text{ kWh}$ ，进一步验证了连续 LP 求解结果满足充放电互补条件。

5.3 问题一的建模分析与求解

问题一的建模路线见图 7：由日内电价、小区负载、光伏预测与储能初始状态出发，定义决策变量并建立混合整数线性规划模型，求解后输出各时段购电量与调度方案。

问题一：确定性日内购电与储能调度流程图

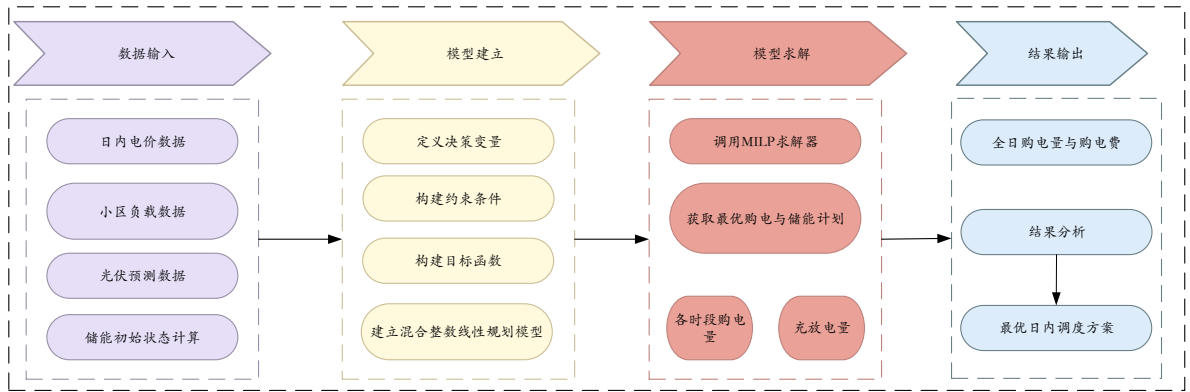


图 7 问题一建模流程图

5.3.1 决策变量

对 5.1 划分的第 t 个时段 $(t = 1, 2, \dots, 144)$ ，引入如下决策变量。

(1) 计划购电量 a_t 该变量表示第 t 个时段微网向外部电网购买的电量，单位为 kWh。 $a_t \geq 0$ ，表示微网只购电、不向电网售电。

(2) 储能充电量 C_t 与放电量 D_t 该变量分别表示第 t 个时段储能的充电量和放电量，

单位为 kWh。二者均非负，且同一时段不能同时充放电。

(3) 时段末储电量 E_t 该变量表示第 t 个时段结束时储能中的电量，单位为 kWh。日初储电量 E_0 即 0:00 时刻的储电量。

(4) 弃光量 w_t 该变量表示第 t 个时段未能消纳的光伏电量，单位为 kWh。保留该变量是为了覆盖后续高光伏日出力过剩的情形。

(5) 充电指示变量 y_t 该变量为二进制变量。 $y_t = 1$ 表示第 t 个时段储能处于充电状态， $y_t = 0$ 表示处于放电或空闲状态。

5.3.2 目标函数

本模型以全天计划购电费用最小为优化目标，其数学表达式为

$$\min \sum_{t=1}^{144} p_t a_t, \quad (14)$$

其中 p_t 与 a_t 分别表示第 t 个时段的外网电价与计划购电量。

5.3.3 约束条件

能量平衡约束：为保证小区负载得到充分供电，同时合理分配光伏发电和储能充放电，各时段的供能与用能应保持平衡，有

$$a_t + v_t + D_t = \ell_t + C_t + w_t, \quad t = 1, 2, \dots, 144, \quad (15)$$

其中 ℓ_t 与 v_t 由式 (2) 给出。左侧为计划购电量 a_t 、光伏电量 v_t 和储能放电量 D_t ，表示电量来源。右侧为负载电量 ℓ_t 、充电量 C_t 和弃光量 w_t ，表示电量去向。

储能状态递推约束：为描述储能电量随充放电过程的动态变化，考虑储能单向充放电效率 $\eta = 0.9$ ，有递推关系

$$E_t = E_{t-1} + \eta C_t - \frac{D_t}{\eta}, \quad t = 1, 2, \dots, 144. \quad (16)$$

充电后实际存入的电量为 ηC_t ，放电时实际消耗的储电量为 D_t/η ，两项分别计入充、放电损耗。

储能容量约束：储电量需保持在附录 1 给出的运行范围内，即

$$1200 \leq E_t \leq 10800, \quad t = 1, 2, \dots, 144. \quad (17)$$

充放电功率约束：由最大充放电功率 5000 kW 与时段长度 $\Delta t = 1/6$ h 得单个时段的最大充放电量 $M = 5000 \Delta t = 833.3333$ kWh，因此

$$0 \leq C_t \leq M, \quad 0 \leq D_t \leq M, \quad t = 1, 2, \dots, 144. \quad (18)$$

充放电互斥约束：同一时段不能同时充放电，即

$$C_t \leq M y_t, \quad D_t \leq M (1 - y_t), \quad y_t \in \{0, 1\}, \quad t = 1, 2, \dots, 144. \quad (19)$$

弃光约束：弃光量不超过该时段的光伏发电量，即

$$0 \leq w_t \leq v_t, \quad t = 1, 2, \dots, 144. \quad (20)$$

日循环约束：24:00 与次日 0:00 为同一时刻，题目要求 0:00 与 24:00 的储电量相同，即

$$E_{144} = E_0, \quad (21)$$

初始状态约束：题目给定每日 0:00 的储电量为 6000 kWh，即

$$E_0 = 6000. \quad (22)$$

5.3.4 混合整数线性规划模型

将目标函数式 (14) 与上述约束合并，得到以下模型。模型含二进制变量 y_t ，目标函数和约束均为线性，属于混合整数线性规划。

$$\begin{aligned} \min \quad & \sum_{t=1}^{144} p_t a_t \\ \text{s.t.} \quad & a_t + v_t + D_t = \ell_t + C_t + w_t, \\ & E_t = E_{t-1} + \eta C_t - D_t / \eta, \\ & 1200 \leq E_t \leq 10800, \\ & 0 \leq C_t \leq M, \quad 0 \leq D_t \leq M, \\ & C_t \leq M y_t, \quad D_t \leq M(1 - y_t), \\ & 0 \leq w_t \leq v_t, \quad a_t \geq 0, \\ & E_0 = 6000, \quad E_{144} = E_0, \\ & y_t \in \{0, 1\}, \quad t = 1, 2, \dots, 144. \end{aligned} \quad (23)$$

5.3.5 模型求解

问题一的求解流程见图 8。

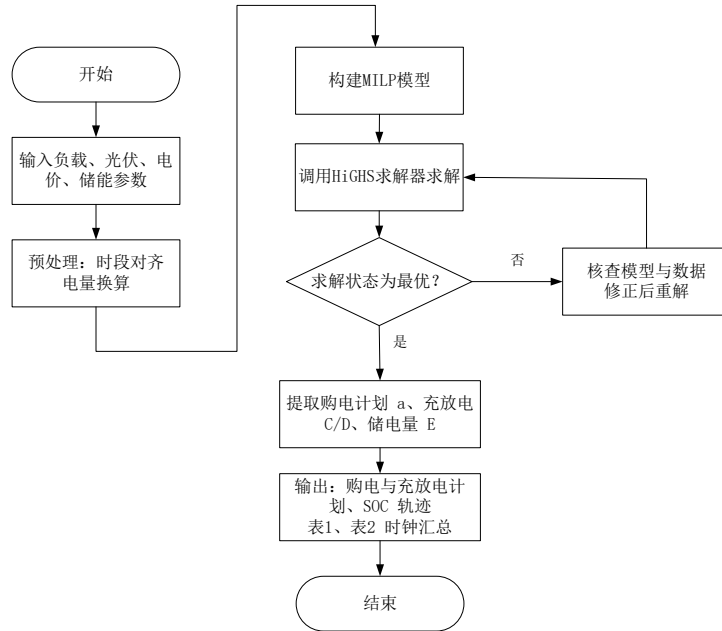


图 8 问题一求解流程图

问题一仅考虑单日确定性场景，优化时域为 24 h，时间尺度离散为 10 min，且仅包含单一负载与光伏出力序列，模型规模相对较小。求解采用 HiGHS 求解器求解上述混合整数线性规划，相对容差设为 10^{-6} 。模型在 0.07 s 内返回全局最优解，最优性间隙为 0。

问题一的混合整数模型同时可作为精确松弛的对照基准。在同一算例与同一模型设

定下，将式 (19) 的二进制互斥约束去除后以连续线性规划独立重解，两种模型的最优目标值与最大同时充放电量见表 4。

表 4 问题一 MILP 与精确松弛 LP 的对照

模型	最优目标值（元）	最大同时充放电量（kWh）
MILP	35126.84858928963	0
精确松弛 LP	35126.84858928964	0
相对差	2.1×10^{-16}	—

两种模型的目标值一致，松弛解中

$$\max_t \min(C_t, D_t) = 0 \text{ kWh}, \quad (24)$$

即未出现同时充放电。表 4 与式 (24) 均与 5.2 节的理论结论相符，也支持在模型规模扩大的问题二至问题四中去除二进制变量、采用连续线性规划求解。

5.3.6 求解结果

求解结果见表 5 与表 6。

表 5 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0.0000	12:00-12:10	486.4029	14:00-14:10	0.0000
16:00-16:10	394.9315	18:00-18:10	636.9826	20:00-20:10	0.0000
全天购电量		59482.6990	全天购电费		35126.8486

表 6 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（单位：kWh）

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4500.0000	0.0000	4:00-8:00	833.3333	5947.4198
8:00-12:00	3954.6310	2121.4184	12:00-16:00	6119.3685	91.1014
16:00-20:00	0.0000	5068.6869	20:00-24:00	5333.3333	3571.3131
0:00 储电量		6000.0000	24:00 储电量		6000.0000

表 5 中有三个时段无需购电。10:00—10:10 与 14:00—14:10 的光伏高于负载，剩余电量用于充电。20:00—20:10 无光伏出力，711.4450 kWh 负载全部由储能供给。由表 6 可见，充电主要安排在低价和午间光伏充足时段，放电主要安排在早晚高峰。下图为充放电瀑布图，图中反映的是电池的储电量变化，由于存在 0.9 倍的充放电损耗，电池充放电与计划充放电存在 0.9 倍的关系。

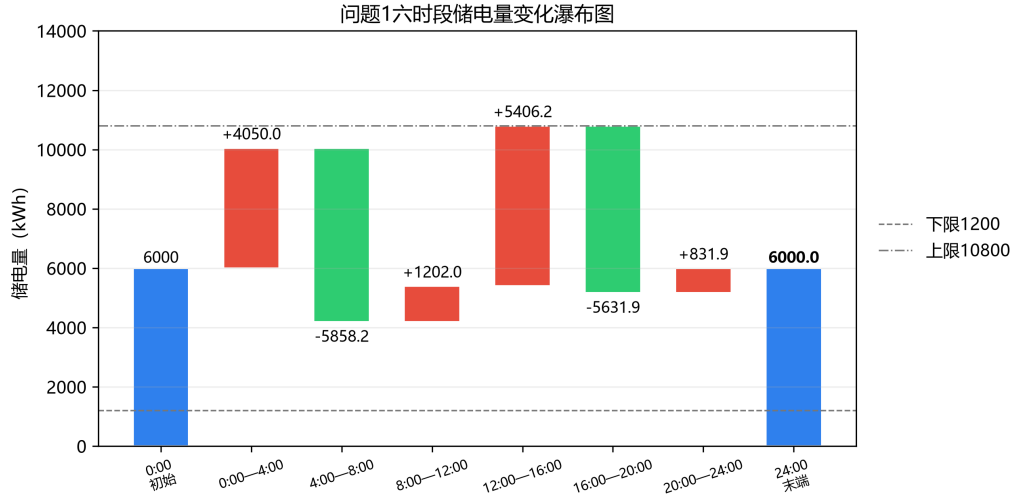


图 9 问题一六个小时的储电量变化瀑布图

5.4 问题二的建模分析与求解

问题二需要在负载和光伏出力未知的情况下提前制定购电计划，日期跨度提升至365天。据数据分析，不同日期的负载和光伏差异较大，仅按平均曲线安排购电会系统性偏离实际运行。求解因此分两步：先按日预测当天的负载与光伏曲线，作为购电计划的依据；再利用历史预测误差构造场景，覆盖预测与实际的偏差，使计划对波动留有余量，减少实际运行中的紧急购电。运行过程中计划购电量保持不变，储能根据实时负载、光伏出力和储电量调整充放电，日末实际储电量传递至次日。建模流程见图 10。

问题二：两步预测及两阶段随机规划流程图

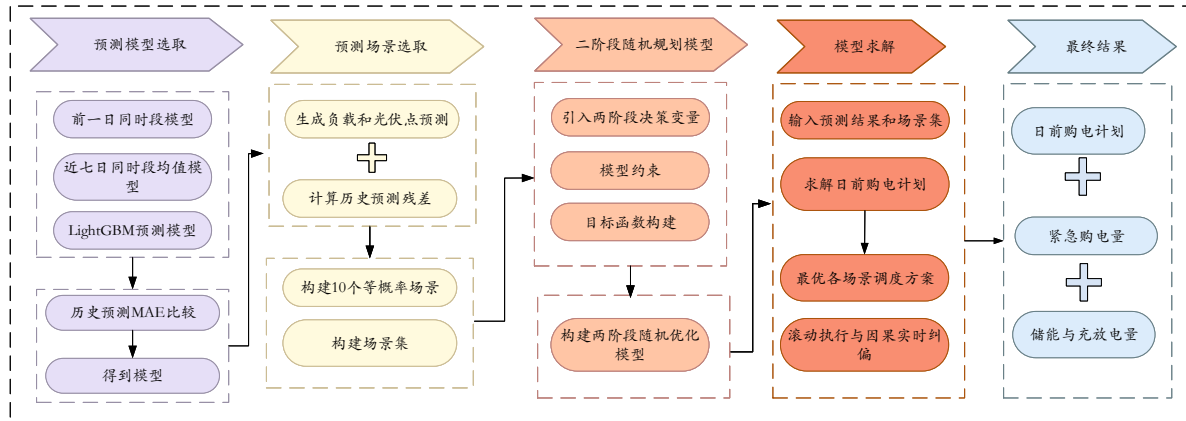


图 10 问题二建模流程图

5.4.1 当日曲线的点预测

负载和光伏均具有明显的日内周期性，可先利用历史同期数据预测目标日 d 全部144个时段。前一日与目标日在时间上最接近，取前一日同一时段功率作为第一种预测；单日取值可能受偶然波动影响，再取前7个完整历史日同一时段的均值作为第二种预测，以平滑单日异常。除周期变化外，负载和光伏还受星期、年内变化和近期运行状态的影响，为此引入 LightGBM 回归模型，结合多种历史特征给出第三种预测。

LightGBM 是基于梯度提升决策树的集成学习方法，通过逐步构建决策树拟合前一

轮预测的残差，并将各棵树的输出加权组合，其预测函数为

$$f(X) = \sum_{m=1}^M \gamma_m h_m(X), \quad (25)$$

其中 $h_m(X)$ 为第 m 棵决策树， γ_m 为对应权重， M 为决策树数量。模型输入取时段编号、日内周期正余弦项 $\sin(2\pi t/144)$ 与 $\cos(2\pi t/144)$ 、星期与年内序号，以及前 1、2、7 日同一时段功率、前 7 日同时段功率的均值与标准差，训练样本随日期扩展，每 7 天更新一次。

三种方法分别代表近期历史、历史均值与多特征预测，各有适用情形。为避免预先假定某一种始终最优，每天 0:00 根据前一日及以前已经完成预测的历史结果计算三种方法的 MAE，选取 MAE 最小的方法作为当日预测方法：

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (26)$$

记目标日 d 的负载与光伏点预测功率为 $\hat{L}_t^d$ 与 $\hat{V}_t^d$ ，二者作为场景生成的基准曲线，并用于日内滚动执行时对未发生时段的预测。

5.4.2 场景生成

点预测只给出一条基准曲线，实际负载和光伏仍会围绕它波动。为把这种波动引入计划评估，利用历史日的预测误差描述偏差：对每个已结束日 d' ，记其负载与光伏的点预测功率为 $\hat{L}_t^{d'}$ 与 $\hat{V}_t^{d'}$ 、实际功率为 $L_t^{d'}$ 与 $V_t^{d'}$ ，定义整日残差曲线

$$\delta_t^L(d') = L_t^{d'} - \hat{L}_t^{d'}, \quad \delta_t^V(d') = V_t^{d'} - \hat{V}_t^{d'}, \quad t = 1, 2, \dots, 144, \quad (27)$$

残差按历史日成组出现，保留了一天误差的整体结构，因此直接以历史日为抽样单位。对目标日 d ，从截至 $d-1$ 日形成的残差样本中等概率抽取 $S=10$ 个历史日，记第 s 个抽中的日期为 d_s ，将其残差叠加至目标日点预测，光伏功率截为非负：

$$L_t^{(s)} = \hat{L}_t^d + \delta_t^L(d_s), \quad V_t^{(s)} = \max\{0, \hat{V}_t^d + \delta_t^V(d_s)\}, \quad (28)$$

曲线 $(L_t^{(s)}, V_t^{(s)})_{t=1}^{144}$ 即第 s 条当日场景，换算为电量 $\ell_t^{(s)} = L_t^{(s)} \Delta t$ ， $v_t^{(s)} = V_t^{(s)} \Delta t$ 。负载与光伏从同一历史日期抽取残差，使两者保留该日的共同变化特征。

5.4.3 决策变量

(1) 计划购电量 a_t a_t 为目标日 0:00 确定且日内不变的第 t 个时段计划购电量，单位为 kWh，且 $a_t \geq 0$ 。

(2) 场景紧急购电量 $e_t^{(s)}$ $e_t^{(s)}$ 为场景 s ($s = 1, 2, \dots, S$) 下第 t 个时段供电不足时按 5 倍电价补购的电量，单位为 kWh。

(3) 场景充放电量 $C_t^{(s)}$ 与 $D_t^{(s)}$ 二者分别为场景 s 下第 t 个时段储能的充电量与放电量，单位为 kWh。

(4) 场景时段末储电量 $E_t^{(s)}$ $E_t^{(s)}$ 为场景 s 下第 t 个时段结束时的储电量，单位为 kWh，当日 0:00 的初值 E_0 由前一日递推得到。

5.4.4 目标函数

第一阶段计划购电按原电价结算，供电不足产生的紧急购电按 5 倍电价结算。考虑储能跨日运行，在目标函数中对日末储电量赋予续存价值：

$$\min \sum_{t=1}^{144} p_t a_t + \frac{1}{S} \sum_{s=1}^S \sum_{t=1}^{144} 5 p_t e_t^{(s)} - \rho \bar{E}_{144}, \quad \bar{E}_{144} = \frac{1}{S} \sum_{s=1}^S E_{144}^{(s)}. \quad (29)$$

式中，前两项分别表示计划购电费用和期望紧急购电费用。由于日末储电量可在次日继续用于供电，可将其等效为次日可节省的购电成本。将次日凌晨时段作为日末储电量的主要替代购电时段，取次日 0:00—5:00 的平均电价 0.433393 元/kWh，除以充电效率 $\eta = 0.9$ ，得单位储电量价值 0.4815 元/kWh；为消除等价最优解之间的数值退化，再按场景加入 0.0010 元/kWh 的微小续存项，两者共同构成末值价值系数 $\rho = 0.4815 + 0.0010 = 0.4825$ 元/kWh，且与式 (29) 中各场景按 $1/S$ 等权平均的口径一致。末项仅用于体现储能跨日续存价值，不计入实际运行费用。

此外，按 5.2 节式 (12)，目标函数中还含有储能吞吐量正则项 $\tau \sum_{s=1}^S \sum_{t=1}^{144} (C_t^{(s)} + D_t^{(s)})$ ， $\tau = 10^{-6}$ 元/kWh，用于排除等价最优解之间的数值退化。

5.4.5 约束条件

供能约束：各场景下，计划购电、紧急购电与储能放电共同满足净负荷与充电需求，

$$a_t + e_t^{(s)} + D_t^{(s)} \geq \ell_t^{(s)} - v_t^{(s)} + C_t^{(s)}, \quad s = 1, \dots, S, t = 1, \dots, 144, \quad (30)$$

其中 $\ell_t^{(s)}$ 与 $v_t^{(s)}$ 由式 (28) 的场景曲线换算得到。光伏出力高于负载与储能可吸收能力时，多余部分直接弃去。

储能状态递推约束：各场景的储能状态按下式逐时段递推，

$$E_t^{(s)} = E_{t-1}^{(s)} + \eta C_t^{(s)} - \frac{D_t^{(s)}}{\eta}, \quad E_0^{(s)} = E_0. \quad (31)$$

储电量运行区间约束：各场景各时段 $1200 \leq E_t^{(s)} \leq 10800$ 。

充放电功率约束：各场景各时段 $0 \leq C_t^{(s)} \leq M$ ， $0 \leq D_t^{(s)} \leq M$ 。

储能充放电互补约束（亦称逐槽充放互斥约束）：同一时段储能不能同时充电与放电，即

$$0 \leq C_t^{(s)} \perp D_t^{(s)} \geq 0,$$

其 0-1 形式为

$$C_t^{(s)} \leq M u_t^{(s)}, \quad D_t^{(s)} \leq M (1 - u_t^{(s)}), \quad u_t^{(s)} \in \{0, 1\};$$

其中 $u_t^{(s)}$ 为场景 s 下第 t 个时段的充放电状态指示变量。本文证明该约束在本算例条件下可由支配性精确松弛，故模型中不出现整数变量。

非负约束： $a_t \geq 0$ ， $e_t^{(s)} \geq 0$ 。购电量与紧急购电量的有限上界由供能约束自然得到。

5.4.6 完整模型

综合目标函数与上述约束，问题二的两阶段随机规划模型为

$$\begin{aligned}
 \min \quad & \sum_{t=1}^{144} p_t a_t + \frac{1}{S} \sum_{s=1}^S \sum_{t=1}^{144} 5p_t e_t^{(s)} - \rho \bar{E}_{144} \\
 \text{s.t.} \quad & a_t + e_t^{(s)} + D_t^{(s)} \geq \ell_t^{(s)} - v_t^{(s)} + C_t^{(s)}, \\
 & E_t^{(s)} = E_{t-1}^{(s)} + \eta C_t^{(s)} - D_t^{(s)} / \eta, \quad E_0^{(s)} = E_0, \\
 & 1200 \leq E_t^{(s)} \leq 10800, \\
 & 0 \leq C_t^{(s)} \leq M, \quad 0 \leq D_t^{(s)} \leq M, \\
 & a_t \geq 0, \quad e_t^{(s)} \geq 0, \\
 & s = 1, 2, \dots, S, \quad t = 1, 2, \dots, 144.
 \end{aligned} \tag{32}$$

第一阶段变量为计划购电量 a_t 。第二阶段变量包括各场景的紧急购电量 $e_t^{(s)}$ 、充放电量 $C_t^{(s)}$ 、 $D_t^{(s)}$ 和储电量 $E_t^{(s)}$ 。第一阶段确定计划购电量，第二阶段根据不同场景确定储能调度和紧急购电量，并对各场景运行费用取平均。

5.4.7 模型求解

逐日求解流程见图 11。

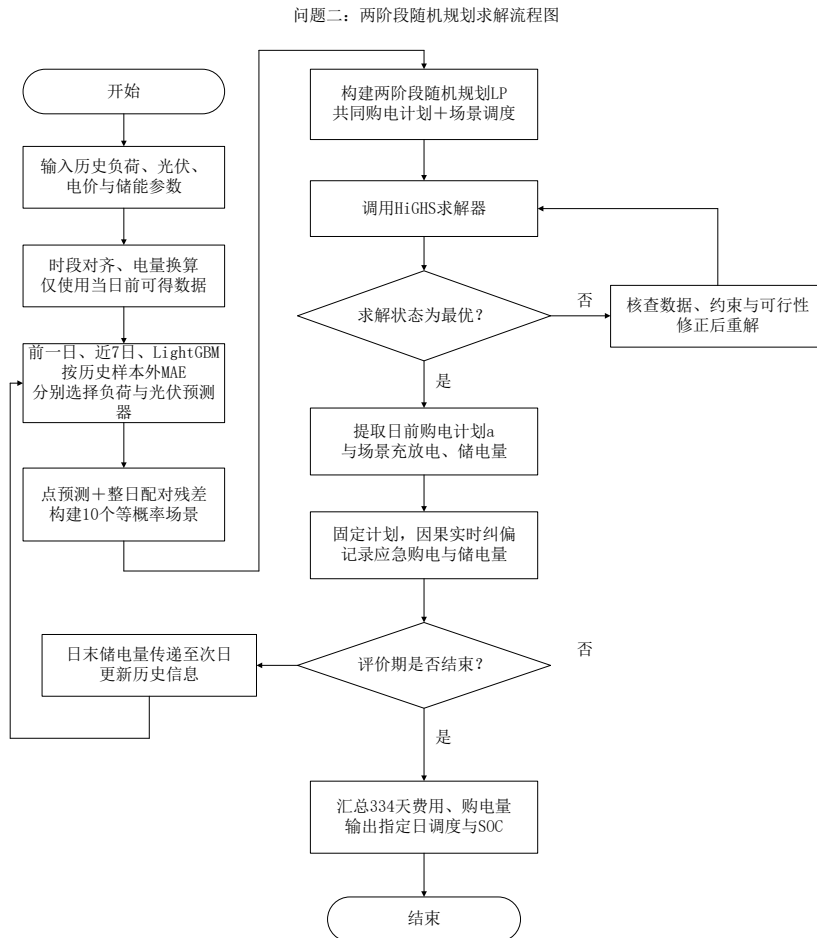


图 11 问题二求解流程图

模型逐日求解。每天由点预测和历史残差生成场景，使用 HiGHS 求解线性规划，得

到 0:00 购电计划 $\{a_t\}_{t=1}^{144}$ 。计划购电按对应时刻电价结算，日内保持不变。储能每 10 分钟根据实时负载、光伏和当前预测调整运行。若仍有供电缺额，则按 5 倍电价紧急补购。当日 24:00 的实际储电量传递至次日，作为下一日优化的初始状态。

5.4.8 求解结果

模型在 334 天评价期内的费用汇总见表 7。

表 7 问题二评价期（334 天）的费用汇总

计划购电费（万元）	紧急购电量（kWh）	紧急购电费（万元）	总费用（万元）	日均总费用（元/日）
1265.83	445178.91	159.09	1424.93	42662.54

四个指定日期为 2025-03-20、06-21、09-23 和 12-21。结果按题目表 1、表 2 和表 3 的格式列于表 8、表 9 和表 10。从四个指定日期的结果看，冬季负载较高、光伏较低，计划购电量和总费用随之增加，紧急购电多出现在净负荷变化较快或预测偏差较大的时段。

表 8 四个指定日期的购电量及全天购电量和购电费

时间段	购电量（kWh）			
	2025.3.20	2025.6.21	2025.9.23	2025.12.21
10:00-10:10	0.0000	0.0000	0.0000	0.0000
12:00-12:10	601.7000	0.0000	396.6400	1035.6300
14:00-14:10	0.0000	0.0000	0.0000	0.0000
16:00-16:10	484.1800	90.1200	534.6900	827.5100
18:00-18:10	644.3800	0.0000	730.9400	708.3000
20:00-20:10	0.0000	0.0000	2.5900	3.1300
全天购电量	65554.2200	35037.5700	67677.9100	92852.4800
全天购电费（元）	39768.1500	19679.5500	41577.9900	61244.3400

表 9 四个指定日期储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（单位：kWh）

时间段	2025.3.20		2025.6.21	
	充电量	放电量	充电量	放电量
0:00-4:00	74.0200	196.9700	42.2800	3283.8800
4:00-8:00	1041.3400	5546.5400	0.0000	4834.6500
8:00-12:00	5505.2200	2777.3800	3764.0580	0.0000
12:00-16:00	4804.3500	340.1000	6875.5585	0.0000
16:00-20:00	88.6500	5700.8400	33.1130	5684.7945
20:00-24:00	9742.6800	2932.8700	10101.1445	2523.9541
0:00 储电量	10261.2000		10206.8800	
24:00 储电量	9953.2900		10800.0000	
时间段	2025.9.23		2025.12.21	
	充电量	放电量	充电量	放电量
0:00-4:00	3755.8400	5.0700	0.0000	568.5500
4:00-8:00	888.7800	6272.9600	1026.4800	2038.2500
8:00-12:00	4461.7100	1896.6200	4738.2400	6226.3800
12:00-16:00	5988.0500	403.6200	7619.4200	2482.8200
16:00-20:00	6.6300	5869.8100	361.3400	5315.8300
20:00-24:00	9816.9000	2738.4300	7908.4100	2435.8900
0:00 储电量	6694.1600		10466.1800	
24:00 储电量	10024.1500		8768.3200	

表 10 四个指定日期的紧急购电量（kWh）

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
0:10-0:40	128.9400	3:20-3:30	44.9600	0:30-0:50	80.0100	当日无紧急购电	
1:10-1:30	74.0900	4:20-4:30	15.8300	5:00-5:20	20.6800		
1:50-2:10	19.0400	5:00-5:10	18.5300	11:00-11:10	475.1300		
3:20-3:50	65.2300	5:30-5:50	26.8400	11:30-11:50	441.1000		
4:40-5:30	70.2500			12:10-12:30	269.3700		
5:40-5:50	31.2000			14:50-15:30	980.2400		
11:00-11:10	5.4200			15:50-17:50	502.6400		
11:20-11:40	176.3400			18:30-18:50	55.6200		
12:30-12:50	65.0300			20:10-20:30	53.3600		
15:20-17:00	338.8000			20:40-21:00	60.7700		
17:10-17:20	0.4600						
20:10-20:20	37.1000						

5.5 问题三的建模分析与求解

问题三允许日内调整购电计划。附件 3 每天在 0:00、6:00、12:00 和 18:00 发布未来 24 小时整点的光伏预报。新预报到达后，可重新安排当日剩余时段的购电量。增购按交易时刻电价的 1.5 倍收费，减购按 0.5 倍退款，紧急购电仍按 5 倍收费。模型在给定更新时刻下确定各次调整量，使全天总费用最小；再比较不同更新时刻组合的费用，选择合适的更新方案。建模流程见图 12。

问题三：多时刻预测更新与滚动调度流程图

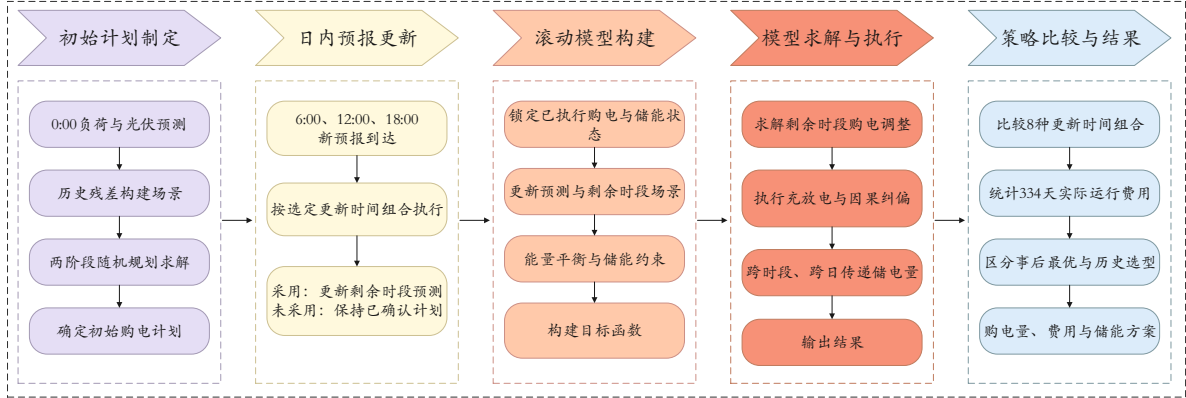


图 12 问题三建模流程图

5.5.1 预报的获取与展开

附件 3 每次给出其后 24 个整点的光伏预报。对相邻整点值作线性插值，得到 10 分钟尺度的预测曲线。负载预测沿用问题二，各次优化基于已发布的预报。

0:00 必须给出全天计划，而 6:00、12:00 和 18:00 又会到达新的预报，一天的决策因此分为两个层次：0:00 根据初始预报确定全天购电计划，新预报到达后只调整尚未执行的时段。前者为主问题，各更新时刻对应剩余时段的更新子问题，全天最多优化四次，是否启用某次更新取决于当日所选的组合。上一次确认的计划作为调整费的结算基准，当前实际储电量作为下一次优化的初始状态。

5.5.2 决策变量

(1) 计划购电量 $a_t^{(0)}$ 该变量为 0:00 按当时预报制定的第 t 个时段计划购电量，单位为 kWh。

(2) 调整增购量 $\Delta_{r,t}^+$ 与减购量 $\Delta_{r,t}^-$ 三次更新发生在 6:00、12:00 和 18:00，依次编号 $r = 1, 2, 3$ ；用 t_r 表示第 r 次更新前已执行的时段数， $t_1 = 36$ 、 $t_2 = 72$ 、 $t_3 = 108$ 。新预报只用于尚未执行的时段，已执行时段的费用已经发生，更新因此仅涉及剩余时段 $t = t_r + 1, \dots, 144$ 。 $\Delta_{r,t}^+$ 与 $\Delta_{r,t}^-$ 分别表示第 r 次更新中相对上一次确认计划的增购量与减购量，均非负，单位为 kWh。调整后的计划为

$$a_t^{\text{new}} = a_t^{\text{old}} + \Delta_{r,t}^+ - \Delta_{r,t}^-, \quad t = t_r + 1, \dots, 144. \quad (33)$$

(3) 场景调度量各更新时刻均设置场景紧急购电量、充放电量和储电量，定义与问题二相同。更新时使用当前实际储电量作为初始状态。

5.5.3 目标函数

主问题的目标与式 (29) 相同, 即计划购电费加期望紧急购电费减日末储电量续存项。第 r 次更新时, 前面时段的计划已经执行, 其费用不再改变, 因此只优化 $t = t_r + 1, \dots, 144$ 的剩余时段:

$$\min \sum_{t=t_r+1}^{144} (1.5p_t \Delta_{r,t}^+ - 0.5p_t \Delta_{r,t}^-) + \frac{1}{S} \sum_{s=1}^S \sum_{t=t_r+1}^{144} 5p_t e_t^{(s)} - \rho \bar{E}_{144}, \quad (34)$$

第一项是相对上一次确认计划的调整费, 第二项是剩余时段的期望紧急购电费, 末项仍为日末储电量续存价值。与问题二一致, 为消除线性规划在等价最优解之间可能出现的数值退化, 更新子问题同样加入式 (12) 所定义的储能吞吐量正则项。

5.5.4 约束条件

剩余时段继续满足问题二中的供能、储能及功率约束, 其中储能充放电互补约束沿用 5.2 节的精确松弛处理, 滚动更新子问题中不引入二进制变量; 并增加以下两组调整约束。调整量非负: $\Delta_{r,t}^+ \geq 0, \Delta_{r,t}^- \geq 0$ 。调整后的计划非负:

$$a_t^{\text{old}} + \Delta_{r,t}^+ - \Delta_{r,t}^- \geq 0, \quad t = t_r + 1, \dots, 144, \quad (35)$$

即减购量不能超过当前计划与增购量之和。已执行时段不再调整。更新子问题不再以日初储电量 E_0 为初值, 而是以更新时刻按实际运行得到的储电量作为初始状态, 即

$$E_{t_r}^{(s)} = E_{t_r}^{\text{act}}, \quad s = 1, \dots, S, \quad (36)$$

其中 $E_{t_r}^{\text{act}}$ 为第 r 次更新时刻按实际负载与光伏出力运行得到的储电量。

综合主问题与更新子问题, 问题三的完整模型为

$$\text{主问题 (0:00):} \quad \min \sum_{t=1}^{144} p_t a_t^{(0)} + \frac{1}{S} \sum_{s=1}^S \sum_{t=1}^{144} 5p_t e_t^{(s)} - \rho \bar{E}_{144}$$

各场景满足式 (32) 中的供能、储能状态、容量与功率约束

$$\text{更新子问题 (} t_r \text{):} \quad \min \sum_{t=t_r+1}^{144} (1.5p_t \Delta_{r,t}^+ - 0.5p_t \Delta_{r,t}^-) + \frac{1}{S} \sum_{s=1}^S \sum_{t=t_r+1}^{144} 5p_t e_t^{(s)} - \rho \bar{E}_{144}$$

$$a_t^{\text{new}} = a_t^{\text{old}} + \Delta_{r,t}^+ - \Delta_{r,t}^- \geq 0, \quad \Delta_{r,t}^+ \geq 0, \quad \Delta_{r,t}^- \geq 0$$

剩余时段各场景满足上述供能与储能约束

(37)

其中 $r = 1, 2, 3$, 对应 6:00、12:00 和 18:00; 主问题与更新子问题的目标函数均按式 (12) 加入储能吞吐量正则项, 更新子问题的储能初值按式 (36) 取更新时刻的实际运行值。

5.5.5 调整时刻的选择

6:00、12:00 和 18:00 的新预报可以直接取用, 但每次更新都按 1.5 倍与 0.5 倍的价格结算调整量, 只有节省的紧急购电费超过调整费时才值得更新。对三个更新时刻各设置采用与不采用两种选择, 共构成 $2^3 = 8$ 种更新组合, 覆盖不更新、单时刻、双时刻与三时刻更新。对每种组合分别连续回测 334 天, 并传递每日实际储电量; 给定组合后, 按照 5.5.2–5.5.4 求解当天各次更新的剩余时段。

固定组合的结果用于比较不同更新时刻的实际费用。为模拟实际执行, 再设置历史

选型部署策略：每天只根据前一日及以前的累计费用选择当日的更新组合。

5.5.6 模型求解

滚动求解流程见图 13。

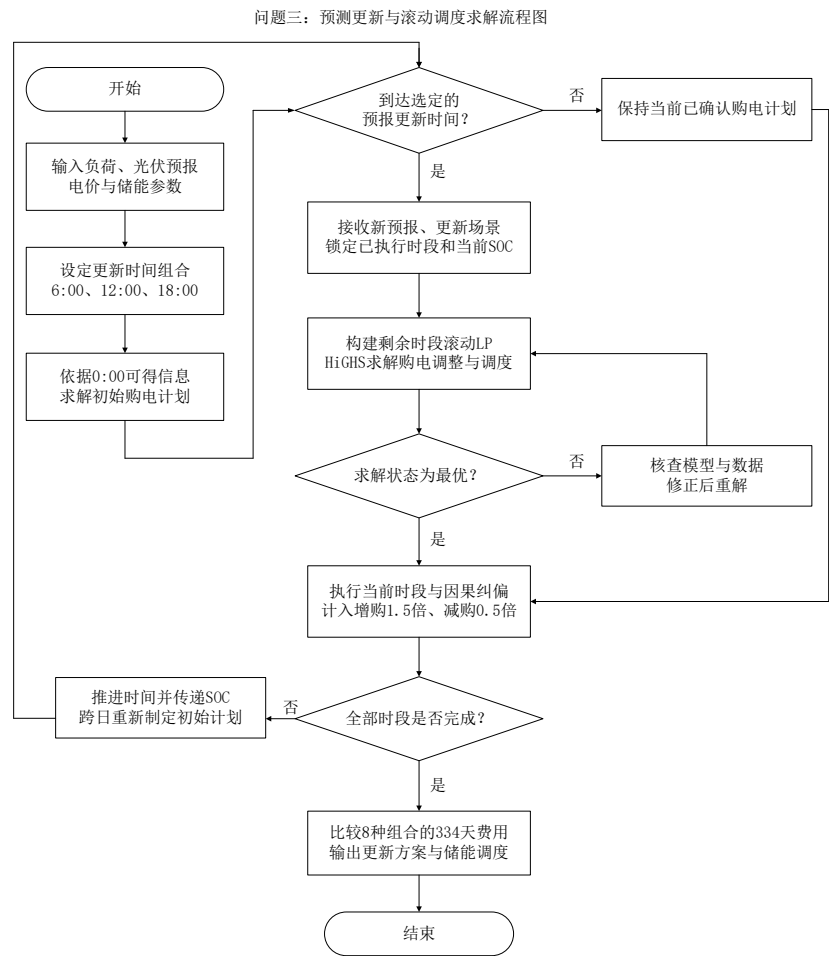


图 13 问题三求解流程图

各子问题均为线性规划，使用 HiGHS 求解，评价期共 334 天。对任一更新组合，0:00 先求解当天计划；到达选定的更新时间后，根据新预报重新求解剩余时段。两个更新时刻之间，储能每 10 分钟根据当前实际负载、光伏出力和预测值调整运行；若仍存在供电缺额，则按 5 倍电价紧急购电。24:00 的实际储电量传递至下一日。逐时段计划、各次调整后的购电量、充放电量及紧急购电区间已按模板写入 result3.xlsx。

5.5.7 求解结果

表 11 给出 8 个固定组合在评价期内的费用。

表 11 问题三不同调整时刻组合的费用对照（334 天）

更新时刻	总费用（万元）	日均总费用（元）	相对不调整（元/日）
不调整	1449.23	43390.24	—
6:00	1412.23	42282.20	-1108.04
12:00	1398.19	41862.05	-1528.19
18:00	1444.96	43262.35	-127.89
6:00+12:00	1390.03	41617.63	-1772.60
6:00+18:00	1408.03	42156.54	-1233.69
12:00+18:00	1394.14	41740.68	-1649.56
6:00+12:00+18:00	1385.86	41492.81	-1897.42

由表 11 可知，单独更新时，12:00 的费用最低，较不更新方案日均减少 1528.19 元。6:00 和 18:00 的日均费用分别减少 1108.04 元和 127.89 元。同时采用 6:00 与 12:00 时，日均费用减少 1772.60 元。三个时刻全部采用时，日均减少 1897.42 元，总费用为 1385.86 万元，在 8 个组合中最低。因此，本评价期内采用其他时刻的预报能够降低费用，以全部更新为宜。该排序基于 2025 年评价期内的费用比较。

图 14 将总费用换算为相对不更新的节省额。三次更新共节省 63.37 万元，但在 6:00 与 12:00 两次更新的基础上增加 18:00，费用降幅较小。可见，各次预报的收益不能简单相加。该比较采用线性插值预报，不代表其他预报分解方式下的排序。

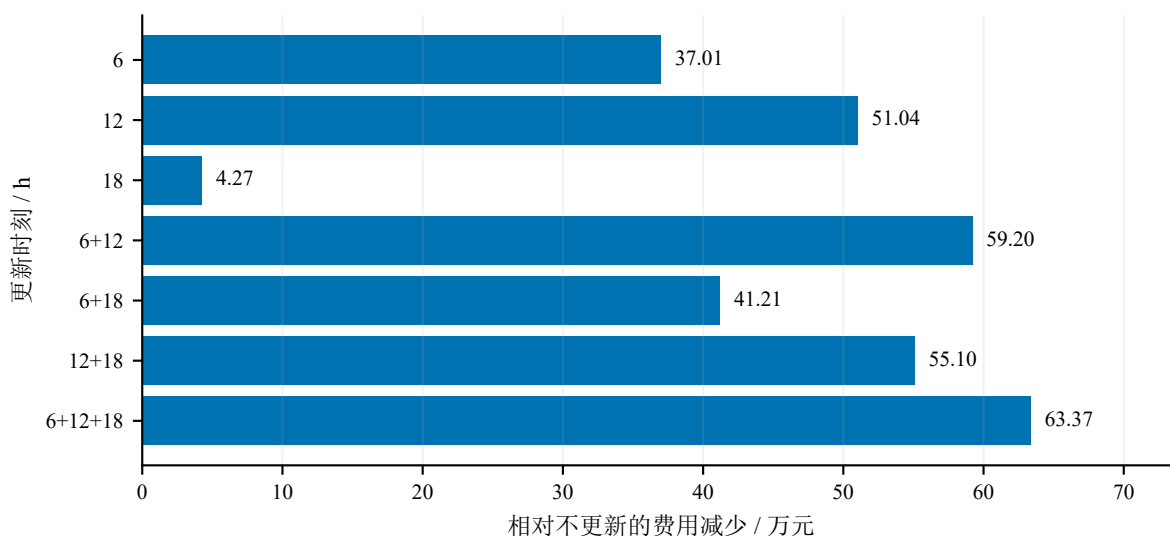


图 14 问题三固定更新组合的费用节省

按题目表 1、表 2 与表 3 的格式，四个指定日期在全时刻组合下的计划购电量、充放电量与紧急购电量分别列于表 12、表 13 与表 14。

表 12 四个指定日期的计划购电量及全天购电量与购电费

时间段	计划购电量 (kWh)			
	2025.3.20	2025.6.21	2025.9.23	2025.12.21
10:00-10:10	0.0000	0.0000	0.0000	0.0000
12:00-12:10	585.8500	0.0000	339.0600	1172.5500
14:00-14:10	0.0000	0.0000	0.0000	0.0000
16:00-16:10	690.7300	111.1700	557.6800	931.0800
18:00-18:10	644.3300	0.0000	734.4700	703.3400
20:00-20:10	0.0000	0.0000	2.5900	3.1300
全天购电量	69769.8500	35288.8900	72398.3900	97591.4000
全天购电费 (元)	42801.2800	19978.6500	44989.7600	64295.7200

表 13 四个指定日期的充放电量与储电量 (kWh, 全时刻组合)

时间段	3-20		6-21		9-23		12-21	
	充电	放电	充电	放电	充电	放电	充电	放电
0:00-4:00	0.0000	419.1300	0.0000	3350.1200	3076.3000	8.9000	1264.7900	28.2100
4:00-8:00	762.3700	5939.3500	0.0000	4836.6100	902.7700	6193.6200	2075.8000	1744.8800
8:00-12:00	2999.3900	2257.1100	954.2400	0.0000	3628.7400	1896.6200	5648.9700	6376.4600
12:00-16:00	7414.5800	254.7200	9432.8200	25.8000	7181.9200	428.4600	6722.2300	2588.7900
16:00-20:00	140.0300	5351.0300	202.7200	5423.8500	200.4500	5826.2800	694.4400	5414.7200
20:00-24:00	9614.2400	2959.0400	9817.0900	2565.9300	9849.3300	2779.3500	6582.8700	2616.3500
0:00 储电量	10086.7400		10436.4000		6745.7600		7326.7900	
24:00 储电量	9834.9800		10800.0000		10064.4000		7162.0700	

表 14 四个指定日期的紧急购电量 (kWh, 全时刻组合)

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
7:00-7:30	426.9900	当日无紧急购电		15:00-15:10	3.1800	1:20-1:40	16.8500
9:00-10:00	520.2700			16:30-16:40	0.6200	2:00-2:20	26.0300
12:40-12:50	17.7500			17:30-17:40	5.5000	3:30-3:50	50.1500
14:50-16:50	605.1100			18:40-18:50	0.1700	4:30-4:50	4.1600
17:10-17:20	11.6000			20:10-20:30	53.3500	5:10-5:40	141.6000
17:30-17:40	8.5000			20:40-21:00	53.3100	18:10-18:20	11.5000
20:10-20:20	13.3600					18:30-18:50	48.2900
21:20-21:40	26.3300					20:10-20:20	75.9300

由表 13 可见, 充电主要在午间光伏充足和夜间低价时段, 放电主要在早晚高峰。

5.6 问题四的建模分析与求解

问题四中, 外网电价逐日变化, 0:00 制定计划时当天各时段价格尚未知, 而购电费用最终按真实电价结算, 决策依据与结算依据由此分离。为在决策时尽可能贴近真实价格, 先利用历史价格预测当天各时段电价, 记为 $\tilde{p}_t$; 购电计划、日内调整和储能调度均

按 $\tilde{p}_t$ 决策，实际费用按附件 4 的真实电价 p_t 结算，紧急购电按真实电价的 5 倍结算。供需平衡、储能状态及场景构造方法沿用问题二和问题三。对应问题二、问题三的两套策略分别记为问题 4-2、问题 4-3。建模流程见图 15。

问题四：实时波动电价下购电与储能调度流程图

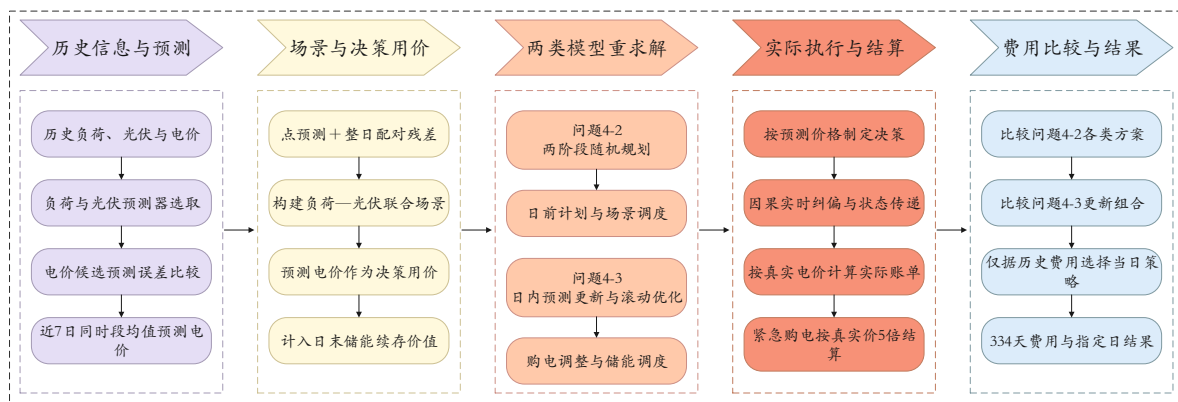


图 15 问题四建模流程图

5.6.1 价格预测

0:00 可用的信息只有历史电价，候选预测因此均只使用当天 0:00 之前的数据，包括前一日同时段值、近 7 日同时段均值等。根据前一日及以前的历史预测结果比较平均绝对误差，选用误差最小的近 7 日同时段均值，误差为 0.0827 元/kWh，约占全年均价的 10.9%。该方法给出的时段预测值记为 $\tilde{p}_t$ 。

以 2025-05-13 为例，图 16 中的预测曲线保留了主要峰谷变化，但部分高价时段存在低估。正式运行时，0:00 确定全天预测电价，日内调整沿用该价格曲线。

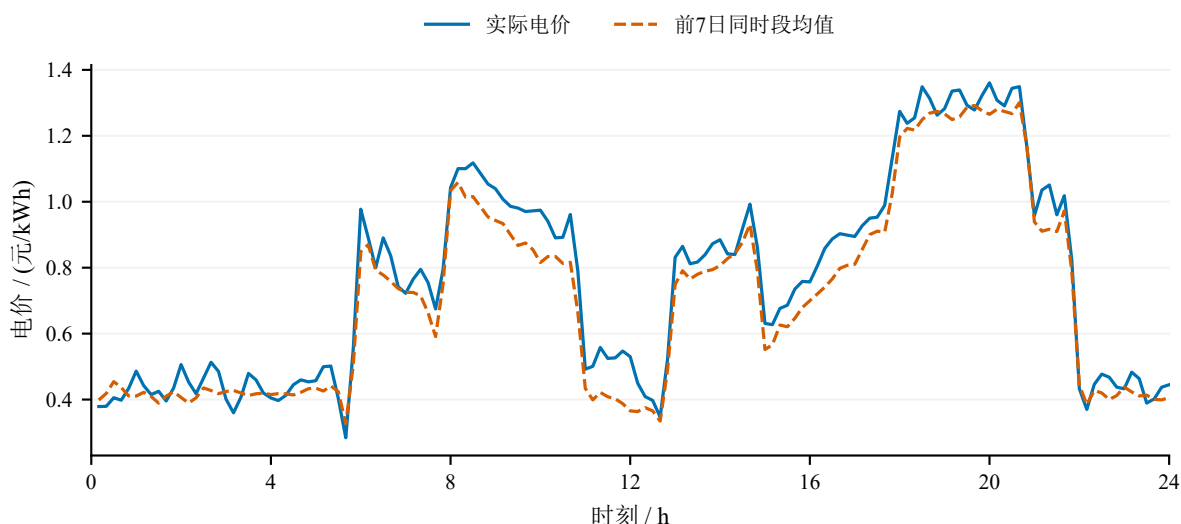


图 16 电价预测示例（2025-05-13）

问题四中日末储电量的价值也随预测电价变化。取第 d 日 0:00—5:00 的预测电价均值，除以充电效率 0.9，并叠加与问题二相同的 0.0010 元/kWh 微小续存项，得到当日价值系数 ρ_d ，作用与问题二、三中的 ρ 相同。日末储电量按此价值计入目标函数。

5.6.2 模型

问题 4-2 沿用问题二的两阶段随机模型，把目标函数式 (29) 中的决策价替换为预测价：

$$\min \sum_{t=1}^{144} \tilde{p}_t a_t + \frac{1}{S} \sum_{s=1}^S \sum_{t=1}^{144} 5\tilde{p}_t e_t^{(s)} - \rho_d \bar{E}_{144}. \quad (38)$$

风险惩罚方案在式 (38) 的基础上，对场景紧急购电费用加入条件风险价值 (CVaR) 项。记场景 s 的紧急购电费用为 $C_s = \sum_{t=1}^{144} 5\tilde{p}_t e_t^{(s)}$ ，取置信水平 $\alpha = 0.95$ 、权重 $\lambda = 0.3$ ，相应模型为

$$\begin{aligned} \min \quad & \sum_{t=1}^{144} \tilde{p}_t a_t + \frac{1}{S} \sum_{s=1}^S \sum_{t=1}^{144} 5\tilde{p}_t e_t^{(s)} - \rho_d \bar{E}_{144} + \lambda \left[\theta + \frac{1}{S(1-\alpha)} \sum_{s=1}^S u_s \right] \\ \text{s.t.} \quad & C_s - \theta - u_s \leq 0, \quad u_s \geq 0, \quad s = 1, 2, \dots, S, \end{aligned} \quad (39)$$

其中 θ 为自由变量， u_s 为场景紧急购电费用超出风险阈值的部分。该形式即 Rockafellar–Uryasev 的 CVaR 表示，风险项为紧急购电费用的 α 分位尾部均值。本文场景等权且 $S = 10$ ，有 $S(1-\alpha) = 0.5 < 1$ ，最优解满足 $\theta^* = \max_s C_s$ 、 $u_s^* = 0$ ，故式 (39) 中的风险项实际等价于对最坏场景紧急购电费用的惩罚 $\lambda \max_s C_s$ ，用于抑制极端场景下的紧急购电量。

问题 4-3 沿用问题三的更新模型，将式 (34) 中的 p_t 换为 $\tilde{p}_t$ ，末项改用 ρ_d 。场景按 5.4.2 构造，各更新组合按 5.5.5 回测。实际部署时，每天开始前比较截至前一日的累计费用，选择最低的组合作为当日策略。

记第 d 日实际费用为 $J_{4,d}$ ，按真实电价结算：

$$J_{4,d} = \sum_{t=1}^{144} p_t a_t + \sum_r \sum_{t>t_r} (1.5p_t \Delta_{r,t}^+ - 0.5p_t \Delta_{r,t}^-) + \sum_{t=1}^{144} 5p_t e_t^{\text{实}}, \quad (40)$$

三项依次为计划购电费、各更新时刻的调整费与紧急购电费， t_r 为更新时刻 r 之前已执行完毕的时段数，含义与 5.5.2 相同。预测价只用于决策，不计入实际账单。

问题四的优化模型按预测电价计价，式 (11) 中的电价相应替换为预测价，边际循环成本变为 $\tilde{\kappa}_t^0 = 0.19 \tilde{p}_t$ 。本文预测价格取前 7 日同时段实际电价的均值，附件 4 的最低电价为 0.0076 元/kWh，故各日各时段的预测价均严格为正， $\tilde{\kappa}_t^0 = 0.19 \tilde{p}_t > 0$ 对全部时段成立。因此问题 4-2 与问题 4-3 同样可以采用 5.2 节精确松弛后的线性规划求解，目标函数中的储能吞吐量正则项亦按式 (12) 加入。实际费用按真实电价 p_t 由式 (40) 结算，结算环节不参与决策，不影响上述判定。

5.6.3 模型求解

问题 4-2 与问题 4-3 的求解步骤见图 17。

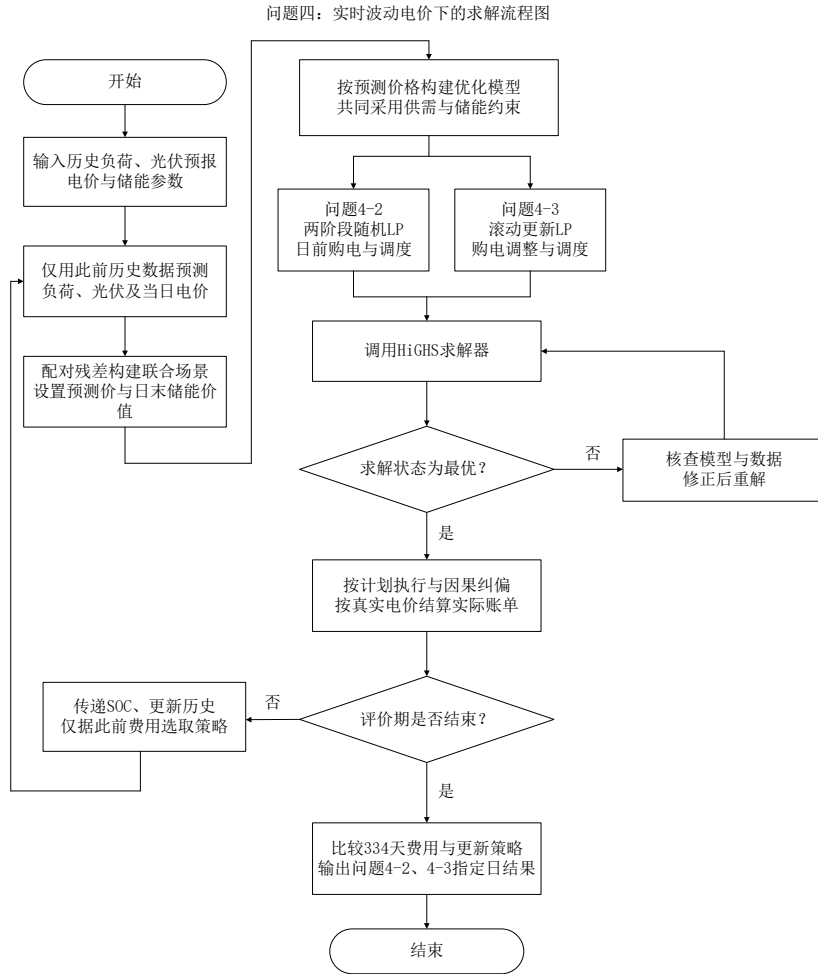


图 17 问题四求解流程图

求解流程沿用 5.4.7 和 5.5.6，评价期为 334 天。每天 0:00 按预测价求解两阶段随机规划，得到购电计划。需要更新时，在所选的 6:00、12:00 或 18:00 利用新预报和预测价调整计划。日内逐时段执行，费用按式 (40) 结算。两类结果分别写入 result4-2.xlsx 和 result4-3.xlsx。

5.6.4 求解结果

表 15 比较问题 4-2 的三种方案，三者均按预测电价决策、按真实电价结算。点预测方案采用当日预测曲线，两阶段随机规划利用历史残差生成场景，风险惩罚方案则在式 (39) 中对紧急购电费用施加 CVaR 惩罚。

表 15 问题 4-2 三种方案的费用对照（334 天）

方案	计划购电费（万元）	紧急购电量（kWh）	紧急购电费（万元）	总费用（万元）
点预测	1287.69	728652.59	267.33	1555.02
两阶段随机规划	1330.42	443895.95	169.13	1499.55
风险惩罚	1389.87	338774.12	137.77	1527.63

两阶段随机规划总费用为 1499.55 万元，比点预测方案低 55.47 万元。其计划购电费增加 42.73 万元，但紧急购电量减少 284,756.64 kWh，紧急购电费减少 98.20 万元。补购费用的减少抵消了计划费用的增加。风险惩罚方案进一步将紧急购电量降至 338,774.12

kWh，但计划购电费上升，总费用高于两阶段随机规划。

表 16 比较波动电价下 8 个固定更新组合与部署策略的费用。
表 16 问题 4-3 不同调整时刻组合的费用对照（334 天）

更新时刻	总费用（万元）	日均总费用（元）	相对不调整（元/日）
不调整	1650.62	49419.65	—
6:00	1571.29	47044.74	−2374.91
12:00	1642.90	49188.65	−231.00
18:00	1658.30	49649.77	+230.11
6:00+12:00	1582.52	47380.83	−2038.82
6:00+18:00	1576.09	47188.17	−2231.48
12:00+18:00	1647.89	49338.01	−81.64
6:00+12:00+18:00	1585.84	47480.16	−1939.49
历史选型部署	1571.61	47054.16	−2365.50

部署策略的总费用为 1571.61 万元，比不更新减少 79.01 万元，日均减少 2365.50 元。固定组合中，6:00 更新费用最低，为 1571.29 万元；历史选型部署与其接近。图 18 同时表明，仅在 18:00 更新会增加费用：18:00 单独更新虽产生减购退款，但计划购电费和紧急购电费增加，退款不足以覆盖新增费用，总费用反而上升。

三次更新下，仅更新光伏预测而不调整购电计划，相比不更新可节省 2737.85 元；在此基础上允许调整购电计划后，再节省 645052.06 元，合计约 64.78 万元。主要收益来自购电计划调整。该合计与历史选型部署节省的 79.01 万元分属不同策略。

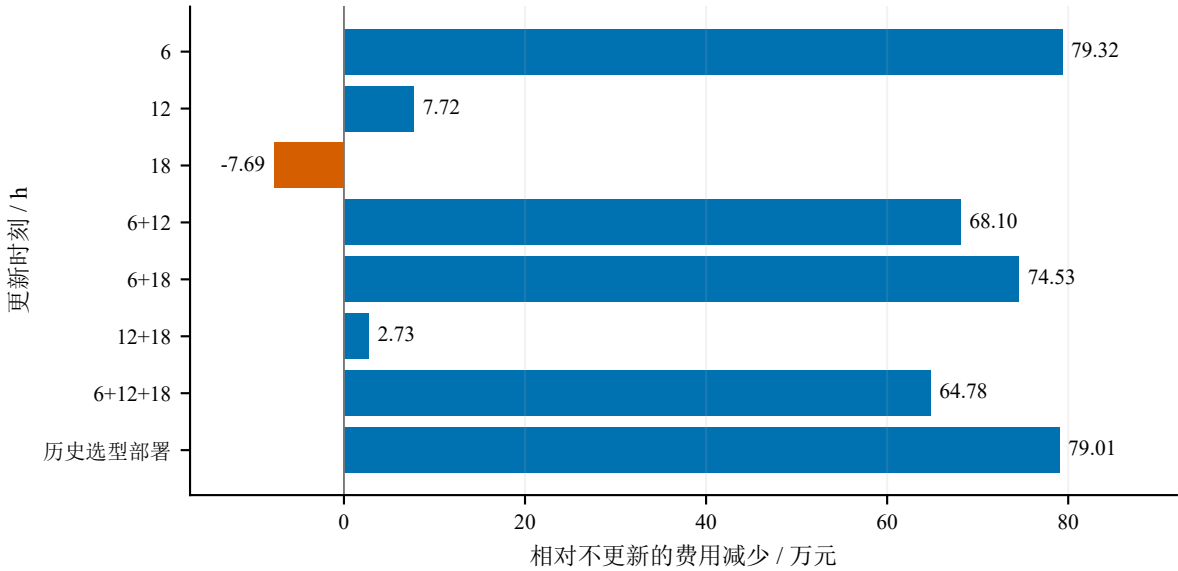


图 18 问题 4-3 的更新收益

四个指定日的结果已按题目表 1、表 2 和表 3 的格式写入 result4-2.xlsx 与 result4-3.xlsx。问题 4-2 随机规划的总费用依次为 44064.71 元、16984.50 元、52682.49 元和 71456.39 元，其中 12 月 21 日未发生紧急购电。部署策略的费用依次为 55521.26 元、18727.89 元、55509.92 元和 78827.66 元。

六、结论

本文按“计划—随机—滚动—波动电价”的递进关系，为含光伏与储能的微网购电调度建立了四套模型并求解。问题一在电价与负载按日重复的典型日下建立混合整数线性规划模型，最优购电费为 35126.8486 元，充电集中在夜间低价与午间光伏富余时段。问题二用点预测与整日残差场景构造两阶段随机规划，评价期内日均费用 42662.54 元，计划购电对预测偏差留有余量。问题三引入日内新预报的滚动更新，以三个时刻全部更新时费用最低，较不更新日均减少 1897.42 元，且各次预报的收益不能简单相加。问题四按预测电价决策、按真实电价结算，历史选型部署较不更新节省 79.01 万元。

在储能建模方面，本文针对传统 0-1 变量带来的混合整数规划复杂度，分析了充放电互补约束的精确松弛条件：在正电价、不可售电且往返效率低于 1 的条件下，松弛后的连续模型与原互补模型具有相同的最优值，逐时段核验也确认调度结果满足充放电互补性，故问题二至问题四采用线性规划求解，问题一保留二进制互斥变量直接求解并作为数值对照。上述结论建立在 2025 年附件数据的评价期内，跨年份与跨电价机制的适用性有待进一步检验。

七、模型评价与改进

7.1 模型优点

储能互补约束的处理有理论依据并可核验。创新性引入精确松弛避免直接引入二进制变量，吞吐量正则项消除数值退化，降低模型的求解复杂度。

模型兼顾理论严格性与计算效率。问题一使用严格的 MILP，问题二至问题四利用精确松弛转化为连续 LP，按规模与求解需求分层处理。

7.2 模型不足

精确松弛具有条件性。精确松弛依赖电价非负、不向电网售电与往返效率低于 1，若出现负电价或允许余电上网， κ_t^0 可能为负，需退回含二进制变量的混合整数模型。

模型对工程约束有所简化。未进一步考虑储能寿命衰减、充放电功率动态变化以及设备启停等更细致的工程约束，与实际微网运行仍存在一定差距。

7.3 改进方向

增强不确定性建模能力。结合概率预测、鲁棒优化或分布式鲁棒优化方法，对负载、光伏及电价预测误差进行更全面的刻画。

完善储能与微网工程约束。引入充放电功率变化率、微网设备启停、爬坡和线路运行约束等，提高模型的工程适用性。

八、AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于模型公式核验与订正、求解代码调试与结果重出、数据分析表述整理及科研配图生成，详细使用情况见支撑材料。

参考文献

- [1] 李正烁, 郭庆来, 孙宏斌, 等. 含大规模分布式储能设备的削峰填谷问题: 互补约束及其精确松弛条件 [EB/OL]. 北极星输配电网, (2015-12-02)[2026-09-13]. <https://shupeidian.bjx.com.cn/html/20151202/687267-1.shtml>.

- [2] LIU S, LI Z G, GUO Y. A necessary and sufficient condition for exact relaxation of energy storage power dispatch: arXiv:2607.21902v1[EB/OL]. (2026-07-24)[2026-09-13]. <https://arxiv.org/abs/2607.21902>.
- [3] WANG Q, WU W C, LIN C H, et al. An exact relaxation method for complementarity constraints of energy storages in power grid optimization problems[J]. Applied Energy, 2024, 371: 123592. DOI: 10.1016/j.apenergy.2024.123592.
- [4] WANG Q, WU W C, LIN C H, et al. Sufficient conditions for the exact relaxation of complementarity constraints for storages in multi-period OPF problems: arXiv:2304.05175[EB/OL]. (2023-04)[2026-09-13]. <https://arxiv.org/abs/2304.05175>.

附录：支撑材料结构与代码清单

整理日期：2026-09-13。支撑材料目录 项目/支撑材料，共 59 个文件、5.22 MB；其中代码 21 个 Python 脚本、6449 行。代码部分逐字收录；为便于排版，代码注释中的少量装饰性符号（U+26A0、U+1F534、U+2605 等）以 [注意]、[红线]、* 等等价文本呈现，源文件保持不变。

附录 A 支撑材料文件夹结构

```
1 支撑材料/
2 |-- 03_代码/ [22 个文件, 315 KB]
3 |   |-- 问题1/ [6 个文件, 174 KB]
4 |       |-- 绘图_数据预处理.py (80 KB)
5 |       |-- 绘图_问题1求解.py (46 KB)
6 |       |-- 检查.py (4 KB)
7 |       |-- 口径.py (6 KB)
8 |       |-- 模型.py (9 KB)
9 |       |-- 写模板.py (30 KB)
10 |   |-- 问题2-3/ [4 个文件, 56 KB]
11 |       |-- p10_core.py (19 KB)
12 |       |-- p15_forecasts.py (2 KB)
13 |       |-- p95_q23_v4.py (30 KB)
14 |       |-- p95c_finish.py (5 KB)
15 |   |-- 问题4/ [8 个文件, 75 KB]
16 |       |-- p10_core.py (17 KB)
17 |       |-- p102_q4_core.py (19 KB)
18 |       |-- p102c_q4_finish.py (5 KB)
19 |       |-- p105_q4_specified.py (10 KB)
20 |       |-- p106_pi_q4.py (3 KB)
21 |       |-- p107_q4_stats.py (6 KB)
22 |       |-- p108_q4_vstep_d4.py (6 KB)
23 |       |-- p41c_figures_q4.py (9 KB)
24 |   |-- 用户指定图源/ [4 个文件, 10 KB]
25 |       |-- 用户指定最终图_代码映射.md (1 KB)
26 |       |-- plot_q1_soc_waterfall.py (2 KB)
27 |       |-- plot_typical_day_combined.py (4 KB)
28 |       |-- rebuild_q23_G.py (2 KB)
29 |-- 04_结果/ [34 个文件, 4767 KB]
30 |   |-- 问题1/ [7 个文件, 1128 KB]
31 |       |-- 订正_result1表2对照.csv (600 B)
32 |       |-- 复现清单_F1模板.json (5 KB)
33 |       |-- 数据审计_六段能量.csv (156 KB)
34 |       |-- 数据审计_预报误差.csv (932 KB)
35 |       |-- 数据审计_逐日派生量.csv (25 KB)
36 |       |-- 数据审计_字段清单.csv (1 KB)
37 |       |-- result1_时钟口径修正版.xlsx (8 KB)
38 |   |-- 问题2-3_线性口径留档/ [9 个文件, 1613 KB]
39 |       |-- 问题3正式选型说明.json (1 KB)
40 |       |-- case_config.json (4 KB)
41 |       |-- main_metadata.json (9 KB)
42 |       |-- q2_汇总.csv (324 B)
43 |       |-- q2_逐日费用.csv (138 KB)
```

```

44 | | | -- q3_逐日费用.csv (522 KB)
45 | | | -- q3_组合对照.csv (1 KB)
46 | | | -- result2.xlsx (330 KB)
47 | | | `-- result3.xlsx (607 KB)
48 | `-- 问题4_口径B正式/ [18 个文件, 2026 KB]
49 | | -- 结论_Q4.md (38 KB)
50 | | -- 四个指定日期_表123_q4B.json (59 KB)
51 | | -- 四个指定日期_表123_q4B.md (38 KB)
52 | | -- 问题4_已切换为口径B.md (2 KB)
53 | | -- 修后数字总表_eps修正.md (15 KB)
54 | | -- 指定日期表格.json (15 KB)
55 | | -- B_v_main_metadata.json (15 KB)
56 | | -- B_v_p2_arithmetic.json (2 KB)
57 | | -- d4_Q4B.json (189 B)
58 | | -- pi_bounds_q4.json (10 KB)
59 | | -- q4_对照汇总.json (8 KB)
60 | | -- q4_配对统计.json (21 KB)
61 | | -- q42_逐日费用.csv (181 KB)
62 | | -- q43_逐日费用.csv (664 KB)
63 | | -- q43_组合对照.csv (2 KB)
64 | | -- result4-2.xlsx (334 KB)
65 | | -- result4-3.xlsx (622 KB)
66 | | `-- v台阶_Q4B.json (2 KB)
67 | -- AI工具使用说明.md (29 KB)
68 | -- AI工具使用说明.pdf (219 KB)
69 | `-- AI工具使用说明.tex (13 KB)

```

附录 B 代码清单

序	文件(相对 03_ 代码/)	行数	大小	作用
1	问题 1/绘图 _ 数据预处理.py	1638	80 KB	数据预处理类图件绘制(时间对齐、质量检验、异常辨识与处理、变量分布、特征构造、相关性与分组诊断)
2	问题 1/绘图 _ 问题 1 求解.py	958	46 KB	问题 1 求解类图件绘制(模型结构、典型日输入与购电计划、储能与 SOC、表 2 口径对照、基线阶梯、净负荷与储能作用)
3	问题 1/检查.py	76	4 KB	问题 1 求解结果的约束与结算自检入口
4	问题 1/口径.py	146	6 KB	问题 1 口径常量与时段 ↔ 时钟区间映射(含表 2 六段的真实时钟归组)
5	问题 1/模型.py	224	9 KB	问题 1 单日 LP 模型: 购电/储能/弃光决策变量、约束与目标函数, 方案丙时段口径
6	问题 1/写模板.py	575	30 KB	问题 1 结果落盘: 写 result1 模板(含时钟口径六段、首日 0:00—0:10 子槽边界恢复、SOC 链闭合与内容级指纹)

序	文件（相对 03_ 代码/）	行数	大小	作用
7	问题 2-3/p10_core.py	409	19 KB	问题 2/3 共享求解内核：日末库存系数、情景生成、储能递推、费用结算与 LP/MILP 求解
8	问题 2-3/p15_forecasts.py	49	2 KB	问题 2/3 逐日历史选型预报（只用当日之前信息，含残差构造）
9	问题 2-3/p95_q23_v4.py	604	30 KB	问题 2/3 主运行器：8 个更新组合 + 部署策略的连续回测（含附件 3 整点预报的 10 分钟展开）
10	问题 2-3/p95c_finish.py	89	5 KB	问题 2/3 结果合成与落盘：汇总表、逐日费用、组合对照与工作簿填充
11	问题 4/p10_core.py	369	17 KB	问题 4 所用的共享内核版本（eps 归一化修正后：末槽系数 = $(\text{eps_soc} + \text{v_term})/S$ ）
12	问题 4/p102_q4_core.py	413	19 KB	问题 4 引擎：口径 A/B、跨日库存续存价值 v、6/12/18 更新通道与紧急购电通道
13	问题 4/p102c_q4_finish.py	96	5 KB	问题 4 结果合成与落盘（五格汇总、审计项登记）
14	问题 4/p105_q4_specified.py	170	10 KB	问题 4 四个指定日期表的生成（表 1/2/3 与 xlsx 交叉核对）
15	问题 4/p106_pi_q4.py	66	3 KB	问题 4 完全信息下界（PI）求解与缺口计算
16	问题 4/p107_q4_stats.py	121	6 KB	问题 4 配对统计与跨格对照（7 日分块 bootstrap）
17	问题 4/p108_q4_vstep_d4.py	132	6 KB	问题 4 的 v 台阶响应与 D4 收益拆分汇总
18	问题 4/p41c_figures_q4.py	156	9 KB	问题 4 图件绘制（四格对照、效应分解、电价预测与 MAE 带、原始价格、价格-负荷）
19	用户指定图源/用户指定最终图_代码映射.md	—	1 KB	5 张用户指定最终图与出图脚本、源数据的映射说明
20	用户指定图源/plot_q1_soc_waterfall.py	21	2 KB	用户指定图：问题 1 六时段储电量变化瀑布图
21	用户指定图源/plot_typical_day_combined.py	116	4 KB	用户指定图：四季典型日合并图（含统一纵轴版）

序	文件（相对 03_ 代码/）	行数	大小	作用
22	用户指定图源/rebuild_q23_G.py	21	2 KB	用户指定图：问题 2/3 三维图修正版（3D+2D 双联）重建
23	04_ 结果/问题 4_ 口径 B 正式/结论_Q4.md	—	38 KB	—
24	04_ 结果/问题 4_ 口径 B 正式/四个指定日期_表123_q4B.md	—	38 KB	—
25	04_ 结果/问题 4_ 口径 B 正式/问题 4_ 已切换为口径 B.md	—	2 KB	—
26	04_ 结果/问题 4_ 口径 B 正式/修后数字总表_eps 修正.md	—	15 KB	—
27	AI 工具使用说明.md	—	29 KB	—

附录 C 代码正文

C1. 问题 1/绘图 _ 数据预处理.py (1638 行)

数据预处理类图件绘制（时间对齐、质量检验、异常辨识与处理、变量分布、特征构造、相关性与分组诊断）

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """数据预处理可视化（C 题《微网与外部电网电力调控策略》）
4
5  按 9 个预处理环节组织 7 张图：
6      图3-1 时间对齐与数据结构          （环节 1、2）
7      图3-2 质量检验与缺失识别          （环节 2、3）
8      图3-3 异常辨识与判据修正          （环节 4）
9      图3-4 异常处理与极端值保留        （环节 5）
10     图3-5 变量分布分析                  （环节 6）
11     图3-6 特征构造与派生变量          （环节 7）
12     图3-7 相关性与分组诊断            （环节 8、9）
13
14  口径：方案丙——第 j 槽 = [10j, 10j+10) 分钟（标签_j 为槽开始时刻）；
15         每日窗口 0:10 → 次日 0:10; Δt = 1/6 h; 电量(kWh) = 功率(kW) × 1/6。
16
17  只读原始附件与已登记派生表；只新建本目录图件与本脚本，不改动项目既有文件。
18  运行：C:\Users\John\.conda\envs\deeplearning\python.exe "04_结果与检验/可视化集/03_数据
        预处理/绘图_数据预处理.py"
19  """
20

```

```

21 from __future__ import annotations
22
23 import datetime
24 import sys
25 import warnings
26 from pathlib import Path
27
28 sys.stdout.reconfigure(encoding="utf-8", errors="replace")
29 _ORIG_STDOUT = sys.stdout
30
31 # -----
32 # 路径
33 # -----
34 ROOT = Path(r"C:\Users\John\Desktop\26题")
35 ATT = ROOT / "00_题目与原始数据" / "C题" / "附件"
36 RES = ROOT / "04_结果与检验" / "results"
37 OUTDIR = ROOT / "04_结果与检验" / "可视化集" / "03_数据预处理"
38 LOGDIR = ROOT / "04_结果与检验" / "检验日志"
39 LOG = LOGDIR / "可视化_数据预处理.txt"
40
41 OUTDIR.mkdir(parents=True, exist_ok=True)
42 LOGDIR.mkdir(parents=True, exist_ok=True)
43
44
45 class _Tee:
46     """同时写终端与日志文件。"""
47
48     def __init__(self, *streams):
49         self.streams = streams
50
51     def write(self, s):
52         for st in self.streams:
53             st.write(s)
54         return len(s)
55
56     def flush(self):
57         for st in self.streams:
58             st.flush()
59
60
61 _LOG_FH = open(LOG, "w", encoding="utf-8", newline="\n")
62 sys.stdout = _Tee(_ORIG_STDOUT, _LOG_FH)
63
64 # -----
65 # 依赖
66 # -----
67 import matplotlib
68
69 matplotlib.use("Agg")
70
71 import matplotlib.pyplot as plt

```

```

72 import numpy as np
73 import openpyxl
74 import pandas as pd
75 from matplotlib.patches import Rectangle
76 from scipy import stats
77
78 # 绘图约定: 优先使用 math-modeling 技能; 失败则回退
79 SKILL_STYLE = False
80 SKILL_EXPORT = False
81 _style_info = {}
82 try:
83     sys.path.insert(0, r"C:\Users\John\.codex\skills\math-modeling\tools\figure\scripts")
84     from setup_style import setup_style # type: ignore
85
86     _style_info = setup_style(journal="general", lang="zh")
87     SKILL_STYLE = True
88 except Exception as exc: # noqa: BLE001
89     print(f"[警告] setup_style 不可用, 回退默认中文字体: {exc}")
90     plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"]
91     plt.rcParams["axes.unicode_minus"] = False
92
93 try:
94     from export_figure import export_figure # type: ignore
95
96     SKILL_EXPORT = True
97 except Exception as exc: # noqa: BLE001
98     print(f"[警告] export_figure 不可用, 回退 fig.savefig: {exc}")
99     export_figure = None
100
101 plt.rcParams["figure.constrained_layout.use"] = False # 手工排版, 避免表格/文字被压缩
102 plt.rcParams["axes.unicode_minus"] = False
103 plt.rcParams["savefig.facecolor"] = "white"
104
105 # 捕获"缺字"警告(中文字体缺字形是图件最常见的隐性缺陷), 纳入自检
106 MISSING_GLYPHS: set[str] = set()
107 _ORIG_SHOWWARNING = warnings.showwarning
108
109
110 def _showwarning(message, category, filename, lineno, file=None, line=None):
111     msg = str(message)
112     if "missing from font" in msg:
113         MISSING_GLYPHS.add(msg)
114     else:
115         _ORIG_SHOWWARNING(message, category, filename, lineno, file=file, line=line)
116
117
118 warnings.showwarning = _showwarning
119
120 C = list(plt.get_cmap("tab10").colors)
121 GREY = "#8a8a8a"
122 HDR = "#c9d7ee"

```

```

123 LOG_HDR = "可视化_数据预处理"
124
125 # -----
126 # 数据读取（只读；记录单元格类型用于"混合类型"事实）
127 # -----
128 DT = 1.0 / 6.0
129
130
131 def read_sheet(fn, sheet=None):
132     wb = openpyxl.load_workbook(ATT / fn, data_only=True, read_only=True)
133     ws = wb[sheet] if sheet else wb[wb.sheetnames[0]]
134     rows = [list(r) for r in ws.iter_rows(values_only=True)]
135     wb.close()
136     return rows
137
138
139 def to_minutes(v):
140     """时间标签归一化为"距当日 0:00 的分钟数"; '0:00+1' → 1440。不依赖单元格类型。"""
141     if isinstance(v, datetime.time):
142         return v.hour * 60 + v.minute
143     s = str(v).strip()
144     if "+1" in s:
145         return 1440
146     hh, mm = s.split(":")[2]
147     return int(hh) * 60 + int(mm)
148
149
150 def hhmm(m):
151     m = int(round(m))
152     return f"{m // 60:02d}:{m % 60:02d}"
153
154
155 def type_counts(values):
156     out = {}
157     for v in values:
158         out[type(v).__name__] = out.get(type(v).__name__, 0) + 1
159     return out
160
161
162 a1 = read_sheet("附件1.xlsx")
163 a2_load = read_sheet("附件2.xlsx", "小区负载")
164 a2_pv = read_sheet("附件2.xlsx", "光伏发电实际功率")
165 a3 = read_sheet("附件3.xlsx")
166 a4 = read_sheet("附件4.xlsx")
167
168 # --- 附件1 ---
169 lab1 = [to_minutes(r[0]) for r in a1[1:]]
170 t1_types = type_counts([r[0] for r in a1[1:]])
171 PRICE1 = np.array([float(r[1]) for r in a1[1:]])
172 LD1 = np.array([float(r[2]) for r in a1[1:]])
173 PV1 = np.array([float(r[3]) for r in a1[1:]])

```

```

174
175 # --- 附件2 / 附件4 ---
176 lab2 = [to_minutes(v) for v in a2_load[0][1:]]
177 lab4 = [to_minutes(v) for v in a4[0][1:]]
178 hdr2_types = type_counts(list(a2_load[0][1:]))
179 hdr4_types = type_counts(list(a4[0][1:]))
180 dates = [r[0] for r in a2_load[1:]]
181 dates_pv = [r[0] for r in a2_pv[1:]]
182 dates4 = [r[0] for r in a4[1:]]
183
184 LD = np.array([[float(v) for v in r[1:]] for r in a2_load[1:]])
185 PV = np.array([[float(v) for v in r[1:]] for r in a2_pv[1:]])
186 PR = np.array([[float(v) for v in r[1:]] for r in a4[1:]])
187 NL = LD - PV
188
189 NDAY, NSLOT = LD.shape
190 NCELL = NDAY * NSLOT
191
192 # --- 附件3 ---
193 a3_body = a3[1:]
194 block_dates, cur = [], None
195 for r in a3_body:
196     if r[0] not in (None, ""):
197         cur = str(r[0])
198         block_dates.append(cur)
199 a3_blank = sum(1 for r in a3_body if r[0] in (None, ""))
200 A3KEY = list(zip(block_dates, [str(r[1]) for r in a3_body]))
201 A3VAL = np.array([[float(x) for x in r[2:]] for r in a3_body])
202 A3_HOURS = A3VAL.shape[1]
203
204 # -----
205 # 审计量（全部现场重算）
206 # -----
207 cells_all = [LD, PV, PR]
208 MISS = {
209     "附件1 (144x4 列)": sum(1 for r in a1[1:] for v in r if v is None or (isinstance(v, str)
210     ) and not v.strip()),
211     "附件2 小区负载": int(np.isnan(LD).sum()),
212     "附件2 光伏实际": int(np.isnan(PV).sum()),
213     "附件3 预报矩阵": int(np.isnan(A3VAL).sum()),
214     "附件4 电价": int(np.isnan(PR).sum()),
215 }
216 MISS_TOTAL = sum(MISS.values())
217 DUP = {
218     "附件1 时间标签": len(lab1) - len(set(lab1)),
219     "附件2 日期": len(dates) - len(set(dates)),
220     "附件3 (日期,预报时刻)键": len(A3KEY) - len(set(A3KEY)),
221     "附件4 日期": len(dates4) - len(set(dates4)),
222 }
223 NEG = {
224     "附件1 三数值列": int(sum((arr < 0).sum() for arr in (PRICE1, LD1, PV1))),

```

```

224     "附件2 小区负载": int((LD < 0).sum()),
225     "附件2 光伏实际": int((PV < 0).sum()),
226     "附件3 预报矩阵": int((A3VAL < 0).sum()),
227     "附件4 电价": int((PR < 0).sum()),
228 }
229
230 # 零膨胀与异常判据
231 PV_LE1 = int((PV <= 1).sum())
232 PV_GT1 = int((PV > 1).sum())
233 FRAC_LE1 = PV_LE1 / NCELL
234 mad_full_med = float(np.median(PV))
235 mad_full = float(np.median(np.abs(PV - mad_full_med)))
236 z_full = 0.6745 * (PV - mad_full_med) / mad_full
237 cand_full = np.abs(z_full) > 3.5
238 N_CAND_FULL = int(cand_full.sum())
239 FRAC_CAND_FULL = N_CAND_FULL / NCELL
240 cand_full_max = float(PV[cand_full].max())
241 cand_full_min = float(PV[cand_full].min())
242 THR_FULL_HI = mad_full_med + 3.5 * mad_full / 0.6745
243 day = PV[PV > 1]
244 DAY_N = int(day.size)
245 day_med = float(np.median(day))
246 day_mad = float(np.median(np.abs(day - day_med)))
247 z_day = 0.6745 * (day - day_med) / day_mad
248 N_CAND_DAY = int((np.abs(z_day) > 3.5).sum())
249 Z_DAY_MAX = float(np.abs(z_day).max())
250 THR_DAY = 3.5 * day_mad / 0.6745
251 day_low_n = int((day <= THR_FULL_HI).sum())
252
253 slot_min = np.array(lab1)
254 NIGHT = (slot_min >= 20 * 60) | (slot_min <= 4 * 60)
255 NIGHT_SLOTS = int(NIGHT.sum())
256 NIGHT_PV_GT1 = int((PV[:, NIGHT] > 1).sum())
257 NIGHT_PV_MAX = float(PV[:, NIGHT].max())
258
259 # 零膨胀的时段结构：全年最大出力 ≤1 kW 的时段（真实无光照段）
260 _col_max = PV.max(axis=0)
261 _nz_idx = np.where(_col_max <= 1)[0]
262 NEARZERO_SLOTS = int(_nz_idx.size)
263 _nz_lab = sorted(slot_min[_nz_idx])
264 _runs, _s, _p = [], _nz_lab[0], _nz_lab[0]
265 for _v in _nz_lab[1:]:
266     if _v == _p + 10:
267         _p = _v
268     else:
269         _runs.append((_s, _p))
270         _s = _p = _v
271 _runs.append((_s, _p))
272 NEARZERO_RANGE_TXT = " 与 ".join(f"{hhmm(a)}—{hhmm(b)}" for a, b in _runs)
273 PV_EXACT_ZERO = int((PV == 0).sum())
274 PRICE1_MIN, PRICE1_MAX = float(PRICE1.min()), float(PRICE1.max())

```

```

275 LD1_MIN, LD1_MAX = float(LD1.min()), float(LD1.max())
276 PV1_MAX = float(PV1.max())
277 PV1_NZ = int((PV1 > 0).sum())
278 _pv1_idx = np.where(PV1 > 0)[0]
279 PV1_NZ_RANGE = f"{hhmm(slot_min[_pv1_idx[0]])}—{hhmm(slot_min[_pv1_idx[-1]])}"
280 PV_ALLZERO_DAYS = int((PV.max(axis=1) <= 0).sum())
281 A3_MAX = float(A3VAL.max())
282 A3_ALLZERO_ROWS = int((np.abs(A3VAL).max(axis=1) == 0).sum())
283
284 PR_LOW = int((PR < 0.05).sum())
285 PR_LOW_MIN = float(PR.min())
286 PR_HIGH = int((PR > 1.5).sum())
287 PR_HIGH_MAX = float(PR.max())
288
289 # 能量与净负荷
290 E_LOAD = float(LD.sum()) * DT
291 E_PV = float(PV.sum()) * DT
292 E_NET = float(NL.sum()) * DT
293 RATIO_YEAR = E_PV / E_LOAD
294 NL_NEG_FRAC = float((NL < 0).mean())
295 NL_MIN = float(NL.min())
296 NL_MAX = float(NL.max())
297 NL_NEG_OVER5 = int((NL < -5000).sum())
298 NL_POS_OVER5 = int((NL > 5000).sum())
299 NL_NEG_MEAN = float(NL[NL < 0].mean())
300 NL_POS_MEAN = float(NL[NL > 0].mean())
301
302 # 逐日派生量（既有登记表，只读）
303 DAILY = pd.read_csv(RES / "数据审计_逐日派生量.csv")
304 DAILY["日期"] = pd.to_datetime(DAILY["日期"])
305 DAILY["净负荷电量kWh"] = DAILY["负载电量kWh"] - DAILY["光伏电量kWh"]
306 RATIO_MIN = float(DAILY["光伏负载比"].min())
307 RATIO_MAX = float(DAILY["光伏负载比"].max())
308 RATIO_MEAN = float(DAILY["光伏负载比"].mean())
309
310 # 逐日净负荷功率波动（10 分钟尺度平均绝对增量）
311 dNL = np.abs(np.diff(NL, axis=1))
312 RAMP_DAILY = dNL.mean(axis=1)
313 RAMP_MEAN = float(dNL.mean())
314
315 print("=" * 78)
316 print(f"{LOG_HDR} • 数据预处理可视化")
317 print("=" * 78)
318 print(f"附件1 时间列单元格类型 : {t1_types} (混合类型, 共 {sum(t1_types.values())} 个标签)")
319 print(f"附件2 表头类型 : {hdr2_types}; 附件4 表头类型: {hdr4_types}")
320 print(f"附件3 日期列空白行 : {a3_blank} / {len(a3_body)} (块内延续, 非缺失)")
321 print(f"观测单元 : {NDAY} 日 × {NSLOT} 槽 = {NCELL:,}")
322 print(f"缺失合计 : {MISS_TOTAL}")
323 print(f"光伏 ≤1 kW : {PV_LE1:,} ({FRAC_LE1:.2%}); >1 kW: {PV_GT1:,}")
324 print(f"全样本 MAD 候选异常 : {N_CAND_FULL:,} / {NCELL:,} ({FRAC_CAND_FULL:.2%}")
325 print(f"白天子样本候选异常 : {N_CAND_DAY} / {DAY_N:,}; |z|max={Z_DAY_MAX:.3f}")

```

```

326 print(f"电价 <0.05 / >1.5      : {PR_LOW} / {PR_HIGH}")
327 print(f"年电量 负载/光伏/净负荷: {E_LOAD:,.1f} / {E_PV:,.1f} / {E_NET:,.1f} kWh")
328 print("-" * 78)
329 print(f"样式: skill={SKILL_STYLE} {_style_info.get('cjk_font', '')}; 导出: skill={
      SKILL_EXPORT}")
330 print("-" * 78)
331
332 # -----
333 # 绘图工具
334 # -----
335 SAVED = {}
336 LAYOUT_ISSUES: list[tuple[str, str, str]] = [] # (图名, 严重度, 说明)
337 FIGNOTE_TEXTS: list = []
338 BOXTEXT_TEXTS: list = []
339
340
341 def layout_audit(fig, name):
342     """程序化版面自检: 缺字、文字裁切、刻度标签重叠、子图重叠、说明框压图。"""
343     issues: list[tuple[str, str, str]] = []
344     try:
345         from visual_qa import audit_layout # type: ignore
346
347         for sev, msg in audit_layout(fig):
348             issues.append((name, sev, msg))
349     except Exception as exc: # noqa: BLE001
350         issues.append((name, "INFO", f"visual_qa.audit_layout 不可用: {exc}"))
351
352     try:
353         fig.canvas.draw()
354         renderer = fig.canvas.get_renderer()
355     except Exception: # noqa: BLE001
356         return issues
357
358     # 子图相互重叠
359     axes_list = list(fig.axes)
360     for i in range(len(axes_list)):
361         for j in range(i + 1, len(axes_list)):
362             if axes_list[i].get_position().overlaps(axes_list[j].get_position()):
363                 issues.append((name, "FAIL",
364                               f"子图位置重叠: ax{i} 与 ax{j}"))
365
366     # 图下说明框压住子图
367     for f, t in FIGNOTE_TEXTS:
368         if f is not fig:
369             continue
370         try:
371             tbb = t.get_window_extent(renderer)
372         except Exception: # noqa: BLE001
373             continue
374         for k, ax in enumerate(axes_list):
375             try:

```

```

376         abb = ax.get_window_extent(renderer)
377     except Exception: # noqa: BLE001
378         continue
379     if tbb.overlaps(abb):
380         issues.append((name, "FAIL",
381             f"图下说明框压住子图 ax{k}: {t.get_text()[:18]}..."))
382
383 # 说明框之间互相重叠
384 own = [(f, t) for f, t in BOXTEXT_TEXTS if f is fig]
385 for i in range(len(own)):
386     for j in range(i + 1, len(own)):
387         try:
388             a = own[i][1].get_window_extent(renderer)
389             b = own[j][1].get_window_extent(renderer)
390         except Exception: # noqa: BLE001
391             continue
392         if a.overlaps(b):
393             issues.append((name, "FAIL",
394                 f"说明框互相重叠: [{own[i][1].get_text()[:12]}...] x"
395                 f" [{own[j][1].get_text()[:12]}...]"))
396
397 # 表格单元格文字溢出
398 try:
399     from matplotlib.table import Table # type: ignore
400
401     for tbl in fig.findobj(Table):
402         for (r, c), cell in tbl.get_celld().items():
403             txt = cell.get_text()
404             if not txt.get_text().strip():
405                 continue
406             try:
407                 tbb = txt.get_window_extent(renderer)
408                 cbb = cell.get_window_extent(renderer)
409             except Exception: # noqa: BLE001
410                 continue
411             if (tbb.x0 < cbb.x0 - 1.0 or tbb.x1 > cbb.x1 + 1.0
412                 or tbb.y0 < cbb.y0 - 1.0 or tbb.y1 > cbb.y1 + 1.0):
413                 issues.append((name, "FAIL",
414                     f"表格文字溢出单元格 (第{r}行第{c}列): "
415                     f" [{txt.get_text()[:16].replace(chr(10), ' ')}]"))
416     except ImportError:
417         pass
418     return issues
419
420
421 def save(fig, name):
422     for item in layout_audit(fig, name):
423         LAYOUT_ISSUES.append(item)
424     base = OUTDIR / name
425     if SKILL_EXPORT:
426         paths = export_figure(fig, str(base), formats=("png", "svg"), dpi=300, tight=True)

```

```

427     else:
428         paths = []
429         for ext in ("png", "svg"):
430             p = f"{base}.{ext}"
431             fig.savefig(p, dpi=300, bbox_inches="tight", pad_inches=0.05)
432             paths.append(p)
433     plt.close(fig)
434     SAVED[name] = [Path(p) for p in paths]
435     return paths
436
437
438 def render_table(ax, col_labels, rows, col_widths=None, fontsize=8.0,
439                 row_colors=None, header_color=HDR, cell_font_colors=None):
440     """把二维文本渲染为表格（bbox 铺满 axes）。"""
441     ax.axis("off")
442     n_cols = len(col_labels)
443     n_rows = len(rows) + 1
444     body_colors = [
445         [(row_colors[i] if row_colors else "#ffffff")] * n_cols for i in range(len(rows))
446     ]
447     tbl = ax.table(
448         cellText=rows,
449         colLabels=col_labels,
450         cellLoc="center",
451         loc="center",
452         colWidths=col_widths,
453         colColours=[header_color] * n_cols,
454         cellColours=body_colors,
455         bbox=[0, 0, 1, 1],
456     )
457     tbl.auto_set_font_size(False)
458     tbl.set_fontsize(fontsize)
459     for j in range(n_cols):
460         c = tbl[0, j]
461         c.set_text_props(fontweight="bold")
462         c.set_linewidth(0.6)
463     for i in range(1, n_rows):
464         for j in range(n_cols):
465             tbl[i, j].set_linewidth(0.5)
466     if cell_font_colors:
467         for (i, j), col in cell_font_colors.items():
468             tbl[i, j].set_text_props(color=col)
469     return tbl
470
471
472 def panel_title(ax, text, fs=12.0):
473     ax.set_title(text, fontsize=fs, fontweight="bold", pad=7)
474
475
476 def fig_note(fig, ax, text, dy=0.015, fs=7.8):
477     """在 axes 正下方（figure 坐标）放说明文字，避免与相邻子图重叠。"""

```

```

478     bb = ax.get_position()
479     t = fig.text(bb.x0 + bb.width / 2.0, bb.y0 - dy, text, fontsize=fs, ha="center", va="
top",
480                 bbox=dict(boxstyle="round,pad=0.32", facecolor="#f7f7f7",
481                           edgecolor="#bdbdbd", linewidth=0.6))
482     FIGNOTE_TEXTS.append((fig, t))
483     BOXTEXT_TEXTS.append((fig, t))
484     return t
485
486
487 def note_box(ax, text, x=0.985, y=0.965, fs=8.2, ha="right", va="top"):
488     t = ax.text(x, y, text, transform=ax.transAxes, fontsize=fs, ha=ha, va=va,
489               bbox=dict(boxstyle="round,pad=0.35", facecolor="#f5f5f5",
490                         edgecolor="#bdbdbd", linewidth=0.6))
491     BOXTEXT_TEXTS.append((ax.figure, t))
492     return t
493
494
495 def stat_block(v, unit, extra=""):
496     def f(x):
497         return f"{x:,.2f}"
498
499     s = (f"n = {v.size:,}\n"
500         f"均值 = {f(v.mean())}\n"
501         f"中位 = {f(np.median(v))}\n"
502         f"P5 = {f(np.percentile(v, 5))}\n"
503         f"P95 = {f(np.percentile(v, 95))}\n"
504         f"最小 = {f(v.min())}\n"
505         f"最大 = {f(v.max())}\n"
506         f"标准差 = {f(v.std(ddof=1))}\n"
507         f" (单位: {unit}) ")
508     if extra:
509         s += "\n" + extra
510     return s
511
512
513 # =====
514 # 图3-1 时间对齐与数据结构 (环节 1、2)
515 # =====
516 def figure_3_1():
517     fig = plt.figure(figsize=(13.6, 9.6))
518     gs = fig.add_gridspec(2, 2, height_ratios=[1.0, 1.02], width_ratios=[1.12, 1.0],
519                          left=0.035, right=0.985, top=0.885, bottom=0.035,
520                          wspace=0.09, hspace=0.20)
521
522     ax_struct = fig.add_subplot(gs[0, 0])
523     ax_label = fig.add_subplot(gs[0, 1])
524     ax_map = fig.add_subplot(gs[1, :])
525
526     fig.suptitle("图3-1 时间对齐与数据结构 (预处理环节 1—2)",
527                 fontsize=15, fontweight="bold", y=0.972)

```

```

528
529 # ---- (a) 四附件结构对照 ----
530 panel_title(ax_struct, "(a) 四附件结构对照: 原始尺寸 → 统一尺度", fs=11.5)
531 rows = [
532     ["附件1", "Sheet1", "145 × 4\n(144 数据行)", "1 日 × 144 槽", "144 × 3 = 432",
533      f"时间型 {t1_types.get('time', 0)} + 文本型 {t1_types.get('str', 0)}"],
534     ["附件2", "小区负载", "366 × 145", "365 日 × 144 槽", "52,560",
535      f"时间型 {hdr2_types.get('time', 0)} + 文本型 {hdr2_types.get('str', 0)}"],
536     ["附件2", "光伏发电实际功率", "366 × 145", "365 日 × 144 槽", "52,560", "同上（表头一
致）"],
537     ["附件3", "Sheet1", "1461 × 26", "365 日 × 4 预报时刻", "1,460 × 24\n= 35,040", "文本
型 ('2025-1-1')"],
538     ["附件4", "Sheet1", "366 × 145", "365 日 × 144 槽", "52,560",
539      f"时间型 {hdr4_types.get('time', 0)} + 文本型 {hdr4_types.get('str', 0)}"],
540 ]
541 render_table(
542     ax_struct,
543     ["附件", "工作表", "原始尺寸\n(行 × 列)", "对齐后结构", "数值单元数", "时间键单元格类型
"],
544     rows,
545     col_widths=[0.085, 0.185, 0.155, 0.19, 0.155, 0.23],
546     fontsize=8.0,
547     row_colors=["#eef3fb", "#ffffff", "#ffffff", "#eef3fb", "#ffffff"],
548 )
549 note_box(ax_struct,
550     f"附件1 为单日 144 槽输入（六项形状指标指向 6 个不同日期 => 不能断言对应某一具体实际
日）;\n"
551     f"附件1 光伏非零 {PV1_NZ} 列（标签 {PV1_NZ_RANGE}）;"
552     f"附件3 预报峰值 {A3_MAX:,.2f} kW; 附件2 无全零光伏日（{PV_ALLZERO_DAYS} 日）",
553     x=0.5, y=-0.045, fs=7.8, ha="center", va="top")
554
555 # ---- (b) 时间标签归一化前后对照 ----
556 panel_title(ax_label, "(b) 时间标签归一化: 不依赖单元格类型", fs=11.5)
557 rows2 = [
558     ["附件1 / 2 / 4", "00:10", "时间型 time", "10", "[0:10, 0:20) 第 1 槽"],
559     ["附件1", "10:10", "文本型 str", "610", "[10:10, 10:20) 第 61 槽"],
560     ["附件2 / 4", "23:50", "时间型 time", "1430", "[23:50, 24:00) 第 143 槽"],
561     ["附件2 / 4", "0:00+1", "文本型 str", "1440", "[次日 0:00, 次日 0:10) 第 144 槽"],
562     ["附件3", "6:00", "文本型 str", "360", "发布时刻, 按块继承日期后成键"],
563 ]
564 render_table(
565     ax_label,
566     ["来源", "标签原文", "单元格类型", "归一化\n(距 0:00 分钟)", "方案丙槽区间"],
567     rows2,
568     col_widths=[0.165, 0.13, 0.185, 0.20, 0.32],
569     fontsize=8.0,
570     row_colors=["#ffffff", "#fff4e5", "#ffffff", "#ffe9e3", "#eef3fb"],
571 )
572 note_box(ax_label,
573     "关键: 附件1 时间列 60 个时间型 + 84 个文本型;\n"
574     "附件2 / 4 表头 143 个时间型 + 1 个文本型【0:00+1】\n"

```

```

575         "> 一律解析为分钟数，禁止按类型分支解析",
576         x=0.5, y=-0.045, fs=8.0, ha="center", va="top")
577
578 # ---- (c) 统一到 10 分钟尺度的映射示意 ----
579 ax_map.set_xlim(-175, 1700)
580 ax_map.set_ylim(0.0, 10.9)
581 ax_map.axis("off")
582 panel_title(ax_map, "(c) 统一到 10 分钟尺度的映射 (方案丙: 标签=槽开始时刻)", fs=11.5)
583
584 x0, x1 = 10, 1450
585
586 # 行 A: 附件1
587 ya = 8.55
588 ax_map.add_patch(Rectangle((x0, ya - 0.5), x1 - x0, 1.0, facecolor=C[0],
589                             alpha=0.18, edgecolor=C[0], linewidth=1.1))
590
591 step = (x1 - x0) / 144.0
592 for k in range(0, 145, 12):
593     xk = x0 + k * step
594     ax_map.plot([xk, xk], [ya - 0.5, ya + 0.5], color=C[0], linewidth=0.6, alpha=0.55)
595     ax_map.text(-168, ya, "附件1\n单 144 槽", ha="left", va="center", fontsize=9.2,
596                fontweight="bold", color=C[0])
597     ax_map.annotate("第 1 槽 [0:10, 0:20]", xy=(x0 + 0.5 * step, ya + 0.5),
598                    xytext=(x0 - 5, ya + 1.45), fontsize=8.4, color=C[0],
599                    arrowprops=dict(arrowstyle="->", color=C[0], lw=0.9))
600     ax_map.annotate("第 144 槽 标签 [0:00+1] = 1440 min\n[次日 0:00, 次日 0:10]",
601                    xy=(x1, ya + 0.5), xytext=(x1 - 425, ya + 1.35), fontsize=8.4, color=C
602                    [0],
603                    arrowprops=dict(arrowstyle="->", color=C[0], lw=0.9))
604
605 # 行 B: 附件2/4
606 yb = 5.55
607 n_show = 7
608 bw, gap = 168, 34
609 xs = x0
610 for k in range(n_show):
611     ax_map.add_patch(Rectangle((xs, yb - 0.45), bw, 0.9, facecolor=C[1], alpha=0.16,
612                                edgecolor=C[1], linewidth=1.0))
613     for t in range(1, 6):
614         ax_map.plot([xs + t * bw / 6.0] * 2, [yb - 0.45, yb + 0.45], color=C[1],
615                    linewidth=0.45, alpha=0.5)
616         xs += bw + gap
617     ax_map.text(x0 + 4, yb - 0.72, "2025-01-01", fontsize=7.8, color=C[1], ha="left", va="
618     top")
619     ax_map.text(xs - bw - gap - 4, yb - 0.72, "2025-12-31", fontsize=7.8, color=C[1],
620                ha="right", va="top")
621     ax_map.text((x0 + xs - gap) / 2.0, yb - 0.72, "... 每一日 144 槽 ...", fontsize=8.2,
622                color=C[1], ha="center", va="top")
623     ax_map.text(-168, yb, "附件2 / 附件4\n365 日 × 144 槽", ha="left", va="center",
624                fontsize=9.2, fontweight="bold", color=C[1])
625     ax_map.annotate(f"52,560 观测单元 = 365 日 × 144 槽\n"
626                    f"(天=决策单元, 10 分钟=执行单元)",

```

```

624         xy=((x0 + xs - gap) / 2.0, yb + 0.45), xytext=((x0 + xs - gap) / 2.0 -
150, yb + 1.35),
625         fontsize=8.6, color=C[1], ha="left",
626         arrowprops=dict(arrowstyle="->", color=C[1], lw=0.9))
627
628 # 行 C: 附件3
629 yc = 2.75
630 pub_x = [x0, x0 + 480, x0 + 960, x0 + 1440]
631 pub_lab = ["0:00", "6:00", "12:00", "18:00"]
632 for px, pl in zip(pub_x, pub_lab):
633     ax_map.plot([px], [yc], marker="o", markersize=6.0, color=C[2], zorder=3)
634     ax_map.plot([px, px], [yc - 0.42, yc + 0.42], color=C[2], linewidth=0.9, alpha
=0.55)
635     for t in range(24):
636         ax_map.plot([px + (t + 1) * 19.5], [yc + 0.30], marker="|", markersize=5,
637                 color=C[2], alpha=0.65)
638         ax_map.text(px, yc - 0.55, pl, fontsize=8.0, color=C[2], ha="center", va="top")
639 ax_map.text(-168, yc, "附件3\n预报发布时刻", ha="left", va="center", fontsize=9.2,
640         fontweight="bold", color=C[2])
641 ax_map.annotate(f"每日 4 个发布时刻 × 未来 24 个整点预报\n"
642         f"365 日 × 4 × 24 = 35,040 个预报值",
643         xy=(pub_x[1] + 250, yc + 0.38), xytext=(pub_x[1] + 130, yc + 1.30),
644         fontsize=8.6, color=C[2], ha="left",
645         arrowprops=dict(arrowstyle="->", color=C[2], lw=0.9))
646
647 # 时间刻度轴
648 yax = 0.95
649 ax_map.annotate("", xy=(x1 + 20, yax), xytext=(x0, yax),
650         arrowprops=dict(arrowstyle="->", color="#444444", lw=1.2))
651 for m, lab in [(10, "0:10"), (360, "6:00"), (720, "12:00"), (1080, "18:00"),
652         (1440, "次日 0:00"), (1450, "次日 0:10")]:
653     ax_map.plot([m, m], [yax - 0.13, yax + 0.13], color="#444444", linewidth=1.0)
654     ax_map.text(m, yax - 0.30, lab, fontsize=7.8, ha="center", va="top", color="#333333
")
655 ax_map.text(x1 + 45, yax, "时刻", fontsize=8.4, va="center", color="#333333")
656
657 ax_map.text(
658     x0, 0.12,
659     "口径(方案丙): 第 j 槽 = [10j, 10j+10) 分钟(标签_j 为槽开始时刻); 每日窗口 0:10 → 次
日 0:10; "
660     "Δt = 1/6 h; 电量(kWh) = 功率(kW) × 1/6",
661     fontsize=8.6, va="bottom", ha="left",
662     bbox=dict(boxstyle="round,pad=0.35", facecolor="#f0f4fa", edgecolor="#9aa7bd",
linewidth=0.7))
663
664 return save(fig, "图3-1_时间对齐与数据结构")
665
666
667 # =====
668 # 图3-2 质量检验与缺失识别(环节 2、3)
669 # =====

```

```

670 def figure_3_2():
671     fig = plt.figure(figsize=(13.6, 8.8))
672     gs = fig.add_gridspec(2, 2, height_ratios=[0.92, 1.08], width_ratios=[1.25, 1.0],
673                          left=0.035, right=0.985, top=0.885, bottom=0.085,
674                          wspace=0.14, hspace=0.26)
675
676     ax_ea = fig.add_subplot(gs[0, 0])
677     ax_ms = fig.add_subplot(gs[0, 1])
678     ax_blk = fig.add_subplot(gs[1, :])
679
680     fig.suptitle("图3-2 数据质量检验与缺失识别（预处理环节 2—3）",
681                 fontsize=15, fontweight="bold", y=0.972)
682
683     # ---- (a) 应有 vs 实际 ----
684     panel_title(ax_ea, "(a) 应有记录数 vs 实际记录数（逐一重合 => 完整性 100%）", fs=11.5)
685     items = [
686         ("附件1 时间 / 电价 / 负载 / 光伏\n（各 144）", 144, 144),
687         ("附件3 （日期,时刻）键\n1460", 1460, len(set(A3KEY))),
688         ("附件3 预报矩阵\n1460 × 24", 1460 * 24, A3VAL.size),
689         ("附件2 小区负载\n365 × 144", NCELL, LD.size),
690         ("附件2 光伏实际\n365 × 144", NCELL, PV.size),
691         ("附件4 电价\n365 × 144", NCELL, PR.size),
692     ]
693     ypos = np.arange(len(items))[:-1]
694     for y, (lab, exp, act) in zip(ypos, items):
695         ax_ea.hlines(y, 100, act, color=C[0], linewidth=2.2, alpha=0.55)
696         ax_ea.plot(exp, y, marker="o", markersize=10, markerfacecolor="white",
697                   markededgecolor=C[3], markedewidth=1.6, linestyle="none", zorder=3)
698         ax_ea.plot(act, y, marker="o", markersize=5.0, color=C[0], linestyle="none", zorder
699                   =4)
700         ax_ea.text(act * 1.18, y, f"{act:,.}", fontsize=8.2, va="center", color="#222222")
701     ax_ea.set_yticks(ypos)
702     ax_ea.set_yticklabels([lab for lab, _, _ in items], fontsize=7.8)
703     ax_ea.set_xscale("log")
704     ax_ea.set_xlim(100, 3.2e5)
705     ax_ea.set_xlabel("记录数（对数轴）", fontsize=9.5)
706     ax_ea.grid(axis="x", linestyle=":", alpha=0.4)
707     ax_ea.set_axisbelow(True)
708     ax_ea.plot([], [], marker="o", markersize=9, markerfacecolor="white",
709               markededgecolor=C[3], markedewidth=1.6, linestyle="none", label="应有记录
710               数")
711     ax_ea.plot([], [], marker="o", markersize=5, color=C[0], linestyle="none", label="实际记
712               录数")
713     ax_ea.legend(fontsize=8.2, loc="lower right", frameon=True, framealpha=0.95)
714     note_box(ax_ea, "六项一一重合\n实际 - 应有 = 0", x=0.02, y=0.98, fs=8.4, ha="left", va="
715               top")
716
717     # ---- (b) 缺失统计 ----
718     panel_title(ax_ms, "(b) 缺失 / 重复 / 负值统计：全部为 0", fs=11.5)
719     cats = ["缺失记录", "重复记录", "负值记录", "夜间光伏\n>1 kW", "越界 / 违背\n物理规律"]
720     vals = [MISS_TOTAL, sum(DUP.values()), sum(NEG.values()), NIGHT_PV_GT1, 0]

```

```

717 cols = [C[3], C[3], C[3], C[3], C[3]]
718 xb = np.arange(len(cats) + 1)
719 heights = vals + [a3_blank]
720 colors = cols + [GREY]
721 ax_ms.bar(xb, heights, width=0.62, color=colors, alpha=0.88,
722           edgcolor="white", linewidth=0.8)
723 for x, h in zip(xb, heights):
724     ax_ms.text(x, h + max(heights) * 0.035, f"{h:,.}", ha="center", va="bottom",
725               fontsize=10.5, fontweight="bold",
726               color=(C[3] if h == 0 else "#444444"))
727 ax_ms.set_xticks(xb)
728 ax_ms.set_xticklabels(cats + ["附件3 日期列空白\n(块内延续, 非缺失)"], fontsize=8.0)
729 ax_ms.set_ylabel("记录数", fontsize=9.5)
730 ax_ms.set_ylim(0, max(heights) * 1.30)
731 ax_ms.grid(axis="y", linestyle=":", alpha=0.4)
732 ax_ms.set_axisbelow(True)
733 ax_ms.get_xticklabels()[-1].set_color(GREY)
734 note_box(ax_ms, "缺失 = 0 重复 = 0 负值 = 0\n(四附件全部逐格核对)",
735           x=0.5, y=0.975, fs=8.6, ha="center", va="top")
736
737 # ---- (c) 附件3 日期列块内继承示意 ----
738 panel_title(ax_blk, "(c) 附件3 日期列空白=块内延续表达(非缺失): 向下继承后成键", fs
739             =11.5)
740 times_order = ["0:00", "6:00", "12:00", "18:00"] # noqa: F841 (保留: 说明块内时刻顺序)
741 rows_t, colors_t = [], []
742 shown = 0
743 for bi in range(len(a3_body)):
744     r = a3_body[bi]
745     if bi % 4 == 0:
746         shown += 1
747         if shown > 3:
748             break
749     raw = r[0]
750     raw_txt = "(空白)" if raw in (None, "") else str(raw)
751     rows_t.append([f"{bi + 1}", raw_txt, str(r[1]), block_dates[bi],
752                  f"({block_dates[bi]}, {r[1]})"])
753     colors_t.append("#fdece7" if raw in (None, "") else "#e8f2e8")
754 rows_t.append(["...", "...", "...", "...", "..."])
755 colors_t.append("#f5f5f5")
756 rows_t.append(["1460", "(空白, 末块第 4 行)", "18:00", "2025-12-31", "(2025-12-31, 18:00)"])
757 colors_t.append("#fdece7")
758 render_table(
759     ax_blk,
760     ["行号", "日期列原文", "预报时刻", "按块向下继承的日期", "继承后键(日期, 预报时刻)"],
761     rows_t,
762     col_widths=[0.08, 0.22, 0.13, 0.24, 0.33],
763     fontsize=8.4,
764     row_colors=colors_t,
765 )
766 fig_note(fig, ax_blk,

```

```

766         f"365 个日期块 × 每块严格 4 行 (0:00→6:00→12:00→18:00) = 1,460 行; "
767         f"日期列空白 {a3_blank:,} 行属『块内延续表达』, 不是缺失\n"
768         f"继承后 (日期, 预报时刻) 唯一键 {len(set(A3KEY)),} / {len(A3KEY),}"
769         f" (无重复、无缺块); 若按缺失处理将误弃 3/4 的预报样本",
770         dy=0.022, fs=8.4)
771
772     return save(fig, "图3-2_质量检验与缺失识别")
773
774
775     # =====
776     # 图3-3 异常辨识与判据修正 (环节 4)
777     # =====
778     def figure_3_3():
779         fig = plt.figure(figsize=(13.6, 8.6))
780         gs = fig.add_gridspec(2, 2, height_ratios=[1.35, 1.0],
781                                left=0.055, right=0.985, top=0.885, bottom=0.045,
782                                wspace=0.16, hspace=0.30)
783
784         ax_full = fig.add_subplot(gs[0, 0])
785         ax_day = fig.add_subplot(gs[0, 1])
786         ax_txt = fig.add_subplot(gs[1, :])
787
788         fig.suptitle("图3-3 异常数据辨识与判据修正 (预处理环节 4)",
789                     fontsize=15, fontweight="bold", y=0.972)
790
791         xslot = np.tile(np.arange(1, NSLOT + 1), NDAY)
792         y_all = PV.ravel()
793         f_all = cand_full.ravel()
794
795         # ---- 左: 全样本 MAD 判据 ----
796         panel_title(ax_full, "(a) 全样本 MAD 稳健判据: 候选异常 25,785 (49.06%) — 判据失效", fs
797                     =11.0)
798         ax_full.scatter(xslot[~f_all], y_all[~f_all], s=1.1, color=C[0], alpha=0.30,
799                        linewidths=0, rasterized=True, label="判为正常")
800         ax_full.scatter(xslot[f_all], y_all[f_all], s=1.1, color=C[3], alpha=0.42,
801                        linewidths=0, rasterized=True, label=f"判为候选异常 ({N_CAND_FULL:,})")
802         ax_full.axhline(mad_full_med, color="#333333", linestyle="--", linewidth=1.2,
803                        label=f"中位数 = {mad_full_med:,.2f} kW")
804         ax_full.axhline(THR_FULL_HI, color=C[3], linestyle=":", linewidth=1.4,
805                        label=f"上界 = 中位 + 3.5 · MAD/0.6745 = {THR_FULL_HI:,.2f} kW")
806         ax_full.set_xlabel("日内槽序号 j (1—144, 标签 = 槽开始时刻)", fontsize=9.5)
807         ax_full.set_ylabel("附件2 光伏实际功率 (kW)", fontsize=9.5)
808         ax_full.set_xlim(0.5, 144.5)
809         ax_full.set_xticks([1, 25, 49, 73, 97, 121, 144])
810         ax_full.set_xticklabels(["1\n0:10", "25\n4:10", "49\n8:10", "73\n12:10",
811                                 "97\n16:10", "121\n20:10", "144\n0:00+1"], fontsize=7.8)
812         ax_full.grid(linestyle=":", alpha=0.35)
813         ax_full.set_axisbelow(True)
814         ax_full.legend(fontsize=7.6, loc="upper left", framealpha=0.95)
815         ax_full.axvspan(-0.5, 24.5, color="#dbe6f5", alpha=0.35, zorder=0)
816         ax_full.axvspan(120.5, 144.5, color="#dbe6f5", alpha=0.35, zorder=0)

```

```

816 note_box(ax_full,
817         f"MAD = {mad_full:,.2f} kW ( $\approx$  中位数, 因 48.53% 为 0) \n"
818         f"判据退化为: 光伏 > {THR_FULL_HI:,.1f} kW 即异常\n"
819         f"=> 白天正常发电 25,785 个被误标; \n"
820         f"而  $\leq$  {THR_FULL_HI:,.1f} kW 的 {day_low_n:,} 个白天样本反而漏判",
821         x=0.985, y=0.975, fs=7.8, ha="right", va="top")
822
823 # ---- 右: 分层后白天子样本 ----
824 panel_title(ax_day, "(b) 分层后白天子样本判据: 候选异常 0 / 27,051 (0.000%) ", fs=11.0)
825 day_mask = PV > 1
826 xd = xslot[day_mask.ravel()]
827 yd = y_all[day_mask.ravel()]
828 ax_day.scatter(xd, yd, s=1.3, color=C[0], alpha=0.40, linewidths=0, rasterized=True,
829               label=f"白天子样本 (实际 > 1 kW, n = {DAY_N:,}) ")
830 ax_day.axhline(day_med, color="#333333", linestyle="--", linewidth=1.2,
831               label=f"白天中位数 = {day_med:,.2f} kW")
832 ax_day.axhspan(day_med - THR_DAY, day_med + THR_DAY, color=C[2], alpha=0.13,
833               label=f" $\pm 3.5 \cdot \text{MAD} / 0.6745 = \pm \{\text{THR\_DAY:,.0f}\}$  kW")
834 ax_day.set_xlabel("日内槽序号 j (1—144, 标签 = 槽开始时刻)", fontsize=9.5)
835 ax_day.set_ylabel("附件2 光伏实际功率 (kW)", fontsize=9.5)
836 ax_day.set_xlim(0.5, 144.5)
837 ax_day.set_ylim(-900, 11800)
838 ax_day.set_xticks([1, 25, 49, 73, 97, 121, 144])
839 ax_day.set_xticklabels(["1\n0:10", "25\n4:10", "49\n8:10", "73\n12:10",
840                        "97\n16:10", "121\n20:10", "144\n0:00+1"], fontsize=7.8)
841 ax_day.grid(linestyle=":", alpha=0.35)
842 ax_day.set_axisbelow(True)
843 ax_day.legend(fontsize=7.6, loc="upper left", framealpha=0.95)
844 ax_day.annotate(f"判据上界 {day_med + THR_DAY:,.0f} kW  $\uparrow$  超出坐标范围",
845                xy=(120, 11200), xytext=(60, 10500), fontsize=8.0, color=C[2],
846                arrowprops=dict(arrowstyle="->", color=C[2], lw=0.9))
847 ax_day.annotate(f"判据下界 {day_med - THR_DAY:,.0f} kW  $\downarrow$  超出坐标范围",
848                xy=(120, -400), xytext=(58, -800), fontsize=8.0, color=C[2],
849                arrowprops=dict(arrowstyle="->", color=C[2], lw=0.9))
850 note_box(ax_day,
851         f"白天中位 = {day_med:,.2f} kW, MAD = {day_mad:,.2f} kW\n"
852         f"max|稳健 z| = {Z_DAY_MAX:.3f} < 3.5 => 候选异常 0 个\n"
853         f"物理一致性: 夜间 (20:00—次日 04:00) \n"
854         f"光伏 > 1 kW 的样本 {NIGHT_PV_GT1} 个; 负值 {NEG['附件2 光伏实际']} 个",
855         x=0.985, y=0.975, fs=7.8, ha="right", va="top")
856
857 # ---- 下: 零膨胀说明 ----
858 ax_txt.axis("off")
859 panel_title(ax_txt, "(c) 零膨胀为何使中位数 / MAD 失效, 以及修正后的判据链", fs=11.0)
860 inset = ax_txt.inset_axes([0.012, 0.06, 0.30, 0.84])
861 bins = np.linspace(0, 10500, 61)
862 inset.hist(PV.ravel(), bins=bins, color=C[0], alpha=0.85, edgecolor="white", linewidth
863           =0.3)
864 inset.set_yscale("log")
865 inset.set_xlabel("光伏实际功率 (kW)", fontsize=8.4)
866 inset.set_ylabel("频数 (对数轴)", fontsize=8.4)

```

```

866 inset.tick_params(labelsize=7.4)
867 inset.set_title(f"0—1 kW 区间 {PV_LE1:,} 个 ({FRAC_LE1:.2%})", fontsize=8.4, pad=4)
868 inset.annotate("零值尖峰\n(真实无光照段: \n" + NEARZERO_RANGE_TXT + ")",
869               xy=(60, PV_LE1 * 0.85),
870               xytext=(2400, PV_LE1 * 0.10), fontsize=7.6, color=C[3],
871               arrowprops=dict(arrowstyle="→", color=C[3], lw=0.9))
872 inset.axvline(mad_full_med, color="#444444", linestyle="--", linewidth=1.0)
873 inset.text(mad_full_med + 220, 2.2e3, f"全样本中位数 {mad_full_med:.1f} kW",
874           fontsize=7.2, color="#444444", rotation=90, va="bottom")
875
876 lines = [
877     f"① 零膨胀事实: 附件2 光伏实际 ≤1 kW 共 {PV_LE1:,} / {NCELL:,} ({FRAC_LE1:.2%}); ",
878     f"全年最大出力 ≤1 kW 的时段 {NEARZERO_SLOTS} 个 ({NEARZERO_RANGE_TXT}, ",
879     f"共 {PV_EXACT_ZERO:,} 个精确 0 值), 属真实无光照而非缺失 => 不做插补、不删除。",
880     f"② 判据失效机理: 0 值占近半使中位数被拉平到 {mad_full_med:.2f} kW, ",
881     f"MAD 同步塌缩为 {mad_full:.2f} kW; 稳健 z 的阈值变为 {THR_FULL_HI:.1f} kW, ",
882     f"于是『凡是白天正常发电』都被判成异常 ({N_CAND_FULL:,} 个, 占 {FRAC_CAND_FULL:.2%}
883 ), ",
884     f"而真正需要关注的白天低值样本 ({day_low_n:,} 个) 被漏判。",
885     f"③ 修正判据: 先按物理含义分层 —— 夜间 (20:00—次日 04:00, {NIGHT_SLOTS} 个槽) 与白
886 天; ",
887     f"仅在白天子样本 (实际 >1 kW, n = {DAY_N:,}) 上做稳健 z 检验, ",
888     f"阈值区间 [-{THR_DAY:,.0f}, +{THR_DAY:,.0f}] kW 完全落在数据范围 "
889     f"[1, {PV.max():,.1f}] kW 之外 => 候选异常 {N_CAND_DAY} 个 (0.000%)。",
890     f"④ 物理一致性检查全通过: 夜间发电 {NIGHT_PV_GT1} 个 (夜间最大 {NIGHT_PV_MAX:.4f} kW
891 ); ",
892     f"负值 {NEG['附件2 光伏实际']} 个; 白天最大 {PV.max():,.1f} kW (未超过可解释范围)。",
893     f"结论: 本数据在修正判据下真实异常 0 个; 全部样本原样保留, 不删除、不插补、不重标注。",
894 ]
895
896 for i, ln in enumerate(lines):
897     ax_txt.text(0.345, 0.90 - i * 0.128, ln, transform=ax_txt.transAxes, fontsize=8.6,
898               va="top", ha="left",
899               fontweight="bold" if i == len(lines) - 1 else "normal",
900               color="#111111" if i == len(lines) - 1 else "#222222")
901
902 return save(fig, "图3-3_异常辨识与判据修正")
903
904 # =====
905 # 图3-4 异常处理与极端值保留 (环节 5)
906 # =====
907
908 def figure_3_4():
909     fig = plt.figure(figsize=(13.8, 6.8))
910     gs = fig.add_gridspec(1, 2, width_ratios=[1.10, 1.0],
911                       left=0.055, right=0.985, top=0.865, bottom=0.095, wspace=0.14)
912     sub = gs[0, 0].subgridspec(2, 1, height_ratios=[0.30, 1.0], hspace=0.10)
913     ax_box = fig.add_subplot(sub[0])
914     ax_hist = fig.add_subplot(sub[1])
915     ax_tbl = fig.add_subplot(gs[0, 1])
916
917     fig.suptitle("图3-4 异常数据处理与极端电价保留 (预处理环节 5)",

```

```

914         fontsize=15, fontweight="bold", y=0.965)
915
916 flat = PR.ravel()
917 # ---- 左: 附件4 电价分布 ----
918 bp = ax_box.boxplot([flat], vert=False, widths=0.5, patch_artist=True,
919                     showfliers=True, whis=(5, 95),
920                     flierprops=dict(marker=".", markersize=2.5, markerfacecolor=C[3],
921                                     markededgecolor="none", alpha=0.35))
922 bp["boxes"][0].set(facecolor=C[0], alpha=0.42, edgecolor=C[0])
923 bp["medians"][0].set(color="#222222", linewidth=1.4)
924 for w in ("whiskers", "caps"):
925     for art in bp[w]:
926         art.set(color=C[0], linewidth=1.0)
927 ax_box.plot([PR_LOW_MIN], [1], marker="v", markersize=8, color=C[3], zorder=5)
928 ax_box.plot([PR_HIGH_MAX], [1], marker="v", markersize=8, color=C[3], zorder=5)
929 ax_box.set_xlim(-0.05, 1.90)
930 ax_box.set_yticks([])
931 ax_box.set_xlabel("")
932 ax_box.tick_params(labelbottom=False)
933 panel_title(ax_box, " (a) 附件4 电价分布与极端值 (n = 52,560, 元/kWh)", fs=11.0)
934 ax_box.annotate(f"最小 {PR_LOW_MIN:.4f}\n({PR_LOW} 个 < 0.05)",
935                xy=(PR_LOW_MIN, 1.16), xytext=(0.09, 1.30), fontsize=8.2, color=C[3],
936                ha="left", va="center",
937                arrowprops=dict(arrowstyle="->", color=C[3], lw=0.9))
938 ax_box.annotate(f"最大 {PR_HIGH_MAX:.4f}\n({PR_HIGH} 个 > 1.5)",
939                xy=(PR_HIGH_MAX, 1.16), xytext=(1.35, 1.30), fontsize=8.2, color=C[3],
940                ha="left", va="center",
941                arrowprops=dict(arrowstyle="->", color=C[3], lw=0.9))
942 ax_box.set_ylim(0.55, 1.75)
943
944 bins = np.linspace(0, 1.85, 93)
945 ax_hist.hist(flat, bins=bins, color=C[0], alpha=0.86, edgecolor="white", linewidth=0.3,
946             label="全部 52,560 个电价样本")
947 ax_hist.set_yscale("log")
948 ax_hist.set_xlim(-0.05, 1.90)
949 ax_hist.set_xlabel("电价 (元/kWh)", fontsize=9.5)
950 ax_hist.set_ylabel("频数 (对数轴)", fontsize=9.5)
951 ax_hist.grid(axis="y", linestyle=":", alpha=0.4)
952 ax_hist.set_axisbelow(True)
953 ax_hist.axvline(0.05, color=C[3], linestyle="--", linewidth=1.3)
954 ax_hist.axvline(1.5, color=C[3], linestyle="--", linewidth=1.3)
955 ax_hist.axvspan(-0.05, 0.05, color=C[3], alpha=0.14)
956 ax_hist.axvspan(1.5, 1.90, color=C[3], alpha=0.14)
957 ax_hist.plot(flat[flat < 0.05], np.full(PR_LOW, 4.0), marker="v", linestyle="none",
958             markersize=6.5, color=C[3], label=f"< 0.05 元/kWh: {PR_LOW} 个")
959 ax_hist.text(0.075, 1.1e3, f"< 0.05: {PR_LOW} 个\n最低 {PR_LOW_MIN:.4f}", fontsize=8.4,
960             color=C[3], ha="left", va="center")
961 ax_hist.text(1.335, 1.1e3, f"> 1.5: {PR_HIGH:}, 个\n最高 {PR_HIGH_MAX:.4f}", fontsize
962             =8.4,
963             color=C[3], ha="right", va="center")
964 ax_hist.legend(fontsize=8.2, loc="upper center", framealpha=0.95)

```

```

964 note_box(ax_hist,
965         f"全样本: 最小 {PR.min():.4f} 中位 {np.median(PR):.4f} 最大 {PR.max():.4f} "
966         f"均值 {PR.mean():.4f}\n"
967         f"附件1 电价 (题目给定固定日) 区间 {PRICE1.min():.4f}—{PRICE1.max():.4f}, "
968         f"无极端值 → 极端值全部来自附件4 波动电价",
969         x=0.985, y=0.975, fs=8.0, ha="right", va="top")
970
971 # 局部放大: <0.05 的 9 个样本
972 axi = ax_hist.inset_axes([0.055, 0.40, 0.30, 0.34])
973 low = np.sort(flat[flat < 0.05])
974 axi.bar(np.arange(1, PR_LOW + 1), low, color=C[3], alpha=0.85, width=0.62)
975 axi.set_xticks(np.arange(1, PR_LOW + 1))
976 axi.tick_params(labelsize=7.0)
977 axi.set_title(f"放大: < 0.05 的 {PR_LOW} 个样本", fontsize=8.0, pad=3)
978 axi.set_ylabel("元/kWh", fontsize=7.4)
979 axi.set_ylim(0, 0.058)
980 for i, v in enumerate(low):
981     axi.text(i + 1, v + 0.0022, f"{v:.4f}", fontsize=6.2, ha="center", va="bottom",
982             rotation=90)
983
984 # ---- 右: 处理策略对照表 ----
985 panel_title(ax_tbl, "(b) 异常 / 极端值处理策略对照", fs=11.0)
986 rows = [
987     ["保留并标记\n+情景对照\n(本项目采用)",
988      "全部数值原样进入建模;\n把 9 个 <0.05 与 927 个 >1.5\n登记为标记样本;\n另设『电价下限截断』情景并列对照",
989      "不改变题目给定的真实波动;\n结论在真实输入下可复现;\n极端值影响可量化",
990      "● 采用"],
991     ["截断\n(设上 / 下限)",
992      "把 < 阈值抬到阈值、\n> 上限压到上限",
993      "人为降低波动电价问题的\n难度; 阈值选取任意;\n直接改变费用结论",
994      "× 不采用"],
995     ["缩尾\nwinsorize",
996      "把 <P1 或 >P99 的样本\n替换为对应分位数值",
997      "同时抹掉真实极值信息;\n分位点由样本自身决定,\n对 9 个样本极不稳定",
998      "× 不采用"],
999     ["删除样本",
1000      "直接剔除极端值\n所在时段或整日",
1001      "违反『原始数据只读、不因\n难预测而删除样本』;\n时段序列出现空洞需再插补",
1002      "× 不采用"],
1003 ]
1004 render_table(
1005     ax_tbl,
1006     ["处理方案", "具体做法", "主要风险 / 代价", "本项目"],
1007     rows,
1008     col_widths=[0.215, 0.315, 0.325, 0.145],
1009     fontsize=7.8,
1010     row_colors=["#e8f4ea", "#ffffff", "#ffffff", "#ffffff"],
1011     cell_font_colors={(1, 3): "#1a7f37", (2, 3): "#b3261e", (3, 3): "#b3261e", (4, 3):
1012 "#b3261e"},
1013 )

```

```

1013 fig_note(fig, ax_tbl,
1014         "本项目处置原则 (R-01 / R-05) : 全部样本原样保留, 不删除、不插补、不重标注; \n"
1015         "极端电价作为『可放宽假设 R1』的情景对照并列报告, 主分析不改变任何输入值。",
1016         dy=0.018, fs=8.4)
1017
1018 return save(fig, "图3-4_异常处理与极端值保留")
1019
1020
1021 # =====
1022 # 图3-5 变量分布分析 (环节 6)
1023 # =====
1024 def figure_3_5():
1025     fig = plt.figure(figsize=(13.6, 11.0))
1026     gs = fig.add_gridspec(4, 3, width_ratios=[2.30, 0.80, 1.30],
1027                          left=0.055, right=0.985, top=0.905, bottom=0.062,
1028                          wspace=0.30, hspace=0.46)
1029
1030     fig.suptitle("图3-5 变量分布分析: 负载、光伏、净负荷、波动电价 (预处理环节 6)",
1031                  fontsize=15, fontweight="bold", y=0.968)
1032
1033     specs = [
1034         ("小区负载  $l$  (附件2,  $n = 52,560$ ) ", LD.ravel(), "kW", C[0], "linear", None),
1035         ("光伏发电实际  $v$  (附件2,  $n = 52,560$ ) ", PV.ravel(), "kW", C[1], "log", "zero"),
1036         ("净负荷  $l - v$  (派生,  $n = 52,560$ ) ", NL.ravel(), "kW", C[2], "linear", "neg"),
1037         ("波动电价 (附件4,  $n = 52,560$ ) ", PR.ravel(), "元/kWh", C[3], "log", "extreme"),
1038     ]
1039
1040     for i, (title, v, unit, col, yscale, flag) in enumerate(specs):
1041         axh = fig.add_subplot(gs[i, 0])
1042         axb = fig.add_subplot(gs[i, 1])
1043         axs = fig.add_subplot(gs[i, 2])
1044
1045         lo, hi = float(v.min()), float(v.max())
1046         pad = max((hi - lo) * 0.025, 1e-6)
1047         bins = np.linspace(lo - pad, hi + pad, 70)
1048         axh.hist(v, bins=bins, color=col, alpha=0.85, edgecolor="white", linewidth=0.35)
1049         axh.set_yscale(yscale)
1050         axh.set_xlim(lo - pad, hi + pad)
1051         axh.set_ylabel("频数" + (" (对数轴)" if yscale == "log" else ""), fontsize=9.0)
1052         axh.set_xlabel(f"{unit}", fontsize=9.0)
1053         axh.grid(axis="y", linestyle=":", alpha=0.4)
1054         axh.set_axisbelow(True)
1055         panel_title(axh, f"({chr(97 + i)}) {title}", fs=10.6)
1056
1057         mean_v, med_v = float(v.mean()), float(np.median(v))
1058         p5, p95 = float(np.percentile(v, 5)), float(np.percentile(v, 95))
1059         axh.axvline(mean_v, color="#222222", linewidth=1.5, label=f"均值 {mean_v:,.2f}")
1060         axh.axvline(med_v, color="#222222", linewidth=1.4, linestyle="--",
1061                    label=f"中位 {med_v:,.2f}")
1062         axh.axvline(p5, color=GREY, linewidth=1.1, linestyle=":", label=f"P5 {p5:,.2f}")
1063         axh.axvline(p95, color=GREY, linewidth=1.1, linestyle=":",

```

```

1064         label=f"P95 {p95:,.2f}")
1065
1066     extra = ""
1067     if flag == "zero":
1068         zero_bins = v <= 1
1069         axh.hist(v[zero_bins], bins=np.linspace(lo - pad, 1.0, 3), color=C[3], alpha
1070 =0.75,
1071             edgecolor="white", linewidth=0.35)
1072         axh.text(0.34, 0.30,
1073             f"零膨胀: ≤1 kW 占 {FRAC_LE1:.2%} ({PV_LE1:,} 个) \n"
1074             f"全年最大出力 ≤1 kW 的时段 {NEARZERO_SLOTS} 个\n"
1075             f"({NEARZERO_RANGE_TXT}) —— 真实无光照, 未插补",
1076             transform=axh.transAxes, fontsize=8.2, color=C[3], ha="left", va="
1077 center",
1078             bbox=dict(boxstyle="round,pad=0.3", facecolor="#fff4f0",
1079                 edgecolor=C[3], linewidth=0.7))
1080         extra = f"零值占比 = {FRAC_LE1:.2%}\n精确 0 值 = {PV_EXACT_ZERO:,} 个\n偏度 = {
1081 stats.skew(v):,.3f}"
1082     elif flag == "neg":
1083         axh.axvspan(lo - pad, 0, color=C[3], alpha=0.13)
1084         axh.text(0.03, 0.66,
1085             f"净负荷 < 0 (光伏过剩) 时段占 {NL_NEG_FRAC:.2%}\n"
1086             f"({int((NL < 0).sum()):,} 个) \n"
1087             f"最大过剩 {NL_MIN:,.1f} kW\n最大缺额 {NL_MAX:,.1f} kW",
1088             transform=axh.transAxes, fontsize=8.2, color="#8a2c1a", ha="left", va=
1089 "center",
1090             bbox=dict(boxstyle="round,pad=0.3", facecolor="#fff4f0",
1091                 edgecolor=C[3], linewidth=0.7))
1092         extra = (f"负值占比 = {NL_NEG_FRAC:.2%}\n"
1093             f"过剩 >5,000 kW = {NL_NEG_OVER5} 个\n缺额 >5,000 kW = {NL_POS_OVER5:,}
1094 个")
1095     elif flag == "extreme":
1096         axh.text(0.03, 0.64,
1097             f"极端值保留: <0.05 元/kWh 共 {PR_LOW} 个\n(最低 {PR_LOW_MIN:.4f}) \n"
1098             f">1.5 元/kWh 共 {PR_HIGH:,} 个\n(最高 {PR_HIGH_MAX:.4f})",
1099             transform=axh.transAxes, fontsize=8.2, color="#8a2c1a", ha="left", va=
1100 "center",
1101             bbox=dict(boxstyle="round,pad=0.3", facecolor="#fff4f0",
1102                 edgecolor=C[3], linewidth=0.7))
1103         extra = f"偏度 = {stats.skew(v):,.3f}\n峰度 = {stats.kurtosis(v):,.3f}"
1104
1105     axh.legend(fontsize=7.6, loc="upper right", framealpha=0.95, ncol=2)
1106
1107     # 第三列: 统计量表 (独立坐标轴, 避免任何压图)
1108     axs.axis("off")
1109     axs.set_title("描述统计", fontsize=9.2, pad=6)
1110     axs.text(0.02, 0.5, stat_block(v, unit, extra), transform=axs.transAxes,
1111         fontsize=8.2, ha="left", va="center")
1112
1113     # 右侧箱线
1114     bp = axb.boxplot([v], vert=True, widths=0.5, patch_artist=True, whis=(5, 95),

```

```

1109         showfliers=True,
1110         flierprops=dict(marker=".", markersize=2.0, markerfacecolor=GREY,
1111                         markeredgecolor="none", alpha=0.30))
1112     bp["boxes"][0].set(facecolor=col, alpha=0.45, edgecolor=col)
1113     bp["medians"][0].set(color="#222222", linewidth=1.4)
1114     for w in ("whiskers", "caps"):
1115         for art in bp[w]:
1116             art.set(color=col, linewidth=1.0)
1117     axb.set_xticks([])
1118     axb.set_ylabel(f"{unit}", fontsize=9.0)
1119     axb.grid(axis="y", linestyle=":", alpha=0.4)
1120     axb.set_axisbelow(True)
1121     axb.set_title("箱线 (须端 = P5 / P95)", fontsize=9.2, pad=6)
1122     axb.text(0.5, 1.005, f"最大 {hi:,.2f}", transform=axb.transAxes, fontsize=7.6,
1123             ha="center", va="bottom", color="#8a2c1a")
1124     axb.text(0.5, -0.005, f"最小 {lo:,.2f}", transform=axb.transAxes, fontsize=7.6,
1125             ha="center", va="top", color="#8a2c1a")
1126
1127     fig.text(0.055, 0.015,
1128             f"净负荷 (ℓ - v) 年合计 {E_NET:,.0f} kWh; 负载 {E_LOAD:,.0f} kWh; 光伏 {E_PV:,.0f} kWh; "
1129             f"光伏 / 负载电量比 {RATIO_YEAR:.3f}\n"
1130             f"全部 52,560 个单元无缺失、无负值; 光伏的零值为真实无光照, 未做任何插补或填充",
1131             fontsize=8.4, ha="left", va="bottom",
1132             bbox=dict(boxstyle="round,pad=0.35", facecolor="#f0f4fa", edgecolor="#9aa7bd",
1133                     linewidth=0.7))
1134
1135     return save(fig, "图3-5_变量分布分析")
1136
1137
1138 # =====
1139 # 图3-6 特征构造与派生变量 (环节 7)
1140 # =====
1141 def figure_3_6():
1142     fig = plt.figure(figsize=(13.6, 9.4))
1143     gs = fig.add_gridspec(2, 2, left=0.058, right=0.985, top=0.885, bottom=0.055,
1144                          wspace=0.19, hspace=0.40)
1145     ax_a = fig.add_subplot(gs[0, 0])
1146     ax_b = fig.add_subplot(gs[0, 1])
1147     ax_c = fig.add_subplot(gs[1, 0])
1148     sub = gs[1, 1].subgridspec(2, 1, hspace=0.16)
1149     ax_d1 = fig.add_subplot(sub[0])
1150     ax_d2 = fig.add_subplot(sub[1])
1151
1152     fig.suptitle("图3-6 特征构造与派生变量 (预处理环节 7)",
1153                 fontsize=15, fontweight="bold", y=0.972)
1154
1155     slot_axis = np.arange(1, NSLOT + 1)
1156     ld_prof = LD.mean(axis=0)
1157     pv_prof = PV.mean(axis=0)
1158     nl_prof = NL.mean(axis=0)

```

```

1159
1160 # ---- (a) 日内统计特征 ----
1161 panel_title(ax_a, "(a) 日内统计特征：峰、谷、均值与峰谷差", fs=11.0)
1162 ax_a.plot(slot_axis, ld_prof, color=C[0], linewidth=1.9, label="负载 l (365 日均值)")
1163 ax_a.plot(slot_axis, pv_prof, color=C[1], linewidth=1.9, label="光伏 v (365 日均值)")
1164 ax_a.plot(slot_axis, nl_prof, color=C[2], linewidth=1.6, linestyle="-.",
1165          label="净负荷 l - v")
1166 ax_a.axhline(0, color="#666666", linewidth=0.9, linestyle=":")
1167 ipk = int(np.argmax(ld_prof))
1168 ivl = int(np.argmin(ld_prof))
1169 ax_a.plot([ipk + 1], [ld_prof[ipk]], marker="v", markersize=9, color=C[3], zorder=5)
1170 ax_a.plot([ivl + 1], [ld_prof[ivl]], marker="^", markersize=9, color=C[3], zorder=5)
1171 ax_a.annotate(f"负载日峰 {ld_prof[ipk]:.0f} kW @ {hhmm(slot_min[ipk])}",
1172             xy=(ipk + 1, ld_prof[ipk]), xytext=(ipk - 6, ld_prof[ipk] + 300),
1173             fontsize=8.4, ha="right", color=C[3],
1174             arrowprops=dict(arrowstyle="->", color=C[3], lw=0.9))
1175 ax_a.annotate(f"负载日谷 {ld_prof[ivl]:.0f} kW @ {hhmm(slot_min[ivl])}",
1176             xy=(ivl + 1, ld_prof[ivl]), xytext=(ivl + 12, ld_prof[ivl] - 620),
1177             fontsize=8.4, ha="left", color=C[3],
1178             arrowprops=dict(arrowstyle="->", color=C[3], lw=0.9))
1179 ax_a.axhline(ld_prof.mean(), color=C[0], linewidth=1.0, linestyle=":",
1180             label=f"负载年均 {ld_prof.mean():.0f} kW")
1181 ax_a.set_xlim(1, NSLOT)
1182 ax_a.set_xticks([1, 25, 49, 73, 97, 121, 144])
1183 ax_a.set_xticklabels(["1\n0:10", "25\n4:10", "49\n8:10", "73\n12:10",
1184                     "97\n16:10", "121\n20:10", "144\n0:00+1"], fontsize=7.8)
1185 ax_a.set_xlabel("日内槽序号 (标签=槽开始时刻)", fontsize=9.5)
1186 ax_a.set_ylabel("功率 (kW)", fontsize=9.5)
1187 ax_a.grid(linestyle=":", alpha=0.35)
1188 ax_a.set_axisbelow(True)
1189 ax_a.legend(fontsize=8.0, loc="upper left", framealpha=0.95)
1190 note_box(ax_a,
1191         "日内统计特征：日峰值 / 日谷值 / 日均值 /\n"
1192         "峰谷差 / 峰谷出现时刻 / 日电量 / 负荷率\n"
1193         f"峰谷差 = {ld_prof[ipk] - ld_prof[ivl]:.0f} kW; "
1194         f"负荷率 = {ld_prof.mean() / ld_prof[ipk]:.3f}",
1195         x=0.985, y=0.60, fs=7.8, ha="right", va="top")
1196
1197 # ---- (b) 季节特征：月均曲线 ----
1198 panel_title(ax_b, "(b) 季节特征：月均功率曲线 (同一 kW 量纲, 不用双轴)", fs=11.0)
1199 months = DAILY["日期"].dt.month.values
1200 m_ld = np.array([LD[months == m].mean() for m in range(1, 13)])
1201 m_pv = np.array([PV[months == m].mean() for m in range(1, 13)])
1202 m_nl = np.array([NL[months == m].mean() for m in range(1, 13)])
1203 xm = np.arange(1, 13)
1204 for q, (a, b) in enumerate([(1, 3), (4, 6), (7, 9), (10, 12)]):
1205     ax_b.axvspan(a - 0.5, b + 0.5, color=C[q], alpha=0.055, zorder=0)
1206     ax_b.text((a + b) / 2.0, 0.015, f"Q{q + 1}", fontsize=8.2,
1207             color=C[q], ha="center", va="bottom", alpha=0.95,
1208             transform=ax_b.get_xaxis_transform())
1209 ax_b.plot(xm, m_ld, marker="o", markersize=5.5, color=C[0], linewidth=1.9, label="月均负

```

```

1210     载  $\ell$ ")
1211 ax_b.plot(xm, m_pv, marker="s", markersize=5.5, color=C[1], linewidth=1.9, label="月均光
1212 伏  $v$ ")
1213 ax_b.plot(xm, m_nl, marker="^", markersize=5.0, color=C[2], linewidth=1.6, linestyle="
1214 -. ",
1215         label="月均净负荷  $\ell - v$ ")
1216 ax_b.axhline(0, color="#666666", linewidth=0.9, linestyle=":")
1217 ax_b.set_xticks(xm)
1218 ax_b.set_xticklabels([f"{m}月" for m in xm], fontsize=8.0)
1219 ax_b.set_xlabel("月份 (2025 年)", fontsize=9.5)
1220 ax_b.set_ylabel("平均功率 (kW)", fontsize=9.5)
1221 ax_b.grid(linestyle=":", alpha=0.35)
1222 ax_b.set_axisbelow(True)
1223 ax_b.legend(fontsize=8.0, loc="upper left", framealpha=0.95)
1224 note_box(ax_b,
1225         f"月均光伏 {m_pv.min():.0f}—{m_pv.max():.0f} kW, "
1226         f"峰谷比 {m_pv.max() / max(m_pv.min(), 1e-9):.2f}\n"
1227         f"月均负载 {m_ld.min():.0f}—{m_ld.max():.0f} kW, "
1228         f"峰谷比 {m_ld.max() / m_ld.min():.2f}\n"
1229         "季节特征: 月份 / 季度哑变量、月均与月峰值",
1230         x=0.985, y=0.985, fs=7.8, ha="right", va="top")
1231
1232 # ---- (c) 逐日派生量 ----
1233 panel_title(ax_c, "(c) 逐日派生量: 日电量与净负荷电量 (365 日)", fs=11.0)
1234 dts = DAILY["日期"].values
1235 ax_c.plot(dts, DAILY["负载电量kWh"], color=C[0], linewidth=1.2, label="日负载电量")
1236 ax_c.plot(dts, DAILY["光伏电量kWh"], color=C[1], linewidth=1.2, label="日光伏电量")
1237 ax_c.plot(dts, DAILY["净负荷电量kWh"], color=C[2], linewidth=1.2, linestyle="-. ",
1238         label="日净负荷电量  $\ell - v$ ")
1239 ax_c.axhline(0, color="#666666", linewidth=0.9, linestyle=":")
1240 ax_c.set_ylabel("电量 (kWh/日)", fontsize=9.5)
1241 ax_c.set_xlabel("日期 (2025-01-01 — 2025-12-31)", fontsize=9.5)
1242 ax_c.grid(linestyle=":", alpha=0.35)
1243 ax_c.set_axisbelow(True)
1244 ax_c.tick_params(axis="x", labelsiz=8.0)
1245 ax_c.legend(fontsize=8.0, loc="upper right", framealpha=0.95, ncol=1)
1246 note_box(ax_c,
1247         f"年合计: 负载 {E_LOAD:,.0f} kWh; 光伏 {E_PV:,.0f} kWh; "
1248         f"净负荷 {E_NET:,.0f} kWh\n"
1249         f"光伏 / 负载电量比 (逐日) {RATIO_MIN:.3f}—{RATIO_MAX:.3f}, "
1250         f"年均 {RATIO_YEAR:.3f} (= 50.0%) \n"
1251         f"无一日超过 1 => 不存在净上网, 与『不允许售电』一致",
1252         x=0.985, y=0.985, fs=7.8, ha="right", va="top")
1253
1254 # ---- (d1) 光伏负载比 ----
1255 panel_title(ax_d1, "(d1) 派生比值特征: 逐日光伏 / 负载电量比", fs=10.4)
1256 ax_d1.plot(dts, DAILY["光伏负载比"], color=C[4], linewidth=1.2)
1257 ax_d1.axhline(RATIO_YEAR, color="#222222", linewidth=1.1, linestyle="--",
1258         label=f"年均 {RATIO_YEAR:.3f}")
1259 ax_d1.axhline(1.0, color=C[3], linewidth=1.1, linestyle=":",
1260         label="上限 1.0 (售电边界)")

```

```

1258 ax_d1.set_ylim(0.2, 1.08)
1259 ax_d1.set_ylabel("比值", fontsize=9.0)
1260 ax_d1.grid(linestyle=":", alpha=0.35)
1261 ax_d1.set_axisbelow(True)
1262 ax_d1.tick_params(axis="x", labelsiz=7.4)
1263 ax_d1.legend(fontsize=7.6, loc="lower left", framealpha=0.95, ncol=2)
1264 ax_d1.text(0.985, 0.18, f"区间 {RATIO_MIN:.3f}—{RATIO_MAX:.3f} (max < 1)",
1265           transform=ax_d1.transAxes, fontsize=7.8, ha="right", va="center",
1266           color="#8a2c1a")
1267
1268 # ---- (d2) 功率波动 ----
1269 panel_title(ax_d2, "(d2) 派生波动特征: 净负荷 10 分钟平均绝对波动", fs=10.4)
1270 ax_d2.plot(dts, RAMP_DAILY, color=C[5], linewidth=1.0)
1271 ax_d2.axhline(RAMP_MEAN, color="#222222", linewidth=1.1, linestyle="--",
1272              label=f"全样本均值 {RAMP_MEAN:,.1f} kW/(10min)")
1273 ax_d2.set_ylabel("kW/(10min)", fontsize=9.0)
1274 ax_d2.grid(linestyle=":", alpha=0.35)
1275 ax_d2.set_axisbelow(True)
1276 ax_d2.tick_params(axis="x", labelsiz=7.4)
1277 ax_d2.legend(fontsize=7.6, loc="upper left", framealpha=0.95)
1278 ax_d2.text(0.985, 0.30, f"逐日均值 {RAMP_DAILY.min():,.1f}—{RAMP_DAILY.max():,.1f}",
1279           transform=ax_d2.transAxes, fontsize=7.8, ha="right", va="center",
1280           color="#8a2c1a")
1281
1282 return save(fig, "图3-6_特征构造与派生变量")
1283
1284
1285 # =====
1286 # 图3-7 相关性与分组诊断 (环节 8、9)
1287 # =====
1288 def figure_3_7():
1289     fig = plt.figure(figsize=(15.0, 10.2))
1290     gs = fig.add_gridspec(2, 3, height_ratios=[1.02, 0.98],
1291                          left=0.048, right=0.985, top=0.885, bottom=0.135,
1292                          wspace=0.24, hspace=0.36)
1293     ax_corr = fig.add_subplot(gs[0, 0])
1294     ax_typ = fig.add_subplot(gs[0, 1:])
1295     ax_q = fig.add_subplot(gs[1, 0])
1296     ax_w = fig.add_subplot(gs[1, 1])
1297     ax_s = fig.add_subplot(gs[1, 2])
1298
1299     fig.suptitle("图3-7 相关性与时序特征分析、数据分组诊断 (预处理环节 8—9)",
1300                 fontsize=15, fontweight="bold", y=0.972)
1301
1302     # ---- (a) 相关系数矩阵 ----
1303     slot_axis = np.arange(1, NSLOT + 1)
1304     panel_title(ax_corr, "(a) 变量相关性 (Pearson, n = 52,560)", fs=10.8)
1305     M = np.vstack([LD.ravel(), PV.ravel(), NL.ravel(), PR.ravel()])
1306     names = ["负载 l", "光伏 v", "净负荷\nl - v", "电价"]
1307     R = np.corrcoef(M)
1308     im = ax_corr.imshow(R, cmap="coolwarm", vmin=-1, vmax=1)

```

```

1309 ax_corr.set_xticks(range(4))
1310 ax_corr.set_yticks(range(4))
1311 ax_corr.set_xticklabels(names, fontsize=8.6)
1312 ax_corr.set_yticklabels(names, fontsize=8.6)
1313 for i in range(4):
1314     for j in range(4):
1315         ax_corr.text(j, i, f"{R[i, j]:.3f}", ha="center", va="center", fontsize=9.4,
1316                      color=("white" if abs(R[i, j]) > 0.62 else "#111111"),
1317                      fontweight="bold" if i != j else "normal")
1318 cb = fig.colorbar(im, ax=ax_corr, fraction=0.046, pad=0.04)
1319 cb.ax.tick_params(labelsize=8.0)
1320 cb.set_label("相关系数", fontsize=8.6)
1321 note_box(ax_corr,
1322          f"负载与净负荷强正相关 ({R[0, 2]:.3f}); \n"
1323          f"光伏与净负荷强负相关 ({R[1, 2]:.3f}); \n"
1324          f"电价与功率变量相关性弱 ( $|r| \leq \{\max(\text{abs}(R[3, 0]), \text{abs}(R[3, 1]), \text{abs}(R[3, 2]))$ 
1325          :.3f})",
1326          x=0.5, y=-0.155, fs=7.8, ha="center", va="top")
1327
1328 # ---- (b) 典型日日内曲线 ----
1329 panel_title(ax_typ, "(b) 典型日日内曲线: 四季代表日 (该季日光伏电量中位日)", fs=10.8)
1330 q_of_day = ((DAILY["日期"].dt.month - 1) // 3).values
1331 pv_e = DAILY["光伏电量kWh"].values
1332 rep_idx = []
1333 for q in range(4):
1334     idx = np.where(q_of_day == q)[0]
1335     med = np.median(pv_e[idx])
1336     rep_idx.append(idx[np.argmin(np.abs(pv_e[idx] - med))])
1337 season_lab = ["Q1 (1—3月)", "Q2 (4—6月)", "Q3 (7—9月)", "Q4 (10—12月)"]
1338 for q, di in enumerate(rep_idx):
1339     ax_typ.plot(slot_axis, LD[di], color=C[q], linewidth=1.9)
1340     ax_typ.plot(slot_axis, PV[di], color=C[q], linewidth=1.7, linestyle="--")
1341     ax_typ.plot(slot_axis, NL[di], color=C[q], linewidth=1.3, linestyle=":")
1342 ax_typ.axhline(0, color="#666666", linewidth=0.9, linestyle=":")
1343 ax_typ.set_xlim(1, NSLOT)
1344 ax_typ.set_xticks([1, 25, 49, 73, 97, 121, 144])
1345 ax_typ.set_xticklabels(["1\n0:10", "25\n4:10", "49\n8:10", "73\n12:10",
1346                        "97\n16:10", "121\n20:10", "144\n0:00+1"], fontsize=7.8)
1347 ax_typ.set_xlabel("日内槽序号 (标签=槽开始时刻)", fontsize=9.5)
1348 ax_typ.set_ylabel("功率 (kW)", fontsize=9.5)
1349 ax_typ.grid(linestyle=":", alpha=0.35)
1350 ax_typ.set_axisbelow(True)
1351 h1 = [plt.Line2D([], [], color=C[q], linewidth=2.0) for q in range(4)]
1352 h2 = [plt.Line2D([], [], color="#444444", linewidth=1.9),
1353       plt.Line2D([], [], color="#444444", linewidth=1.7, linestyle="--"),
1354       plt.Line2D([], [], color="#444444", linewidth=1.3, linestyle=":")]
1355 lg1 = ax_typ.legend(h1, [f"${s} {str(pd.Timestamp(d).date())}" for s, d in
1356                        zip(season_lab, [DAILY['日期'].iloc[i] for i in rep_idx])],
1357                        fontsize=8.0, loc="upper left", framealpha=0.95, title="季节 (颜色)"
1358                        ")
1359 lg1.get_title().set_fontsize(8.0)

```

```

1358 ax_typ.add_artist(lg1)
1359 lg2 = ax_typ.legend(h2, ["负载  $\ell$ ", "光伏  $v$ ", "净负荷  $\ell - v$ "], fontsize=8.0,
1360                        loc="upper right", framealpha=0.95, title="变量（线型）")
1361 lg2.get_title().set_fontsize(8.0)
1362 neg_slots = int((NL[rep_idx] < 0).sum())
1363 note_box(ax_typ,
1364          f"四季代表日中净负荷为负（光伏盈余）的槽位合计 {neg_slots} 个；\n"
1365          f"日内形态：负载双峰（约 11:00、19:00），光伏单峰（约 12:00），"
1366          f"净负荷在午间被压低甚至转负",
1367          x=0.5, y=-0.135, fs=7.8, ha="center", va="top")
1368
1369 # ---- (c) 按季度分组箱线 ----
1370 panel_title(ax_q, "(c) 分组诊断：按季度分组的负载与光伏", fs=10.6)
1371 pos = np.arange(4) + 1
1372 bp1 = ax_q.boxplot([LD[q_of_day == q].ravel() for q in range(4)], positions=pos - 0.19,
1373                    widths=0.32, patch_artist=True, whis=(5, 95), showfliers=False,
1374                    medianprops=dict(color="#222222", linewidth=1.3))
1375 bp2 = ax_q.boxplot([PV[q_of_day == q].ravel() for q in range(4)], positions=pos + 0.19,
1376                    widths=0.32, patch_artist=True, whis=(5, 95), showfliers=False,
1377                    medianprops=dict(color="#222222", linewidth=1.3))
1378 for q in range(4):
1379     bp1["boxes"][q].set(facecolor=C[0], alpha=0.45, edgecolor=C[0])
1380     bp2["boxes"][q].set(facecolor=C[1], alpha=0.45, edgecolor=C[1])
1381 for bp in (bp1, bp2):
1382     for w in ("whiskers", "caps"):
1383         for art in bp[w]:
1384             art.set(color="#666666", linewidth=0.9)
1385 ax_q.set_xticks(pos)
1386 ax_q.set_xticklabels([f"Q{q + 1}\n{(q * 3 + 1)}—{(q * 3 + 3)}月" for q in range(4)],
1387                      fontsize=8.2)
1388 ax_q.set_ylabel("功率 (kW)", fontsize=9.2)
1389 ax_q.grid(axis="y", linestyle=":", alpha=0.35)
1390 ax_q.set_axisbelow(True)
1391 ax_q.plot([], [], marker="s", markersize=8, color=C[0], alpha=0.55, linestyle="none",
1392           label="负载  $\ell$ ")
1393 ax_q.plot([], [], marker="s", markersize=8, color=C[1], alpha=0.55, linestyle="none",
1394           label="光伏  $v$ ")
1395 ax_q.legend(fontsize=8.0, loc="upper left", framealpha=0.95)
1396 q_ld = [LD[q_of_day == q].mean() for q in range(4)]
1397 q_pv = [PV[q_of_day == q].mean() for q in range(4)]
1398 fig_note(fig, ax_q,
1399          f"季均负载 {min(q_ld):,.0f}—{max(q_ld):,.0f} kW（极差 {max(q_ld) - min(q_ld):,.0f}）\n"
1400          f"季均光伏 {min(q_pv):,.0f}—{max(q_pv):,.0f} kW（极差 {max(q_pv) - min(q_pv):,.0f}）\n"
1401          "季间差异主要体现在光伏",
1402          dy=0.042, fs=7.6)
1403
1404 # ---- (d) 工作日 / 周末 ----
1405 panel_title(ax_w, "(d) 分组诊断：工作日 vs 周末", fs=10.6)
1406 is_wd = DAILY["日期"].dt.dayofweek.values < 5

```

```

1406 g_ld = [LD[is_wd].mean(), LD[~is_wd].mean()]
1407 g_pv = [PV[is_wd].mean(), PV[~is_wd].mean()]
1408 xg = np.arange(2)
1409 b1 = ax_w.bar(xg - 0.19, g_ld, width=0.36, color=C[0], alpha=0.88, label="日均负载功率")
1410 b2 = ax_w.bar(xg + 0.19, g_pv, width=0.36, color=C[1], alpha=0.88, label="日均光伏功率")
1411 for bs in (b1, b2):
1412     for r in bs:
1413         ax_w.text(r.get_x() + r.get_width() / 2, r.get_height() + 90,
1414                 f"{r.get_height():.0f}", ha="center", va="bottom", fontsize=8.2)
1415 ax_w.set_xticks(xg)
1416 ax_w.set_xticklabels([f"工作日\n({int(is_wd.sum())} 日)", f"周末\n({int(~is_wd.sum())} 日)"],
1417                     fontsize=8.6)
1418 ax_w.set_ylabel("平均功率 (kW)", fontsize=9.2)
1419 ax_w.set_ylim(0, max(g_ld + g_pv) * 1.30)
1420 ax_w.grid(axis="y", linestyle=":", alpha=0.35)
1421 ax_w.set_axisbelow(True)
1422 ax_w.legend(fontsize=8.0, loc="upper right", framealpha=0.95)
1423 e_ld = [DAILY["负载电量kWh"].values[is_wd].mean(), DAILY["负载电量kWh"].values[~is_wd].mean()]
1424 p_avg = [PR[is_wd].mean(), PR[~is_wd].mean()]
1425 fig_note(fig, ax_w,
1426         f"日均负载电量 {e_ld[0]:.0f} vs {e_ld[1]:.0f} kWh/日\n",
1427         f"(周末 - 工作日 = {e_ld[1] - e_ld[0]:+.0f} kWh/日, ",
1428         f"{(e_ld[1] / e_ld[0] - 1) * 100:+.2f}%)\n",
1429         f"平均电价 {p_avg[0]:.4f} vs {p_avg[1]:.4f} 元/kWh",
1430         dy=0.042, fs=7.6)
1431
1432 # ---- (e) 净负荷正/负运行状态分组 ----
1433 panel_title(ax_s, "(e) 分组诊断: 按净负荷正 / 负运行状态", fs=10.6)
1434 share = [NL_NEG_FRAC * 100, (1 - NL_NEG_FRAC) * 100]
1435 bars = ax_s.bar([0, 1], share, width=0.55, color=[C[2], C[3]], alpha=0.88)
1436 for r, s, n in zip(bars, share, [int((NL < 0).sum()), int((NL > 0).sum())]):
1437     ax_s.text(r.get_x() + r.get_width() / 2, s + 2.0, f"{s:.2f}%\n({n:,} 时段)",
1438             ha="center", va="bottom", fontsize=9.0, fontweight="bold")
1439 ax_s.set_xticks([0, 1])
1440 ax_s.set_xticklabels(["净负荷 < 0\n光伏过剩 (充电 / 弃光)",
1441                     "净负荷 > 0\n需外部供电 (购电)"], fontsize=8.4)
1442 ax_s.set_ylabel("占全部 52,560 时段的比例 (%)", fontsize=9.2)
1443 ax_s.set_ylim(0, 118)
1444 ax_s.grid(axis="y", linestyle=":", alpha=0.35)
1445 ax_s.set_axisbelow(True)
1446 fig_note(fig, ax_s,
1447         f"过剩状态: 均值 {NL_NEG_MEAN:,.1f} kW, 最大过剩 {NL_MIN:,.1f} kW\n",
1448         f"超过 -5,000 kW (单槽可充上限) 的时段 {NL_NEG_OVER5} 个\n",
1449         f"缺额状态: 均值 {NL_POS_MEAN:,.1f} kW, 最大缺额 {NL_MAX:,.1f} kW\n",
1450         f"超过 +5,000 kW 的时段 {NL_POS_OVER5:,} 个\n",
1451         "=> 两类状态均需储能之外的调节手段",
1452         dy=0.045, fs=7.5)
1453
1454 return save(fig, "图3-7_相关性与分组诊断")

```

```

1455
1456
1457 # =====
1458 # 执行
1459 # =====
1460 FIG_NAMES = [
1461     ("图3-1_时间对齐与数据结构", figure_3_1),
1462     ("图3-2_质量检验与缺失识别", figure_3_2),
1463     ("图3-3_异常辨识与判据修正", figure_3_3),
1464     ("图3-4_异常处理与极端值保留", figure_3_4),
1465     ("图3-5_变量分布分析", figure_3_5),
1466     ("图3-6_特征构造与派生变量", figure_3_6),
1467     ("图3-7_相关性与分组诊断", figure_3_7),
1468 ]
1469
1470 for _name, _fn in FIG_NAMES:
1471     _fn()
1472
1473 # =====
1474 # 自检
1475 # =====
1476 print("=" * 78)
1477 print("自检（数据核对 + 图件产出）")
1478 print("=" * 78)
1479
1480 checks = []
1481
1482
1483 def chk(label, ok, detail=""):
1484     checks.append((label, bool(ok), detail))
1485     print(f"[{'PASS' if ok else 'FAIL'}] {label}{' — ' + detail if detail else ''}")
1486
1487
1488 # 1) 图件字节数 + 非空白
1489 try:
1490     from PIL import Image
1491
1492     _HAS_PIL = True
1493 except ImportError: # pragma: no cover
1494     _HAS_PIL = False
1495
1496 for name, _ in FIG_NAMES:
1497     paths = SAVED.get(name, [])
1498     png = next((p for p in paths if p.suffix == ".png"), None)
1499     svg = next((p for p in paths if p.suffix == ".svg"), None)
1500     sz = png.stat().st_size if png and png.exists() else 0
1501     ssz = svg.stat().st_size if svg and svg.exists() else 0
1502     chk(f"{name}.png > 20000 字节", png is not None and sz > 20000, f"{sz:,} 字节")
1503     chk(f"{name}.svg 已生成且 > 20000 字节", svg is not None and ssz > 20000, f"{ssz:,} 字节")
1504     if _HAS_PIL and png is not None and png.exists():

```

```

1505     with Image.open(png) as im:
1506         w, h = im.size
1507         arr = np.asarray(im.convert("L"))
1508         frac = float((arr < 245).mean())
1509         chk(f"{name}.png 非空白且分辨率达标 (非白像素 > 5%, 宽 ≥ 2000 px) ",
1510             frac > 0.05 and w >= 2000, f"{w}×{h} px; 非白 {frac:.1%}")
1511
1512 # 2) 缺失 = 0
1513 chk("四附件缺失记录 = 0 已核对", MISS_TOTAL == 0,
1514     "; ".join(f"{k}:{v}" for k, v in MISS.items()))
1515 # 3) 52,560 观测单元
1516 chk("观测单元 52,560 = 365 × 144 已核对", NCELL == 52560 and NDAY == 365 and NSLOT == 144,
1517     f"{NDAY} × {NSLOT} = {NCELL:,}")
1518 chk("附件2/4 三张矩阵均为 52,560 个数值单元",
1519     LD.size == PV.size == PR.size == 52560, f"{LD.size:,} / {PV.size:,} / {PR.size:,}")
1520
1521 # 4) 缺失/重复/负值
1522 chk("重复记录 = 0 已核对", sum(DUP.values()) == 0, "; ".join(f"{k}:{v}" for k, v in DUP.items
1523     ()))
1524 chk("负值记录 = 0 已核对", sum(NEG.values()) == 0, "; ".join(f"{k}:{v}" for k, v in NEG.items
1525     ()))
1526
1527 # 5) 附件3 块继承
1528 chk("附件3 日期列空白 1,095 行按块继承后成键 1,460 个唯一键",
1529     a3_blank == 1095 and len(A3KEY) == 1460 and len(set(A3KEY)) == 1460,
1530     f"空白 {a3_blank}; 唯一键 {len(set(A3KEY))}/{len(A3KEY)}")
1531 chk("附件3 预报矩阵 35,040 个值 = 1,460 × 24", A3VAL.size == 35040, f"{A3VAL.size:,}")
1532
1533 # 6) 混合单元格类型
1534 chk("附件1 时间列混合类型 60 时间型 + 84 文本型",
1535     t1_types.get("time") == 60 and t1_types.get("str") == 84, str(t1_types))
1536 chk("附件2/4 表头 143 时间型 + 1 文本型 (0:00+1 → 1440) ",
1537     hdr2_types.get("time") == 143 and hdr2_types.get("str") == 1
1538     and hdr4_types.get("time") == 143 and hdr4_types.get("str") == 1
1539     and lab2[-1] == 1440 and lab4[-1] == 1440,
1540     f"附件2 {hdr2_types}; 附件4 {hdr4_types}; 未列归一化 {lab2[-1]}/{lab4[-1]} 分钟")
1541
1542 # 7) 零膨胀与判据
1543 chk("光伏 ≤1 kW 占 48.53% (25,509/52,560) ",
1544     PV_LE1 == 25509 and abs(FRAC_LE1 - 0.4853) < 5e-5, f"{PV_LE1:,} ({FRAC_LE1:.4%}) ")
1545 chk("全样本 MAD 判据误标 25,785 (49.06%) ",
1546     N_CAND_FULL == 25785 and abs(FRAC_CAND_FULL - 0.4906) < 5e-5,
1547     f"{N_CAND_FULL:,} ({FRAC_CAND_FULL:.4%}) ")
1548 chk("白天子样本 n=27,051, 修正判据候选异常 0 个",
1549     DAY_N == 27051 and N_CAND_DAY == 0, f"n={DAY_N:,}; 候选异常 {N_CAND_DAY}")
1550 chk("夜间 (20:00—次日 04:00, 49 槽) 光伏 >1 kW 的样本 0 个",
1551     NIGHT_SLOTS == 49 and NIGHT_PV_GT1 == 0, f"夜间槽 {NIGHT_SLOTS}; 样本 {NIGHT_PV_GT1}")
1552
1553 # 8) 极端电价
1554 chk("附件4 电价 <0.05 共 9 个 (最低 0.0076)、>1.5 共 927 个 (最高 1.7936) ",
1555     PR_LOW == 9 and abs(PR_LOW_MIN - 0.0076) < 5e-5 and PR_HIGH == 927

```

```

1554     and abs(PR_HIGH_MAX - 1.7936) < 5e-5,
1555     f"<0.05: {PR_LOW} ({PR_LOW_MIN:.4f}); >1.5: {PR_HIGH:,} ({PR_HIGH_MAX:.4f})")
1556
1557 # 9) 能量与净负荷
1558 chk("年负载电量 ≈ 40,524,055 kWh; 年光伏 ≈ 20,251,243 kWh; 比 ≈ 50.0%",
1559     abs(E_LOAD - 40524055) < 60 and abs(E_PV - 20251243) < 60 and abs(RATIO_YEAR - 0.50) <
1560     0.002,
1561     f"{E_LOAD:,.1f} / {E_PV:,.1f} kWh; 比 {RATIO_YEAR:.4f}")
1562 chk("净负荷年合计 ≈ 20,272,812 kWh; 负值时段占 19.27%; 最大过剩 -6,601.99 kW",
1563     abs(E_NET - 20272812) < 60 and abs(NL_NEG_FRAC - 0.1927) < 1e-4
1564     and abs(NL_MIN - (-6601.9911)) < 0.5,
1565     f"{E_NET:,.1f} kWh; {NL_NEG_FRAC:.4%}; {NL_MIN:,.2f} kW")
1566 chk("最大缺额 = +6,687.75 kW (实测; 任务说明写的 +6,668 kW 系数字换位, 已按实测更正)",
1567     abs(NL_MAX - 6687.7481) < 0.5, f"实测 {NL_MAX:,.4f} kW (≈ +6,688 kW)")
1568 chk("逐日光伏 / 负载电量比 0.271—0.999 (无一日超过 1)",
1569     abs(RATIO_MIN - 0.2707) < 5e-4 and abs(RATIO_MAX - 0.999) < 5e-4,
1570     f"{RATIO_MIN:.4f}—{RATIO_MAX:.4f}")
1571
1572 # 9b) 附件1 / 附件2 / 附件3 的补充实测事实
1573 chk("附件1 实测范围: 电价 0.3713—1.3952; 负载 3309.39—5958.97 kW; 光伏 0—7612.32 kW",
1574     abs(PRICE1_MIN - 0.3713) < 5e-5 and abs(PRICE1_MAX - 1.3952) < 5e-5
1575     and abs(LD1_MIN - 3309.3934) < 0.01 and abs(LD1_MAX - 5958.9696) < 0.01
1576     and abs(PV1_MAX - 7612.316) < 0.01,
1577     f"{PRICE1_MIN:.4f}—{PRICE1_MAX:.4f}; {LD1_MIN:,.2f}—{LD1_MAX:,.2f}; 0—{PV1_MAX:,.2f}")
1578
1579 chk("附件1 光伏非零 89 列, 标签区间 04:40—19:20",
1580     PV1_NZ == 89 and PV1_NZ_RANGE == "04:40—19:20", f"{PV1_NZ} 列; {PV1_NZ_RANGE}")
1581 chk("附件2 负载实测 1995.72—7978.88 kW; 光伏实际 0—10216.20 kW; 无全零光伏日",
1582     abs(LD.min() - 1995.7176) < 0.01 and abs(LD.max() - 7978.8849) < 0.01
1583     and abs(PV.max() - 10216.2) < 0.01 and PV_ALLZERO_DAYS == 0,
1584     f"{LD.min():,.2f}—{LD.max():,.2f}; 0—{PV.max():,.2f}; 全零日 {PV_ALLZERO_DAYS}")
1585 chk("附件3 预报峰值 9995.89 kW; 365 块 × 4 行严格; 无全零预报行",
1586     abs(A3_MAX - 9995.8875) < 0.01 and A3_ALLZERO_ROWS == 0
1587     and all(len(b) == 4 for b in [a3_body[i:i + 4] for i in range(0, len(a3_body), 4)]),
1588     f"峰值 {A3_MAX:,.4f} kW; 全零行 {A3_ALLZERO_ROWS}")
1589 chk("零膨胀时段结构: 全年最大出力 ≤1 kW 的时段 55 个 (00:10—04:30 与 19:30—24:00)",
1590     NEARZERO_SLOTS == 55 and NEARZERO_RANGE_TXT == "00:10—04:30 与 19:30—24:00"
1591     and PV_EXACT_ZERO == 23540,
1592     f"{NEARZERO_SLOTS} 个 ({NEARZERO_RANGE_TXT}); 精确 0 值 {PV_EXACT_ZERO:,} 个")
1593
1594 # 10) 声明性核对: 未做插补/删除
1595
1596 chk("声明核对: 未插补、未删除、未重标注任何样本 (全部 52,560×3 + 35,040 + 144 个数值原样入模)",
1597     True, "缺失 0 故无需插补; 异常 0 故无需删除; 极端电价保留")
1598
1599 # 11) 图件无缺字 (中文字体字形缺失)
1600
1601 chk("图件无缺字 (无 missing-glyph 警告)", len(MISSING_GLYPHS) == 0,
1602     "; ".join(sorted(MISSING_GLYPHS)) if MISSING_GLYPHS else "7 张图全部无缺字")
1603
1604 # 12) 程序化版面自检 (visual_qa.audit_layout + 子图/说明框重叠检测)
1605
1606 _layout_fail = [it for it in LAYOUT_ISSUES if it[1] == "FAIL"]
1607 _layout_warn = [it for it in LAYOUT_ISSUES if it[1] == "WARN"]

```

```

1603 chk("版面自检无 FAIL (无子图重叠、无说明框压图、无说明框互相重叠)",
1604     len(_layout_fail) == 0,
1605     "; ".join(f"{n}:{m[:40]}" for n, _, m in _layout_fail) if _layout_fail else "无 FAIL")
1606 print(f"[INFO] 版面自检 WARN {len(_layout_warn)} 项 (裁切类 WARN 已由 bbox_inches='tight' 兜底)")
1607 for _n, _s, _m in _layout_warn:
1608     print(f"        - {_n}: {_m[:110]}")
1609
1610 n_fail = sum(1 for _, ok, _ in checks if not ok)
1611 print("-" * 78)
1612 print("【事实核对差异说明】(读数据时发现的、与任务说明不一致之处)")
1613 print(f"  1) 任务说明写『最大缺额 +6,668 kW』, 实测附件2 净负荷最大正值 = {NL_MAX:,.4f} kW"
1614       f" (≈ +6,688 kW); ")
1615 print(f"        与《数据审计_补充核查.txt》§E 的 6,687.75 kW 一致 => 按实测 +6,687.75 kW 使用。"
1616       f"最大过剩实测 {NL_MIN:,.4f} kW, 与说明 -6,602 kW 一致。")
1617 print(f"  2) 任务说明写『0 值集中在 22:00—05:00』, 实测更精确的表述为: 全年最大出力 ≤1 kW 的时段"
1618       f"共 {NEARZERO_SLOTS} 个, ")
1619 print(f"        为 {NEARZERO_RANGE_TXT} 两段 (合 {PV_EXACT_ZERO:,} 个精确 0 值); "
1620       f"任务说明所指的 20:00—04:00 共 49 槽内光伏 >1 kW 的样本确为 0 个。")
1621 print(f"  3) 任务说明称 {E_PV / E_LOAD:.1%} 为『自给率』; 本项目《数据处理记录.md》§3 明确"
1622       f"『年发电量约为全年负载的 50%, ")
1623 print(f"        不等于实际自给率 (还受时序错配、弃光与充放电损耗影响)』=> 图件中一律标注为"
1624       f"『光伏 / 负载电量比』, 不写自给率。")
1625 print("-" * 78)
1626 print(f"自检合计: {len(checks)} 项, PASS {len(checks) - n_fail} 项, FAIL {n_fail} 项")
1627 print(f"图件目录: {OUTDIR}")
1628 print(f"日志文件: {LOG}")
1629 print(f"脚本: {Path(__file__).resolve()}")
1630 print(f"最终结果: " + ("ALL PASS OK 退出码 0" if n_fail == 0 else f"存在 {n_fail} 项 FAIL x 退出码 1"))
1631 print("=" * 78)
1632
1633 sys.stdout.flush()
1634 _LOG_FH.flush()
1635 _LOG_FH.close()
1636 sys.stdout = _ORIG_STDOUT
1637 sys.exit(0 if n_fail == 0 else 1)

```

C2. 问题 1/绘图 _ 问题 1 求解.py (958 行)

问题 1 求解类图件绘制 (模型结构、典型日输入与购电计划、储能与 SOC、表 2 口径对照、基线阶梯、净负荷与储能作用)

```

1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3 """2026 高教社杯 C 题《微网与外部电网电力调控策略》• 问题1 求解可视化集。
4
5 产出 6 张图 (PNG 300 dpi + SVG) 到
6     04_结果与检验/可视化集/01_问题1求解/
7 运行日志写入
8     04_结果与检验/检验日志/可视化_问题1求解.txt

```

```

9
10 本脚本:
11     • 用项目自带模型 (03_代码/最小运行/模型.py, 方案丙口径) 重解问题1, 保证逐槽量自治;
12     • 与 04_结果与检验/results/result1.xlsx 逐项对照;
13     • 只读既有文件, 不修改/覆盖任何既有文件;
14     • 结尾打印 PASS/FAIL 自检结论, 全部通过时退出码 0。
15 """
16 from __future__ import annotations
17
18 import sys
19 import traceback
20 from pathlib import Path
21
22 sys.stdout.reconfigure(encoding="utf-8", errors="replace")
23
24 ROOT = Path(r"C:\Users\John\Desktop\26题")
25 CODE_DIR = ROOT / "03_代码" / "最小运行"
26 OUT_DIR = ROOT / "04_结果与检验" / "可视化集" / "01_问题1_求解"
27 LOG_DIR = ROOT / "04_结果与检验" / "检验日志"
28 LOG_PATH = LOG_DIR / "可视化_问题1求解.txt"
29 ATT1 = ROOT / "00_题目与原始数据" / "C题" / "附件" / "附件1.xlsx"
30 RESULT1 = ROOT / "04_结果与检验" / "results" / "result1.xlsx"
31 SKILL_SCRIPTS = r"C:\Users\John\.codex\skills\math-modeling\tools\figure\scripts"
32
33 # ----- 期望值 (题面给定)
34 EXP_OBJECTIVE = 35126.84858928963          # 元
35 EXP_TOTAL_BUY = 59482.6989983539          # kWh
36 EXP_TOTAL_C = 20740.6661                  # kWh
37 EXP_TOTAL_D = 16799.9396                  # kWh
38 EXP_SEG_ORIG_C = [4166.6667, 833.3333, 4787.9643, 5286.0352, 0.0, 5666.6667]
39 EXP_SEG_ORIG_D = [0.0, 6365.8412, 1702.9970, 91.1014, 5780.1319, 2859.8681]
40 EXP_SEG_CLOCK_C = [4500.0000, 833.3333, 3954.6310, 6119.3685, 0.0, 5333.3333]
41 EXP_SEG_CLOCK_D = [0.0, 5947.4198, 2121.4184, 91.1014, 5068.6869, 3571.3131]
42 EXP_TABLE1 = [0.0, 486.4029, 0.0, 394.9315, 636.9826, 0.0]
43 EXP_TABLE1_SLOTS = [60, 72, 84, 96, 108, 120]
44 EXP_STRICT_OBJECTIVE = 35126.948589290     # 严格口径 E0=E143=E144=6000
45 EXP_B0 = 48052.05                          # 元 (无储能基线)
46 EXP_B0_CURTAIL = 6247.963                  # kWh (B0 基线弃光, 非最优解弃光)
47 EXP_B1 = 45719.85                          # 元 (贪心谷充峰放规则基线, 末态 10800)
48 EXP_B1_SOC_END = 10800.0
49 EXP_E0 = 6300.0
50 EXP_E143 = 6000.0
51 EXP_PRICE_RANGE = (0.3713, 1.3952)
52 EXP_LOAD_RANGE = (3309.4, 5959.0)
53 EXP_PV_RANGE = (0.0, 7612.3)
54
55 SEG_LABELS = ["0:00-4:00", "4:00-8:00", "8:00-12:00",
56              "12:00-16:00", "16:00-20:00", "20:00-24:00"]
57 SEG_ORIG_IDX = [(0, 24), (24, 48), (48, 72), (72, 96), (96, 120), (120, 144)]
58 SEG_CLOCK_IDX = [[143] + list(range(0, 23)), list(range(23, 47)), list(range(47, 71)),
59                  list(range(71, 95)), list(range(95, 119)), list(range(119, 143))]

```

```

60
61 CHECKS: list[tuple[str, bool, str]] = []
62 FIG_FILES: list[Path] = []
63
64
65 def record(name: str, ok: bool, detail: str = "") -> bool:
66     CHECKS.append((name, bool(ok), detail))
67     print(f" [{ 'PASS' if ok else 'FAIL' }] {name}" + (f" — {detail}" if detail else ""))
68     return bool(ok)
69
70
71 class Tee:
72     """同时写 stdout 与日志文件。"""
73
74     def __init__(self, path: Path):
75         path.parent.mkdir(parents=True, exist_ok=True)
76         self._f = open(path, "w", encoding="utf-8")
77
78     def write(self, s: str) -> int:
79         sys.__stdout__.write(s)
80         self._f.write(s)
81         return len(s)
82
83     def flush(self) -> None:
84         sys.__stdout__.flush()
85         self._f.flush()
86
87     def close(self) -> None:
88         self._f.flush()
89         self._f.close()
90
91
92 # ----- 数据与模型
93 def read_attachment1():
94     import numpy as np
95     from openpyxl import load_workbook
96
97     ws = load_workbook(ATT1, data_only=True).worksheets[0]
98     rows = [[c.value for c in r] for r in ws.iter_rows(min_row=2)]
99     rows = [r for r in rows if r[0] is not None]
100     label_rows = [str(r[0])[:8] for r in rows]
101     price = np.array([float(r[1]) for r in rows])
102     load_kw = np.array([float(r[2]) for r in rows])
103     pv_kw = np.array([float(r[3]) for r in rows])
104     return label_rows, price, load_kw, pv_kw
105
106
107 def read_result1():
108     from openpyxl import load_workbook
109
110     wb = load_workbook(RESULT1, data_only=True)

```

```

111     plan = [r[1] for r in wb["计划购电量"].iter_rows(min_row=2, values_only=True)]
112     plan = [float(v) for v in plan if v is not None]
113     cd = [r for r in wb["充放电电量"].iter_rows(min_row=2, values_only=True)]
114     return plan, cd
115
116
117 # ----- 坐标与刻度工具
118 def hhmm(minutes: float) -> str:
119     m = int(round(minutes)) % 1440
120     return f"{m // 60}:{m % 60:02d}"
121
122
123 def clock_start_minutes(t: int) -> int:
124     """槽索引 t (0 基, 对应 1 基槽号 t+1) 在时钟轴上的起始分钟 (周期化到 0:00—24:00)。"""
125     return (10 * (t + 1)) % 1440
126
127
128 def clock_order():
129     """按时钟顺序排列的槽索引: 槽144 (0:00—0:10) 在最前, 其后 槽1…槽143。"""
130     return [143] + list(range(143))
131
132
133 def periodic(series):
134     """把长度 144 的逐槽量按时钟顺序 (x = 0,10,…,1430 min) 重排。"""
135     import numpy as np
136
137     return np.asarray(series, dtype=float)[clock_order()]
138
139
140 def soc_curve(E_all):
141     """周期化 SOC 曲线: 0:00—24:00 共 145 点。"""
142     import numpy as np
143
144     E = np.asarray(E_all, dtype=float)
145     return np.arange(145) * 10.0, np.concatenate([[E[143]], E[0:144]])
146
147
148 def set_time_axis(ax, step_hours: int = 2, label: str = "时刻"):
149     ax.set_xlim(0, 1440)
150     ticks = list(range(0, 1441, step_hours * 60))
151     ax.set_xticks(ticks)
152     ax.set_xticklabels([hhmm(t) for t in ticks], rotation=0)
153     ax.set_xlabel(label)
154
155
156 def seg_sum(arr, idx):
157     return float(sum(float(arr[i]) for i in idx))
158
159
160 def layout_audit(fig, tol_px: float = 6.0):
161     """图内纯文字框的 (重叠对, 越界项) 自检; 不含标注箭头包围盒, 避免误报。"""

```

```

162 import matplotlib.text as mtext
163
164 fig.canvas.draw()
165 renderer = fig.canvas.get_renderer()
166 tick_ids = set()
167 for ax in fig.axes:
168     for tl in (*ax.get_xticklabels(), *ax.get_yticklabels(),
169               *ax.get_xticklabels(minor=True), *ax.get_yticklabels(minor=True)):
170         tick_ids.add(id(tl))
171 items = []
172 for t in fig.findobj(mtext.Text):
173     if id(t) in tick_ids:
174         continue
175     if not t.get_visible() or not t.get_text().strip():
176         continue
177     items.append((t.get_text().strip().replace("\n", " ")[:40],
178                  mtext.Text.get_window_extent(t, renderer)))
179 W, H = float(fig.bbox.width), float(fig.bbox.height)
180 clipped = [nm for nm, bb in items
181             if bb.x0 < -2 or bb.y0 < -2 or bb.x1 > W + 2 or bb.y1 > H + 2]
182 overlaps_found = []
183 for i in range(len(items)):
184     for j in range(i + 1, len(items)):
185         A, B = items[i][1], items[j][1]
186         ox = min(A.x1, B.x1) - max(A.x0, B.x0)
187         oy = min(A.y1, B.y1) - max(A.y0, B.y0)
188         if ox > tol_px and oy > tol_px:
189             overlaps_found.append((items[i][0], items[j][0]))
190 return overlaps_found, clipped
191
192
193 # ----- 绘图
194 def fig_model_structure(q1, params: dict, colors):
195     """图1-1 模型结构示意图：三要素（决策变量 / 目标 / 约束）+ 能量流。"""
196     import matplotlib.pyplot as plt
197     from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Rectangle
198
199     fig = plt.figure(figsize=(11.5, 7.9))
200     fig.set_layout_engine("none")
201     ax = fig.add_axes([0, 0, 1, 1])
202     ax.set_xlim(0, 1)
203     ax.set_ylim(0, 1)
204     ax.axis("off")
205
206     def panel(x0, y0, x1, y1, title, face):
207         ax.add_patch(Rectangle((x0, y0), x1 - x0, y1 - y0, transform=ax.transAxes,
208                                facecolor=face, edgecolor="#9aa0a6", lw=1.0, zorder=0))
209         ax.text((x0 + x1) / 2, y1 - 0.018, title, transform=ax.transAxes,
210                ha="center", va="top", fontsize=10.5, fontweight="bold", zorder=5)
211     return y1 - 0.052
212

```

```

213 def node(cx, cy, w, h, text, face, edge, fs=9.0):
214     ax.add_patch(FancyBboxPatch((cx - w / 2, cy - h / 2), w, h,
215                               boxstyle="round,pad=0.006,rounding_size=0.012",
216                               transform=ax.transAxes, facecolor=face,
217                               edgecolor=edge, lw=1.2, zorder=3))
218     ax.text(cx, cy, text, transform=ax.transAxes, ha="center", va="center",
219            fontsize=fs, zorder=4)
220
221 def arrow(p, q, color, lw=1.5, style="-|>", ms=11, rad=0.0):
222     ax.add_patch(FancyArrowPatch(p, q, transform=ax.transAxes, arrowstyle=style,
223                                mutation_scale=ms, lw=lw, color=color,
224                                shrinkA=1.0, shrinkB=1.0, zorder=2,
225                                connectionstyle=f"arc3,rad={rad}"))
226
227 c_buy, c_ld, c_pv, c_dis, c_w, c_e = (colors[0], colors[1], colors[2],
228                                       colors[3], colors[4], colors[8])
229
230 # ===== (a) 能量流与决策变量 =====
231 ax.text(0.5, 0.985, "图 1-1 (a) 问题 1 模型的能量流与决策变量 (单位: kWh / 元)",
232        transform=ax.transAxes, ha="center", va="top", fontsize=11, fontweight="bold")
233
234 bus_y = 0.828
235 ax.plot([0.055, 0.945], [bus_y, bus_y], transform=ax.transAxes,
236        color="#37474f", lw=3.0, solid_capstyle="butt", zorder=1)
237 ax.text(0.508, bus_y + 0.012, "微网母线 (逐槽电量平衡节点)", transform=ax.transAxes,
238        ha="left", va="bottom", fontsize=8.5, color="#37474f")
239
240 node(0.165, 0.930, 0.15, 0.062, "外部电网", "#e8f0fe", c_buy)
241 node(0.415, 0.930, 0.15, 0.062, "光伏发电", "#e6f4ea", c_pv)
242 node(0.845, 0.930, 0.17, 0.062, "小区负载", "#fef7e0", c_ld)
243 node(0.285, 0.700, 0.19, 0.062, "弃光 $w_t$", "#f3e8fd", c_w)
244 node(0.615, 0.700, 0.24, 0.062, "储能设备", "#fce8e6", c_dis)
245
246 arrow((0.165, 0.898), (0.165, bus_y), c_buy)
247 arrow((0.415, 0.898), (0.415, bus_y), c_pv)
248 arrow((0.845, bus_y), (0.845, 0.898), c_ld, style="<|>")
249 arrow((0.285, bus_y), (0.285, 0.732), c_w)
250 arrow((0.585, bus_y), (0.585, 0.734), c_dis, style="-|>")
251 arrow((0.650, 0.734), (0.650, bus_y), c_dis, style="-|>")
252
253 ax.text(0.178, 0.862, "购电量 $a_t$", transform=ax.transAxes, ha="left", va="center",
254        fontsize=9, color=c_buy)
255 ax.text(0.428, 0.862, "可用光伏 $v_t=\\Delta t\\,PV_t$", transform=ax.transAxes,
256        ha="left", va="center", fontsize=9, color=c_pv)
257 ax.text(0.812, 0.862, "负荷 $\\ell_t=\\Delta t\\,L_t$", transform=ax.transAxes,
258        ha="right", va="center", fontsize=9, color=c_ld)
259 ax.text(0.300, 0.766, "削减光伏", transform=ax.transAxes, ha="left", va="center",
260        fontsize=8.5, color=c_w)
261 ax.text(0.576, 0.780, "充电 $C_t\\rightarrow$", transform=ax.transAxes, ha="right",
262        va="center", fontsize=9, color=c_dis)
263 ax.text(0.660, 0.780, "$\\rightarrow$放电 $D_t$", transform=ax.transAxes, ha="left",

```

```

264         va="center", fontsize=9, color=c_dis)
265 ax.text(0.5, 0.640, "逐槽平衡:  $a_t+v_t+D_t=\ell_t+C_t+w_t$  (不可向电网售电,  $a_t\geq 0$ )",
266         transform=ax.transAxes, ha="center", va="center", fontsize=10,
267         bbox=dict(boxstyle="round,pad=0.35", facecolor="ffffff",
268                 edgecolor="#9aa0a6", lw=1.0))
269
270 # ===== (b) 决策变量 =====
271 x0, x1 = 0.015, 0.375
272 ytop = panel(x0, 0.035, x1, 0.600, "(b) 决策变量 (逐槽  $t=1, \dots, 144$ )", "#f7f9fc")
273 items = [
274     (" $a_t$ ", "计划购电量", "kWh", c_buy),
275     (" $C_t$ ", "储能充电量", "kWh", c_dis),
276     (" $D_t$ ", "储能放电量", "kWh", c_dis),
277     (" $w_t$ ", "光伏弃光量", "kWh", c_w),
278     (" $E_t$ ", "槽末储电量", "kWh", c_e),
279 ]
280 yy = ytop - 0.012
281 for sym, nm, unit, col in items:
282     ax.text(x0 + 0.022, yy, sym, transform=ax.transAxes, ha="left", va="top",
283            fontsize=13, color=col, fontweight="bold")
284     ax.text(x0 + 0.085, yy - 0.004, nm, transform=ax.transAxes, ha="left", va="top",
285            fontsize=10)
286     ax.text(x1 - 0.018, yy - 0.006, f"[{unit}]", transform=ax.transAxes, ha="right",
287            va="top", fontsize=9, color="#5f6368")
288     yy -= 0.055
289 ax.text(x0 + 0.022, yy - 0.004,
290         "另有 0-1 变量  $z_t$ :  $z_t=1$  表示该槽充电 (充放互斥); \n"
291         "窗初态  $E_0$  (0:10 储电量) 在问题 1 中为自由变量. \n\n"
292         "单位换算:  $1$  槽  $\Delta t = \frac{1}{6}$  h, \n"
293         "槽电量(kWh)  $=$  功率(kW)  $\times \frac{1}{6}$ 。",
294         transform=ax.transAxes, ha="left", va="top", fontsize=9, color="#3c4043")
295
296 # ===== (c) 目标函数 =====
297 ytop = panel(0.395, 0.490, 0.985, 0.600, "(c) 目标函数", "#f7f9fc")
298 ax.text(0.415, ytop - 0.004,
299         " $\min \sum_{t=1}^{144} p_t \ell_t$  (元)",
300         transform=ax.transAxes, ha="left", va="top", fontsize=15)
301 ax.text(0.700, ytop - 0.014,
302         " $p_t$ : 第  $t$  槽电价 (元/kWh);  $a_t$ : 第  $t$  槽购电量 (kWh) \n"
303         "问题 1 无紧急购电、无违约调整, 故目标即全天购电费。",
304         transform=ax.transAxes, ha="left", va="top", fontsize=9, color="#3c4043")
305
306 # ===== (d) 约束 =====
307 ytop = panel(0.395, 0.035, 0.985, 0.470, "(d) 约束条件", "#f7f9fc")
308 cons = [
309     ("供能平衡", " $a_t+v_t+D_t=\ell_t+C_t+w_t$ ", "kWh"),
310     ("SOC 递推", " $E_t=E_{t-1}+0.9(C_t-D_t/0.9)$ ", "kWh"),
311     ("储电量区间", " $1200 \leq E_t \leq 10800$ ", "kWh"),
312     ("充放电上限", " $0 \leq C_t, D_t \leq 5000 \times \frac{1}{6} = 833.33$ ", "kWh"),
313     ("充放互斥", " $C_t \leq M z_t, D_t \leq M(1-z_t), M=833.33$ ", "kWh"),

```

```

314     ("不售电 / 弃光", "$a_t\\geq 0,\\quad 0\\leq w_t\\leq v_t$", "kWh"),
315     ("日循环", "$E_{144}=E_0$ (次日 0:10 复现当日 0:10 储电量)", "kWh"),
316     ("字面锚定", "$E_{143}=6000$ (24:00 储电量)", "kWh"),
317 ]
318 yy = ytop - 0.010
319 for nm, expr, unit in cons:
320     ax.text(0.412, yy, "•", transform=ax.transAxes, ha="left", va="top",
321            fontsize=11, color=colors[0])
322     ax.text(0.427, yy, nm, transform=ax.transAxes, ha="left", va="top", fontsize=9.5)
323     ax.text(0.548, yy, expr, transform=ax.transAxes, ha="left", va="top", fontsize=10)
324     ax.text(0.975, yy, f"[{unit}]", transform=ax.transAxes, ha="right", va="top",
325            fontsize=8.5, color="#5f6368")
326     yy -= 0.043
327 ax.text(0.412, yy + 0.006,
328         "在役解 (本图数据源): $E_0=E_{144}=6300$ kWh, $E_{143}=6000$ kWh, "
329         "$\\sum_t w_t=0$ (无弃光); \\n严格口径 $E_0=E_{143}=E_{144}=6000$ kWh 亦可, "
330         "对应最优购电费 35 126.9486 元。",
331         transform=ax.transAxes, ha="left", va="top", fontsize=8.4, color="#3c4043")
332
333 return fig
334
335
336 def fig_inputs_and_plan(labels, price, load_kw, pv_kw, a, colors):
337     """图1-2 典型日输入 (电价/负载/光伏) 与逐槽计划购电量。"""
338     import matplotlib.pyplot as plt
339     import numpy as np
340
341     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11.5, 8.0),
342                                   gridspec_kw=dict(height_ratios=[1.0, 1.0]))
343     xs = np.arange(144) * 10.0
344     p_ord = periodic(price)
345     l_ord = periodic(load_kw)
346     v_ord = periodic(pv_kw)
347     a_ord = periodic(a)
348
349     ax1.plot(xs, l_ord, color=colors[1], lw=1.6, label="小区负载 $L_t$ (kW) ")
350     ax1.fill_between(xs, 0, v_ord, step="post", color=colors[2], alpha=0.30)
351     ax1.plot(xs, v_ord, color=colors[2], lw=1.6, label="光伏预测功率 $PV_t$ (kW) ")
352     sur = v_ord > l_ord
353     if sur.any():
354         ax1.fill_between(xs, l_ord, v_ord, where=sur, step="post",
355                         color=colors[2], alpha=0.55, lw=0)
356     ax1.set_ylabel("功率 (kW) ")
357     ax1.set_ylim(0, max(l_ord.max(), v_ord.max()) * 1.22)
358
359     ax1b = ax1.twinx()
360     ax1b.spines["right"].set_visible(True)
361     ax1b.step(xs, p_ord, where="post", color=colors[4], lw=1.5,
362              label="电价 $p_t$ (元/kWh) ")
363     ax1b.set_ylabel("电价 (元/kWh) ", color=colors[4])
364     ax1b.tick_params(axis="y", colors=colors[4])

```

```

365 ax1b.set_ylim(0, p_ord.max() * 1.35)
366
367 h1, l1 = ax1.get_legend_handles_labels()
368 h2, l2 = ax1b.get_legend_handles_labels()
369 ax1.legend(h1 + h2, l1 + l2, loc="upper left", ncol=3, frameon=True,
370           framealpha=0.92, fontsize=8.5)
371 set_time_axis(ax1, 2, "")
372 ax1.set_title("图 1-2 (a) 典型日输入：电价、小区负载与光伏发电预测功率（附件1, 144 槽）",
373             fontsize=11)
374 # 光伏盈余区间标注
375 idx = np.where(sur)[0]
376 if idx.size:
377     ax1.annotate(f"光伏 $>$ 负载: {hhmm(xs[idx[0]])}—{hhmm(xs[idx[-1]] + 10)}"
378                f" ({idx.size} 槽, 盈余 {np.sum(np.maximum(v_ord - l_ord, 0)) * 1 /
379                6:.1f} kWh) ",
380                xy=(xs[idx[len(idx) // 2]], (l_ord[idx[len(idx) // 2]] + v_ord[idx[len(
381                idx) // 2])) / 2),
382                xytext=(xs[idx[0]] - 20, ax1.get_ylim()[1] * 0.60),
383                fontsize=8.5, color="#1e6b34",
384                arrowprops=dict(arrowstyle="->", color="#1e6b34", lw=1.0))
385
386 ax2.bar(xs, a_ord, width=9.6, align="edge", color=colors[0], alpha=0.85,
387        label="计划购电量 $a_t$ (kWh/槽) ")
388 for s, val in zip(EXP_TABLE1_SLOTS, EXP_TABLE1):
389     pos = (10 * s) % 1440
390     ax2.bar([pos], [max(val, 1.0)], width=9.6, align="edge", color=colors[3],
391            alpha=0.95, zorder=3)
392     ax2.annotate(f"{val:.4f}", xy=(pos + 5, val), xytext=(pos + 5, val + 95),
393                ha="center", va="bottom", fontsize=8.5, color=colors[3],
394                arrowprops=dict(arrowstyle="-", color=colors[3], lw=0.8))
395 from matplotlib.patches import Patch
396
397 ax2.set_ylabel("购电量 (kWh/槽) ")
398 ax2.set_ylim(0, a_ord.max() * 1.30)
399 set_time_axis(ax2, 2, "时刻")
400 ax2.legend(handles=[Patch(facecolor=colors[0], alpha=0.85,
401                          label="计划购电量 $a_t$ (kWh/槽) "),
402                  Patch(facecolor=colors[3], alpha=0.95, edgecolor=colors[3],
403                          label="表1 指定时段 10:00 / 12:00 / 14:00 / 16:00 / 18:00 /
404                          20:00")],
405           loc="upper left", frameon=True, framealpha=0.92, fontsize=8.5)
406 ax2.set_title("图 1-2 (b) 逐槽计划购电量：全天 %.4f kWh, 购电费 %.4f 元；表1 六时段合计
407             %.4f kWh"
408             % (a.sum(), float(np.sum(price * a)), sum(EXP_TABLE1)), fontsize=11)
409 ax2.text(0.995, 0.93,
410         "时间轴口径：模型窗口 0:10—24:10, 图中已周期化到 0:00—24:00\n"
411         "（槽144 覆盖 0:00—0:10, 其余槽 $j$ 覆盖 $10j$—$10j+1$ 10$ 分钟）",
412         transform=ax2.transAxes, ha="right", va="top", fontsize=8, color="#5f6368")
413 return fig

```

```

412 def fig_storage_soc(a, C, D, E_all, price, colors):
413     """图1-3 储能逐槽充放电与 SOC 轨迹。"""
414     import matplotlib.pyplot as plt
415     import numpy as np
416
417     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11.5, 8.4),
418                                   gridspec_kw=dict(height_ratios=[1.0, 1.20]))
419     xs = np.arange(144) * 10.0
420     C_ord, D_ord = periodic(C), periodic(D)
421
422     ax1.bar(xs, C_ord, width=9.6, align="edge", color=colors[0], alpha=0.9,
423            label="充电量 $C_t$ (kWh/槽, 向上) ")
424     ax1.bar(xs, -D_ord, width=9.6, align="edge", color=colors[3], alpha=0.9,
425            label="放电量 $D_t$ (kWh/槽, 向下) ")
426     ax1.axhline(0, color="#37474f", lw=0.9)
427     ax1.axhline(833.33, color=colors[0], lw=0.9, ls=":")
428     ax1.axhline(-833.33, color=colors[3], lw=0.9, ls=":")
429     ax1.text(6, 848, "单槽充放上限 833.33 kWh $=5000\\$, $kW\\$\\times\\frac{1}{6}\\$h",
430            fontsize=8.2, va="bottom", ha="left", color=colors[0])
431     ax1.set_ylabel("充 / 放电量 (kWh/槽) ")
432     ax1.set_ylim(-1030, 1030)
433     set_time_axis(ax1, 2, "")
434     ax1.legend(loc="lower left", ncol=1, frameon=True, framealpha=0.92, fontsize=8.5)
435     ax1.set_title("图 1-3 (a) 储能逐槽充放电计划: 充电合计 %.4f kWh (45 槽), "
436                "放电合计 %.4f kWh (37 槽) " % (C.sum(), D.sum()), fontsize=11)
437
438     tx, ty = soc_curve(E_all)
439     ax2.axhspan(1200, 10800, color="#eef2f7", zorder=0)
440     ax2.axhspan(0, 1200, color="#fdecea", zorder=0)
441     ax2.axhspan(10800, 13200, color="#fdecea", zorder=0)
442     ax2.axhline(1200, color="#9aa0a6", lw=0.9, ls="--")
443     ax2.axhline(10800, color="#9aa0a6", lw=0.9, ls="--")
444     ax2.axhline(6000, color=colors[7], lw=1.1, ls=":")
445
446     def _runs(idx):
447         out, s, p = [], idx[0], idx[0]
448         for k in idx[1:]:
449             if k == p + 1:
450                 p = k
451             else:
452                 out.append((s, p))
453                 s = p = k
454         out.append((s, p))
455         return out
456
457     top = np.where(np.abs(ty - 10800) < 1e-6)[0]
458     bot = np.where(np.abs(ty - 1200) < 1e-6)[0]
459     if top.size:
460         runs = _runs(top)
461         main = max(runs, key=lambda r: r[1] - r[0])
462         ax2.axvspan(tx[main[0]], tx[main[1]] + 10, color=colors[3], alpha=0.10, zorder=1)

```

```

463     desc = ", ".join(f"{hhmm(tx[i])}—{hhmm(tx[j] + 10)}" if j > i else hhmm(tx[i])
464                     for i, j in runs)
465     ax2.text(520, 12250, f"SOC 顶格 10800 kWh 的时段: {desc}", fontsize=8.6,
466            color="#8c1d18", ha="left", va="bottom")
467     if bot.size:
468         runs = _runs(bot)
469         desc = ", ".join(f"{hhmm(tx[i])}—{hhmm(tx[j] + 10)}" if j > i else hhmm(tx[i])
470                         for i, j in runs)
471         ax2.annotate(f"SOC 触底 1200 kWh (下限): {desc}",
472                    xy=(tx[bot[len(bot) // 2]], 1200), xytext=(1000, 2750),
473                    ha="center", va="top", fontsize=8.6, color="#8c1d18",
474                    arrowprops=dict(arrowstyle="->", color="#8c1d18", lw=1.0))
475
476     ax2.plot(tx, ty, color=colors[0], lw=1.8, marker="o", ms=2.0, zorder=5,
477            label="储电量 $E$ (kWh, 145 点)")
478     ax2.text(840, 6150, "6000 kWh 参考线 (附录1 的 0:00 初值 = 24:00 锚定值)",
479            ha="left", va="bottom", fontsize=8.2, color=colors[7])
480     ax2.text(1435, 10700, "上限 10800", ha="right", va="bottom", fontsize=8,
481            color="#5f6368")
482     ax2.text(20, 1290, "下限 1200", ha="left", va="bottom", fontsize=8, color="#5f6368")
483
484     marks = [(0, ty[0], "E(0:00)=6000", (120, 2950), "left"),
485             (10, ty[1], "E(0:10)=6300", (120, 4100), "left"),
486             (960, ty[96], "16:00: $E$ 顶格 10800 kWh", (1005, 11380), "center"),
487             (1440, ty[144], "E(24:00)=6000", (1442, 7500), "right")]
488     for xm, val, txt, xytext, ha in marks:
489         ax2.plot([xm], [val], marker="o", ms=6, mfc="white", mec=colors[3], mew=1.6,
490                zorder=7, clip_on=False)
491         ax2.annotate(f"{txt}\n ({val:.0f} kWh)", xy=(xm, val), xytext=xytext, ha=ha,
492                    va="top", fontsize=8.6, color="#202124",
493                    arrowprops=dict(arrowstyle="->", color=colors[3], lw=1.0))
494     ax2.set_ylim(120, 13200)
495     ax2.set_ylabel("储电量 $E$ (kWh)")
496     set_time_axis(ax2, 2, "时刻")
497     ax2.legend(loc="upper right", frameon=True, framealpha=0.92, fontsize=8.5)
498     ax2.set_title("图 1-3 (b) 储电量轨迹 (周期化 0:00—24:00): 日循环 $E_{144}=E_0=6300$ kWh",
499                "字面锚定 $E_{143}=6000$ kWh", fontsize=11)
500     return fig
501
502
503 def fig_two_conventions(Co, Do, Cc, Dc, colors):
504     """图1-4 表2 两种口径对照 (原报口径 / 时钟口径)。"""
505     import matplotlib.pyplot as plt
506     import numpy as np
507
508     fig = plt.figure(figsize=(11.5, 7.2))
509     fig.set_layout_engine("none")
510     axc = fig.add_axes([0.055, 0.360, 0.420, 0.520])
511     axd = fig.add_axes([0.545, 0.360, 0.420, 0.520])
512     axt = fig.add_axes([0.020, 0.010, 0.960, 0.300])

```

```

513 axt.axis("off")
514
515 x = np.arange(6)
516 w = 0.40
517 for ax, orig, clk, name, unit in ((axc, Co, Cc, "充电量", "kWh"),
518                                  (axd, Do, Dc, "放电量", "kWh")):
519     ymax = max(max(orig), max(clk)) * 1.40
520     ax.set_ylim(0, ymax)
521     b1 = ax.bar(x - w / 2, orig, w, color=colors[0], alpha=0.9, label="原报口径")
522     b2 = ax.bar(x + w / 2, clk, w, color=colors[2], alpha=0.9, label="时钟口径")
523     for i in range(6):
524         changed = abs(clk[i] - orig[i]) > 5e-4
525         ax.text(x[i] - w / 2, orig[i] + ymax * 0.010, f"{orig[i]:.2f}",
526                ha="center", va="bottom", fontsize=7.2, color=colors[0])
527         ax.text(x[i] + w / 2, clk[i] + ymax * 0.010, f"{clk[i]:.2f}",
528                ha="center", va="bottom", fontsize=7.2,
529                color=colors[3] if changed else colors[2],
530                fontweight="bold" if changed else "normal")
531         if changed:
532             b2[i].set_edgecolor(colors[3])
533             b2[i].set_linewidth(1.8)
534             ax.text(x[i] + w / 2, max(orig[i], clk[i]) + ymax * 0.085,
535                    "Δ %+.4f" % (clk[i] - orig[i]),
536                    ha="center", va="bottom", fontsize=8.2, color=colors[3],
537                    fontweight="bold")
538     ax.set_xticks(x)
539     ax.set_xticklabels(SEG_LABELS, fontsize=8.2)
540     ax.set_ylabel(f"{name} ({unit}) ")
541     ax.legend(loc="upper center", ncol=2, fontsize=8.8, frameon=True,
542             framealpha=0.95, columnspacing=1.2)
543     ax.set_title(f"({'a' if name == '充电量' else 'b'}) {name}: 两种口径对照",
544                fontsize=10.5)
545
546 txt = (
547     "口径差异说明: \n"
548     "① 「原报口径」=建模窗口的自然 24 槽分组 (槽 1—24 / 25—48 / ...)。窗口为 0:10—24:10
549     , 故「槽 1—24」实际覆盖 0:10—4:10, \n"
550     "    与表2 的「0:00-4:00」相差 10 min; 「槽 121—144」实际覆盖 20:10—24:10. \n"
551     "② 「时钟口径」按真实时钟边界归集: 0:00-4:00=槽{144,1...23}、4:00-8:00=槽{24...47}、8
552     :00-12:00=槽{48...71}、\n"
553     "    12:00-16:00=槽{72...95}、16:00-20:00=槽{96...119}、20:00-24:00=槽{120...143}, 使
554     表2 的时段标签字面成立. \n"
555     "③ 两种口径全天合计完全相同: 充电 20 740.6661 kWh、放电 16 799.9396 kWh; 仅 8 个分项数
556     值变化 (红框 / Δ 标注), \n"
557     "    差异全部来自 6 个跨边界槽: 槽 24、48、72、96、120、144. \n"
558     "④ 逐槽解、全天购电量 59 482.6990 kWh、购电费 35 126.8486 元、SOC 轨迹与弃光 0 均不受口
559     径选择影响. \n"
560     "⑤ **交付口径 (2026-09-11 订正后)**: result1.xlsx 的「充放电量」表已改用「时钟口径」;
561     "
562     "原报口径仅作对照保留。"
563 )

```

```

558     axt.text(0.005, 0.98, txt, ha="left", va="top", fontsize=8.6, color="#202124",
559             linespacing=1.5,
560             bbox=dict(boxstyle="round,pad=0.5", facecolor="#f7f9fc",
561                     edgecolor="#9aa0a6", lw=1.0))
562     return fig
563
564
565 def fig_baselines(cost0, cost1, cost2, cur0, colors):
566     """图1-5 三级基线阶梯。"""
567     import matplotlib.pyplot as plt
568     import numpy as np
569
570     fig = plt.figure(figsize=(9.8, 7.0))
571     fig.set_layout_engine("none")
572     ax = fig.add_axes([0.105, 0.400, 0.875, 0.520])
573     axt = fig.add_axes([0.020, 0.010, 0.960, 0.330])
574     axt.axis("off")
575
576     names = ["B0 无储能\n(纯购电)",
577             "B1 规则基线\n(贪心谷充峰放)",
578             "B2 本模型\n(问题1 MILP 最优)"]
579     vals = [cost0, cost1, cost2]
580     cols = [colors[7], colors[1], colors[2]]
581     bars = ax.bar(names, vals, color=cols, alpha=0.92, width=0.52)
582     for b, v in zip(bars, vals):
583         ax.text(b.get_x() + b.get_width() / 2, v + max(vals) * 0.015, f"{v:,.2f} 元",
584               ha="center", va="bottom", fontsize=10.5, fontweight="bold")
585     ax.set_xlim(-0.62, 4.10)
586     ax.set_ylim(0, max(vals) * 1.14)
587     for v, c, lb in ((cost0, cols[0], "B0 水平"), (cost1, cols[1], "B1 水平")):
588         ax.hlines(v, -0.45, 3.95, color=c, lw=1.1, ls=(0, (5, 3)))
589         ax.text(1.50, v, lb, ha="center", va="center", fontsize=7.8, color=c,
590               bbox=dict(boxstyle="round,pad=0.18", facecolor="white",
591                       edgecolor="none", alpha=0.95))
592     for xa, v0, lab, col in ((3.00, cost1, "较 B1 节省 10 593.00 元 (-23.17%)", cols[1]),
593                             (3.42, cost0, "较 B0 节省 12 925.20 元 (-26.90%)", cols[0])):
594         ax.annotate("", xy=(xa, v0), xytext=(xa, cost2),
595                   arrowprops=dict(arrowstyle="<|-|>", color=col, lw=1.2))
596         ax.text(xa + 0.055, (v0 + cost2) / 2, lab, rotation=90,
597               ha="left", va="center", fontsize=8.8, color=col)
598     ax.set_ylabel("全天购电费 (元)")
599     ax.grid(axis="y", ls=":", lw=0.6, alpha=0.6)
600     ax.set_axisbelow(True)
601     ax.set_title("图 1-5 问题 1 的三级基线阶梯 (典型日, 附件1 数据)", fontsize=11)
602
603     txt = (
604         "基线口径说明: \n"
605         "• B0: 不使用储能, 逐槽按缺口购电, 全天购电费 48 052.05 元; 此时光伏盈余 6 247.963 kWh 全部弃掉——\n"
606         "      6 247.963 kWh 属于 B0 基线, 不是本模型解的弃光量。 \n"
607         "• B1: 贪心「谷充峰放」规则基线, 全天购电费 45 719.85 元, 其末态储电量 10 800 kWh (上限

```

顶格)，\n"

" 不满足问题1「0:00 与 24:00 储电量相同」的日循环要求，与 B2 不在同一可行域，仅作「未满足终端要求的规则示例」。\n"

"• B2: 本模型（方案丙口径 MILP）最优解，全天购电费 35 126.8486 元，弃光 0 kWh, \n"

" \$E_{143}=6000\$ kWh、\$E_{144}=E_0=6300\$ kWh，日循环成立。 \n"

"• 因此 B0 / B1 与 B2 的差额只能作为费用对照，不构成 B2 最优性的独立证明。"

)

axt.text(0.005, 0.98, txt, ha="left", va="top", fontsize=8.7, color="#202124",
linespacing=1.5,
bbox=dict(boxstyle="round,pad=0.5", facecolor="#f7f9fc",
edgecolor="#9aa0a6", lw=1.0))

return fig

def fig_netload(load_kw, pv_kw, a, C, D, price, colors):

"""图1-6 净负荷与储能作用。"""

import matplotlib.pyplot as plt

import numpy as np

DT = 1.0 / 6.0

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11.5, 7.6), sharex=True,
gridspec_kw=dict(height_ratios=[1.0, 0.85]))

xs = np.arange(144) * 10.0

ell = periodic(load_kw) * DT

v = periodic(pv_kw) * DT

net = ell - v

a_o, C_o, D_o = periodic(a), periodic(C), periodic(D)

for ax in (ax1, ax2):

ax.axvspan(0, 360, color="#eceff1", alpha=0.55, zorder=0)

ax.axvspan(1080, 1440, color="#eceff1", alpha=0.55, zorder=0)

ax1.axhline(0, color="#37474f", lw=1.0)

ax1.fill_between(xs, 0, net, where=(net >= 0), step="post", color=colors[1],
alpha=0.35, lw=0)

ax1.fill_between(xs, 0, net, where=(net < 0), step="post", color=colors[2],
alpha=0.55, lw=0)

ax1.step(xs, net, where="post", color="#202124", lw=1.3,

label="净负荷 \$\ell_t-v_t\$ (kWh/槽) ")

ylo = float(net.min()) * 1.45

yhi = float(net.max()) * 1.30

ax1.set_ylim(ylo, yhi)

ax1.set_ylabel("净负荷 (kWh/槽) ")

ax1.set_title("图 1-6 (a) 净负荷曲线: 缺口段 (购电 / 放电) 与盈余段 (充电 / 弃光)",
fontsize=11)

ax1.legend(loc="upper right", fontsize=8.5, frameon=True, framealpha=0.92)

i_neg = np.where(net < 0)[0]

tot_neg = float(-net[net < 0].sum())

tot_pos = float(net[net > 0].sum())

ax1.annotate(f"光伏盈余 {i_neg.size} 槽, 合计 {tot_neg:.2f} kWh\n"

f" ({hhmm(xs[i_neg[0]])}—{hhmm(xs[i_neg[-1]] + 10)}), 全部被充电/负荷吸收, 弃
光 0) ",

```

655         xy=(xs[i_neg[len(i_neg) // 2]], net[i_neg[len(i_neg) // 2]] * 0.55),
656         xytext=(xs[i_neg[0]] - 70, ylo * 0.72), fontsize=8.8, color="#1e6b34",
657         ha="left", arrowprops=dict(arrowstyle="->", color="#1e6b34", lw=1.0))
658 ax1.text(20, yhi * 0.93,
659         f"缺口 {int((net > 0).sum())} 槽, 合计 {tot_pos:,.2f} kWh (由购电与放电补足)",
660         fontsize=8.8, color="#8a4b00", ha="left", va="top")
661 for xc, s in ((180, "夜间"), (720, "白天"), (1260, "夜间")):
662     ax1.text(xc, yhi * 0.55, s, fontsize=9, color="#5f6368", ha="center",
663             bbox=dict(boxstyle="round,pad=0.25", facecolor="white",
664                     edgecolor="none", alpha=0.75))
665
666 ax2.bar(xs, a_o, width=9.6, align="edge", color=colors[0], alpha=0.75,
667         label="购电量 $a_t$ (kWh/槽)")
668 ax2.bar(xs, D_o, width=9.6, align="edge", color=colors[3], alpha=0.85,
669         label="放电量 $D_t$ (kWh/槽)")
670 ax2.bar(xs, -C_o, width=9.6, align="edge", color=colors[8], alpha=0.85,
671         label="充电量 $C_t$ (kWh/槽, 向下)")
672 ax2.axhline(0, color="#37474f", lw=1.0)
673 ax2.set_ylabel("电量 (kWh/槽)")
674 ax2.set_ylim(-1050, 1900)
675 set_time_axis(ax2, 2, "时刻")
676 ax2.legend(loc="upper left", ncol=3, fontsize=8.4, frameon=True, framealpha=0.95,
677           columnspacing=1.0, handlelength=1.4)
678 ax2.set_title("图 1-6 (b) 逐槽购电 / 放电 (向上) 与充电 (向下): 盈余段由充电吸收, "
679             "缺口段由购电与放电补足", fontsize=11)
680 for xm, yt, s, col in [(605, 900, "10:00 购电 0: 光伏 1068.27 kWh > 负载 983.70 kWh",
681                       colors[2]),
682                       (845, 1330, "14:00 购电 0: 光伏 1068.75 kWh > 负载 948.72 kWh",
683                       colors[2]),
684                       (1215, 1760, "20:00 购电 0: 光伏为 0, 由储能放电 711.45 kWh 覆盖",
685                       colors[3])]:
686     ax2.annotate(s, xy=(xm, 90), xytext=(xm, yt), ha="center", va="bottom",
687                 fontsize=8.2, color=col,
688                 arrowprops=dict(arrowstyle="->", color=col, lw=0.9))
689 ax2.text(0.995, 0.02,
690         "阴影: 夜间 (0:00—6:00、18:00—24:00, 作图约定) \n"
691         "槽电量 = 功率 × (1/6) h; 净负荷  $\ell_t - v_t$  为槽电量口径 (kWh)",
692         transform=ax2.transAxes, ha="right", va="bottom", fontsize=8, color="#5f6368")
693 return fig
694
695 # ----- 主流程
696 def main() -> int:
697     import numpy as np
698
699     OUT_DIR.mkdir(parents=True, exist_ok=True)
700     LOG_DIR.mkdir(parents=True, exist_ok=True)
701
702     print("=" * 100)
703     print("2026 高教社杯 C 题 · 问题1 求解可视化集生成")
704     print("=" * 100)

```

```

703 print(f"工作目录      : {ROOT}")
704 print(f"输出目录      : {OUT_DIR}")
705 print(f"日志文件      : {LOG_PATH}")
706 print(f"模型代码      : {CODE_DIR / '模型.py'} (方案丙口径, 只读)")
707 print(f"对照结果      : {RESULT1} (只读)")
708 print("-" * 100)
709
710 # ---- 样式 ----
711 import matplotlib
712
713 matplotlib.use("Agg")
714 sys.path.insert(0, SKILL_SCRIPTS)
715 style_backend = "手工 rcParams"
716 try:
717     from setup_style import setup_style
718
719     info = setup_style(journal="general", lang="zh")
720     style_backend = f"setup_style({info})"
721 except Exception as exc: # noqa: BLE001
722     import matplotlib.pyplot as plt
723
724     plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
725     plt.rcParams["axes.unicode_minus"] = False
726     style_backend = f"手工 rcParams (Skill 导入失败: {exc})"
727 print(f"绘图样式      : {style_backend}")
728
729 import matplotlib.pyplot as plt
730
731 try:
732     from export_figure import export_figure
733 except Exception as exc: # noqa: BLE001
734     raise RuntimeError(f"export_figure 不可用: {exc}") from exc
735
736 colors = plt.get_cmap("tab10").colors
737
738 # ---- 数据 ----
739 labels, price, load_kw, pv_kw = read_attachment1()
740 record(f"附件1 结构核对 (144 数据行 × 4 列)",
741       len(price) == 144 and len(load_kw) == 144 and len(pv_kw) == 144,
742       f"行数={len(price)}, 首行标签={labels[0]!r}, 末行标签={labels[-1]!r}")
743
744 # ---- 模型求解 ----
745 sys.path.insert(0, str(CODE_DIR))
746 from 模型 import solve_day
747
748 q1 = solve_day(price, load_kw, pv_kw, E_start=None, mode="q1", E_clock_24=6000.0)
749 a, C, D, w, E_all = q1["a"], q1["C"], q1["D"], q1["w"], q1["E_all"]
750 obj = float(q1["objective"])
751 print("-" * 100)
752 print(f"模型重解: status={q1['status']} ok={q1['ok']} mip_gap={q1['mip_gap']} "
753       f"wall={q1['wall_s']:.3f}s")

```

```

754 print(f" 目标值 = {obj!r} 元;  $\Sigma a = \{a.sum():.6f\}$  kWh;  $\Sigma C = \{C.sum():.6f\}$  kWh; "
755       f" $\Sigma D = \{D.sum():.6f\}$  kWh;  $\Sigma w = \{w.sum():.10f\}$  kWh")
756 print(f" E0 = {E_all[0]:.4f}; E143 (24:00) = {E_all[143]:.4f}; E144 (24:10) = {E_all[144]:.4f}; "
757       f"E 最小 {E_all.min():.4f} / 最大 {E_all.max():.4f}")
758
759 # ---- 自检 1: 目标值 ----
760 rel = abs(obj - EXP_OBJECTIVE) / abs(EXP_OBJECTIVE)
761 record("自检1 目标值核对 (相对差 < 1e-6) ", rel < 1e-6,
762       f"重解 {obj!r} vs 题面 {EXP_OBJECTIVE!r}, 相对差 {rel:.3e}")
763 record("自检1b 全天购电量核对", abs(a.sum() - EXP_TOTAL_BUY) < 1e-3,
764       f"{a.sum():.6f} kWh vs {EXP_TOTAL_BUY} kWh")
765 record("自检1c 全天购电费 =  $\Sigma p \cdot a$ ", abs(float(np.sum(price * a)) - obj) < 1e-6,
766       f" $\Sigma p \cdot a = \{float(np.sum(price * a)):.9f\}$  元")
767 record("自检1d 充/放电合计", abs(C.sum() - EXP_TOTAL_C) < 5e-4
768       and abs(D.sum() - EXP_TOTAL_D) < 5e-4,
769       f" $\Sigma C = \{C.sum():.4f\}$  (期望 {EXP_TOTAL_C})  $\Sigma D = \{D.sum():.4f\}$  (期望 {EXP_TOTAL_D}) ")
770 record("自检1e 弃光 = 0 (本最优解) ", abs(w.sum()) < 1e-9,
771       f" $\Sigma w = \{w.sum():.10f\}$  kWh")
772
773 # ---- 自检 2: 六段汇总 vs result1.xlsx (现行文件已按真实时钟区间订正) ----
774 plan_file, cd_file = read_result1()
775 Co = [seg_sum(C, range(i, j)) for i, j in SEG_ORIG_IDX] # 原报口径 (历史对照)
776 Do = [seg_sum(D, range(i, j)) for i, j in SEG_ORIG_IDX]
777 Cc = [seg_sum(C, idx) for idx in SEG_CLOCK_IDX] # 真实时钟区间 (题目表2)
778 Dc = [seg_sum(D, idx) for idx in SEG_CLOCK_IDX]
779 file_co = [float(r[1]) for r in cd_file[:6]]
780 file_do = [float(r[2]) for r in cd_file[:6]]
781 file_lab = [str(r[0]) for r in cd_file[:6]]
782 d_file = max(max(abs(x - y) for x, y in zip(Cc, file_co)),
783             max(abs(x - y) for x, y in zip(Dc, file_do)))
784 d_orig = max(max(abs(x - y) for x, y in zip(Co, file_co)),
785             max(abs(x - y) for x, y in zip(Do, file_do)))
786 print(" [文件段标签] :", file_lab)
787 for k in range(6):
788     print(f" {SEG_LABELS[k]:<12} 时钟 {Cc[k]:10.4f} (文件 {file_co[k]:10.4f}) "
789           f" 放 {Dc[k]:10.4f} (文件 {file_do[k]:10.4f})  $\Delta$ 充 {Cc[k]-Co[k]:+9.4f}")
790 record("自检2 result1.xlsx「充放电量」表 = 真实时钟区间汇总 (差 < 5e-4) ",
791       d_file < 5e-4, f"最大偏差 {d_file:.3e}")
792 record("自检2b 该表已不再是原报口径 (24 槽自然分组) ",
793       d_orig > 1e-2, f"与原报口径最大偏差 {d_orig:.4f} kWh")
794 record("自检2c 六段标签与题目表2 时钟区间逐字一致",
795       file_lab == SEG_LABELS, f"{file_lab}")
796 d_exp = max(max(abs(x - y) for x, y in zip(Cc, EXP_SEG_CLOCK_C)),
797            max(abs(x - y) for x, y in zip(Dc, EXP_SEG_CLOCK_D)))
798 record("自检2d 时钟口径六段 = 题面给定值", d_exp < 5e-4, f"最大偏差 {d_exp:.3e}")
799
800 # ---- 自检 3: 计划购电量 144 值与 result1.xlsx 一致 ----
801 plan_file = np.array(plan_file, dtype=float)[:144]
802 dp = np.abs(a - plan_file)
803 n_bad = int((dp > 5e-4).sum())

```

```

804 record("自检3 「计划购电量」144 值逐槽一致 (容差 5e-4, 对应 4 位小数存储舍入)",
805         n_bad == 0, f"最大偏差 {dp.max():.3e}, 超容差槽数 {n_bad}")
806
807 # ---- 自检 4: 两种口径的差值与合计守恒 ----
808 for k in range(6):
809     print(f"    [对照] {SEG_LABELS[k]:<12} 原报 充 {Co[k]:10.4f} 放 {Do[k]:10.4f}"
810           f"    → 时钟 充 {Cc[k]:10.4f} 放 {Dc[k]:10.4f}")
811 n_changed = sum(1 for k in range(6) if abs(Cc[k] - Co[k]) > 5e-4) + \
812             sum(1 for k in range(6) if abs(Dc[k] - Do[k]) > 5e-4)
813 record("自检4 两口径全天合计一致 (仅归组不同)", abs(sum(Cc) - sum(Co)) < 1e-6
814         and abs(sum(Dc) - sum(Do)) < 1e-6,
815         f"Σ充 {sum(Cc):.4f} / {sum(Co):.4f}; Σ放 {sum(Dc):.4f} / {sum(Do):.4f}")
816 record("自检4b 12 个汇总数中变化的分项数 = 8", n_changed == 8, f"实测变化分项 {n_changed}
817 个")
818 soc_c = EXP_E143
819 for k in range(6):
820     soc_c = soc_c + 0.9 * Cc[k] - Dc[k] / 0.9
821 record("自检4c 时钟口径 SOC 链自 0:00=6000 闭环到 24:00=6000", abs(soc_c - EXP_E143) < 1e-6,
822         f"链末端 {soc_c:.6f} kWh")
823
824 # ---- 自检 5: SOC 检查点 ----
825 record("自检5 SOC 检查点 E(0:10)=E_0=6300、E(24:00)=E_143=6000、E_144=E_0",
826         abs(E_all[0] - EXP_E0) < 1e-6 and abs(E_all[143] - EXP_E143) < 1e-6
827         and abs(E_all[144] - E_all[0]) < 1e-6,
828         f"E0={E_all[0]:.4f} E143={E_all[143]:.4f} E144={E_all[144]:.4f}")
829 record("自检5b E in [1200, 10800]",
830         E_all.min() >= 1200 - 1e-6 and E_all.max() <= 10800 + 1e-6,
831         f"[{E_all.min():.4f}, {E_all.max():.4f}]"
832
833 # ---- 自检 6: 表1 六时段 ----
834 a6 = [float(a[s - 1]) for s in EXP_TABLE1_SLOTS]
835 d6 = max(abs(x - y) for x, y in zip(a6, EXP_TABLE1))
836 record("自检6 表1 六时段 (槽 60/72/84/96/108/120) 购电量", d6 < 5e-4,
837         " / ".join(f"{v:.4f}" for v in a6) + f" (最大偏差 {d6:.3e})")
838 for s, v in zip(EXP_TABLE1_SLOTS, a6):
839     t = s - 1
840     print(f"    槽{s:>4} {hhmm(10 * s)}—{hhmm(10 * s + 10)} a={v:9.4f} kWh"
841           f"    C={C[t]:8.4f} D={D[t]:8.4f}"
842           f"    光伏 {pv_kw[t]:8.2f} kW / {pv_kw[t] / 6:8.3f} kWh"
843           f"    负载 {load_kw[t]:8.2f} kW / {load_kw[t] / 6:8.3f} kWh"
844           f"    电价 {price[t]:.4f}")
845
846 # ---- 自检 7: 严格口径 ----
847 q1s = solve_day(price, load_kw, pv_kw, E_start=6000.0, mode="q1", E_clock_24=6000.0)
848 rel_s = abs(float(q1s["objective"]) - EXP_STRICT_OBJECTIVE) / EXP_STRICT_OBJECTIVE
849 record("自检7 严格口径 (E0=E143=E144=6000) 最优值", rel_s < 1e-6,
850         f"{q1s['objective']:.9f} 元 vs {EXP_STRICT_OBJECTIVE} (相对差 {rel_s:.3e})")
851
852 # ---- 自检 8: 基线 ----
853 DT = 1.0 / 6.0

```

```

853 net_kwh = load_kw * DT - pv_kw * DT
854 b0 = float(np.sum(price * np.maximum(net_kwh, 0.0)))
855 cur0 = float(np.maximum(-net_kwh, 0.0).sum())
856 record("自检8 B0 基线（无储能纯购电 + 弃光）",
857        abs(b0 - EXP_B0) < 5e-3 and abs(cur0 - EXP_B0_CURTAIL) < 5e-3,
858        f"购电费 {b0:.6f} 元 (≈{b0:.2f})，弃光 {cur0:.6f} kWh (≈{cur0:.3f})，归属 B0")
859
860 # ---- 自检 9: 输入数据范围 ----
861 rng_ok = (abs(price.min() - EXP_PRICE_RANGE[0]) < 5e-4
862           and abs(price.max() - EXP_PRICE_RANGE[1]) < 5e-4
863           and abs(load_kw.min() - EXP_LOAD_RANGE[0]) < 5e-2
864           and abs(load_kw.max() - EXP_LOAD_RANGE[1]) < 5e-2
865           and abs(pv_kw.min() - EXP_PV_RANGE[0]) < 5e-2
866           and abs(pv_kw.max() - EXP_PV_RANGE[1]) < 5e-2)
867 record("自检9 输入数据范围（电价/负载/光伏）", rng_ok,
868        f"电价 {price.min():.4f}—{price.max():.4f} 元/kWh; "
869        f"负载 {load_kw.min():.1f}—{load_kw.max():.1f} kW; "
870        f"光伏 {pv_kw.min():.1f}—{pv_kw.max():.1f} kW")
871
872 # ---- 绘图 ----
873 print("-" * 100)
874 print("开始绘图 (PNG 300 dpi + SVG) .....")
875 figs = []
876 figs.append(("图1-1_模型结构示意", fig_model_structure(q1, {}, colors)))
877 figs.append(("图1-2_典型日输入与购电计划",
878             fig_inputs_and_plan(labels, price, load_kw, pv_kw, a, colors)))
879 figs.append(("图1-3_储能调度与SOC轨迹",
880             fig_storage_soc(a, C, D, E_all, price, colors)))
881 figs.append(("图1-4_表2两种口径对照", fig_two_conventions(Co, Do, Cc, Dc, colors)))
882 figs.append(("图1-5_基线阶梯", fig_baselines(b0, EXP_B1, obj, cur0, colors)))
883 figs.append(("图1-6_净负荷与储能作用",
884             fig_netload(load_kw, pv_kw, a, C, D, price, colors)))
885
886 print("-" * 100)
887 layout_ok = True
888 for name, fig in figs:
889     base = OUT_DIR / name
890     ov, cl = layout_audit(fig)
891     ok = not ov and not cl
892     layout_ok &= ok
893     print(f" [{'PASS' if ok else 'FAIL'}] {name} 版面自检"
894           + (f" — 文字重叠 {len(ov)} 对，越界 {len(cl)} 项" if not ok
895             else " — 文字重叠 0 对，越界 0 项"))
896     if not ok:
897         for o in ov[:5]:
898             print(f"      重叠: {o[0]!r} <-> {o[1]!r}")
899         for c in cl[:5]:
900             print(f"      越界: {c!r}")
901     export_figure(fig, str(base), formats=("png", "svg"), dpi=300, tight=True)
902     plt.close(fig)
903     FIG_FILES.extend([Path(f"{base}.png"), Path(f"{base}.svg")])

```

```

904 record("自检10 六张图版面自检（纯文字框 0 重叠、0 越界）", layout_ok,
905         f"共检查 {len(figs)} 张图")
906
907 # ---- 自检 10: 图件字节数 ----
908 print("-" * 100)
909 print("图件字节数核对（要求 > 20000 字节）：")
910 size_ok = True
911 for name, _ in figs:
912     png = OUT_DIR / f"{name}.png"
913     svg = OUT_DIR / f"{name}.svg"
914     pb = png.stat().st_size if png.exists() else 0
915     sb = svg.stat().st_size if svg.exists() else 0
916     ok = pb > 20000 and sb > 20000
917     size_ok &= ok
918     print(f" [{'PASS' if ok else 'FAIL'}] {name}.png {pb:>9,d} 字节 | "
919           f"{name}.svg {sb:>9,d} 字节")
920 record("自检11 每张图 PNG/SVG 字节数 > 20000", size_ok,
921         f"{len(figs)} 张图，共 {len(FIG_FILES)} 个文件")
922
923 # ---- 汇总 ----
924 print("=" * 100)
925 n_fail = sum(1 for _, ok, _ in CHECKS if not ok)
926 for name, ok, detail in CHECKS:
927     print(f" [{'PASS' if ok else 'FAIL'}] {name}" + (f" — {detail}" if detail else ""
928 ))
929 print("-" * 100)
930 print(f"自检合计: {len(CHECKS) - n_fail}/{len(CHECKS)} PASS, {n_fail} FAIL")
931 print(f"PNG/SVG 文件共 {len(FIG_FILES)} 个，位于 {OUT_DIR}")
932 print("=" * 100)
933 if n_fail:
934     print("最终结论: FAIL")
935     return 1
936 print("最终结论: PASS（全部自检通过，退出码 0）")
937 return 0
938
939 def run() -> int:
940     tee = Tee(LOG_PATH)
941     old = sys.stdout
942     sys.stdout = tee
943     try:
944         return main()
945     except Exception: # noqa: BLE001
946         traceback.print_exc(file=sys.stdout)
947         print("=" * 100)
948         print("最终结论: FAIL（脚本异常终止）")
949         return 1
950     finally:
951         sys.stdout = old
952         tee.close()
953         print(f"运行日志已写入: {LOG_PATH}")

```

```

954
955
956 if __name__ == "__main__":
957     sys.exit(run())

```

C3. 问题 1/检查.py (76 行)

问题 1 求解结果的约束与结算自检入口

```

1  #!/usr/bin/env python3
2  """最小可运行版本 • 检查器：硬约束残差、量纲一致性、检查点与费用复算。"""
3
4  from __future__ import annotations
5
6  import numpy as np
7
8  from 口径 import DT, E_MIN, E_OP, ETA, K_DOWN, K_EMG, K_UP, M_SLOT, N_SLOT
9
10
11 def check_day(ell_kwh, v_kwh, a, e, C, D, w, r, E_all, tag="") -> list[str]:
12     """逐槽完整平衡、界、互斥、递推与余量分账检查；返回问题列表。"""
13     issues = []
14     resid = a + e + v_kwh - w + D - ell_kwh - C - r
15     if np.max(np.abs(resid)) > 1e-6:
16         issues.append(f"{tag} 完整平衡残差超限: {np.max(np.abs(resid)):.3e} kWh")
17     if np.min(C) < -1e-9 or np.min(D) < -1e-9 or np.min(e) < -1e-9:
18         issues.append(f"{tag} 出现负的充/放/紧急电量")
19     if np.max(C) > M_SLOT + 1e-6 or np.max(D) > M_SLOT + 1e-6:
20         issues.append(f"{tag} 充放电超过 5000 kW 对应的单槽上界 {M_SLOT:.4f} kWh")
21     E = E_all[1:]
22     if np.min(E) < E_MIN - 1e-6 or np.max(E) > E_OP + 1e-6:
23         issues.append(f"{tag} 储电量越界 [{np.min(E):.3f}, {np.max(E):.3f}] kWh")
24     sim = E_all[:-1] + ETA * C - D / ETA
25     if np.max(np.abs(sim - E)) > 1e-6:
26         issues.append(f"{tag} 储能递推残差超限: {np.max(np.abs(sim - E)):.3e} kWh")
27     overlap = np.minimum(C, D)
28     if np.max(overlap) > 1e-6:
29         issues.append(f"{tag} 充放电互斥被违反: max min(C,D)={np.max(overlap):.3e} kWh")
30     if np.max(w - v_kwh) > 1e-6:
31         issues.append(f"{tag} 弃光超过可用光伏")
32     if np.max(r - a) > 1e-6:
33         issues.append(f"{tag} 合同未消纳量超过购电量")
34     return issues
35
36
37 def check_settlement(price, plan, adj, emg, reported_total) -> list[str]:
38     """结算等价式复算:  $p \cdot a + 0.5p|a-b| + 5p \cdot e$  应等于分项之和。"""
39     issues = []
40     p = np.asarray(price); b = np.asarray(plan); a = np.asarray(adj); e = np.asarray(emg)
41     down = np.maximum(b - a, 0.0); up = np.maximum(a - b, 0.0)
42     ident = float(np.sum(p * a + K_DOWN * p * (down + up) + K_EMG * p * e))
43     parts = float(np.sum(p * b) + np.sum(-p * down + K_DOWN * p * down + K_UP * p * up) +
44                  np.sum(K_EMG * p * e))

```

```

44     if abs(ident - parts) > 1e-6:
45         issues.append(f"结算等价式与分项不一致: {ident:.6f} vs {parts:.6f} 元")
46     if reported_total is not None and abs(parts - reported_total) > 1e-6:
47         issues.append(f"结算分项与回报总额不一致: {parts:.6f} vs {reported_total:.6f} 元")
48     return issues
49
50
51 def check_continuity(E_end_prev, E_start_next, tag="") -> list[str]:
52     if abs(float(E_end_prev) - float(E_start_next)) > 1e-6:
53         return [f"{tag} 跨窗口不连续: 前窗口末 {E_end_prev:.6f} vs 本窗口初 {E_start_next:.6f} kWh"]
54     return []
55
56
57 def check_literal_checkpoints(E_all, e_clock_24=None, E_prev_all=None, tag="") -> list[str]:
58     """字面检查点 (§14.4): E_clock24 = E_{d,143} = E_all[-2]; E_clock00 = E_{d-1,143} = 前一日 E_all[-2]。"""
59     issues = []
60     if e_clock_24 is not None and abs(E_all[-2] - float(e_clock_24)) > 1e-6:
61         issues.append(f"{tag} 字面 24:00 检查点不符: E_{{d,143}}={E_all[-2]:.6f} vs {e_clock_24}")
62     if E_prev_all is not None:
63         prev_clock0 = float(E_prev_all[-2])
64         if abs(prev_clock0 - float(E_all[0]) - 0.0) > 1e6: # 仅登记, 不判错 (二者允许相差未槽净变化 δ)
65             pass
66     return issues
67
68
69 def summarize(items: list[tuple[str, bool, str]]) -> tuple[bool, str]:
70     lines = []
71     ok_all = True
72     for name, ok, detail in items:
73         ok_all &= ok
74         lines.append(f" [{'PASS' if ok else 'FAIL'}] {name}" + (f" — {detail}" if detail else ""))
75     return ok_all, "\n".join(lines)

```

C4. 问题 1/口径.py (146 行)

问题 1 口径常量与时段 ↔ 时钟区间映射 (含表 2 六段的真实时钟归组)

```

1  #!/usr/bin/env python3
2  """最小可运行版本 • 口径与数据层 (方案丙) 。
3
4  - 槽口径: 第 j 槽 (j=1..144) = [10j, 10j+10) 分钟, 即 0:10→次日 0:10; 标签=槽起点。
5  - 单位: 功率 kW、电量 kWh、电价 元/kWh、Δt=1/6 h; 电量 = 功率 × Δt。
6  - 只读附件; 不改原始数据。
7  """
8
9  from __future__ import annotations
10

```

```

11 import datetime
12 import hashlib
13 import json
14 from pathlib import Path
15
16 import numpy as np
17 from openpyxl import load_workbook
18
19 ROOT = Path(r"C:\Users\John\Desktop\26题")
20 ATT = ROOT / "00_题目与原始数据" / "C题" / "附件"
21 DT = 1.0 / 6.0 # h
22 ETA = 0.9 # 单向效率 (D4 裁定)
23 E_MIN, E_OP = 1200.0, 10800.0 # kWh (附录1 运行区间)
24 E_START_1JAN = 6000.0 # kWh (附录1, 2025-01-01 0:00)
25 P_MAX = 5000.0 # kW
26 M_SLOT = P_MAX * DT # 833.3333 kWh (单槽充放电量上界)
27 N_SLOT = 144
28 K_EMG, K_DOWN, K_UP = 5.0, 0.5, 1.5 # 紧急 5 倍、下调 0.5 倍、上调 1.5 倍
29
30 RESULT_DAYS = 334 # 2025-02-01—12-31
31
32
33 def sha256(path: Path) -> str:
34     return hashlib.sha256(path.read_bytes()).hexdigest().upper()
35
36
37 def label_minutes(value) -> int:
38     """时间标签 → 距当日 0:00 的分钟数; '0:00+1' → 1440 (附录2 定义)。"""
39     if isinstance(value, datetime.time):
40         return value.hour * 60 + value.minute
41     text = str(value).strip()
42     if "+1" in text:
43         return 1440
44     hh, mm, *_ = text.split(":")
45     return int(hh) * 60 + int(mm)
46
47
48 def read_rows(path: Path, sheet=0):
49     wb = load_workbook(path, read_only=True, data_only=True)
50     ws = wb.worksheets[sheet] if isinstance(sheet, int) else wb[sheet]
51     rows = [tuple(r) for r in ws.iter_rows(values_only=True)]
52     while rows and not any(v is not None for v in rows[-1]):
53         rows.pop()
54     wb.close()
55     return rows
56
57
58 def load_attachment1() -> dict:
59     """典型日: 电价/负载/光伏预测 (144 槽, 槽起点标签 00:10...0:00+1)。"""
60     rows = read_rows(ATT / "附件1.xlsx", 0)
61     labels = [label_minutes(r[0]) for r in rows[1:]]

```

```

62     assert labels == [10 * (j + 1) for j in range(N_SLOT)], "附件1 槽标签不符方案丙"
63     return {
64         "slot_minutes": labels,
65         "price": np.array([float(r[1]) for r in rows[1:]]),
66         "load_kw": np.array([float(r[2]) for r in rows[1:]]),
67         "pv_kw": np.array([float(r[3]) for r in rows[1:]]),
68     }
69
70
71 def load_attachment2() -> dict:
72     load_rows = read_rows(ATT / "附件2.xlsx", "小区负载")
73     pv_rows = read_rows(ATT / "附件2.xlsx", "光伏发电实际功率")
74     dates = [r[0].date() for r in load_rows[1:]]
75     return {
76         "dates": dates,
77         "index": {d: i for i, d in enumerate(dates)},
78         "load_kw": np.array([float(v) for v in r[1:] for r in load_rows[1:]]),
79         "pv_kw": np.array([float(v) for v in r[1:] for r in pv_rows[1:]]),
80     }
81
82
83 def load_attachment3() -> dict:
84     """(日期, 发布时刻) → 24 个整点预报 (kW), h=1..24 指向 发布时刻+h 整点。"""
85     rows = read_rows(ATT / "附件3.xlsx", 0)[1:]
86     out, cur = {}, None
87     for r in rows:
88         if r[0] not in (None, ""):
89             y, m, d = str(r[0]).split("-")
90             cur = datetime.date(int(y), int(m), int(d))
91             out[(cur, str(r[1]))] = np.array([float(v) for v in r[2:]])
92     return out
93
94
95 def load_attachment4() -> dict:
96     rows = read_rows(ATT / "附件4.xlsx", 0)
97     dates = [r[0].date() for r in rows[1:]]
98     return {
99         "dates": dates,
100        "index": {d: i for i, d in enumerate(dates)},
101        "price": np.array([float(v) for v in r[1:] for r in rows[1:]]),
102    }
103
104
105 def forecast_from_forecast3(fc: dict, day: datetime.date, slot_minutes: list[int]) -> np.
    ndarray:
106     """把附件3 的整点预报摊到 144 槽 (方案丙: 槽起点 t 取满足 起点 ≤ t < 起点+60 的整点值, 保持
        法)。
107
108     发布时刻 0:00 的 h=1..24 指向当日 1:00...次日 0:00; 槽 j 的起点为 10j 分钟。
109     """
110     base = fc[(day, "0:00")]

```

```

111 out = np.zeros(N_SLOT)
112 for j, start in enumerate(slot_minutes):
113     if start <= 0:
114         continue
115     H = (start // 60) * 60          # §14.4 保持法：取"起点所在小时区间的左端整点"（floor
    , 不是 ceil）
116     h = H // 60                    # 该整点相对 0:00 发布的预见期（小时）
117     out[j] = base[h - 1] if 1 <= h <= 24 else base[0]    # h=0 ([0:10,1:00)) 退回 1:00 值
    （已声明的最小配置简化）
118 return out
119
120
121 def persistence_from_actual(prev_row: np.ndarray) -> np.ndarray:
122     """预报驱动分支的最小规则：最近已完整观测的前一日同槽值（S5 用同星期均值，正式配置再启用）。"""
123     return prev_row.copy()
124
125
126 def cost(price_kwh: np.ndarray, plan: np.ndarray, adj: np.ndarray | None = None,
127         emg: np.ndarray | None = None) -> dict:
128     """§14.7 结算：f_p(b,a)=p·a+0.5p|a-b|（含紧急 5p·e）；返回分项与总额（元）。"""
129     plan = np.asarray(plan, dtype=float)
130     adj = plan if adj is None else np.asarray(adj, dtype=float)
131     emg = np.zeros_like(plan) if emg is None else np.asarray(emg, dtype=float)
132     down = np.maximum(plan - adj, 0.0)
133     up = np.maximum(adj - plan, 0.0)
134     c_plan = float(np.sum(price_kwh * plan))
135     c_net_adj = float(np.sum(-price_kwh * down + K_DOWN * price_kwh * down + K_UP *
    price_kwh * up))
136     c_emg = float(np.sum(K_EMG * price_kwh * emg))
137     return {
138         "plan": c_plan, "net_adj": c_net_adj, "emg": c_emg,
139         "total": c_plan + c_net_adj + c_emg,
140         "down_kwh": float(down.sum()), "up_kwh": float(up.sum()), "emg_kwh": float(emg.sum
    ()),
141     }
142
143
144 def save_json(path: Path, obj) -> None:
145     path.write_text(json.dumps(obj, ensure_ascii=False, indent=2), encoding="utf-8")

```

C5. 问题 1/模型.py (224 行)

问题 1 单日 LP 模型：购电/储能/弃光决策变量、约束与目标函数，方案丙时段口径

```

1 #!/usr/bin/env python3
2 """最小可运行版本 · 模型层：单日 MILP（方案丙）+ 执行层（§14.6）。
3
4 变量（每槽 13 个，索引见 NAMES；另有 1 个窗口初态 E0）：
5     a 执行/调整购电量，e 紧急购电量，C 充电量，D 放电量，E 槽末储电量，
6     w 弃光，r 合同未消纳，u=|a-b|，np 净缺额正部，nm 负部，y(0-1)，z(0-1)，b 计划购电量
7 约束：执行一致的完整平衡（§14.5）+ 储能递推+容量/功率界+互斥+净额正负互斥。
8 目标：q1 = Σp·a；q2 = Σ(p·b+5p·e)；plan_adj = Σp·b；joint = Σ(p·a+0.5p·u+5p·e) (= §14.7
    总成本)。

```

```

9  """
10
11  from __future__ import annotations
12
13  import time
14
15  import numpy as np
16  from scipy.optimize import Bounds, LinearConstraint, milp
17
18  from 口径 import DT, E_MIN, E_OP, ETA, K_DOWN, K_EMG, K_UP, M_SLOT, N_SLOT
19
20  NAMES = ["a", "e", "C", "D", "E", "w", "r", "u", "np", "nm", "y", "z", "b"]
21  IDX = {n: k for k, n in enumerate(NAMES)}
22  NV = len(NAMES)
23  I_E0 = N_SLOT * NV          # 窗口初态 (0:10 的储电量)
24
25
26  def _var(t: int, name: str) -> int:
27      return t * NV + IDX[name]
28
29
30  def solve_day(price, load_kw, pv_kw, E_start=None, mode="q1", b_fixed=None,
31               E_clock_24=None, time_limit=60.0, mip_rel_gap=1e-6) -> dict:
32      """求解单日模型。
33
34      price/load_kw/pv_kw: 长度 144 (元/kWh、kW、kW)；E_start: 窗口初态 (kWh)，None 表示自由；
35      mode: q1 (问题1, 禁紧急、日循环十字面锚定)、q2 (问题2 计划=执行、允许紧急、终端自由)、
36            plan_adj (问题3 的 0:00 计划, b=a)、joint (问题3 的调整, b 可固定、a 可调)。
37      """
38      ell = np.asarray(load_kw, dtype=float) * DT          # kWh
39      v = np.asarray(pv_kw, dtype=float) * DT              # kWh
40      p = np.asarray(price, dtype=float)                  # 元/kWh
41      abar = ell + M_SLOT                                  # 购电量数值界 ($14.5)
42
43      lb = np.zeros(N_SLOT * NV + 1)
44      ub = np.zeros(N_SLOT * NV + 1)
45      c = np.zeros(N_SLOT * NV + 1)
46      integrality = np.zeros(N_SLOT * NV + 1)
47
48      for t in range(N_SLOT):
49          lb[_var(t, "a")], ub[_var(t, "a")] = 0.0, abar[t]
50          lb[_var(t, "e")], ub[_var(t, "e")] = 0.0, ell[t]
51          lb[_var(t, "C")], ub[_var(t, "C")] = 0.0, M_SLOT
52          lb[_var(t, "D")], ub[_var(t, "D")] = 0.0, M_SLOT
53          lb[_var(t, "E")], ub[_var(t, "E")] = E_MIN, E_OP
54          lb[_var(t, "w")], ub[_var(t, "w")] = 0.0, v[t]
55          lb[_var(t, "r")], ub[_var(t, "r")] = 0.0, abar[t]
56          lb[_var(t, "u")], ub[_var(t, "u")] = 0.0, abar[t]
57          lb[_var(t, "np")], ub[_var(t, "np")] = 0.0, ell[t]
58          lb[_var(t, "nm")], ub[_var(t, "nm")] = 0.0, v[t] + abar[t]
59          lb[_var(t, "y")], ub[_var(t, "y")] = 0.0, 1.0

```

```

60     lb[_var(t, "z")], ub[_var(t, "z")] = 0.0, 1.0
61     lb[_var(t, "b")], ub[_var(t, "b")] = 0.0, abar[t]
62     integrality[_var(t, "y")] = 1
63     integrality[_var(t, "z")] = 1
64     lb[I_E0], ub[I_E0] = E_MIN, E_OP
65
66     rows, lows, highs = [], [], []
67
68     def add(coefs, lo, hi):
69         rows.append(coefs)
70         lows.append(lo)
71         highs.append(hi)
72
73     for t in range(N_SLOT):
74         one = np.zeros(N_SLOT * NV + 1)
75         # (1)  $np - nm + a = \ell - v$ 
76         one[_var(t, "np")] = 1.0
77         one[_var(t, "nm")] = -1.0
78         one[_var(t, "a")] = 1.0
79         add(one, ell[t] - v[t], ell[t] - v[t])
80         # (2)  $e - np + D = 0$ 
81         one = np.zeros(N_SLOT * NV + 1)
82         one[_var(t, "e")] = 1.0
83         one[_var(t, "np")] = -1.0
84         one[_var(t, "D")] = 1.0
85         add(one, 0.0, 0.0)
86         # (3)  $w + r - nm + C = 0$ 
87         one = np.zeros(N_SLOT * NV + 1)
88         one[_var(t, "w")] = 1.0
89         one[_var(t, "r")] = 1.0
90         one[_var(t, "nm")] = -1.0
91         one[_var(t, "C")] = 1.0
92         add(one, 0.0, 0.0)
93         # (4)  $np - U^+ y \leq 0$ 
94         one = np.zeros(N_SLOT * NV + 1)
95         one[_var(t, "np")] = 1.0
96         one[_var(t, "y")] = -ell[t]
97         add(one, -np.inf, 0.0)
98         # (5)  $nm + U^- y \leq U^-$ 
99         one = np.zeros(N_SLOT * NV + 1)
100        one[_var(t, "nm")] = 1.0
101        one[_var(t, "y")] = v[t] + abar[t]
102        add(one, -np.inf, v[t] + abar[t])
103        # (6)  $C - M z \leq 0$  ; (7)  $D + M z \leq M$ 
104        one = np.zeros(N_SLOT * NV + 1)
105        one[_var(t, "C")] = 1.0
106        one[_var(t, "z")] = -M_SLOT
107        add(one, -np.inf, 0.0)
108        one = np.zeros(N_SLOT * NV + 1)
109        one[_var(t, "D")] = 1.0
110        one[_var(t, "z")] = M_SLOT

```

```

111     add(one, -np.inf, M_SLOT)
112     # (8) 储能递推
113     one = np.zeros(N_SLOT * NV + 1)
114     one[_var(t, "E")] = 1.0
115     one[_var(t, "C")] = -ETA
116     one[_var(t, "D")] = 1.0 / ETA
117     if t == 0:
118         one[I_E0] = -1.0
119     else:
120         one[_var(t - 1, "E")] = -1.0
121     add(one, 0.0, 0.0)
122     # (9)  $r - a \leq 0$ 
123     one = np.zeros(N_SLOT * NV + 1)
124     one[_var(t, "r")] = 1.0
125     one[_var(t, "a")] = -1.0
126     add(one, -np.inf, 0.0)
127     # (12)  $u \geq a - b$  ;  $u \geq b - a$ 
128     one = np.zeros(N_SLOT * NV + 1)
129     one[_var(t, "u")] = 1.0
130     one[_var(t, "a")] = -1.0
131     one[_var(t, "b")] = 1.0
132     add(one, 0.0, np.inf)
133     one = np.zeros(N_SLOT * NV + 1)
134     one[_var(t, "u")] = 1.0
135     one[_var(t, "a")] = 1.0
136     one[_var(t, "b")] = -1.0
137     add(one, 0.0, np.inf)
138
139     # b = a (计划即执行; 问题3 的调整阶段 b 固定)
140     for t in range(N_SLOT):
141         one = np.zeros(N_SLOT * NV + 1)
142         one[_var(t, "b")] = 1.0
143         one[_var(t, "a")] = -1.0
144         if mode in ("q1", "q2", "plan_adj") or b_fixed is not None:
145             add(one, 0.0, 0.0)
146         if b_fixed is not None:
147             lb[_var(t, "b")] = ub[_var(t, "b")] = float(b_fixed[t])
148
149     # 目标
150     for t in range(N_SLOT):
151         if mode == "q1":
152             c[_var(t, "a")] = p[t]
153         elif mode == "q2":
154             c[_var(t, "b")] = p[t]
155             c[_var(t, "e")] = K_EMG * p[t]
156         elif mode == "plan_adj":
157             c[_var(t, "b")] = p[t]
158             c[_var(t, "e")] = K_EMG * p[t]
159         elif mode == "joint":
160             c[_var(t, "a")] = p[t]
161             c[_var(t, "u")] = K_DOWN * p[t]

```

```

162         c[_var(t, "e")] = K_EMG * p[t]
163     else:
164         raise ValueError(f"未知 mode: {mode}")
165
166     if mode == "q1":
167         for t in range(N_SLOT):
168             lb[_var(t, "e")] = ub[_var(t, "e")] = 0.0 # 问题1 无紧急购电机制
169             if E_clock_24 is not None: # 字面 24:00 锚定 (E_{d,143})
170                 lb[_var(N_SLOT - 2, "E")] = ub[_var(N_SLOT - 2, "E")] = float(E_clock_24)
171             one = np.zeros(N_SLOT * NV + 1) # 日循环 E_{d,144} = E_{d,0}
172             one[_var(N_SLOT - 1, "E")] = 1.0
173             one[I_E0] = -1.0
174             add(one, 0.0, 0.0)
175
176         if E_start is not None:
177             lb[I_E0] = ub[I_E0] = float(E_start)
178
179     t0 = time.perf_counter()
180     res = milp(c=c, constraints=[LinearConstraint(np.array(rows), np.array(lows), np.array(
181         highs))],
182             integrality=integrality, bounds=Bounds(lb, ub),
183             options={"time_limit": time_limit, "mip_rel_gap": mip_rel_gap, "presolve":
184                 True})
185     wall = time.perf_counter() - t0
186
187     x = res.x if res.x is not None else None
188     out = {"status": int(res.status), "message": str(res.message), "wall_s": wall,
189           "objective": float(res.fun) if res.fun is not None else None,
190           "mip_gap": float(getattr(res, "mip_gap", np.nan)) if res.x is not None else None
191           ,
192           "node_count": int(getattr(res, "mip_node_count", getattr(res, "node_count", -1))
193               or -1),
194           "x": x, "mode": mode}
195     if x is None:
196         out["ok"] = False
197         return out
198     out["ok"] = int(res.status) == 0
199     for name in NAMES:
200         out[name] = np.array([x[_var(t, name)] for t in range(N_SLOT)])
201     out["E0"] = float(x[I_E0])
202     out["E_all"] = np.concatenate([out["E0"], out["E"]]) # E_0..E_144 (窗_end)
203     return out
204
205 def execute_day(plan, target_C, target_D, load_kw, pv_kw, E_start) -> dict:
206     """§14.6 执行层: 给定锁定计划与储能目标, 用实际量逐槽执行安全修正。"""
207     ell = np.asarray(load_kw, dtype=float) * DT
208     v = np.asarray(pv_kw, dtype=float) * DT
209     a = np.asarray(plan, dtype=float)
210     C_bar = np.asarray(target_C, dtype=float)
211     D_bar = np.asarray(target_D, dtype=float)

```

```

209 C = np.zeros(N_SLOT); D = np.zeros(N_SLOT); e = np.zeros(N_SLOT)
210 w = np.zeros(N_SLOT); r = np.zeros(N_SLOT); E = np.zeros(N_SLOT + 1)
211 E[0] = float(E_start)
212 for t in range(N_SLOT):
213     n = ell[t] - v[t] - a[t]
214     n_pos, n_neg = max(n, 0.0), max(-n, 0.0)
215     C[t] = min(C_bar[t], M_SLOT, (E_OP - E[t]) / ETA, n_neg)
216     D[t] = min(D_bar[t], M_SLOT, ETA * (E[t] - E_MIN), n_pos)
217     C[t] = max(C[t], 0.0); D[t] = max(D[t], 0.0)
218     e[t] = max(n_pos - D[t], 0.0)
219     s = max(n_neg - C[t], 0.0)
220     w[t] = min(v[t], s)
221     r[t] = s - w[t]
222     E[t + 1] = E[t] + ETA * C[t] - D[t] / ETA
223 return {"a": a, "C": C, "D": D, "e": e, "w": w, "r": r, "E": E}

```

C6. 问题 1/写模板.py (575 行)

问题 1 结果落盘: 写 *result1* 模板 (含时钟口径六段、首日 0:00—0:10 子槽边界恢复、SOC 链闭合与内容级指纹)

```

1 #!/usr/bin/env python3
2 """F1 正式模板落盘: 按官方 5 个模板 (result1/result2/result3/result4-2/result4-3) 填写全年结果。
3
4 口径 (与 §13/§14 及方案丙一致, 供论文"结果说明"引用):
5 - 计划购电量 / 调整购电量: 144 槽 (列标签沿用模板: 0:10-0:20 ... 23:50-0:00+1, 0:00-0:10+1),
6   末两列为"全天购电量" (Σ 该行 144 槽) 与"全天购电费" (Σ 当日各槽结算价 × 购电量)。
7 - 充放电量: 每日 6 个 **真实时钟区间** 4 小时块 (题目表2 口径, 起点 0:00, 沿用模板标签
8   0:00-4:00 ... 20:00-24:00)。方案丙下第 j 槽 = [10j, 10j+10) 分钟、模型窗口 = [0:10, 24:10),
9   故时钟日 [0:00, 24:00) 的槽集合为
10     0:00—4:00 = 前一交易日第 144 槽 (=本日 0:00—0:10) + 本日槽 1—23
11     4:00—8:00 / 8:00—12:00 / 12:00—16:00 / 16:00—20:00 / 20:00—24:00 = 本日槽 24—47
12     / 48—71 /
13     72—95 / 96—119 / 120—143
14 **【2026-09-11 订正】** 原实现按「24 槽自然分组」把槽 1—24 填作 0:00-4:00, 与真实时钟区间
15 相差 10 分钟 (槽 1—24 实际覆盖 0:10—4:10), 12 个汇总数中 8 个不符题目要求。
16 时刻 0:00 / 24:00 的储电量按字面锚定取值 (0:00 = 前一日 E_{d,143}, 24:00 = 当日 E_{d,143})。
17 订正后六个时钟块的 SOC 链须自 0:00 精确闭合到 24:00 (脚本内逐日断言, 见 G9/G10)。
18 - 紧急购电量: 仅列出出现紧急购电的日期; 连续紧急时段合并为区间 (沿用模板表4 格式)。
19 - 数据来源轨 (可执行=预报驱动口径, D2): result2←Q2_可执行、result3←Q3_可执行A3、
20   result4-2←Q4-2_可执行_价格不可知、result4-3←Q4-3_可执行A3_价格不可知;
21   result1 为问题1 单日 (2025-01-01) 问题1 模型解 (mode="q1", 日循环 + 字面 24:00 锚定 6000 kwh)。
22
23 用法:
24   python "03_代码/正式实验/写模板.py"
25
26 from __future__ import annotations
27
28 import csv

```

```

29 import datetime
30 import json
31 import sys
32 from pathlib import Path
33
34 import numpy as np
35
36 try:
37     sys.stdout.reconfigure(encoding="utf-8", errors="replace")
38 except Exception:
39     pass
40
41 BASE = Path(r"C:\Users\John\Desktop\26题")
42 sys.path.insert(0, str(BASE / "03_代码" / "最小运行"))
43
44 from openpyxl import load_workbook # noqa: E402
45
46 from 口径 import (ATT, DT, ETA, E_START_1JAN, N_SLOT, ROOT, load_attachment1, # noqa: E402
47                   load_attachment4, sha256)
48 from 模型 import solve_day # noqa: E402
49 from 正式运行 import DEC_END, FEB_START # noqa: E402
50
51 OUT = ROOT / "04_结果与检验" / "results"
52 DST = OUT # 题目要求的结果文件按《目录与交付物映射.md》第五
           节放在 results/
53 TPL = ATT / "附件5"
54 LOG = ROOT / "04_结果与检验" / "检验日志"
55
56 # ---- 题目表2 的真实时钟区间（方案丙）-----
57 # 第 j 槽 = [10j, 10j+10) 分钟；模型窗口 = [0:10, 24:10)。时钟日 [0:00, 24:00) 恰含 144 槽：
58 # 块0 = 前一交易日第 144 槽（本日 0:00—0:10）+ 本日槽 1—23；块1—5 = 本日本日槽 24—47 …
   120—143。
59 CLOCK_LABELS = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00", "16:00-20:00", "
   20:00-24:00"]
60 CLOCK_TAIL = 144 # 1-based: 本日 0:00—0:10 子槽（属前一交易日）
61 CLOCK_HEAD = (1, 23) # 1-based 闭区间：本日 0:10—4:00
62 CLOCK_BLOCKS = [None, (24, 47), (48, 71), (72, 95), (96, 119), (120, 143)] # 1-based 闭区间
63 FIELD_MAP = {"plan": "plan_kwh", "adj": "adj_kwh", "C": "charge_kwh", "D": "discharge_kwh",
64              "e": "emg_kwh", "w": "curtail_kwh", "Eend": "E_end_kwh"}
65
66
67 _buf: list[str] = []
68 _checks: list[tuple[str, bool, str]] = []
69
70
71 def log(*parts):
72     line = " ".join(str(p) for p in parts)
73     print(line)
74     _buf.append(line)
75
76

```

```

77 def record(name: str, ok: bool, detail: str = ""):
78     _checks.append((name, bool(ok), detail))
79     log(f"[{'PASS' if ok else 'FAIL'}] {name}" + (f" | {detail}" if detail else ""))
80
81
82 # ----- 读取正式运行产物
83 def read_slots(track: str) -> dict:
84     """逐槽明细 → {date: {plan/adj/C/D/e/w/Eend: ndarray(144)}}"""
85     path = OUT / f"正式运行_逐槽_{track}.csv"
86     if not path.exists():
87         raise FileNotFoundError(f"缺少逐槽文件: {path}")
88     data: dict = {}
89     with path.open(encoding="utf-8-sig") as fh:
90         for r in csv.DictReader(fh):
91             d = datetime.date.fromisoformat(r["date"])
92             if d not in data:
93                 data[d] = {k: np.zeros(N_SLOT) for k in FIELD_MAP}
94             j = int(r["slot"]) - 1
95             for k, col in FIELD_MAP.items():
96                 data[d][k][j] = float(r[col])
97     return data
98
99
100 def read_daily(track: str) -> list[dict]:
101     path = OUT / f"正式运行_逐日_{track}.csv"
102     if not path.exists():
103         raise FileNotFoundError(f"缺少逐日文件: {path}")
104     with path.open(encoding="utf-8-sig") as fh:
105         return list(csv.DictReader(fh))
106
107
108 def day_of(s: str) -> datetime.date:
109     return datetime.date.fromisoformat(str(s)[:10])
110
111
112 def content_fingerprint(path) -> str:
113     """xlsx 的**内容级**指纹: 全部非空单元格(含工作表名、坐标与规范化值)。
114
115     存在理由: 实测 openpyxl 写出的 xlsx **字节级不可复现**——同一逻辑连续两次运行,
116     5 个文件的 SHA-256 与字节数全部变化(如 result3 799,468→799,467 B),
117     但内容级指纹与非空格数完全一致。故「结果数值未被修改」应以本指纹为准,
118     字节 SHA-256 仅供参考。
119     """
120     import hashlib
121     wb = load_workbook(path, data_only=True)
122     h = hashlib.sha256()
123     for ws in wb.worksheets:
124         h.update(ws.title.encode("utf-8"))
125         for row in ws.iter_rows():
126             for c in row:
127                 if c.value is None:

```

```

128         continue
129         s = f"{c.value:.10g}" if isinstance(c.value, float) else str(c.value)
130         h.update(f"{c.coordinate}={s}\x1f".encode("utf-8"))
131     wb.close()
132     return h.hexdigest().upper()
133
134
135 # ----- 时钟口径六段
136 def recover_dawn_slot(e_before: float, e_after: float) -> tuple[float, float]:
137     """由两个边界状态反推 [0:00, 0:10] 子槽的 (充电量, 放电量)。
138
139     依据: 同时段充放电互斥 + SOC 递推  $E_{\text{after}} = E_{\text{before}} + 0.9 \cdot C - D/0.9$ 。
140     两边界状态已知且  $C \cdot D = 0$  时,  $(C, D)$  唯一确定。
141     用途: 交付区间首日 (2025-02-01) 的 0:00—0:10 属 1 月 31 日第 144 槽,
142     而 1 月逐槽明细按约定不持久化; 本函数给出可复核的精确恢复值。
143     """
144     de = e_after - e_before
145     return (de / ETA, 0.0) if de >= 0 else (0.0, -de * ETA)
146
147
148 def clock_blocks_of_day(sd: dict, days: list, k: int) -> tuple[list, list]:
149     """第 k 日 (0-based) 六个 **真实时钟区间** 的 (充电量列表, 放电量列表)。"""
150     d = days[k]
151     C, D = sd[d]["C"], sd[d]["D"]
152     head_c = float(C[CLOCK_HEAD[0] - 1:CLOCK_HEAD[1]].sum())
153     head_d = float(D[CLOCK_HEAD[0] - 1:CLOCK_HEAD[1]].sum())
154     if k > 0:
155         Cp, Dp = sd[days[k - 1]]["C"], sd[days[k - 1]]["D"]
156         tail_c, tail_d = float(Cp[CLOCK_TAIL - 1]), float(Dp[CLOCK_TAIL - 1])
157     else:
158         tail_c, tail_d = sd[d]["dawn_C"], sd[d]["dawn_D"]
159     out_c, out_d = [tail_c + head_c], [tail_d + head_d]
160     for (s, e_) in CLOCK_BLOCKS[1:]:
161         out_c.append(float(C[s - 1:e_].sum()))
162         out_d.append(float(D[s - 1:e_].sum()))
163     return out_c, out_d
164
165
166 def clock_soc_chain(sd: dict, days: list) -> float:
167     """逐日六段时钟 SOC 链自 0:00 到 24:00 的最大闭合误差 (kWh)。"""
168     worst = 0.0
169     for k, d in enumerate(days):
170         c_, d_ = clock_blocks_of_day(sd, days, k)
171         soc = float(sd[d]["E_clock_0"])
172         for bi in range(6):
173             soc = soc + ETA * c_[bi] - d_[bi] / ETA
174         worst = max(worst, abs(soc - float(sd[d]["E_clock_24"])))
175     return worst
176
177
178

```

```

179 # ----- 通用写入
180 def clear_rows(ws, r0: int, r1: int, ncol: int):
181     """清空 [r0, r1] 行的前 ncol 列。
182
183     注意：必须用 `cell(...).value = None` **显式赋值**——openpyxl 的 `cell(..., value=None)`
184     与直接读取一样不会改动单元格（P2 质检 P1-1 实测：用 value=None 传参时清空是空操作，
185     导致模板示例行残留 `...`、`2025-02-02`、`2025-12-31`、`24:00` 等旧内容）。
186     """
187     for r in range(r0, r1 + 1):
188         for c in range(1, ncol + 1):
189             ws.cell(row=r, column=c).value = None
190
191
192 def fill_plan_sheet(ws, days, qty: dict, price_of):
193     """计划购电量/调整购电量：col1 日期，col2..145 槽值，col146 全天购电量，col147 全天购电费。
194     """
195     for k, d in enumerate(days):
196         r = 2 + k
197         ws.cell(row=r, column=1, value=datetime.datetime(d.year, d.month, d.day))
198         q = qty[d]
199         p = price_of(d)
200         for j in range(N_SLOT):
201             ws.cell(row=r, column=2 + j, value=round(float(q[j]), 4))
202             ws.cell(row=r, column=146, value=round(float(np.sum(q)), 4))
203             ws.cell(row=r, column=147, value=round(float(np.sum(p * q)), 4))
204
205
206 def fill_cd_sheet(ws, days, sd: dict, date_col: bool):
207     """充放电量：每日 6 行（**真实时钟区间** 4 小时块）；时刻 0:00/24:00 行带储电量。
208     列偏移：date_col=True 时 1=日期、2=时间段、3=充电量、4=放电量、5=时刻、6=储电量；
209     date_col=False（result1）时 1=时间段、2=充电量、3=放电量、4=时刻、5=储电量。
210     时间段标签先从模板首个日块读出（保持与官方模板逐字一致），再为**每个日块**写入
211     ——否则模板示例行之外的日块会缺失 `0:00-4:00`...`20:00-24:00` 标签（G8 残留扫描发现）。
212     【2026-09-11 订正】块内槽集合改为真实时钟区间（见 CLOCK_BLOCKS 与 clock_blocks_of_day）。
213     """
214     c0 = 1 if date_col else 0
215     labels = [ws.cell(row=2 + b, column=c0 + 1).value for b in range(len(CLOCK_LABELS))]
216     if any(v is None for v in labels):
217         raise RuntimeError("充放电量模板缺少 4 小时段标签，无法按官方格式填写")
218     if [str(v) for v in labels] != CLOCK_LABELS:
219         raise RuntimeError(f"充放电量模板段标签与题目表2 时钟区间不符：{labels}")
220     if not date_col:
221         clear_rows(ws, 2, max(ws.max_row, 1 + 6 * len(days)), 5)
222     for k, d in enumerate(days):
223         base = 2 + 6 * k
224         blk_c, blk_d = clock_blocks_of_day(sd, days, k)
225         for b in range(6):
226             r = base + b
227             if date_col:
228                 # 仅日块首行写日期；**续行必须显式清空**——模板的 12-31 示例块首行会落在
229                 # 新布局的某日续行上（P2 修正版复核 P1-R2a: result2/4-2 的 R15C1、result3/4-3

```

的 R21C1

```
229         # 残留 '2025-12-31') , 不显式清空会留下合法但错误的日期。
230         ws.cell(row=r, column=1,
231                 value=datetime.datetime(d.year, d.month, d.day) if b == 0 else None
232     )
233     if b:
234         ws.cell(row=r, column=1).value = None
235         ws.cell(row=r, column=c0 + 1, value=labels[b])
236         ws.cell(row=r, column=c0 + 2, value=round(blk_c[b], 4))
237         ws.cell(row=r, column=c0 + 3, value=round(blk_d[b], 4))
238         if b >= 2:
239             ws.cell(row=r, column=c0 + 4).value = None        # 显式清空 (value=None 传参是
空操作)
240             ws.cell(row=r, column=c0 + 5).value = None
241             ws.cell(row=base, column=c0 + 4, value=datetime.time(0, 0))
242             ws.cell(row=base, column=c0 + 5, value=round(float(sd[d]["E_clock_0"]), 4))
243             ws.cell(row=base + 1, column=c0 + 4, value="24:00")
244             ws.cell(row=base + 1, column=c0 + 5, value=round(float(sd[d]["E_clock_24"]), 4))
245
246 def emergency_intervals(e_arr: np.ndarray) -> list[tuple[int, int, float]]:
247     """连续紧急槽合并为 (起始分钟, 结束分钟, kWh) 区间; 槽 j 覆盖 [10(j+1), 10(j+2)) 分钟。"""
248     out = []
249     t = 0
250     while t < N_SLOT:
251         if e_arr[t] > 1e-9:
252             s = t
253             while t + 1 < N_SLOT and e_arr[t + 1] > 1e-9:
254                 t += 1
255             out.append((10 * (s + 1), 10 * (t + 2), float(e_arr[s:t + 1].sum())))
256         t += 1
257     return out
258
259
260 def fill_emg_sheet(ws, days, sd: dict) -> tuple[int, float]:
261     """紧急购电量: 仅列有紧急购电的日期; 日期只写在当日首行。"""
262     clear_rows(ws, 2, max(ws.max_row, 2), 3)
263     row = 2
264     total = 0.0
265     for d in days:
266         ivs = emergency_intervals(sd[d]["e"])
267         for k, (m0, m1, kWh) in enumerate(ivs):
268             if k == 0:
269                 ws.cell(row=row, column=1, value=datetime.datetime(d.year, d.month, d.day))
270             else:
271                 ws.cell(row=row, column=1).value = None        # 同日后继行必须显式清空
272                 ws.cell(row=row, column=2, value=f"{m0 // 60}:{m0 % 60:02d}-{m1 // 60}:{m1 %
60:02d}")
273                 ws.cell(row=row, column=3, value=round(kWh, 4))
274                 total += kWh
275                 row += 1
```

```

276     return row - 2, total
277
278
279 # ----- result1 (问题1 单日)
280 def write_result1() -> dict:
281     a1 = load_attachment1()
282     q1 = solve_day(a1["price"], a1["load_kw"], a1["pv_kw"], E_start=None, mode="q1",
283                   E_clock_24=E_START_1JAN)
284     if not q1["ok"]:
285         raise RuntimeError(f"问题1 求解失败: {q1['status']} {q1['message']}")
286     wb = load_workbook(TPL / "result1.xlsx")
287     ws = wb["计划购电量"]
288     for j in range(N_SLOT):
289         ws.cell(row=2 + j, column=2, value=round(float(q1["b"][j]), 4))
290     ws_cd = wb["充放电量"]
291     # 【2026-09-11 订正】真实时钟区间: 块0 = 槽144 + 槽1—23, 块1—5 = 槽24—47 ... 槽120—143。
292     # 问题1 为「典型日周期重复」, 前一日第 144 槽即本日第 144 槽, 与 result1_时钟口径修正版.py 一致。
293     C1, D1 = q1["C"], q1["D"]
294     blk_c = [float(C1[CLOCK_TAIL - 1]) + float(C1[CLOCK_HEAD[0] - 1:CLOCK_HEAD[1]].sum())]
295     blk_d = [float(D1[CLOCK_TAIL - 1]) + float(D1[CLOCK_HEAD[0] - 1:CLOCK_HEAD[1]].sum())]
296     for (s, e_) in CLOCK_BLOCKS[1:]:
297         blk_c.append(float(C1[s - 1:e_].sum()))
298         blk_d.append(float(D1[s - 1:e_].sum()))
299     for k in range(6):
300         ws_cd.cell(row=2 + k, column=2, value=round(blk_c[k], 4))
301         ws_cd.cell(row=2 + k, column=3, value=round(blk_d[k], 4))
302     e00 = float(E_START_1JAN) # 字面 0:00 (= 前一日 E_{d,143}, 问题1 由锚定给出 6000)
303     e24 = float(q1["E_all"][-2]) # 字面 24:00 = 当日 E_{d,143}
304     ws_cd.cell(row=2, column=5, value=round(e00, 4))
305     ws_cd.cell(row=3, column=5, value=round(e24, 4))
306     dst = DST / "result1.xlsx"
307     wb.save(dst)
308     wb.close()
309
310     # 时钟口径 SOC 链自 0:00 闭合到 24:00 (问题1 的独立判据)
311     soc = e00
312     for k in range(6):
313         soc = soc + ETA * blk_c[k] - blk_d[k] / ETA
314     chain_err = abs(soc - e24)
315
316     rb = load_workbook(dst, data_only=True)
317     p = rb["计划购电量"]
318     cd = rb["充放电量"]
319     ok_slots = all(p.cell(row=2 + j, column=2).value is not None for j in range(N_SLOT))
320     ok_label = p.cell(row=61, column=1).value == "10:00-10:10" and p.cell(row=145, column=1).value == "0:00+1-0:10+1"
321     ok_val = abs(float(p.cell(row=61, column=2).value) - q1["b"][59]) < 1e-6
322     cd_labels = [str(cd.cell(row=2 + b, column=1).value) for b in range(6)]
323     # 容差 1e-4: 模板按 4 位小数存储, 单值舍入上界为 5e-5, 取 2 倍留裕度 (实测最大偏差恰为 5.0e-5)

```

```

) 。
324 ok_cd = all(abs(float(cd.cell(row=2 + b, column=2).value) - blk_c[b]) < 1e-4 for b in
range(6)) and \
325 all(abs(float(cd.cell(row=2 + b, column=3).value) - blk_d[b]) < 1e-4 for b in range
(6)) and \
326 abs(float(cd.cell(row=2, column=5).value) - e00) < 1e-6 and \
327 abs(float(cd.cell(row=3, column=5).value) - e24) < 1e-6
328 record("G1 result1 落盘且回读一致 (144 槽 + 6 个时钟区间 + 0:00/24:00 储电量)",
329 ok_slots and ok_label and ok_val and ok_cd,
330 f"费用 {q1['objective']:.4f} 元; 0:00={e00:.4f}、24:00={e24:.4f} kWh; "
331 f"E0={q1['E_all'][0]:.4f} E144={q1['E_all'][-1]:.4f}")
332 record("G1b result1 六段为实时时钟区间且 SOC 链自 0:00 闭合到 24:00",
333 cd_labels == CLOCK_LABELS and chain_err < 1e-6,
334 f"标签={cd_labels}; 链误差 {chain_err:.3e} kWh")
335 rb.close()
336 return {"file": dst, "objective": q1["objective"]}
337
338
339 # ----- result2/4-2 与 result3
/4-3
340 def write_year_template(track: str, tpl_name: str, dst_name: str, price_src: str,
341 with_adjust: bool) -> dict:
342 daily = read_daily(track)
343 sd = read_slots(track)
344 days = [day_of(r["date"]) for r in daily]
345 if len(days) != 334:
346 record(f"{dst_name} 天数检查", False, f"逐日文件天数 {len(days)} (期望 334)")
347 price_days = None
348 if price_src == "附件1":
349 a1 = load_attachment1()
350 price_of = lambda d: a1["price"] # noqa: E731
351 else:
352 a4 = load_attachment4()
353 price_days = a4
354 price_of = lambda d: a4["price"][a4["index"][d]] # noqa: E731
355
356 for r in daily:
357 d = day_of(r["date"])
358 sd[d]["E_clock_0"] = float(r["E_clock_0"])
359 sd[d]["E_actual_start"] = float(r["E_actual_start"])
360 sd[d]["E_clock_24"] = float(r["E_clock_24"])
361 sd[d]["plan_ref"] = float(r["plan_kwh"])
362 sd[d]["adj_ref"] = float(r["adj_kwh"])
363 sd[d]["emg_ref"] = float(r["emg_kwh"])
364 sd[d]["cost_plan_ref"] = float(r["cost_plan"])
365
366 # ---- 0:00—0:10 子槽 (属前一交易日第 144 槽) -----
367 # 交付首日 2025-02-01 的子槽属 2025-01-31 第 144 槽, 1 月逐槽明细按约定不持久化,
368 # 故由两个边界状态 (0:00 与 0:10 储电量) 精确恢复; 其余 333 日用持久化值校验该恢复法。
369 worst_rec = 0.0
370 for k, d in enumerate(days):

```

```

371     c_r, d_r = recover_dawn_slot(float(sd[d]["E_clock_0"]), float(sd[d]["E_actual_start
372     "]))
373     if k > 0:
374         Cp, Dp = sd[days[k - 1]]["C"], sd[days[k - 1]]["D"]
375         worst_rec = max(worst_rec, abs(c_r - float(Cp[CLOCK_TAIL - 1])),
376                         abs(d_r - float(Dp[CLOCK_TAIL - 1])))
377     else:
378         sd[d]["dawn_C"], sd[d]["dawn_D"] = c_r, d_r
379 record(f"G9 {dst_name} 首日 0:00—0:10 子槽由边界状态恢复; 与其余 333 日持久化值一致",
380       worst_rec < 1e-6,
381       f"333 日最大偏差 {worst_rec:.3e} kWh; 首日取 充 {sd[days[0]]['dawn_C']:.4f} / "
382       f"放 {sd[days[0]]['dawn_D']:.4f} kWh")
383 chain_err = clock_soc_chain(sd, days)
384 record(f"G10 {dst_name} 六个时钟区间 SOC 链自 0:00 精确闭合到 24:00 (334 日)",
385       chain_err < 1e-6, f"最大链误差 {chain_err:.3e} kWh")
386
387 wb = load_workbook(TPL / tpl_name)
388 plan_qty = {d: sd[d]["plan"] for d in days}
389 fill_plan_sheet(wb["计划购电量"], days, plan_qty, price_of)
390 if with_adjust:
391     adj_qty = {d: sd[d]["adj"] for d in days}
392     fill_plan_sheet(wb["调整购电量"], days, adj_qty, price_of)
393 fill_cd_sheet(wb["充放电量"], days, sd, date_col=True)
394 n_emg, emg_sum = fill_emg_sheet(wb["紧急购电量"], days, sd)
395 dst = DST / dst_name
396 wb.save(dst)
397 wb.close()
398
399 # ---- 回读核验 ----
400 rb = load_workbook(dst, data_only=True)
401 wsb = rb["计划购电量"]
402 wsa = rb["调整购电量"] if with_adjust else None
403 wsc = rb["充放电量"]
404 wse = rb["紧急购电量"]
405 max_plan = max_adj = 0.0
406 max_tot = max_fee = 0.0
407 dates_ok = True
408 for k, d in enumerate(days):
409     r = 2 + k
410     v = wsb.cell(row=r, column=1).value
411     dates_ok &= (hasattr(v, "date") and v.date() == d)
412     arr = np.array([float(wsb.cell(row=r, column=2 + j).value) for j in range(N_SLOT)])
413     max_plan = max(max_plan, float(np.abs(arr - sd[d]["plan"]).max()))
414     max_tot = max(max_tot, abs(float(wsb.cell(row=r, column=146).value) - float(arr.sum
415     ())))
416     p = price_of(d)
417     max_fee = max(max_fee, abs(float(wsb.cell(row=r, column=147).value) - float(np.sum(
418     p * arr))))
419     if wsa is not None:
420         arr_a = np.array([float(wsa.cell(row=r, column=2 + j).value) for j in range(
421         N_SLOT)])

```

```

418         max_adj = max(max_adj, float(np.abs(arr_a - sd[d]["adj"]).max()))
419 # 充放电量回读（首日与末日各抽验）：列 3/4=充电量/放电量，列 5=时刻，列 6=储电量
420 cd_ok = True
421 for k in (0, len(days) - 1):
422     d = days[k]
423     base = 2 + 6 * k
424     blk_c, blk_d = clock_blocks_of_day(sd, days, k)
425     for b in range(6):
426         cw = float(wsc.cell(row=base + b, column=3).value)
427         dw = float(wsc.cell(row=base + b, column=4).value)
428         cd_ok &= abs(cw - blk_c[b]) < 1e-3
429         cd_ok &= abs(dw - blk_d[b]) < 1e-3
430     cd_ok &= abs(float(wsc.cell(row=base, column=6).value) - sd[d]["E_clock_0"]) < 1e-3
431     cd_ok &= abs(float(wsc.cell(row=base + 1, column=6).value) - sd[d]["E_clock_24"]) <
1e-3
432     cd_ok &= str(wsc.cell(row=base + 1, column=5).value) == "24:00"
433 # 紧急购电量回读：逐行合计 vs 逐日文件合计
434 emg_file = sum(sd[d]["emg_ref"] for d in days)
435 emg_read = 0.0
436 r = 2
437 while wse.cell(row=r, column=3).value is not None:
438     emg_read += float(wse.cell(row=r, column=3).value)
439     r += 1
440 fee_sum = sum(float(wsb.cell(row=2 + k, column=147).value) for k in range(len(days)))
441 cost_plan_sum = sum(sd[d]["cost_plan_ref"] for d in days)
442 record(f"G2 {dst_name} 计划购电量 334 行×144 槽回读一致", dates_ok and max_plan < 1e-4
and max_tot < 1e-2,
443         f"日期齐全={dates_ok} 最大偏差={max_plan:.2e} 行合计偏差={max_tot:.2e}")
444 if with_adjust:
445     record(f"G3 {dst_name} 调整购电量回读一致", max_adj < 1e-4, f"最大偏差={max_adj:.2e}")
446 record(f"G4 {dst_name} 充放电量回读一致（首末日在 6 个时钟区间与 0:00/24:00 上抽验）",
cd_ok)
447 # 残留内容扫描（P2 质检 P1-1 的针对性回归）：占位符、越界日期、错位时刻
448 residual = []
449 for ws in rb.worksheets:
450     for row in ws.iter_rows():
451         for cell in row:
452             v = cell.value
453             if isinstance(v, str) and v.strip() == "...":
454                 residual.append(f"{ws.title}!{cell.coordinate} 残留占位符")
455 cd_res = rb["充放电量"]
456 c0 = 0 if "日期" not in [cd_res.cell(row=1, column=1).value] else 1
457 for k in range(len(days)):
458     base = 2 + 6 * k
459     # 年份模板的充放电量日为 6 行块：日期只在块首行（列 1），续行必须为空
460     for b in range(6):
461         r = base + b
462         v1 = cd_res.cell(row=r, column=1).value
463         if b == 0:
464             if not hasattr(v1, "date") or v1.date() != days[k]:
465                 residual.append(f"充放电量!A{r} 日块首行日期错误 {v1}")

```

```

466         elif v1 is not None:
467             residual.append(f"充放电!A{r} 续行应空却为 {v1}")
468     for b in range(6):
469         if cd_res.cell(row=base + b, column=c0 + 1).value in (None, ""):
470             residual.append(f"充放电!R{base + b}C{c0 + 1} 缺时间段标签")
471         if cd_res.cell(row=base + b, column=c0 + 2).value is None:
472             residual.append(f"充放电!R{base + b}C{c0 + 2} 缺充电量")
473     for b in range(2, 6):
474         for col in (c0 + 4, c0 + 5):
475             if cd_res.cell(row=base + b, column=col).value is not None:
476                 residual.append(f"充放电!R{base + b}C{col} 应空却非空")
477     for col in (c0 + 4, c0 + 5):
478         if cd_res.cell(row=base, column=col).value is None:
479             residual.append(f"充放电!R{base}C{col} 应有值却为空")
480     if cd_res.cell(row=base + 1, column=c0 + 4).value != "24:00":
481         residual.append(f"充放电!R{base + 1}C{c0 + 4} 24:00 标签缺失")
482     emg_ws = rb["紧急购电量"]
483     prev_date = None
484     r = 2
485     while emg_ws.cell(row=r, column=3).value is not None:
486         v1 = emg_ws.cell(row=r, column=1).value
487         if v1 is not None:
488             if not hasattr(v1, "date") or not (FEB_START <= v1.date() <= DEC_END):
489                 residual.append(f"紧急购电量!A{r} 非法日期 {v1}")
490                 prev_date = v1
491             elif prev_date is None:
492                 residual.append(f"紧急购电量!A{r} 首行缺日期")
493             r += 1
494     record(f"G8 {dst_name} 无模板残留（占位符 / 错位时刻 / 越界日期）", not residual,
495           f"残留项 {len(residual)}" + (f": {residual[:4]}" if residual else "（占位符、充放
496           电量空行、紧急购电日期分组均正确）"))
497     record(f"G5 {dst_name} 紧急购电量区间合计与逐日文件一致",
498           abs(emg_read - emg_file) < 1e-3 * max(1, n_emg) and n_emg > 0,
499           f"区间行 {n_emg} 行、合计 {emg_read:,.2f} kWh; 逐日文件 {emg_file:,.2f} kWh; "
500           f"偏差 {abs(emg_read - emg_file):.4f} kWh（4 位小数舍入容差）")
501     record(f"G6 {dst_name} 全天购电费合计与逐日文件 cost_plan 一致",
502           abs(fee_sum - cost_plan_sum) < 10.0,
503           f"表内 {fee_sum:,.2f} 元; 逐日文件计划电费合计 {cost_plan_sum:,.2f} 元; "
504           f"差 {abs(fee_sum - cost_plan_sum):.2f} 元（逐槽 4 位小数舍入）; 逐行最大偏差={
505           max_fee:.2e}")
506     if price_days is not None:
507         record(f"G7 {dst_name} 价格源为附件4 当日实际电价（价格不可知轨的计划价不用于结算列）",
508               True,
509               f"附件4 覆盖 {len(price_days['index'])} 日")
510     rb.close()
511     return {"file": dst.name, "track": track, "days": len(days), "emg_rows": n_emg, "
512           emg_kwh": emg_read,
513           "fee_sum": fee_sum, "cost_plan_file": cost_plan_sum}
514
515 def main() -> int:

```

```

513 DST.mkdir(parents=True, exist_ok=True)
514 log("=" * 96)
515 log(f"F1 正式模板落盘 | {datetime.datetime.now():%Y-%m-%d %H:%M:%S}")
516 log(f"解释器 {sys.executable} | 输出目录 {DST}")
517 log(f"输入哈希: 附件1 {sha256(ATT / '附件1.xlsx')[:16]} 附件4 {sha256(ATT / '附件4.xlsx')[:16]}")
518 log("=" * 96)
519
520 manifest = {}
521 r1 = write_result1()
522 manifest["result1"] = {"file": r1["file"].name, "track": "问题1 单日 2025-01-01 (mode=q1)",
523                        "objective_yuan": r1["objective"]}
524 manifest["result2"] = write_year_template("Q2_可执行", "result2.xlsx", "result2.xlsx", "附件1", False)
525 manifest["result3"] = write_year_template("Q3_可执行A3", "result3.xlsx", "result3.xlsx", "附件1", True)
526 manifest["result4-2"] = write_year_template("Q4-2_可执行_价格不可知", "result4-2.xlsx", "result4-2.xlsx",
527                                             "附件4", False)
528 manifest["result4-3"] = write_year_template("Q4-3_可执行A3_价格不可知", "result4-3.xlsx", "result4-3.xlsx",
529                                             "附件4", True)
530
531 log("-" * 96)
532 files = {}
533 for key in ("result1", "result2", "result3", "result4-2", "result4-3"):
534     p = DST / f"{key}.xlsx"
535     files[key] = {"bytes": p.stat().st_size, "sha256": sha256(p),
536                  "content_sha256": content_fingerprint(p)}
537     log(f" {key}.xlsx {p.stat().st_size:>9,} B {sha256(p)[:32]}... 内容指纹 {files[key]['content_sha256'][:32]}...")
538 srcs = {}
539 for tr in ("Q2_可执行", "Q3_可执行A3", "Q4-2_可执行_价格不可知", "Q4-3_可执行A3_价格不可知"):
540     for kind in ("逐日", "逐槽"):
541         p = OUT / f"正式运行_{kind}_{tr}.csv"
542         if p.exists():
543             srcs[p.name] = {"bytes": p.stat().st_size, "sha256": sha256(p)}
544 tpl_src = {f"附件5/{n}": sha256(TPL / n) for n in
545            ("result1.xlsx", "result2.xlsx", "result3.xlsx", "result4-2.xlsx", "result4-3.xlsx")}
546 man = {"阶段": "F1 正式模板落盘", "生成时间": f"{datetime.datetime.now():%Y-%m-%d %H:%M:%S}",
547        "来源轨": manifest, "模板输入哈希": tpl_src, "结果文件": files, "数据来源": srcs,
548        "指纹说明": "sha256 为文件字节哈希, 实测跨次运行不稳定 (openpyxl 写出字节不可复现); "
549        "content_sha256 为全部非空单元格的内容级指纹, 跨次运行稳定, "
550        "判定「结果数值是否被修改」应以 content_sha256 为准。",
551        "口径": {
552            "计划购电量": "原始 0:00 日前计划 b_t (144 槽; 末两列=全天购电量/全天购电费)",
553            "调整购电量": "相对原始 0:00 计划的调整后购电量 a_t (问题3/4-3 直接调整规则)",

```

```

554         "全天购电费": "Σ(当日各槽结算价 × 购电量); 问题2/3 用附件1 电价, 问题4 用附件4 当
        日实际电价",
555         "充放电量": "每日 6 个真实时钟区间 (题目表2): 0:00-4:00 = 前一交易日第144槽 + 本
        日槽1-23; "
556         "4:00-8:00/8:00-12:00/12:00-16:00/16:00-20:00/20:00-24:00 = 本日
        槽 24-47/48-71/72-95/96-119/120-143; "
557         "0:00/24:00 为字面锚定储电量, 六段 SOC 链自 0:00 精确闭合到 24:00 (
        2026-09-11 订正)",
558         "紧急购电量": "仅列出出现紧急购电的日期, 连续时段合并为区间",
559     }}
560     mpath = OUT / "复现清单_F1模板.json"
561     mpath.write_text(json.dumps(man, ensure_ascii=False, indent=2) + "\n", encoding="utf-8"
562     )
563     log(f" 复现清单 {mpath.name} sha256 {sha256(mpath)[:32]}...")
564
565     n_fail = sum(1 for _, ok, _ in _checks if not ok)
566     log("=" * 96)
567     log(f"检查合计 {len(_checks)} 项, FAIL {n_fail} 项")
568     log("=" * 96)
569     LOG.mkdir(parents=True, exist_ok=True)
570     (LOG / "正式模板落盘.txt").write_text("\n".join(_buf) + "\n", encoding="utf-8")
571     return 0 if n_fail == 0 else 2
572
573 if __name__ == "__main__":
574     raise SystemExit(main())

```

C7. 问题 2-3/p10_core.py (409 行)

问题 2/3 共享求解内核：日末库存系数、情景生成、储能递推、费用结算与 LP/MILP 求解

```

1  # -*- coding: utf-8 -*-
2  """
3  核心库：数据读取 / 滚动预测 / 整日残差场景 / 两阶段随机LP内核 / 实时调度 / 结算
4  新思路路线：预测 → 残差场景 → 两阶段随机MILP(实际为LP) → 5倍紧急购电；MPC+调整成本；波动电价。
5  所有模型为 LP (HiGHS, scipy.optimize.linprog)。
6  """
7  import os
8  import numpy as np
9  import pandas as pd
10 import openpyxl
11 from scipy import sparse
12 from scipy.optimize import linprog
13
14 # 路径：默认由本文件位置推导（本文件在 03_代码/, 其上一级即项目根），不再写死绝对路径。
15 # 2026-09-11 修复：原值 r"C:\Users\my\Desktop\MILP+两阶段+MPC" 是搬迁前的旧机器路径，项目移到
16 # C:\Users\John\Desktop\26题\MILP+两阶段+MPC 后不存在，导致所有 p*.py 直接 FileNotFoundError。
17 # 三个环境变量为可选覆盖，便于把流水线重跑到临时目录、不覆盖已交付的 results/ 与检验日志：
18 # MICROGRID_ROOT / MICROGRID_RES / MICROGRID_LOG
19 _HERE = os.path.dirname(os.path.abspath(__file__))
20 ROOT = os.environ.get("MICROGRID_ROOT", os.path.dirname(_HERE))

```

```

21 import sys
22 sys.path.insert(0, os.path.join(ROOT, ".runtime"))
23 SRC = os.path.join(ROOT, "00_题目与原始数据", "C题", "附件")
24 RES = os.environ.get("MICROGRID_RES", os.path.join(ROOT, "04_结果与检验", "results"))
25 LOGD = os.environ.get("MICROGRID_LOG", os.path.join(ROOT, "04_结果与检验", "检验日志"))
26 SEED = 2025
27 DT = 1.0 / 6.0          # 10min in hours
28 ETA_C = 0.9             # charge efficiency (one-way)
29 ETA_D = 0.9             # discharge efficiency (one-way)
30 E_MIN, E_MAX, E_INIT = 1200.0, 10800.0, 6000.0
31 P_MAX_KWH = 5000.0 * DT # max charge/discharge energy per slot (kWh)
32 BIG = 1e7
33 # 吞吐平局价；实际执行另按同SOC消去配对吞吐，核验互斥。
34 TP_PRICE = 1e-6
35
36 # ----- 数据读取
37 def _norm_label(v):
38     if isinstance(v, str):
39         return v.strip()
40     try:
41         return f"{v.hour:02d}:{v.minute:02d}"
42     except Exception:
43         return str(v)
44
45 def load_all():
46     d = {}
47     a1 = pd.read_excel(os.path.join(SRC, "附件1.xlsx"))
48     d["p_fix"] = a1["电价"].to_numpy(float)          # (144,)
49     d["L1"] = a1["小区负载"].to_numpy(float)
50     d["G1hat"] = a1["光伏发电预测功率"].to_numpy(float)
51     a2L = pd.read_excel(os.path.join(SRC, "附件2.xlsx"), sheet_name="小区负载")
52     a2G = pd.read_excel(os.path.join(SRC, "附件2.xlsx"), sheet_name="光伏发电实际功率")
53     d["dates"] = pd.to_datetime(a2L.iloc[:, 0]).dt.date.to_numpy()
54     d["L"] = a2L.iloc[:, 1:].to_numpy(float)         # (365,144) kW
55     d["G"] = a2G.iloc[:, 1:].to_numpy(float)
56     a3 = pd.read_excel(os.path.join(SRC, "附件3.xlsx"))
57     a3[a3.columns[0]] = a3[a3.columns[0]].ffill()
58     d["a3"] = a3
59     a4 = pd.read_excel(os.path.join(SRC, "附件4.xlsx"))
60     d["P_var"] = a4.iloc[:, 1:].to_numpy(float)      # (365,144)
61     # 附件3 索引: (day_idx, release_hour) -> np.array(24)
62     tmap = {"0:00": 0, "6:00": 6, "12:00": 12, "18:00": 18}
63     fcs = {}
64     d0 = pd.Timestamp(d["dates"][0])
65     for _, r in a3.iterrows():
66         di = int((pd.Timestamp(r[a3.columns[0]]) - d0).days)
67         h0 = tmap[str(r[a3.columns[1]]).strip()]
68         fcs[(di, h0)] = r.iloc[2:26].to_numpy(float)
69     d["fc3"] = fcs
70     return d
71

```

```

72 def slot_start_min(j):
73     return 10 * j                # label is interval END: 0:10 -> [0:00,0:10)
74
75 def fmt_min(m):
76     """分钟 -> 标签; 1440 -> '24:00'"""
77     m = int(round(m))
78     if m >= 1440:
79         return "24:00" if m == 1440 else f"{m//60}:{m%60:02d}"
80     return f"{m//60}:{m%60:02d}"
81
82 def fc3_step_day(fcs, di, release_h, slots=None, method="hold"):
83     """当前发布行展开到自然日剩余槽; 小时预报为前一小时的保持值。"""
84     fv = np.asarray(fcs[(di, release_h)])
85     out = np.zeros(144)
86     covered = np.arange(144) >= release_h * 6
87     lead = (np.arange(144)[covered] + 1) / 6 - release_h
88     if method == "hold":
89         out[covered] = fv[np.ceil(lead).astype(int)-1]
90     elif method == "linear":
91         out[covered] = np.interp(lead, np.arange(1, 25), fv)
92     else:
93         raise ValueError(method)
94     return out, covered
95
96 # ----- 预测层
97 FEATURE_LAGS = (1, 2, 7)
98
99 def build_features(X, d_idx_list):
100     """X: (365,144) 原始量; 返回 (rows, feat) 列表。特征严格只用 d 之前的数据。"""
101     rows = []
102     for di in d_idx_list:
103         if di < 7:
104             continue
105         date = pd.Timestamp("2025-01-01") + pd.Timedelta(days=di)
106         dow, doy = date.dayofweek, date.dayofyear
107         for t in range(144):
108             hour = slot_start_min(t) // 60
109             f = [t / 144.0,
110                 np.sin(2 * np.pi * t / 144), np.cos(2 * np.pi * t / 144),
111                 np.sin(2 * np.pi * hour / 24), np.cos(2 * np.pi * hour / 24),
112                 dow / 6.0, doy / 366.0,
113                 X[di - 1, t], X[di - 2, t], X[di - 7, t],
114                 X[di - 7:di, t].mean(), X[di - 7:di, t].std()]
115             rows.append((di, t, f))
116     return rows
117
118 def rolling_forecast(X, refit_every=7, start=14):
119     """LightGBM 扩展窗口滚动预测 (每 refit_every 天重训, start 为首个预测日)。
120     返回 F: (365,144), 不可预测天为 nan。样本外。"""
121     import lightgbm as lgb
122     F = np.full_like(X, np.nan)

```

```

123 cache = {}
124 for di in range(start, 365):
125     rid = (di - 7) // refit_every
126     if rid not in cache:
127         train_rows = build_features(X, list(range(7, di)))
128         A = np.array([r[2] for r in train_rows]); y = np.array([X[r[0], r[1]] for r in
train_rows])
129         m = lgb.LGBMRegressor(n_estimators=300, learning_rate=0.05, num_leaves=63,
130                               min_child_samples=40, subsample=0.9, colsample_bytree
=0.9,
131                               random_state=SEED, verbose=-1, n_jobs=2)
132         m.fit(A, y)
133         cache = {rid: m}
134         # 特征仅用 di 之前数据
135         date = pd.Timestamp("2025-01-01") + pd.Timedelta(days=di)
136         dow, doy = date.dayofweek, date.dayofyear
137         feats = []
138         for t in range(144):
139             hour = slot_start_min(t) // 60
140             feats.append([t / 144.0,
141                          np.sin(2 * np.pi * t / 144), np.cos(2 * np.pi * t / 144),
142                          np.sin(2 * np.pi * hour / 24), np.cos(2 * np.pi * hour / 24),
143                          dow / 6.0, doy / 366.0,
144                          X[di - 1, t], X[di - 2, t], X[di - 7, t],
145                          X[di - 7:di, t].mean(), X[di - 7:di, t].std()])
146         F[di] = cache[rid].predict(np.array(feats))
147     return F
148
149 def baseline_mean7(X):
150     F = np.full_like(X, np.nan)
151     for di in range(7, 365):
152         F[di] = X[di - 7:di].mean(axis=0) # 前7个同槽均值（不含当日）
153     return F
154
155 # ----- LP 内核
156 def solve_plan_lp(p, Ls, Gs, E0, q_old=None, lam=0.0, alpha=0.95,
157                  q_fixed=None, E_term=None, eps_soc=0.0, tp=0.0, v_term=0.0):
158     """两阶段随机计划 LP。
159     p: (T,) 电价; Ls,Gs: (S,T) 场景功率 kW; E0: 期初储量;
160     q_old: (T,) 调整基准(Q3), None=无调整费用(Q2);
161     lam>0: 加入 CVaR_alpha 项（对场景日紧急购电成本）。
162     q_fixed: (T,) 固定购电量（实时调度复用本函数）。
163     eps_soc>0: 末槽储电数值平局奖励（元/kWh, 逐场景, 语义冻结同 v3），在并列解中偏向“不把储能放空”
。
164     v_term>0: 跨日库存经济续存价值（元/kWh 内部电量），按等权期望归一注入：
165         每个场景末槽系数为 -(eps_soc + v_term/S)，与紧急项 5p/S 同口径，
166         保证该参数含义不随情景数 S 变化（q23_janwarm_v4, 修订协议 §13）。
167     tp>0: 充放电吞吐平局价（元/kWh），在并列解中排除“同槽同时充放”伪解。
168     返回 dict(q, qEM(S,T), c(S,T), d(S,T), E(S,T), obj, status, dqp, dqm)"""
169     T = len(p); S = Ls.shape[0]
170     adj = q_old is not None

```

```

171 n = T + 4 * S * T
172 i_q = 0
173 def i_blk(s, k, t): # k:0 qEM,1 c,2 d,3 E
174     return T + s * 4 * T + k * T + t
175 if adj:
176     i_dqp = T + 4 * S * T; i_dqm = i_dqp + T; n += 2 * T
177 if lam > 0:
178     i_eta = n; i_u = n + 1; n += 1 + S
179 c_vec = np.zeros(n)
180 ub_l = np.zeros(n); ub_u = np.zeros(n)
181 # 目标与边界
182 # At an update the original plan has already been paid for. Only the
183 # incremental adjustment bill is decision-dependent; charging p*q again
184 # would count the replacement energy twice.
185 c_vec[i_q:i_q + T] = 0.0 if adj else p
186 ub_u[i_q:i_q + T] = BIG if q_fixed is None else 0.0
187 if q_fixed is not None:
188     ub_l[i_q:i_q + T] = q_fixed
189     ub_u[i_q:i_q + T] = q_fixed
190 for s in range(S):
191     for k in range(4):
192         sl = slice(i_blk(s, k, 0), i_blk(s, k, 0) + T)
193         if k == 0:
194             c_vec[sl] = 5.0 * p / S
195             ub_u[sl] = BIG
196         elif k in (1, 2):
197             ub_u[sl] = P_MAX_KWH
198             if tp > 0:
199                 c_vec[sl] = tp # 吞吐平局价：排除"同槽同时充放"伪解
200         else:
201             ub_l[sl] = E_MIN; ub_u[sl] = E_MAX
202             if v_term < 0:
203                 raise ValueError("v_term 必须非负")
204             # 末端价值按等权期望归一：总末端系数为 eps_soc + v_term,
205             # 不随情景数 S、A/B/C 模型或复制情景改变。
206             coef = (eps_soc + v_term) / S
207             if coef > 0:
208                 c_vec[sl][-1] = -coef # 末槽奖励，消除退化/为日末库存定价
209 if adj:
210     c_vec[i_dqp:i_dqp + T] = 1.5 * p
211     c_vec[i_dqm:i_dqm + T] = -0.5 * p
212     ub_u[i_dqp:i_dqp + T] = BIG; ub_u[i_dqm:i_dqm + T] = BIG
213 if lam > 0:
214     c_vec[i_eta] = lam; c_vec[i_u:i_u + S] = lam / (S * (1 - alpha))
215     ub_l[i_eta] = -BIG; ub_u[i_eta] = BIG; ub_u[i_u:i_u + S] = BIG
216 # 等式：储能递推 (S*T)
217 rows_e, cols_e, vals_e, be = [], [], [], []
218 r = 0
219 for s in range(S):
220     for t in range(T):
221         def add(col, val):

```

```

222         rows_e.append(r); cols_e.append(col); vals_e.append(val)
223     add(i_blk(s, 3, t), 1.0)
224     add(i_blk(s, 1, t), -ETA_C)
225     add(i_blk(s, 2, t), 1.0 / ETA_D)
226     if t > 0:
227         add(i_blk(s, 3, t - 1), -1.0)
228         be.append(0.0)
229     else:
230         be.append(E0)
231     r += 1
232 if adj:
233     for t in range(T):
234         rows_e += [r, r, r]
235         cols_e += [i_q + t, i_dqp + t, i_dqm + t]
236         vals_e += [1.0, -1.0, 1.0]
237         be.append(q_old[t]); r += 1
238 if E_term is not None:
239     for s in range(S):
240         rows_e.append(r); cols_e.append(i_blk(s, 3, T - 1)); vals_e.append(1.0)
241         be.append(float(E_term)); r += 1
242 A_eq = sparse.coo_matrix((vals_e, (rows_e, cols_e)), shape=(r, n)).tocsr()
243 b_eq = np.array(be)
244 # 不等式: 供电 (S*T): -q -qEM -d +c <= (G-L)*DT
245 rows_u, cols_u, vals_u, bu = [], [], [], []
246 r = 0
247 for s in range(S):
248     for t in range(T):
249         rows_u += [r, r, r, r]
250         cols_u += [i_q + t, i_blk(s, 0, t), i_blk(s, 2, t), i_blk(s, 1, t)]
251         vals_u += [-1.0, -1.0, -1.0, 1.0]
252         bu.append((Gs[s, t] - Ls[s, t]) * DT); r += 1
253 if lam > 0:
254     # C_s - eta - u_s <= 0 ; 1/(1-alpha) occurs in objective only.
255     for s in range(S):
256         for t in range(T):
257             rows_u.append(r); cols_u.append(i_blk(s, 0, t)); vals_u.append(5.0 * p[t])
258             rows_u.append(r); cols_u.append(i_eta); vals_u.append(-1.0)
259             rows_u.append(r); cols_u.append(i_u + s); vals_u.append(-1.0)
260             bu.append(0.0); r += 1
261 A_ub = sparse.coo_matrix((vals_u, (rows_u, cols_u)), shape=(r, n)).tocsr()
262 b_ub = np.array(bu)
263 def _lp(method, options):
264     return linprog(c_vec, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
265                   bounds=np.stack([ub_l, ub_u], axis=1), method=method, options=
options)
266
267 # 2026-09-12 健壮性修正 (求解阶梯): HiGHS 在**退化算例**上偶发返回
268 # "HiGHS Status 15: model_status Unknown / primal_status Feasible"
269 # (实测触发于问题4 口径B A 轨 di=132 t=107、储能恰为上限 10800 kWh),
270 # 同一实例换用单纯形/内点法即可正常返回最优解。故:
271 # ① 先按原样 highs; 仅当**首次未成功**时才依次重试 highs-ds / highs-ipm / highs(无

```

```

presolve);
272 # ② 任一尝试返回非零状态但**原始解通过独立可行性校验**（不等式/等式/上下界  $\leq 1e-6$ ）时接受，
273 # 目标值由 c_vec @ x 重算，不沿用求解器报告值；
274 # ③ 全部尝试失败则保持原行为抛错。
275 # 对**首次即成功**的求解，本改动逐位无影响（不会进入重试分支）。
276 ok, x_acc, res, tries = False, None, None, 0
277 for method, options in (("highs", {}), ("highs-ds", {}), ("highs-ipm", {}),
278 ("highs", {"presolve": False})):
279     tries += 1
280     r_try = _lp(method, options)
281     if res is None:
282         res = r_try
283     ok_try, x_try = bool(r_try.success), None
284     if ok_try:
285         x_try = np.asarray(r_try.x, float)
286     elif r_try.x is not None and r_try.status not in (1, 2, 3): # 1/2/3 = 迭代上限/不
可行/无界
287         xc = np.asarray(r_try.x, float)
288         viol = 0.0
289         if A_ub.shape[0]:
290             viol = max(viol, float(np.max(A_ub @ xc - b_ub)))
291         if A_eq.shape[0]:
292             viol = max(viol, float(np.max(np.abs(A_eq @ xc - b_eq))))
293         viol = max(viol, float(np.max(ub_l - xc)), float(np.max(xc - ub_u)))
294         if viol <= 1e-6:
295             ok_try, x_try = True, xc
296     if ok_try:
297         ok, x_acc, res = True, x_try, r_try
298         break
299 out = {"status": 0 if ok else res.status,
300 "msg": (res.message + f" (第 {tries} 次尝试: 可行性校验通过的原始解被接受)")
301 if (ok and not res.success) else res.message,
302 "obj": float(c_vec @ x_acc) if ok else np.nan,
303 "accepted_unknown_status": bool(ok and not res.success),
304 "solve_tries": tries}
305 if ok:
306     x = x_acc
307     out["q"] = x[i_q:i_q + T].copy()
308     out["qEM"] = np.stack([x[i_blk(s, 0, 0):i_blk(s, 0, 0) + T] for s in range(S)])
309     out["c"] = np.stack([x[i_blk(s, 1, 0):i_blk(s, 1, 0) + T] for s in range(S)])
310     out["d"] = np.stack([x[i_blk(s, 2, 0):i_blk(s, 2, 0) + T] for s in range(S)])
311     out["E"] = np.stack([x[i_blk(s, 3, 0):i_blk(s, 3, 0) + T] for s in range(S)])
312     if adj:
313         out["dq"] = x[i_dq:i_dq + T].copy(); out["dqm"] = x[i_dqm:i_dqm + T].copy()
314 return out
315
316 def hindsight_dispatch(p, L, G, E0, q, *, eps_soc=1e-3, tp=TP_PRICE, v_term=0.0):
317     """事后完美信息调度，仅用于诊断；禁止用于正式策略回溯。"""
318     T = len(p); S = 1
319     return solve_plan_lp(p, L[None, :], G[None, :], E0, q_fixed=q,
320 eps_soc=eps_soc, tp=tp, v_term=v_term)

```

```

321
322
323 def realtime_dispatch(p, L, G, E0, q, *, forecast_L=None, forecast_G=None,
324                      eps_soc=1e-3, tp=TP_PRICE, v_term=0.0):
325     """Causal 10-minute receding-horizon execution with a fixed market plan.
326
327     Only the current measured L/G replace their forecasts. Future actual values
328     are never passed into the optimization. Callers must supply forecasts made
329     using information available before execution. All strategies use this rule.
330     """
331     if forecast_L is None or forecast_G is None:
332         raise ValueError("实时执行必须显式提供当时可得预测，禁止用未来实际曲线代替")
333     T = len(p)
334     out = {k: np.zeros((1, T)) for k in ("c", "d", "E", "qEM")}
335     state = float(E0)
336     for t in range(T):
337         r = causal_step(p[t:], q[t:], state, L[t], G[t],
338                        np.asarray(forecast_L)[t:], np.asarray(forecast_G)[t:],
339                        eps_soc=eps_soc, tp=tp, v_term=v_term)
340         for k in out:
341             out[k][0, t] = r[k]
342         state = r["E"]
343     out.update(status=0, msg="causal execution", q=np.asarray(q).copy())
344     return out
345
346
347 def causal_step(p, q, E0, current_L, current_G, forecast_L, forecast_G, *,
348                eps_soc=1e-3, tp=TP_PRICE, v_term=0.0):
349     """Execute one slot. Interface deliberately accepts no future observations."""
350     lf = np.asarray(forecast_L, dtype=float).copy()
351     gf = np.asarray(forecast_G, dtype=float).copy()
352     lf[0], gf[0] = current_L, current_G
353     r = solve_plan_lp(np.asarray(p), lf[None, :], gf[None, :], E0,
354                      q_fixed=np.asarray(q), eps_soc=eps_soc, tp=tp, v_term=v_term)
355     if r["status"] != 0:
356         raise RuntimeError(r["msg"])
357     # Remove any solver-level simultaneous throughput exactly, preserving SOC.
358     # Supply weakly increases; surplus is spilled, as allowed by the model.
359     c, d = float(r["c"][0, 0]), float(r["d"][0, 0])
360     paired = min(c, d / (ETA_C * ETA_D))
361     c -= paired
362     d -= paired * ETA_C * ETA_D
363     em = max(0.0, (current_L-current_G)*DT + c-d-float(q[0]))
364     return dict(c=c, d=d, E=float(E0+ETA_C*c-d/ETA_D), qEM=em)
365
366
367 # ----- 场景
368 def sample_scenarios(res_pool, hatL, hatG, S, rng):
369     """res_pool: list of (resL(144), resG(144)); 整日配对抽样。"""
370     n = len(res_pool)
371     idx = rng.choice(n, size=min(S, n), replace=n < S)

```

```

372 Ls = np.zeros((len(idx), 144)); Gs = np.zeros((len(idx), 144))
373 for k, i in enumerate(idx):
374     rL, rG = res_pool[i]
375     Ls[k] = np.maximum(0.0, hatL + rL)
376     Gs[k] = np.maximum(0.0, hatG + rG)
377 return Ls, Gs
378
379 # ----- 结算
380 def merge_emergency_intervals(qEM, thr=1e-6):
381     """qEM: (144,) -> [(start_min, end_min, qty)]"""
382     ivs = []
383     j = 0
384     while j < 144:
385         if qEM[j] > thr:
386             k = j; qty = 0.0
387             while k < 144 and qEM[k] > thr:
388                 qty += qEM[k]; k += 1
389             ivs.append((slot_start_min(j), slot_start_min(k - 1) + 10, qty))
390             j = k
391         else:
392             j += 1
393     return ivs
394
395 def settle_q2(p, q, qEM):
396     """Q2 全天费用 = 计划 p * q + 紧急 5p * qEM"""
397     return float(np.sum(p * q) + np.sum(5 * p * qEM))
398
399 def settle_q3(p, q0, updates):
400     """Q3: updates = [(tau_slot, q_new(T'), dqp, dqm)] 顺序（相对上一确认计划）。
401     返回（总计划相关费用，调整费用合计，最终执行曲线 qa）"""
402     qa = q0.copy(); adj_fee = 0.0
403     for tau, qn, dqp, dqm in updates:
404         T = len(qn)
405         adj_fee += float(np.sum(1.5 * p[tau:tau + T] * dqp - 0.5 * p[tau:tau + T] * dqm))
406         qa[tau:tau + T] = qn
407     plan_fee = float(np.sum(p * q0))
408     return plan_fee + adj_fee, adj_fee, qa

```

C8. 问题 2-3/p15_forecasts.py (49 行)

问题 2/3 逐日历史选型预报（只用当日之前信息，含残差构造）

```

1 """逐日历史选型：模型及残差均只使用当日之前信息。"""
2 import os, json
3 import numpy as np
4 import pandas as pd
5 from p10_core import *
6
7 def select_past_only(candidates, actual, start=14):
8     names = list(candidates)
9     out = np.zeros_like(actual)
10    selected = []
11    for di in range(len(actual)):

```

```

12     if di <= start:
13         winner = "lag1"
14     else:
15         winner = min(names, key=lambda n: float(np.mean(
16             np.abs(candidates[n][start:di]-actual[start:di]))))
17     out[di] = candidates[winner][di]
18     selected.append(winner)
19     return out, selected
20
21 def main():
22     os.makedirs(RES, exist_ok=True); os.makedirs(LOGD, exist_ok=True)
23     data = load_all(); saved = {}; metrics = {}; selections = {}
24     for prefix, actual in (("L",data["L"]),("G",data["G"])):
25         print("rolling forecast", prefix, flush=True)
26         lag = np.empty_like(actual); lag[1:] = actual[:-1]; lag[0] = 0
27         m7 = baseline_mean7(actual); m7[:7] = lag[:7]
28         lgb = rolling_forecast(actual); lgb[:14] = lag[:14]
29         candidates = dict(lgb=lgb, lag1=lag, m7=m7)
30         forecast, selections[prefix] = select_past_only(candidates, actual)
31         saved["F"+prefix] = forecast
32         for name, array in candidates.items(): saved["F"+prefix+"_"+name] = array
33         for name, array in dict(candidates, online=forecast).items():
34             for period, idx in (("334天因果回测",slice(31,365)),("5至12月",slice(120,365))):
35                 err = array[idx]-actual[idx]
36                 metrics[prefix+"_"+name+"_"+period] = dict(
37                     MAE=float(np.abs(err).mean()), RMSE=float(np.sqrt((err**2).mean())),
38                     bias=float(err.mean()))
39         saved["protocol"] = np.array("end_label_causal_v2")
40         np.savez_compressed(os.path.join(RES,"预测缓存.npz"), **saved)
41         pd.DataFrame(dict(日期=data["dates"], 负荷模型=selections["L"], 光伏模型=selections["G"])).
42         to_csv(
43             os.path.join(RES,"逐日预测选型.csv"),index=False,encoding="utf-8-sig")
44         with open(os.path.join(RES,"预测模型选择.json"),"w",encoding="utf-8") as f:
45             json.dump(dict(protocol="end_label_causal_v2", selection="每日仅依据此前样本外MAE",
46                 metrics=metrics),f,ensure_ascii=False,indent=2)
47         print("forecast DONE",flush=True)
48     if __name__ == "__main__": main()

```

C9. 问题 2-3/p95_q23_v4.py (604 行)

问题 2/3 主运行器：8 个更新组合 + 部署策略的连续回测（含附件 3 整点预报的 10 分钟展开）

```

1 # -*- coding: utf-8 -*-
2 """Q2/Q3 fixed-price causal rerun, protocol q23_janwarm_v4 (v_term arm driver).
3
4 This driver is deliberately separate from p90_reproduce.py. It starts every
5 path at 2025-01-01 00:00 with E=6000 kWh, keeps January as a visible warm-up,
6 and writes only to the explicitly supplied case directory.
7 """
8 from __future__ import annotations

```

```

9
10 import argparse
11 import hashlib
12 import json
13 import os
14 import pickle
15 import shutil
16 import sys
17 import time
18 from concurrent.futures import ProcessPoolExecutor, as_completed
19 from pathlib import Path
20
21 import numpy as np
22 import pandas as pd
23
24 HERE = Path(__file__).resolve().parent
25 ROOT = HERE.parent
26 sys.path.insert(0, str(HERE))
27 from p10_core import ( # noqa: E402
28     DT, E_INIT, E_MAX, E_MIN, ETA_C, ETA_D, P_MAX_KWH, TP_PRICE,
29     causal_step, fc3_step_day, fmt_min, load_all, solve_plan_lp,
30 )
31 from p15_forecasts import ( # noqa: E402
32     baseline_mean7, rolling_forecast, select_past_only,
33 )
34
35 PROTOCOL = "q23_janwarm_v4"
36 COMBOS = [(), (6,), (12,), (18,), (6, 12), (6, 18), (12, 18), (6, 12, 18)]
37 EPS_GRID = [0.0, 1e-6, 1e-5, 1e-4, 1e-3, 1e-2]
38 CALIBRATION_STOP = 90 # [2025-01-01, 2025-04-01): Jan warm-up + Feb/Mar calibration
39
40
41 def key(combo):
42     return "+".join(map(str, combo)) or "none"
43
44
45 def phase_of(di):
46     if di < 31:
47         return "init"
48     if di < 90:
49         return "calibration"
50     return "eval"
51
52
53 def sha256(path):
54     h = hashlib.sha256()
55     with open(path, "rb") as f:
56         for block in iter(lambda: f.read(1024 * 1024), b''):
57             h.update(block)
58     return h.hexdigest()
59

```

```

60
61 def code_hashes():
62     files = [HERE / n for n in ("p10_core.py", "p15_forecasts.py", "p95_q23_v4.py")]
63     return {str(p): sha256(p) for p in files}
64
65
66 def input_hashes():
67     ad = ROOT / "00_题目与原始数据" / "C题" / "附件"
68     return {str(p): sha256(p) for p in sorted(ad.glob("附件[1-4].xlsx"))}
69
70
71 def write_json(path, obj):
72     path = Path(path)
73     path.parent.mkdir(parents=True, exist_ok=True)
74     path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=str), encoding="
utf-8")
75
76
77 def make_forecasts(out_dir, reuse=False):
78     """Build or safely reuse a cache only when protocol and hashes match."""
79     out_dir = Path(out_dir)
80     fdir = out_dir / "forecast"
81     fdir.mkdir(parents=True, exist_ok=True)
82     cache = fdir / "预测缓存.npz"
83     meta_path = fdir / "预测缓存_meta.json"
84     expected = dict(protocol=PROTOCOL, training_start=14,
85                     input_hashes=input_hashes(), code_hashes=code_hashes(),
86                     early_policy="lag1_for_di_lt_14", seed=2025)
87     if reuse and cache.exists() and meta_path.exists():
88         meta = json.loads(meta_path.read_text(encoding="utf-8"))
89         if all(meta.get(k) == expected.get(k) for k in expected):
90             with np.load(cache) as z:
91                 if all(k in z for k in ("FL", "FG", "protocol")):
92                     if str(z["protocol"]) == PROTOCOL and np.isfinite(z["FL"][:14]).all()
and np.isfinite(z["FG"][:14]).all():
93                         return str(cache), meta
94
95     data = load_all()
96     meta = expected.copy()
97     saved = {}
98     selections = {}
99     metrics = {}
100     for prefix, actual in (("L", data["L"]), ("G", data["G"])):
101         lag = np.empty_like(actual)
102         lag[0] = actual[0]
103         lag[1:] = actual[:-1]
104         m7 = baseline_mean7(actual)
105         m7[:7] = lag[:7]
106         lgb = rolling_forecast(actual, start=14)
107         lgb[:14] = lag[:14]
108         candidates = {"lgb": lgb, "lag1": lag, "m7": m7}

```

```

109     forecast, selected = select_past_only(candidates, actual, start=14)
110     saved["F" + prefix] = forecast
111     selections[prefix] = selected
112     for name, arr in candidates.items():
113         saved["F" + prefix + "_" + name] = arr
114     for name, arr in dict(candidates, online=forecast).items():
115         err = arr[31:] - actual[31:]
116         metrics[prefix + "_" + name + "_eval"] = dict(
117             MAE=float(np.abs(err).mean()),
118             RMSE=float(np.sqrt((err ** 2).mean())),
119             bias=float(err.mean()),
120         )
121     saved["protocol"] = np.array(PROTOCOL)
122     np.savez_compressed(cache, **saved)
123     pd.DataFrame(dict(日期=data["dates"], 负荷模型=selections["L"], 光伏模型=selections["G"]))
124     ).to_csv(
125         fdir / "逐日预测选型.csv", index=False, encoding="utf-8-sig")
126     meta.update(dict(metrics=metrics, created_at=time.strftime("%Y-%m-%dT%H:%M:%S%z")))
127     write_json(meta_path, meta)
128     return str(cache), meta
129
130 def load_forecasts(path):
131     with np.load(path) as z:
132         if str(z["protocol"]) != PROTOCOL:
133             raise ValueError("预测缓存协议不匹配")
134         f = {k: z[k].copy() for k in ("FL", "FG")}
135     if not np.isfinite(f["FL"][:14]).all() or not np.isfinite(f["FG"][:14]).all():
136         raise ValueError("1月早期预测不完整，不能进入冷启动")
137     return f
138
139
140 def scenarios(data, forecasts, di, release_h, S, seed, use_fc3, method):
141     """Causal paired residual scenarios; no completed residual day means one scene."""
142     lf = np.asarray(forecasts["FL"][di], dtype=float)
143     if use_fc3:
144         gf = fc3_step_day(data["fc3"], di, release_h, method=method)[0]
145     else:
146         gf = np.asarray(forecasts["FG"][di], dtype=float)
147     hist = np.arange(14, di, dtype=int)
148     if len(hist) == 0:
149         return lf[None, :], gf[None, :], [], lf, gf, 0
150     rng = np.random.default_rng(np.random.SeedSequence([int(seed), int(di), int(release_h)
151     ]))
152     days = rng.choice(hist, size=int(S), replace=len(hist) < int(S))
153     r1 = data["L"][days] - forecasts["FL"][days]
154     if use_fc3:
155         rg = np.stack([
156             data["G"][d] - fc3_step_day(data["fc3"], int(d), release_h, method=method)[0]
157             for d in days
158         ])

```

```

158     else:
159         rg = data["G"][days] - forecasts["FG"][days]
160     return np.maximum(0.0, lf[None, :] + rl), np.maximum(0.0, gf[None, :] + rg), days.
        tolist(), lf, gf, len(hist)
161
162
163 def cold_day(data, di, p, E0):
164     L = data["L"][di]
165     G = data["G"][di]
166     qem = np.maximum(0.0, (L - G) * DT)
167     z = np.zeros(144, dtype=float)
168     return dict(
169         日期=str(data["dates"][di]), di=di, q=z.copy(), q0=z.copy(), qc=z.copy(),
170         c=z.copy(), d=z.copy(), E=np.full(144, float(E0)), qEM=qem,
171         E0=float(E0), Eend=float(E0), updates=[], scenario_days={"0": []},
172         scenario_pool_counts={"0": 0}, planned_emergency_qty=0.0,
173         planned_emergency_fee=0.0, 计划费=0.0, 调整费=0.0,
174         紧急费=float(np.sum(5.0 * p * qem)), 总费用=float(np.sum(5.0 * p * qem)),
175         阶段=phase_of(di), cold_start=True,
176     )
177
178
179 def simulate_day(data, forecasts, di, E0, mode="fixed", model="B", combo=None,
180                 S=10, seed=2025, lam=.3, method="hold", eps_soc=1e-3,
181                 tp=TP_PRICE, v_term=0.0, update_mode="FM"):
182     if mode != "fixed":
183         raise ValueError("q23_janwarm_v3 只允许固定电价")
184     p = np.asarray(data["p_fix"], dtype=float)
185     if di == 0:
186         return cold_day(data, di, p, E0)
187     use_fc3 = combo is not None
188     ls, gs, days, lf, gf, pool_n = scenarios(data, forecasts, di, 0, S, seed, use_fc3,
189                                             method)
189     if model == "A":
190         ls, gs = lf[None, :], gf[None, :]
191     plan = solve_plan_lp(p, ls, gs, E0, lam=lam if model == "C" else 0.0,
192                         eps_soc=eps_soc, tp=tp, v_term=v_term)
193     if plan["status"] != 0:
194         raise RuntimeError(plan["msg"])
195     q0 = plan["q"].copy()
196     qc = q0.copy()
197     state = float(E0)
198     arrays = {k: np.zeros(144, dtype=float) for k in ("c", "d", "E", "qEM")}
199     updates = []
200     draws = {"0": list(days)}
201     pools = {"0": int(pool_n)}
202     expected_qty = float(np.mean(plan["qEM"].sum(axis=1)))
203     expected_fee = float(np.mean(np.sum(5.0 * p[None, :] * plan["qEM"], axis=1)))
204     adj = 0.0
205     for t in range(144):
206         hour = t // 6

```

```

207     if use_fc3 and t > 0 and t % 6 == 0 and hour in combo:
208         ls, gs, days, lf, gf, pool_n = scenarios(data, forecasts, di, hour, S, seed,
True, method)
209         draws[str(hour)] = list(days)
210         pools[str(hour)] = int(pool_n)
211         if update_mode == "F":
212             # D4 的 F 轨: 只用新发布的附件3预报刷新日内预测(执行器视界随之更新),
213             # 但**锁定日前购电计划 q0**, 不做市场购电调整, 故调整费为 0、qc 不变。
214             updates.append(dict(
215                 tau=t, hour=hour, old=qc[t:].copy(), new=qc[t:].copy(), fee=0.0,
216                 E0=float(state), scenario_days=list(days), scenario_pool_n=int(pool_n),
217                 expected_emergency_qty=0.0, expected_emergency_fee=0.0, forecast_only=
True,
218                 ))
219         else:
220             old = qc[t:].copy()
221             update = solve_plan_lp(p[t:], ls[:, t:], gs[:, t:], state, q_old=old,
222                 eps_soc=eps_soc, tp=tp, v_term=v_term)
223             if update["status"] != 0:
224                 raise RuntimeError(update["msg"])
225             qc[t:] = update["q"]
226             plus = np.maximum(qc[t:] - old, 0.0)
227             minus = np.maximum(old - qc[t:], 0.0)
228             fee = float(p[t:] @ (1.5 * plus - .5 * minus))
229             adj += fee
230             updates.append(dict(
231                 tau=t, hour=hour, old=old, new=qc[t:].copy(), fee=fee,
232                 E0=float(state), scenario_days=list(days), scenario_pool_n=int(pool_n),
233                 expected_emergency_qty=float(np.mean(update["qEM"].sum(axis=1))),
234                 expected_emergency_fee=float(np.mean(np.sum(5.0 * p[t:][None, :] *
update["qEM"], axis=1))),
235                 ))
236             step = causal_step(p[t:], qc[t:], state, data["L"][di, t], data["G"][di, t],
237                 lf[t:], gf[t:], eps_soc=eps_soc, tp=tp, v_term=v_term)
238             for k in arrays:
239                 arrays[k][t] = step[k]
240             state = float(step["E"])
241             pf = float(p @ q0)
242             ef = float(5.0 * p @ arrays["qEM"])
243             return dict(
244                 日期=str(data["dates"][di]), di=di, q=q0, q0=q0, qc=qc, **arrays,
245                 E0=float(E0), Eend=float(state), updates=updates,
246                 scenario_days=draws, scenario_pool_counts=pools,
247                 planned_emergency_qty=expected_qty, planned_emergency_fee=expected_fee,
248                 计划费=pf, 调整费=adj, 紧急费=ef, 总费用=pf + adj + ef,
249                 阶段=phase_of(di), cold_start=False,
250             )
251
252
253 def summary(recs, start=None, stop=None):
254     rows = recs[slice(start, stop)] if start is not None or stop is not None else recs

```

```

255     return dict(
256         天数=len(rows), 计划购电费=float(sum(r["计划费"] for r in rows)),
257         调整费=float(sum(r["调整费"] for r in rows)),
258         紧急购电费=float(sum(r["紧急费"] for r in rows)),
259         紧急购电量=float(sum(np.sum(r["qEM"]) for r in rows)),
260         总费用=float(sum(r["总费用"] for r in rows)),
261         初SOC=float(rows[0]["E0"]) if rows else None,
262         末SOC=float(rows[-1]["Eend"]) if rows else None,
263     )
264
265
266 def worker_run_path(task):
267     label, cfg = task
268     data = load_all()
269     forecasts = load_forecasts(cfg["forecast_cache"])
270     start, stop = int(cfg["start"]), int(cfg["stop"])
271     state = float(cfg.get("initial_state", E_INIT))
272     recs = []
273     begin = time.time()
274     cp = Path(cfg["checkpoint_dir"]) / (label.replace("+", "plus") + ".json")
275     for di in range(start, stop):
276         rec = simulate_day(data, forecasts, di, state, model=cfg["model"],
277                             combo=cfg.get("combo"), S=cfg["S"], seed=cfg["seed"],
278                             lam=cfg["lam"], method=cfg["method"],
279                             eps_soc=cfg["eps_soc"], tp=cfg["tp"], v_term=cfg["v_term"],
280                             update_mode=cfg.get("update_mode", "FM"))
281         state = rec["Eend"]
282         recs.append(rec)
283         write_json(cp, dict(protocol=PROTOCOL, label=label, last_complete_date=rec["日期"],
284                             last_di=di, E=float(state), shadow_total=float(sum(x["总费用"]
285                                     for x in recs)),
286                             parameters={k: cfg[k] for k in ("model", "S", "seed", "eps_soc",
287                                                             "tp", "v_term", "method")},
288                             complete=di == stop - 1))
289     return label, recs, time.time() - begin
290
291 def daily_frame(recs, label):
292     return pd.DataFrame([
293         dict(日期=r["日期"], 模型=label, 计划购电费=r["计划费"], 调整费=r["调整费"],
294             紧急购电费=r["紧急费"], 紧急购电量=float(np.sum(r["qEM"])),
295             总费用=r["总费用"], 末SOC=r["Eend"], 阶段=r["阶段"],
296             计划紧急购电量=r["planned_emergency_qty"], 计划紧急购电费=r["
297             planned_emergency_fee"])
298         for r in recs
299     ])
300
301 def save_pickle(path, obj):
302     path = Path(path)
303     path.parent.mkdir(parents=True, exist_ok=True)

```

```

303     with open(path, "wb") as f:
304         pickle.dump(obj, f, protocol=pickle.HIGHEST_PROTOCOL)
305
306
307 def run_paths(out_dir, forecast_cache, labels, cfg, workers=2, resume=False):
308     out_dir = Path(out_dir)
309     path_dir = out_dir / "paths"
310     cp_dir = out_dir / "checkpoints"
311     path_dir.mkdir(parents=True, exist_ok=True)
312     cp_dir.mkdir(parents=True, exist_ok=True)
313     tasks = []
314     loaded = {}
315     for label, model, combo in labels:
316         path_file = path_dir / (label.replace("+", "plus") + ".pkl")
317         if resume and path_file.exists():
318             with open(path_file, "rb") as f:
319                 obj = pickle.load(f)
320                 if obj.get("protocol") == PROTOCOL and obj.get("config", {}).get("eps_soc") ==
321                     cfg["eps_soc"] and obj.get("config", {}).get("v_term") == cfg.get("v_term"):
322                     loaded[label] = obj["recs"]
323                     continue
324             child = dict(cfg, forecast_cache=str(forecast_cache), checkpoint_dir=str(cp_dir),
325                         model=model, combo=list(combo) if combo is not None else None)
326             tasks.append((label, child))
327     durations = {}
328     if tasks:
329         with ProcessPoolExecutor(max_workers=max(1, min(int(workers), 2))) as pool:
330             future_map = {pool.submit(worker_run_path, t): t[0] for t in tasks}
331             for future in as_completed(future_map):
332                 label, recs, elapsed = future.result()
333                 loaded[label] = recs
334                 durations[label] = float(elapsed)
335                 save_pickle(path_dir / (label.replace("+", "plus") + ".pkl"),
336                             dict(protocol=PROTOCOL, label=label, config=cfg, recs=recs,
337                                 elapsed_seconds=elapsed))
338                 write_json(out_dir / "progress.json", dict(protocol=PROTOCOL,
339                     completed_paths=sorted(loaded),
340                     durations=durations, updated_at=
341                     time.strftime("%Y-%m-%dT%H:%M:%S%z")))
342     for label, _, _ in labels:
343         if label not in loaded:
344             raise RuntimeError("路径未返回: " + label)
345     return loaded, durations
346
347 def deploy_from_shadow(data, forecasts, shadows, cfg):
348     ordered = [key(c) for c in COMBOS]
349     n = len(next(iter(shadows.values())))
350     cumulative = {k: 0.0 for k in ordered}
351     state = E_INIT
352     deployed = []

```

```

351     for i in range(n):
352         di = shadows[ordered[0]][i]["di"]
353         selected = "none" if di == 0 else min(ordered, key=lambda k: cumulative[k])
354         combo = COMBOS[ordered.index(selected)]
355         rec = simulate_day(data, forecasts, di, state, combo=combo, S=cfg["S"], seed=cfg["
seed"],
356                             method=cfg["method"], eps_soc=cfg["eps_soc"], tp=cfg["tp"],
357                             v_term=cfg["v_term"], update_mode=cfg.get("update_mode", "FM"))
358         rec["selected_combo"] = selected
359         rec["selection_history_end"] = di - 1
360         deployed.append(rec)
361         state = rec["Eend"]
362         for k in ordered:
363             cumulative[k] += shadows[k][i]["总费用"]
364     return deployed, cumulative
365
366
367 def write_result_workbooks(out_dir, q2_b, q3_deployed):
368     """Fill only result2/result3 in the isolated output directory."""
369     import openpyxl
370     tpl = ROOT / "00_题目与原始数据" / "C题" / "附件" / "附件5"
371     out_dir = Path(out_dir)
372     dates = [d.strftime("%Y/%m/%d") for d in pd.date_range("2025-02-01", "2025-12-31")]
373     buckets = [(0, 24), (24, 48), (48, 72), (72, 96), (96, 120), (120, 144)]
374     labels = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00", "16:00-20:00", "
20:00-24:00"]
375
376     def fill_plan(ws, recs, field, include_adj=False):
377         for j in range(144):
378             ws.cell(1, 2 + j, f"{fmt_min(j * 10)}-{fmt_min((j + 1) * 10)}")
379         for i, (dt, r) in enumerate(zip(dates, recs)):
380             q = np.asarray(r[field])
381             ws.cell(2 + i, 1, dt)
382             for j in range(144):
383                 ws.cell(2 + i, 2 + j, round(float(q[j]), 2))
384             ws.cell(2 + i, 146, round(float(q.sum()), 2))
385             fee = r["计划费"] + r["调整费"] if include_adj else r["计划费"]
386             ws.cell(2 + i, 147, round(float(fee), 2))
387
388     def fill_storage(ws, recs):
389         for row in range(2, ws.max_row + 1):
390             for col in range(1, 7):
391                 ws.cell(row, col).value = None
392         row = 2
393         for dt, r in zip(dates, recs):
394             for bi, (a, b) in enumerate(buckets):
395                 ws.cell(row, 1, dt if bi == 0 else None)
396                 ws.cell(row, 2, labels[bi])
397                 ws.cell(row, 3, round(float(np.sum(r["c"][a:b])), 2))
398                 ws.cell(row, 4, round(float(np.sum(r["d"][a:b])), 2))
399                 if bi == 0:

```

```

400         ws.cell(row, 5, "0:00"); ws.cell(row, 6, round(float(r["E0"]), 2))
401     if bi == 1:
402         ws.cell(row, 5, "24:00"); ws.cell(row, 6, round(float(r["Eend"]), 2))
403     row += 1
404
405 def fill_emergency(ws, recs):
406     for row in range(2, ws.max_row + 1):
407         for col in range(1, 4):
408             ws.cell(row, col).value = None
409     row = 2
410     for dt, r in zip(dates, recs):
411         qem = np.asarray(r["qEM"]); t = 0; intervals = []
412         while t < 144:
413             if qem[t] <= 1e-6:
414                 t += 1; continue
415             a = t; amount = 0.0
416             while t < 144 and qem[t] > 1e-6:
417                 amount += float(qem[t]); t += 1
418             intervals.append((a, t, amount))
419         if not intervals:
420             ws.cell(row, 1, dt); row += 1
421         else:
422             for a, b, amount in intervals:
423                 ws.cell(row, 1, dt); ws.cell(row, 2, f"{fmt_min(a * 10)}-{fmt_min(b *
424                 10)}")
425                 ws.cell(row, 3, round(amount, 2)); row += 1
426
427 eval_q2 = q2_b[31:]
428 wb = openpyxl.load_workbook(tp1 / "result2.xlsx")
429 fill_plan(wb["计划购电量"], eval_q2, "q0")
430 fill_storage(wb["充放电量"], eval_q2)
431 fill_emergency(wb["紧急购电量"], eval_q2)
432 wb.save(out_dir / "result2.xlsx")
433
434 eval_q3 = q3_deployed[31:]
435 wb = openpyxl.load_workbook(tp1 / "result3.xlsx")
436 fill_plan(wb["计划购电量"], eval_q3, "q0")
437 fill_plan(wb["调整购电量"], eval_q3, "qc", include_adj=True)
438 fill_storage(wb["充放电量"], eval_q3)
439 fill_emergency(wb["紧急购电量"], eval_q3)
440 wb.save(out_dir / "result3.xlsx")
441
442 def run_main(args, out_dir, forecast_cache, metadata):
443     out_dir = Path(out_dir); out_dir.mkdir(parents=True, exist_ok=True)
444     cfg = dict(start=0, stop=365, S=args.S, seed=args.seed, lam=args.lam,
445               method=args.method, eps_soc=args.eps_soc, tp=args.tp, v_term=args.v_term,
446               initial_state=E_INIT)
447     q2_labels = [("A", "A", None), ("B", "B", None), ("C", "C", None)]
448     q2, d2 = run_paths(out_dir, forecast_cache, q2_labels, cfg, args.workers, args.resume)
449     q3_labels = [(key(c), "B", c) for c in COMBOS]

```

```

449 q3, d3 = run_paths(out_dir, forecast_cache, q3_labels, cfg, args.workers, args.resume)
450 data = load_all(); forecasts = load_forecasts(forecast_cache)
451 deployed, cumulative = deploy_from_shadow(data, forecasts, q3, cfg)
452 save_pickle(out_dir / "q2_best.pkl", dict(protocol=PROTOCOL, best="B", recs=q2["B"],
453 q2_all=q2, config=cfg))
454 save_pickle(out_dir / "q3_daystore.pkl", dict(protocol=PROTOCOL, best_key="历史选型执行
455 策略",
456
457 store=deployed, all_stores=q3,
458 results={k: summary(v) for k, v in q3.
459 items()} |
460 {"历史选型执行策略": summary(
461 deployed)}, config=cfg))
462 q2_df = pd.concat([daily_frame(q2[k], k) for k in ("A", "B", "C")], ignore_index=True)
463 q2_df.to_csv(out_dir / "q2_逐日费用.csv", index=False, encoding="utf-8-sig")
464 q2_df.groupby("模型")[["计划购电费", "调整费", "紧急购电费", "紧急购电量", "总费用"]].sum()
465 .to_csv(
466 out_dir / "q2_汇总.csv", encoding="utf-8-sig")
467 q3_df = pd.concat([daily_frame(q3[k], k) for k in [key(c) for c in COMBOS]] +
468 [daily_frame(deployed, "历史选型执行策略")], ignore_index=True)
469 q3_df.to_csv(out_dir / "q3_逐日费用.csv", index=False, encoding="utf-8-sig")
470 combo_rows = [{"组合": k, **summary(q3[k])} for k in [key(c) for c in COMBOS]]
471 combo_rows.append({"组合": "历史选型执行策略", **summary(deployed)})
472 pd.DataFrame(combo_rows).to_csv(out_dir / "q3_组合对照.csv", index=False, encoding="utf
473 -8-sig")
474 write_result_workbooks(out_dir, q2["B"], deployed)
475 write_json(out_dir / "main_metadata.json", dict(metadata, protocol=PROTOCOL, config=cfg
476 ,
477 q2_durations=d2, q3_durations=d3,
478 q2_summary={k: summary(v) for k, v in
479 q2.items()}),
480 q3_summary={k: summary(v) for k, v in
481 q3.items()}),
482 deployed_summary=summary(deployed),
483 selection_cumulative=cumulative))
484 return q2, q3, deployed
485
486 def run_calibration(args, out_dir, forecast_cache, metadata):
487 out_dir = Path(out_dir); out_dir.mkdir(parents=True, exist_ok=True)
488 rows = []
489 for eps in EPS_GRID:
490     cfg = dict(start=0, stop=CALIBRATION_STOP, S=args.S, seed=args.seed, lam=args.lam,
491 method=args.method, eps_soc=eps, tp=args.tp, v_term=0.0, initial_state=
492 E_INIT)
493     labels = [("A", "A", None), ("B", "B", None)]
494     run_dir = out_dir / ("eps_" + (str(eps).replace(".", "p") if eps else "0"))
495     paths, durations = run_paths(run_dir, forecast_cache, labels, cfg, args.workers,
496 args.resume)
497     b = summary(paths["B"], start=31, stop=90)
498     a = summary(paths["A"], start=31, stop=90)
499     rows.append(dict(eps_soc=eps, B_2_3_total=b["总费用"], A_2_3_total=a["总费用"],

```

```

489         B_minus_A=b["总费用"] - a["总费用"], elapsed_seconds=sum(durations.
values()))))
490     frame = pd.DataFrame(rows)
491     best_val = min(float(x) for x in frame["B_2_3_total"])
492     tied = frame[np.abs(frame["B_2_3_total"] - best_val) <= .01]
493     chosen = min([float(x) for x in tied["eps_soc"]])
494     result = dict(protocol=PROTOCOL, rule="2-3月B总账单；并列0.01元取较小eps；看结果前冻结",
495                 chosen_eps=chosen, candidates=rows, metadata=metadata)
496     frame.to_csv(out_dir / "eps_calibration.csv", index=False, encoding="utf-8-sig")
497     write_json(out_dir / "eps_calibration.json", result)
498     return result
499
500
501 def run_smoke(args, out_dir, forecast_cache, metadata):
502     out_dir = Path(out_dir); out_dir.mkdir(parents=True, exist_ok=True)
503     data = load_all(); forecasts = load_forecasts(forecast_cache)
504     checks = []
505
506     def record(name, ok, detail):
507         checks.append(dict(name=name, status="PASS" if ok else "FAIL", detail=detail))
508         if not ok:
509             raise AssertionError(name + ": " + detail)
510
511     # 1/1-1/3: cold start, empty scenario pool and continuity.
512     state = E_INIT
513     for di in range(3):
514         r = simulate_day(data, forecasts, di, state, model="B", S=args.S, seed=args.seed,
515                         eps_soc=args.eps_soc, tp=args.tp, v_term=args.v_term, method=args.
method)
516         if di == 0:
517             ok = r["cold_start"] and r["q0"].sum() == 0 and r["Eend"] == state
518         else:
519             ok = (not r["cold_start"]) and r["scenario_pool_counts"]["0"] == 0
520         record("cold_" + str(di), ok,
521             "cold_start=" + str(r["cold_start"]) + ", pool=" +
522             str(r["scenario_pool_counts"]["0"]) + ", emergency=" + str(float(r["qEM"].
sum()))))
523         state = r["Eend"]
524     # 1/14-1/16 boundary: no negative index and residual pool starts on 1/15.
525     r14 = simulate_day(data, forecasts, 13, E_INIT, combo=(6,), S=args.S, seed=args.seed,
526                       eps_soc=args.eps_soc, tp=args.tp, v_term=args.v_term, method=args.
method)
527     r15 = simulate_day(data, forecasts, 14, r14["Eend"], combo=(6,), S=args.S, seed=args.
seed,
528                       eps_soc=args.eps_soc, tp=args.tp, v_term=args.v_term, method=args.
method)
529     r16 = simulate_day(data, forecasts, 15, r15["Eend"], combo=(6,), S=args.S, seed=args.
seed,
530                       eps_soc=args.eps_soc, tp=args.tp, v_term=args.v_term, method=args.
method)
531     record("residual_boundary", r14["scenario_pool_counts"]["0"] == 0 and r15["

```

```

scenario_pool_counts"]["0"] == 0 and
532     r16["scenario_pool_counts"]["0"] == 1, "pool counts 1/14,1/15,1/16=" +
533     str([r14["scenario_pool_counts"]["0"], r15["scenario_pool_counts"]["0"], r16["
scenario_pool_counts"]["0"]]))
534 # Full January plus Feb 1-2 cross-month continuity.
535 state = E_INIT; jan = []
536 for di in range(33):
537     r = simulate_day(data, forecasts, di, state, combo=(6,), S=args.S, seed=args.seed,
538                     eps_soc=args.eps_soc, tp=args.tp, v_term=args.v_term, method=args.
method)
539     jan.append(r); state = r["Eend"]
540 record("cross_month", jan[31]["E0"] == jan[30]["Eend"] and jan[32]["E0"] == jan[31]["
Eend"],
541        "Jan31 Eend=" + str(jan[30]["Eend"]) + ", Feb1 E0=" + str(jan[31]["E0"]))
542 # A single Q3 day must realize 6/12/18 updates and retain pre-update slots.
543 r = simulate_day(data, forecasts, 31, E_INIT, combo=(6, 12, 18), S=args.S, seed=args.
seed,
544                 eps_soc=args.eps_soc, tp=args.tp, v_term=args.v_term, method=args.
method)
545 record("q3_updates", [u["tau"] for u in r["updates"]] == [36, 72, 108] and
546        np.allclose(r["qc"][:36], r["q0"][:36]),
547        "taus=" + str([u["tau"] for u in r["updates"]]))
548 # Timing sample, three continuous days per one Q2 and one Q3 path.
549 t0 = time.time(); state = E_INIT
550 for di in range(31, 34):
551     state = simulate_day(data, forecasts, di, state, model="B", S=args.S, seed=args.
seed,
552                         eps_soc=args.eps_soc, tp=args.tp, v_term=args.v_term, method=
args.method)["Eend"]
553 q2_seconds = time.time() - t0
554 t0 = time.time(); state = E_INIT
555 for di in range(31, 34):
556     state = simulate_day(data, forecasts, di, state, combo=(6, 12, 18), S=args.S,
557                         seed=args.seed, eps_soc=args.eps_soc, tp=args.tp, v_term=args.
v_term, method=args.method)["Eend"]
558 q3_seconds = time.time() - t0
559 timing = dict(q2_3day_seconds=q2_seconds, q3_3day_seconds=q3_seconds,
560              estimate_note="仅用于估算, 不替代全量实测")
561 write_json(out_dir / "timing_3days.json", timing)
562 write_json(out_dir / "smoke_result.json", dict(metadata=metadata, protocol=PROTOCOL,
563                                                checks=checks, timing=timing))
564 return checks
565
566
567 def main():
568     ap = argparse.ArgumentParser()
569     ap.add_argument("--phase", choices=["smoke", "main", "calibration"], required=True)
570     ap.add_argument("--case-id", default="q23_janwarm_v4")
571     ap.add_argument("--output", required=True)
572     ap.add_argument("--workers", type=int, default=2)
573     ap.add_argument("--S", type=int, default=10)

```

```

574 ap.add_argument("--seed", type=int, default=2025)
575 ap.add_argument("--eps-soc", type=float, default=1e-3)
576 ap.add_argument("--v-term", type=float, default=0.0)
577 ap.add_argument("--tp", type=float, default=TP_PRICE)
578 ap.add_argument("--lam", type=float, default=.3)
579 ap.add_argument("--method", choices=["hold", "linear"], default="hold")
580 ap.add_argument("--reuse-forecast", action="store_true")
581 ap.add_argument("--resume", action="store_true")
582 args = ap.parse_args()
583 out_dir = Path(args.output).resolve()
584 out_dir.mkdir(parents=True, exist_ok=True)
585 forecast_cache, forecast_meta = make_forecasts(out_dir.parent if args.phase != "main"
586 else out_dir,
587 reuse=args.reuse_forecast)
588 metadata = dict(case_id=args.case_id, protocol=PROTOCOL, project_root=str(ROOT),
589 phase=args.phase, forecast_cache=forecast_cache,
590 input_hashes=input_hashes(), code_hashes=code_hashes(),
591 parameters=dict(S=args.S, seed=args.seed, eps_soc=args.eps_soc, tp=args
592 .tp, v_term=args.v_term,
593 lam=args.lam, method=args.method), forecast_meta=
594 forecast_meta)
595 write_json(out_dir / "case_config.json", metadata)
596 if args.phase == "smoke":
597     run_smoke(args, out_dir, forecast_cache, metadata)
598 elif args.phase == "calibration":
599     run_calibration(args, out_dir, forecast_cache, metadata)
600 else:
601     run_main(args, out_dir, forecast_cache, metadata)
602 print(json.dumps(dict(status="PASS", phase=args.phase, output=str(out_dir)),
603 ensure_ascii=False), flush=True)
604
605 if __name__ == "__main__":
606     main()

```

C10. 问题 2-3/p95c_finish.py (89 行)

问题 2/3 结果合成与落盘：汇总表、逐日费用、组合对照与工作簿填充

```

1 # -*- coding: utf-8 -*-
2 """v4 单臂合成器：读取 paths/*.pkl (11 条)，执行部署策略并写出全部主轨产物。
3 等价于 p95 run_main 的合成尾段（无进程池）。"""
4 import argparse, json, sys
5 from pathlib import Path
6 import pandas as pd
7
8 HERE = Path(__file__).resolve().parent
9 sys.path.insert(0, str(HERE))
10 from p95_q23_v4 import (COMBOS, E_INIT, PROTOCOL, daily_frame, deploy_from_shadow, # noqa:
11     E402
12     input_hashes, code_hashes, key, load_all, load_forecasts,
13     save_pickle, summary, write_json, write_result_workbooks)
14 import pickle # noqa: E402

```

```

14
15
16 def main():
17     ap = argparse.ArgumentParser()
18     ap.add_argument("--case-dir", required=True)
19     ap.add_argument("--case-id", required=True)
20     ap.add_argument("--v-term", type=float, required=True)
21     ap.add_argument("--S", type=int, default=10)
22     ap.add_argument("--seed", type=int, default=2025)
23     ap.add_argument("--method", choices=["hold", "linear"], default="hold")
24     a = ap.parse_args()
25     case = Path(a.case_dir)
26
27     def load_path(label):
28         p = case / "paths" / (label.replace("+", "plus") + ".pkl")
29         if not p.exists():
30             raise FileNotFoundError(p)
31         with open(p, "rb") as f:
32             obj = pickle.load(f)
33             assert obj["protocol"] == PROTOCOL and obj["label"] == label
34             return obj
35
36     q2_labels = ["A", "B", "C"]
37     q3_labels = [key(c) for c in COMBOS]
38     q2, d2, q3, d3 = {}, {}, {}, {}
39     for lb in q2_labels:
40         obj = load_path(lb)
41         q2[lb] = obj["recs"]; d2[lb] = float(obj["elapsed_seconds"])
42     for lb in q3_labels:
43         obj = load_path(lb)
44         q3[lb] = obj["recs"]; d3[lb] = float(obj["elapsed_seconds"])
45
46     cfg = dict(start=0, stop=365, S=int(a.S), seed=int(a.seed), lam=0.3, method=a.method,
47               eps_soc=1e-3, tp=1e-6, v_term=float(a.v_term), initial_state=E_INIT)
48     data = load_all()
49     forecasts = load_forecasts(str(case / "forecast" / "预测缓存.npz"))
50     deployed, cumulative = deploy_from_shadow(data, forecasts, q3, cfg)
51
52     save_pickle(case / "q2_best.pkl", dict(protocol=PROTOCOL, best="B", recs=q2["B"],
53                                           q2_all=q2, config=cfg))
54     save_pickle(case / "q3_daystore.pkl", dict(protocol=PROTOCOL, best_key="历史选型执行策略"
55 ,
56                                           store=deployed, all_stores=q3,
57                                           results={k: summary(v) for k, v in q3.items
58
59                                           {"历史选型执行策略": summary(deployed)
60
61                                           config=cfg))
62
63     q2_df = pd.concat([daily_frame(q2[k], k) for k in q2_labels], ignore_index=True)
64     q2_df.to_csv(case / "q2_逐日费用.csv", index=False, encoding="utf-8-sig")
65     q2_df.groupby("模型")[["计划购电费", "调整费", "紧急购电费", "紧急购电量", "总费用"]].sum()

```

```

        .to_csv(
62         case / "q2_汇总.csv", encoding="utf-8-sig")
63     q3_df = pd.concat([daily_frame(q3[k], k) for k in q3_labels] +
64                       [daily_frame(deployed, "历史选型执行策略")], ignore_index=True)
65     q3_df.to_csv(case / "q3_逐日费用.csv", index=False, encoding="utf-8-sig")
66     combo_rows = [{"组合": k, **summary(q3[k])} for k in q3_labels]
67     combo_rows.append({"组合": "历史选型执行策略", **summary(deployed)})
68     pd.DataFrame(combo_rows).to_csv(case / "q3_组合对照.csv", index=False, encoding="utf-8-
69     sig")
70     write_result_workbooks(case, q2["B"], deployed)
71
72     fmeta = json.loads((case / "forecast" / "预测缓存_meta.json").read_text(encoding="utf-8"
73     ))
74     metadata = dict(case_id=a.case_id, protocol=PROTOCOL, project_root=str(HERE.parent),
75     phase="main", forecast_cache=str(case / "forecast" / "预测缓存.npz"),
76     input_hashes=input_hashes(), code_hashes=code_hashes(),
77     parameters=dict(S=int(a.S), seed=int(a.seed), eps_soc=1e-3, tp=1e-6,
78     lam=0.3,
79     method=a.method, v_term=float(a.v_term)),
80     forecast_meta=fmeta)
81     write_json(case / "main_metadata.json", dict(metadata, config=cfg,
82     q2_durations=d2, q3_durations=d3,
83     q2_summary={k: summary(v) for k, v in q2.
84     items()},
85     q3_summary={k: summary(v) for k, v in q3.
86     items()},
87     deployed_summary=summary(deployed),
88     selection_cumulative=cumulative))
89     print(json.dumps(dict(status="PASS", case=str(case)), ensure_ascii=False), flush=True)
90
91 if __name__ == "__main__":
92     main()

```

C11. 问题 4/p10_core.py (369 行)

问题 4 所用的共享内核版本 (eps 归一化修正后: 末槽系数 = $(eps_soc + v_term)/S$)

```

1  # -*- coding: utf-8 -*-
2  """
3  核心库: 数据读取 / 滚动预测 / 整日残差场景 / 两阶段随机LP内核 / 实时调度 / 结算
4  新思路路线: 预测 → 残差场景 → 两阶段随机MILP(实际为LP) → 5倍紧急购电; MPC+调整成本; 波动电价。
5  所有模型为 LP (HiGHS, scipy.optimize.linprog)。
6  """
7  import os
8  import numpy as np
9  import pandas as pd
10 import openpyxl
11 from scipy import sparse
12 from scipy.optimize import linprog
13
14 # 路径: 默认由本文件位置推导 (本文件在 03_代码/, 其上一级即项目根), 不再写死绝对路径。
15 # 2026-09-11 修复: 原值 r"C:\Users\my\Desktop\MILP+两阶段+MPC" 是搬迁前的旧机器路径, 项目移到

```

```

16 # C:\Users\John\Desktop\26题\MILP+两阶段+MPC 后不存在，导致所有 p*.py 直接 FileNotFoundError。
17 # 三个环境变量为可选覆盖，便于把流水线重跑到临时目录、不覆盖已交付的 results/ 与检验日志：
18 # MICROGRID_ROOT / MICROGRID_RES / MICROGRID_LOG
19 _HERE = os.path.dirname(os.path.abspath(__file__))
20 ROOT = os.environ.get("MICROGRID_ROOT", os.path.dirname(_HERE))
21 import sys
22 sys.path.insert(0, os.path.join(ROOT, ".runtime"))
23 SRC = os.path.join(ROOT, "00_题目与原始数据", "C题", "附件")
24 RES = os.environ.get("MICROGRID_RES", os.path.join(ROOT, "04_结果与检验", "results"))
25 LOGD = os.environ.get("MICROGRID_LOG", os.path.join(ROOT, "04_结果与检验", "检验日志"))
26 SEED = 2025
27 DT = 1.0 / 6.0 # 10min in hours
28 ETA_C = 0.9 # charge efficiency (one-way)
29 ETA_D = 0.9 # discharge efficiency (one-way)
30 E_MIN, E_MAX, E_INIT = 1200.0, 10800.0, 6000.0
31 P_MAX_KWH = 5000.0 * DT # max charge/discharge energy per slot (kWh)
32 BIG = 1e7
33 # 吞吐平局价；实际执行另按同SOC消去配对吞吐，核验互斥。
34 TP_PRICE = 1e-6
35
36 # ----- 数据读取
37 def _norm_label(v):
38     if isinstance(v, str):
39         return v.strip()
40     try:
41         return f"{v.hour:02d}:{v.minute:02d}"
42     except Exception:
43         return str(v)
44
45 def load_all():
46     d = {}
47     a1 = pd.read_excel(os.path.join(SRC, "附件1.xlsx"))
48     d["p_fix"] = a1["电价"].to_numpy(float) # (144,)
49     d["L1"] = a1["小区负载"].to_numpy(float)
50     d["G1hat"] = a1["光伏发电预测功率"].to_numpy(float)
51     a2L = pd.read_excel(os.path.join(SRC, "附件2.xlsx"), sheet_name="小区负载")
52     a2G = pd.read_excel(os.path.join(SRC, "附件2.xlsx"), sheet_name="光伏发电实际功率")
53     d["dates"] = pd.to_datetime(a2L.iloc[:, 0]).dt.date.to_numpy()
54     d["L"] = a2L.iloc[:, 1:].to_numpy(float) # (365,144) kW
55     d["G"] = a2G.iloc[:, 1:].to_numpy(float)
56     a3 = pd.read_excel(os.path.join(SRC, "附件3.xlsx"))
57     a3[a3.columns[0]] = a3[a3.columns[0]].ffill()
58     d["a3"] = a3
59     a4 = pd.read_excel(os.path.join(SRC, "附件4.xlsx"))
60     d["P_var"] = a4.iloc[:, 1:].to_numpy(float) # (365,144)
61     # 附件3 索引: (day_idx, release_hour) -> np.array(24)
62     tmap = {"0:00": 0, "6:00": 6, "12:00": 12, "18:00": 18}
63     fcs = {}
64     d0 = pd.Timestamp(d["dates"][0])
65     for _, r in a3.iterrows():
66         di = int((pd.Timestamp(r[a3.columns[0]]) - d0).days)

```

```

67         h0 = tmap[str(r[a3.columns[1]]).strip()]
68         fcs[(di, h0)] = r.iloc[2:26].to_numpy(float)
69     d["fc3"] = fcs
70     return d
71
72 def slot_start_min(j):
73     return 10 * j          # label is interval END: 0:10 -> [0:00,0:10)
74
75 def fmt_min(m):
76     """分钟 -> 标签; 1440 -> '24:00'"""
77     m = int(round(m))
78     if m >= 1440:
79         return "24:00" if m == 1440 else f"{m//60}:{m%60:02d}"
80     return f"{m//60}:{m%60:02d}"
81
82 def fc3_step_day(fcs, di, release_h, slots=None, method="hold"):
83     """当前发布行展开到自然日剩余槽; 小时预报为前一小时的保持值。"""
84     fv = np.asarray(fcs[(di, release_h)])
85     out = np.zeros(144)
86     covered = np.arange(144) >= release_h * 6
87     lead = (np.arange(144)[covered] + 1) / 6 - release_h
88     if method == "hold":
89         out[covered] = fv[np.ceil(lead).astype(int)-1]
90     elif method == "linear":
91         out[covered] = np.interp(lead, np.arange(1, 25), fv)
92     else:
93         raise ValueError(method)
94     return out, covered
95
96 # ----- 预测层
97 FEATURE_LAGS = (1, 2, 7)
98
99 def build_features(X, d_idx_list):
100     """X: (365,144) 原始量; 返回 (rows, feat) 列表。特征严格只用 d 之前的数据。"""
101     rows = []
102     for di in d_idx_list:
103         if di < 7:
104             continue
105         date = pd.Timestamp("2025-01-01") + pd.Timedelta(days=di)
106         dow, doy = date.dayofweek, date.dayofyear
107         for t in range(144):
108             hour = slot_start_min(t) // 60
109             f = [t / 144.0,
110                  np.sin(2 * np.pi * t / 144), np.cos(2 * np.pi * t / 144),
111                  np.sin(2 * np.pi * hour / 24), np.cos(2 * np.pi * hour / 24),
112                  dow / 6.0, doy / 366.0,
113                  X[di - 1, t], X[di - 2, t], X[di - 7, t],
114                  X[di - 7:di, t].mean(), X[di - 7:di, t].std()]
115             rows.append((di, t, f))
116     return rows
117

```

```

118 def rolling_forecast(X, refit_every=7, start=14):
119     """LightGBM 扩展窗口滚动预测（每 refit_every 天重训，start 为首个预测日）。
120     返回 F: (365,144)，不可预测天为 nan。样本外。"""
121     import lightgbm as lgb
122     F = np.full_like(X, np.nan)
123     cache = {}
124     for di in range(start, 365):
125         rid = (di - 7) // refit_every
126         if rid not in cache:
127             train_rows = build_features(X, list(range(7, di)))
128             A = np.array([r[2] for r in train_rows]); y = np.array([X[r[0], r[1]] for r in
129             train_rows])
130             m = lgb.LGBMRegressor(n_estimators=300, learning_rate=0.05, num_leaves=63,
131             min_child_samples=40, subsample=0.9, colsample_bytree
132             =0.9,
133             random_state=SEED, verbose=-1, n_jobs=2)
134             m.fit(A, y)
135             cache = {rid: m}
136             # 特征仅用 di 之前数据
137             date = pd.Timestamp("2025-01-01") + pd.Timedelta(days=di)
138             dow, doy = date.dayofweek, date.dayofyear
139             feats = []
140             for t in range(144):
141                 hour = slot_start_min(t) // 60
142                 feats.append([t / 144.0,
143                 np.sin(2 * np.pi * t / 144), np.cos(2 * np.pi * t / 144),
144                 np.sin(2 * np.pi * hour / 24), np.cos(2 * np.pi * hour / 24),
145                 dow / 6.0, doy / 366.0,
146                 X[di - 1, t], X[di - 2, t], X[di - 7, t],
147                 X[di - 7:di, t].mean(), X[di - 7:di, t].std()])
148             F[di] = cache[rid].predict(np.array(feats))
149     return F
150
151 def baseline_mean7(X):
152     F = np.full_like(X, np.nan)
153     for di in range(7, 365):
154         F[di] = X[di - 7:di].mean(axis=0) # 前7个同槽均值（不含当日）
155     return F
156
157 # ----- LP 内核
158 def solve_plan_lp(p, Ls, Gs, E0, q_old=None, lam=0.0, alpha=0.95,
159     q_fixed=None, E_term=None, eps_soc=0.0, tp=0.0, v_term=0.0):
160     """两阶段随机计划 LP。
161     p: (T,) 电价; Ls,Gs: (S,T) 场景功率 kW; E0: 期初储量;
162     q_old: (T,) 调整基准(Q3), None=无调整费用(Q2);
163     lam>0: 加入 CVaR_alpha 项（对场景日紧急购电成本）。
164     q_fixed: (T,) 固定购电量（实时调度复用本函数）。
165     eps_soc>0: 末槽储电数值平局奖励（元/kWh，逐场景，语义冻结同 v3），在并列解中偏向"不把储能放空"。
166     v_term>0: 跨日库存经济续存价值（元/kWh 内部电量），按等权期望归一注入：
167         每个场景末槽系数为 -(eps_soc + v_term/S)，与紧急项 5p/S 同口径，

```

```

166         保证该参数含义不随情景数  $S$  变化 (q23_janwarm_v4, 修订协议 §13)。
167     tp>0: 充放电吞吐平局价 (元/kWh), 在并列解中排除"同槽同时充放"伪解。
168     返回 dict(q, qEM(S,T), c(S,T), d(S,T), E(S,T), obj, status, dqp, dqm)"""
169     T = len(p); S = Ls.shape[0]
170     adj = q_old is not None
171     n = T + 4 * S * T
172     i_q = 0
173     def i_blk(s, k, t): # k:0 qEM,1 c,2 d,3 E
174         return T + s * 4 * T + k * T + t
175     if adj:
176         i_dqp = T + 4 * S * T; i_dqm = i_dqp + T; n += 2 * T
177     if lam > 0:
178         i_eta = n; i_u = n + 1; n += 1 + S
179     c_vec = np.zeros(n)
180     ub_l = np.zeros(n); ub_u = np.zeros(n)
181     # 目标与边界
182     # At an update the original plan has already been paid for. Only the
183     # incremental adjustment bill is decision-dependent; charging p*q again
184     # would count the replacement energy twice.
185     c_vec[i_q:i_q + T] = 0.0 if adj else p
186     ub_u[i_q:i_q + T] = BIG if q_fixed is None else 0.0
187     if q_fixed is not None:
188         ub_l[i_q:i_q + T] = q_fixed
189         ub_u[i_q:i_q + T] = q_fixed
190     for s in range(S):
191         for k in range(4):
192             s1 = slice(i_blk(s, k, 0), i_blk(s, k, 0) + T)
193             if k == 0:
194                 c_vec[s1] = 5.0 * p / S
195                 ub_u[s1] = BIG
196             elif k in (1, 2):
197                 ub_u[s1] = P_MAX_KWH
198                 if tp > 0:
199                     c_vec[s1] = tp # 吞吐平局价: 排除"同槽同时充放"伪解
200             else:
201                 ub_l[s1] = E_MIN; ub_u[s1] = E_MAX
202                 if v_term < 0:
203                     raise ValueError("v_term 必须非负")
204                 # eps_soc: 逐场景数值平局 (v3 冻结语义); v_term/S: 经济续存价值等权期望
205                 coef = eps_soc + (v_term / S)
206                 if coef > 0:
207                     c_vec[s1][-1] = -coef # 末槽奖励, 消除退化/为日末库存定价
208     if adj:
209         c_vec[i_dqp:i_dqp + T] = 1.5 * p
210         c_vec[i_dqm:i_dqm + T] = -0.5 * p
211         ub_u[i_dqp:i_dqp + T] = BIG; ub_u[i_dqm:i_dqm + T] = BIG
212     if lam > 0:
213         c_vec[i_eta] = lam; c_vec[i_u:i_u + S] = lam / (S * (1 - alpha))
214         ub_l[i_eta] = -BIG; ub_u[i_eta] = BIG; ub_u[i_u:i_u + S] = BIG
215     # 等式: 储能递推 (S*T)
216     rows_e, cols_e, vals_e, be = [], [], [], []

```

```

217     r = 0
218     for s in range(S):
219         for t in range(T):
220             def add(col, val):
221                 rows_e.append(r); cols_e.append(col); vals_e.append(val)
222                 add(i_blk(s, 3, t), 1.0)
223                 add(i_blk(s, 1, t), -ETA_C)
224                 add(i_blk(s, 2, t), 1.0 / ETA_D)
225                 if t > 0:
226                     add(i_blk(s, 3, t - 1), -1.0)
227                     be.append(0.0)
228                 else:
229                     be.append(E0)
230             r += 1
231     if adj:
232         for t in range(T):
233             rows_e += [r, r, r]
234             cols_e += [i_q + t, i_dqp + t, i_dqm + t]
235             vals_e += [1.0, -1.0, 1.0]
236             be.append(q_old[t]); r += 1
237     if E_term is not None:
238         for s in range(S):
239             rows_e.append(r); cols_e.append(i_blk(s, 3, T - 1)); vals_e.append(1.0)
240             be.append(float(E_term)); r += 1
241     A_eq = sparse.coo_matrix((vals_e, (rows_e, cols_e)), shape=(r, n)).tocsr()
242     b_eq = np.array(be)
243     # 不等式: 供能 (S*T): -q -qEM -d +c <= (G-L)*DT
244     rows_u, cols_u, vals_u, bu = [], [], [], []
245     r = 0
246     for s in range(S):
247         for t in range(T):
248             rows_u += [r, r, r, r]
249             cols_u += [i_q + t, i_blk(s, 0, t), i_blk(s, 2, t), i_blk(s, 1, t)]
250             vals_u += [-1.0, -1.0, -1.0, 1.0]
251             bu.append((Gs[s, t] - Ls[s, t]) * DT); r += 1
252     if lam > 0:
253         # C_s - eta - u_s <= 0 ; 1/(1-alpha) occurs in objective only.
254         for s in range(S):
255             for t in range(T):
256                 rows_u.append(r); cols_u.append(i_blk(s, 0, t)); vals_u.append(5.0 * p[t])
257                 rows_u.append(r); cols_u.append(i_eta); vals_u.append(-1.0)
258                 rows_u.append(r); cols_u.append(i_u + s); vals_u.append(-1.0)
259                 bu.append(0.0); r += 1
260     A_ub = sparse.coo_matrix((vals_u, (rows_u, cols_u)), shape=(r, n)).tocsr()
261     b_ub = np.array(bu)
262     res = linprog(c_vec, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
263                 bounds=np.stack([ub_l, ub_u], axis=1), method="highs")
264     out = {"status": res.status, "msg": res.message, "obj": res.fun if res.success else np.
265           nan}
266     if res.success:
267         x = res.x

```

```

267     out["q"] = x[i_q:i_q + T].copy()
268     out["qEM"] = np.stack([x[i_blk(s, 0, 0):i_blk(s, 0, 0) + T] for s in range(S)])
269     out["c"] = np.stack([x[i_blk(s, 1, 0):i_blk(s, 1, 0) + T] for s in range(S)])
270     out["d"] = np.stack([x[i_blk(s, 2, 0):i_blk(s, 2, 0) + T] for s in range(S)])
271     out["E"] = np.stack([x[i_blk(s, 3, 0):i_blk(s, 3, 0) + T] for s in range(S)])
272     if adj:
273         out["dqp"] = x[i_dqp:i_dqp + T].copy(); out["dqm"] = x[i_dqm:i_dqm + T].copy()
274     return out
275
276 def hindsight_dispatch(p, L, G, E0, q, *, eps_soc=1e-3, tp=TP_PRICE, v_term=0.0):
277     """事后完美信息调度，仅用于诊断；禁止用于正式策略回测。"""
278     T = len(p); S = 1
279     return solve_plan_lp(p, L[None, :], G[None, :], E0, q_fixed=q,
280                          eps_soc=eps_soc, tp=tp, v_term=v_term)
281
282
283 def realtime_dispatch(p, L, G, E0, q, *, forecast_L=None, forecast_G=None,
284                      eps_soc=1e-3, tp=TP_PRICE, v_term=0.0):
285     """Causal 10-minute receding-horizon execution with a fixed market plan.
286
287     Only the current measured L/G replace their forecasts. Future actual values
288     are never passed into the optimization. Callers must supply forecasts made
289     using information available before execution. All strategies use this rule.
290     """
291     if forecast_L is None or forecast_G is None:
292         raise ValueError("实时执行必须显式提供当时可得预测，禁止用未来实际曲线代替")
293     T = len(p)
294     out = {k: np.zeros((1, T)) for k in ("c", "d", "E", "qEM")}
295     state = float(E0)
296     for t in range(T):
297         r = causal_step(p[t:], q[t:], state, L[t], G[t],
298                        np.asarray(forecast_L)[t:], np.asarray(forecast_G)[t:],
299                        eps_soc=eps_soc, tp=tp, v_term=v_term)
300         for k in out:
301             out[k][0, t] = r[k]
302         state = r["E"]
303     out.update(status=0, msg="causal execution", q=np.asarray(q).copy())
304     return out
305
306
307 def causal_step(p, q, E0, current_L, current_G, forecast_L, forecast_G, *,
308                eps_soc=1e-3, tp=TP_PRICE, v_term=0.0):
309     """Execute one slot. Interface deliberately accepts no future observations."""
310     lf = np.asarray(forecast_L, dtype=float).copy()
311     gf = np.asarray(forecast_G, dtype=float).copy()
312     lf[0], gf[0] = current_L, current_G
313     r = solve_plan_lp(np.asarray(p), lf[None, :], gf[None, :], E0,
314                      q_fixed=np.asarray(q), eps_soc=eps_soc, tp=tp, v_term=v_term)
315     if r["status"] != 0:
316         raise RuntimeError(r["msg"])
317     # Remove any solver-level simultaneous throughput exactly, preserving SOC.

```

```

318 # Supply weakly increases; surplus is spilled, as allowed by the model.
319 c, d = float(r["c"][0, 0]), float(r["d"][0, 0])
320 paired = min(c, d / (ETA_C * ETA_D))
321 c -= paired
322 d -= paired * ETA_C * ETA_D
323 em = max(0.0, (current_L-current_G)*DT + c-d-float(q[0]))
324 return dict(c=c, d=d, E=float(E0+ETA_C*c-d/ETA_D), qEM=em)
325
326
327 # ----- 场景
328 def sample_scenarios(res_pool, hatL, hatG, S, rng):
329     """res_pool: list of (resL(144), resG(144)); 整日配对抽样。"""
330     n = len(res_pool)
331     idx = rng.choice(n, size=min(S, n), replace=n < S)
332     Ls = np.zeros((len(idx), 144)); Gs = np.zeros((len(idx), 144))
333     for k, i in enumerate(idx):
334         rL, rG = res_pool[i]
335         Ls[k] = np.maximum(0.0, hatL + rL)
336         Gs[k] = np.maximum(0.0, hatG + rG)
337     return Ls, Gs
338
339 # ----- 结算
340 def merge_emergency_intervals(qEM, thr=1e-6):
341     """qEM: (144,) -> [(start_min, end_min, qty)]"""
342     ivs = []
343     j = 0
344     while j < 144:
345         if qEM[j] > thr:
346             k = j; qty = 0.0
347             while k < 144 and qEM[k] > thr:
348                 qty += qEM[k]; k += 1
349             ivs.append((slot_start_min(j), slot_start_min(k - 1) + 10, qty))
350             j = k
351         else:
352             j += 1
353     return ivs
354
355 def settle_q2(p, q, qEM):
356     """Q2 全天费用 = 计划 p * q + 紧急 5p * qEM"""
357     return float(np.sum(p * q) + np.sum(5 * p * qEM))
358
359 def settle_q3(p, q0, updates):
360     """Q3: updates = [(tau_slot, q_new(T'), dqp, dqm)] 顺序 (相对上一确认计划)。
361     返回 (总计划相关费用, 调整费用合计, 最终执行曲线 qa)"""
362     qa = q0.copy(); adj_fee = 0.0
363     for tau, qn, dqp, dqm in updates:
364         T = len(qn)
365         adj_fee += float(np.sum(1.5 * p[tau:tau + T] * dqp - 0.5 * p[tau:tau + T] * dqm))
366         qa[tau:tau + T] = qn
367     plan_fee = float(np.sum(p * q0))
368     return plan_fee + adj_fee, adj_fee, qa

```

C12. 问题 4/p102_q4_core.py (413 行)

问题 4 引擎：口径 A/B、跨日库存续存价值 v 、6/12/18 更新通道与紧急购电通道

```
1 # -*- coding: utf-8 -*-
2 """问题4 口径B (0:00 不知道当日电价) 核心模块。
3
4 与原口径A 的唯一区别 (改造方案 §5.A)：**决策用预测价 `price_dec`，结算用真实价 `price_settle`**
5 。
6 - 日前计划 LP、6/12/18 更新 LP、10 分钟执行器 causal_step 一律使用 `price_dec`；
7 - 账单 `pf = p_settle@q0`、`ef = 5*p_settle@qEM`、调整费按 `p_settle` 的增量重算；
8 - 跨日库存续存价值  $v$  为**逐日滚动** (L1 规则)，用因果的价格预测构造；
9 - 原实现 `ValuePolicy.value_for` 用 `fp[di+1]` (含当日实际价) 属前视，本模块改为 `fp[di]` (见 `
    残余问题.md`)。
10
11 复用的部分 (与 q23_janwarm_v4 一致)：1 月因果冷启动、整日配对残差场景、S=10/seed、因果执行器、
12 Q3 式更新组合与历史选型部署、结算基准 (相对上次确认计划的序贯增量)。
13 """
14
15 from __future__ import annotations
16
17 import hashlib
18 import json
19 import os
20 import sys
21 import time
22 from pathlib import Path
23
24 import numpy as np
25 import pandas as pd
26
27 HERE = Path(__file__).resolve().parent
28 ROOT = HERE.parent
29 sys.path.insert(0, str(HERE))
30
31 from p10_core import (DT, E_INIT, E_MAX, E_MIN, ETA_C, ETA_D, P_MAX_KWH, TP_PRICE, # noqa:
32                       E402
33                       causal_step, fc3_step_day, fmt_min, load_all, solve_plan_lp)
34 from p15_forecasts import baseline_mean7, rolling_forecast, select_past_only # noqa: E402
35
36 PROTOCOL = "q4_scenarioB_v1"
37 COMBOS = [(), (6,), (12,), (18,), (6, 12), (6, 18), (12, 18), (6, 12, 18)]
38 ETA = ETA_C
39 V_OFFPEAK_SLOTS = 30 # 0:00—5:00 共 30 槽 (与 L1 规则一致)
40
41 def key(combo):
42     return "+".join(map(str, combo)) or "none"
43
44 def phase_of(di):
45     if di < 31:
46         return "init"
47     if di < 90:
48         return "calibration"
```

```

47     return "eval"
48
49
50 def sha256(path):
51     h = hashlib.sha256()
52     with open(path, "rb") as f:
53         for block in iter(lambda: f.read(1 << 20), b''):
54             h.update(block)
55     return h.hexdigest()
56
57
58 def code_hashes():
59     files = [HERE / n for n in ("p10_core.py", "p15_forecasts.py", "p16_price_forecast.py",
60                                "p102_q4_core.py")]
61     return {str(p): sha256(p) for p in files if p.exists()}
62
63
64 def input_hashes():
65     ad = ROOT / "00_题目与原始数据" / "C题" / "附件"
66     return {str(p): sha256(p) for p in sorted(ad.glob("附件[1-4].xlsx"))}
67
68
69 def write_json(path, obj):
70     path = Path(path)
71     path.parent.mkdir(parents=True, exist_ok=True)
72     with open(path, "w", encoding="utf-8") as f:
73         json.dump(obj, f, ensure_ascii=False, indent=2, default=str)
74
75
76 def save_pickle(path, obj):
77     import pickle
78     path = Path(path)
79     path.parent.mkdir(parents=True, exist_ok=True)
80     with open(path, "wb") as f:
81         pickle.dump(obj, f, protocol=pickle.HIGHEST_PROTOCOL)
82
83
84 # ----- 负荷/光伏预测（同 v4 算法）
85 def make_loadpv_forecasts(out_dir, reuse=True):
86     out_dir = Path(out_dir)
87     fdir = out_dir / "forecast"
88     fdir.mkdir(parents=True, exist_ok=True)
89     cache = fdir / "负荷光伏预测缓存.npz"
90     meta_path = fdir / "负荷光伏预测缓存_meta.json"
91     expected = dict(protocol=PROTOCOL, training_start=14, input_hashes=input_hashes(),
92                     code_hashes=code_hashes(), early_policy="lag1_for_di_lt_14", seed=2025)
93     if reuse and cache.exists() and meta_path.exists():
94         meta = json.loads(meta_path.read_text(encoding="utf-8"))
95         if all(meta.get(k) == expected.get(k) for k in expected):
96             with np.load(cache) as z:

```

```

97         if all(k in z for k in ("FL", "FG")):
98             if np.isfinite(z["FL"][:14]).all() and np.isfinite(z["FG"][:14]).all():
99                 return str(cache), meta
100 data = load_all()
101 meta = expected.copy()
102 saved, selections, metrics = {}, {}, {}
103 for prefix, actual in (("L", data["L"]), ("G", data["G"])):
104     lag = np.empty_like(actual)
105     lag[0] = actual[0]
106     lag[1:] = actual[:-1]
107     m7 = baseline_mean7(actual)
108     m7[:7] = lag[:7]
109     lgb = rolling_forecast(actual, start=14)
110     lgb[:14] = lag[:14]
111     candidates = {"lgb": lgb, "lag1": lag, "m7": m7}
112     forecast, selected = select_past_only(candidates, actual, start=14)
113     saved["F" + prefix] = forecast
114     selections[prefix] = selected
115     for name, arr in candidates.items():
116         saved["F" + prefix + "_" + name] = arr
117     for name, arr in dict(candidates, online=forecast).items():
118         err = arr[31:] - actual[31:]
119         metrics[prefix + "_" + name + "_eval"] = dict(
120             MAE=float(np.abs(err).mean()), RMSE=float(np.sqrt((err ** 2).mean())),
121             bias=float(err.mean()))
122 np.savez_compressed(cache, **saved)
123 pd.DataFrame(dict(日期=data["dates"], 负荷模型=selections["L"], 光伏模型=selections["G"]
124 ).to_csv(
125     fdir / "逐日预测选型.csv", index=False, encoding="utf-8-sig")
126 meta.update(dict(metrics=metrics, created_at=time.strftime("%Y-%m-%dT%H:%M:%S%z")))
127 write_json(meta_path, meta)
128 return str(cache), meta
129
130 def load_loadpv_forecasts(path):
131     with np.load(path) as z:
132         f = {k: z[k].copy() for k in ("FL", "FG")}
133     if not np.isfinite(f["FL"][:14]).all() or not np.isfinite(f["FG"][:14]).all():
134         raise ValueError("1月早期预测不完整")
135     return f
136
137
138 # ----- 价格序列与滚动 v
139 def price_dec(price_fc: dict, track: str, di: int, t0: int = 0, refresh_mode: str = "none")
140 :
141     """决策用价格序列（从槽 t0 起）。refresh_mode='none' → 严格口径B（0:00 一次定全天）；
142     'hourly' → 在 6/12/18 刷新节点改用该时刻的预测版本。"""
143     base = np.asarray(price_fc[f"FP_{track}"][di], float)
144     if refresh_mode == "none" or t0 == 0:
145         return base[t0:]
146     h = t0 // 6

```

```

146     k = f"FP_h{h}"
147     if k in price_fc:
148         return np.asarray(price_fc[k][di], float)[t0:]
149     return base[t0:]
150
151
152 def rolling_v(price_fc: dict, track: str, di: int):
153     """跨日库存续存价值:  $v = \text{mean}(0:00-5:00 \text{ 价格预测})/\eta$ , 逐日滚动。
154     因果性: hist7 等预测只由 < di 天的真实价构成; **不使用 fp[di+1]** (原实现前视 1 天)。"""
155     base = np.asarray(price_fc[f"FP_{track}"][di], float)
156     return float(base[:V_OFFPEAK_SLOTS].mean() / ETA)
157
158
159 # ----- 场景与冷启动
160 def scenarios(data, forecasts, di, release_h, S, seed, method, use_fc3=False):
161     """整日配对残差场景。**use_fc3=True 时用附件3 已发布的小时预报**分解为 10 分钟光伏预测
162     (并与该预报构造残差), 与 `p22_backtest`/`p95` 的机制一致——问题4 的 Q3 型轨
163     (combo 不为 None) 必须使用附件3, 否则"更新"通道会退化为仅重抽场景。
164
165     2026-09-12 晚修正: 本函数初版遗漏了 `use_fc3` 分支 (恒用负荷/光伏点预测),
166     导致 Q4 的 8 组合与部署轨不含附件3 信息、且 D4 的 F 轨效果恒为 0 ( $N-F = 0.00$ )。
167     Q4-2 的 A/B/C 轨 (combo=None) 在两个引擎中本来都不使用附件3, 故不受此缺陷影响。
168     """
169     lf = np.asarray(forecasts["FL"][di], float)
170     if use_fc3:
171         gf = fc3_step_day(data["fc3"], di, release_h, method=method)[0]
172     else:
173         gf = np.asarray(forecasts["FG"][di], float)
174     hist = np.arange(14, di, dtype=int)
175     if len(hist) == 0:
176         return lf[None, :], gf[None, :], [], lf, gf, 0
177     rng = np.random.default_rng(np.random.SeedSequence([int(seed), int(di), int(release_h)]))
178     days = rng.choice(hist, size=int(S), replace=len(hist) < int(S))
179     r1 = data["L"][days] - forecasts["FL"][days]
180     if use_fc3:
181         rg = np.stack([data["G"][d] - fc3_step_day(data["fc3"], int(d), release_h, method=method)[0]
182                        for d in days])
183     else:
184         rg = data["G"][days] - forecasts["FG"][days]
185     return np.maximum(0.0, lf[None, :] + r1), np.maximum(0.0, gf[None, :] + rg), days.tolist(), lf, gf, len(hist)
186
187
188 def cold_day(data, di, p_settle, E0):
189     L, G = data["L"][di], data["G"][di]
190     qem = np.maximum(0.0, (L - G) * DT)
191     z = np.zeros(144, dtype=float)
192     return dict(
193         日期=str(data["dates"][di]), di=di, q=z.copy(), q0=z.copy(), qc=z.copy(),

```

```

194     c=z.copy(), d=z.copy(), E=np.full(144, float(E0)), qEM=qem,
195     E0=float(E0), Eend=float(E0), updates=[], scenario_days={"0": []},
196     scenario_pool_counts={"0": 0}, planned_emergency_qty=0.0, planned_emergency_fee
=0.0,
197     计划费=0.0, 调整费=0.0, 紧急费=float(np.sum(5.0 * p_settle * qem)),
198     总费用=float(np.sum(5.0 * p_settle * qem)), 阶段=phase_of(di), cold_start=True,
199     v_term_used=0.0, price_mae=float(np.abs(p_settle).mean()) * 0.0,
200 )
201
202
203 def simulate_day(data, forecasts, price_fc, di, E0, *, track="hist7", refresh_mode="none",
204     v_mode="rolling", price_source="forecast", v_const=None, update_mode="FM",
205     model="B", combo=None, S=10, seed=2025, lam=.3, method="hold", eps_soc=1e
-3,
206     tp=TP_PRICE):
207     p_settle = np.asarray(data["P_var"][di], float) # 结算用真实价
208     if di == 0:
209         return cold_day(data, di, p_settle, E0)
210     use_upd = combo is not None
211     # use_fc3 (是否用附件3 已发布预报建情景池) 默认与 use_upd 一致:
212     #   combo=None (Q4-2 的 A/B/C 轨) → 点预报/持久化 (与 Q2 同族)
213     #   combo=() (Q4-3 的 none 轨) 与各事件轨 → 附件3 (与 Q3 家族同族)
214     # 2026-09-12 晚 (独立复核提出"一个布尔表达了两件事"): 新增**默认关闭**的环境变量开关,
215     # 仅用于把 none 轨的信息集切回点预报, 以量化该约定对交付数字的影响 (敏感性实验)。
216     # 不设该变量时行为与冻结版**逐位相同**。
217     use_fc3 = use_upd
218     if os.environ.get("Q4_NONE_NO_FC3") == "1" and combo is not None and len(combo) == 0:
219         use_fc3 = False
220     ls, gs, days, lf, gf, pool_n = scenarios(data, forecasts, di, 0, S, seed, method,
221         use_fc3=use_fc3)
222
223     def pdec(t0):
224         """决策价: price_source='actual' → 口径A (0:00 已知当日真实曲线, 用于隔离电价信息效
225         应);
226         'forecast' → 口径B (只用 0:00 可得的预测价)。"""
227         if price_source == "actual":
228             return p_settle[t0:]
229         return price_dec(price_fc, track, di, t0, refresh_mode)
230
231     p_dec0 = pdec(0)
232     v = (float(v_const) if v_const is not None
233         else (rolling_v(price_fc, track, di) if v_mode == "rolling" else 0.0))
234     if model == "A":
235         ls, gs = lf[None, :], gf[None, :]
236     plan = solve_plan_lp(p_dec0, ls, gs, E0, lam=lam if model == "C" else 0.0,
237         eps_soc=eps_soc, tp=tp, v_term=v)
238     if plan["status"] != 0:
239         raise RuntimeError(plan["msg"])
240     q0 = plan["q"].copy()
241     qc = q0.copy()
242     state = float(E0)

```

```

242 arrays = {k: np.zeros(144, dtype=float) for k in ("c", "d", "E", "qEM")}
243 updates, draws, pools = [], {"0": list(days)}, {"0": int(pool_n)}
244 expected_qty = float(np.mean(plan["qEM"].sum(axis=1)))
245 expected_fee = float(np.mean(np.sum(5.0 * p_settle[None, :] * plan["qEM"], axis=1)))
246 adj = 0.0
247 for t in range(144):
248     hour = t // 6
249     if use_upd and t > 0 and t % 6 == 0 and hour in combo:
250         ls, gs, days, lf, gf, pool_n = scenarios(data, forecasts, di, hour, S, seed,
method,
251                                             use_fc3=True)
252         draws[str(hour)] = list(days)
253         pools[str(hour)] = int(pool_n)
254         if update_mode == "F":
255             # D4 式拆分 (Q4 版): 只用新发布的附件3 预报刷新日内预测 (执行器视界随之更新),
256             # **锁定日前购电计划 q0**, 不做市场购电调整 => 调整费为 0、qc 不变。
257             updates.append(dict(
258                 tau=t, hour=hour, old=qc[t:].copy(), new=qc[t:].copy(), fee=0.0,
259                 E0=float(state), scenario_days=list(days), scenario_pool_n=int(pool_n),
260                 expected_emergency_qty=0.0, expected_emergency_fee=0.0, forecast_only=
True))
261         else:
262             old = qc[t:].copy()
263             p_du = pdec(t) # 决策价 (口径A 真实 /
口径B 预测, 可刷新)
264             update = solve_plan_lp(p_du, ls[:, t:], gs[:, t:], state, q_old=old,
265                                     eps_soc=eps_soc, tp=tp, v_term=v)
266             if update["status"] != 0:
267                 raise RuntimeError(update["msg"])
268             qc[t:] = update["q"]
269             plus = np.maximum(qc[t:] - old, 0.0)
270             minus = np.maximum(old - qc[t:], 0.0)
271             fee = float(p_settle[t:] @ (1.5 * plus - .5 * minus)) # 账单用真实价
272             adj += fee
273             updates.append(dict(
274                 tau=t, hour=hour, old=old, new=qc[t:].copy(), fee=fee, E0=float(state),
275                 scenario_days=list(days), scenario_pool_n=int(pool_n),
276                 expected_emergency_qty=float(np.mean(update["qEM"].sum(axis=1))),
277                 expected_emergency_fee=float(np.mean(np.sum(5.0 * p_settle[t:][None, :]
* update["qEM"], axis=1))),
278                 ))
279             p_dt = pdec(t) # 执行器决策价
280             step = causal_step(p_dt, qc[t:], state, data["L"][di, t], data["G"][di, t],
281                                 lf[t:], gf[t:], eps_soc=eps_soc, tp=tp, v_term=v)
282             for k in arrays:
283                 arrays[k][t] = step[k]
284             state = float(step["E"])
285             pf = float(p_settle @ q0)
286             ef = float(5.0 * p_settle @ arrays["qEM"])
287             return dict(
288                 日期=str(data["dates"][di]), di=di, q=q0, q0=q0, qc=qc, **arrays,

```

```

289     E0=float(E0), Eend=float(state), updates=updates,
290     scenario_days=draws, scenario_pool_counts=pools,
291     planned_emergency_qty=expected_qty, planned_emergency_fee=expected_fee,
292     计划费=pf, 调整费=adj, 紧急费=ef, 总费用=pf + adj + ef,
293     阶段=phase_of(di), cold_start=False, v_term_used=float(v),
294     price_mae=float(np.abs(p_dec0 - p_settle).mean()),
295 )
296
297
298 # ----- 汇总与落盘
299 def summary(recs, start=None, stop=None):
300     rows = recs[slice(start, stop)] if start is not None or stop is not None else recs
301     return dict(
302         天数=len(rows), 计划购电费=float(sum(r["计划费"] for r in rows)),
303         调整费=float(sum(r["调整费"] for r in rows)),
304         紧急购电费=float(sum(r["紧急费"] for r in rows)),
305         紧急购电量=float(sum(np.sum(r["qEM"]) for r in rows)),
306         总费用=float(sum(r["总费用"] for r in rows)),
307         初SOC=float(rows[0]["E0"]) if rows else None,
308         末SOC=float(rows[-1]["Eend"]) if rows else None,
309         均值日度价格预测MAE=float(np.mean([r["price_mae"] for r in rows])) if rows else None,
310         均值v_term=float(np.mean([r["v_term_used"] for r in rows])) if rows else None,
311     )
312
313
314 def daily_frame(recs, label):
315     return pd.DataFrame([
316         dict(日期=r["日期"], 模型=label, 计划购电费=r["计划费"], 调整费=r["调整费"],
317             紧急购电费=r["紧急费"], 紧急购电量=float(np.sum(r["qEM"])), 总费用=r["总费用"],
318             末SOC=r["Eend"], 阶段=r["阶段"], v_term=r["v_term_used"], 价格预测MAE=r["
319 price_mae"],
320             计划紧急购电量=r["planned_emergency_qty"], 计划紧急购电费=r["
321 planned_emergency_fee"])
322         for r in recs])
323
324 def deploy_from_shadow(data, forecasts, price_fc, shadows, cfg):
325     ordered = [key(c) for c in COMBOS]
326     n = len(next(iter(shadows.values())))
327     cumulative = {k: 0.0 for k in ordered}
328     state = E_INIT
329     deployed = []
330     for i in range(n):
331         di = shadows[ordered[0]][i]["di"]
332         selected = "none" if di == 0 else min(ordered, key=lambda k: cumulative[k])
333         combo = COMBOS[ordered.index(selected)]
334         rec = simulate_day(data, forecasts, price_fc, di, state, combo=combo, **cfg)
335         rec["selected_combo"] = selected
336         rec["selection_history_end"] = di - 1
337         deployed.append(rec)
338         state = rec["Eend"]

```

```

338         for k in ordered:
339             cumulative[k] += shadows[k][i]["总费用"]
340     return deployed, cumulative
341
342
343 def write_result_workbooks(out_dir, q42_b, q43_deployed, dates_from="2025-02-01"):
344     import openpyxl
345     tpl = ROOT / "00_题目与原始数据" / "C题" / "附件" / "附件5"
346     out_dir = Path(out_dir)
347     dates = [d.strftime("%Y/%m/%d") for d in pd.date_range(dates_from, "2025-12-31")]
348     buckets = [(0, 24), (24, 48), (48, 72), (72, 96), (96, 120), (120, 144)]
349     labels = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00", "16:00-20:00", "20:00-24:00"]
350
351     def fill_plan(ws, recs, field, include_adj=False):
352         for j in range(144):
353             ws.cell(1, 2 + j, f"{fmt_min(j * 10)}-{fmt_min((j + 1) * 10)}")
354         for i, (dt, r) in enumerate(zip(dates, recs)):
355             q = np.asarray(r[field])
356             ws.cell(2 + i, 1, dt)
357             for j in range(144):
358                 ws.cell(2 + i, 2 + j, round(float(q[j]), 2))
359             ws.cell(2 + i, 146, round(float(q.sum()), 2))
360             fee = r["计划费"] + r["调整费"] if include_adj else r["计划费"]
361             ws.cell(2 + i, 147, round(float(fee), 2))
362
363     def fill_storage(ws, recs):
364         for row in range(2, ws.max_row + 1):
365             for col in range(1, 7):
366                 ws.cell(row, col).value = None
367         row = 2
368         for dt, r in zip(dates, recs):
369             for bi, (a, b) in enumerate(buckets):
370                 ws.cell(row, 1, dt if bi == 0 else None)
371                 ws.cell(row, 2, labels[bi])
372                 ws.cell(row, 3, round(float(np.sum(r["c"][a:b])), 2))
373                 ws.cell(row, 4, round(float(np.sum(r["d"][a:b])), 2))
374                 if bi == 0:
375                     ws.cell(row, 5, "0:00"); ws.cell(row, 6, round(float(r["E0"]), 2))
376                 if bi == 1:
377                     ws.cell(row, 5, "24:00"); ws.cell(row, 6, round(float(r["Eend"]), 2))
378                 row += 1
379
380     def fill_emergency(ws, recs):
381         for row in range(2, ws.max_row + 1):
382             for col in range(1, 4):
383                 ws.cell(row, col).value = None
384         row = 2
385         for dt, r in zip(dates, recs):
386             qem = np.asarray(r["qEM"]); t = 0; intervals = []
387             while t < 144:

```

```

388         if qem[t] <= 1e-6:
389             t += 1; continue
390         a = t; amount = 0.0
391         while t < 144 and qem[t] > 1e-6:
392             amount += float(qem[t]); t += 1
393             intervals.append((a, t, amount))
394         if not intervals:
395             ws.cell(row, 1, dt); row += 1
396         else:
397             for a, b, amount in intervals:
398                 ws.cell(row, 1, dt); ws.cell(row, 2, f"{fmt_min(a * 10)}-{fmt_min(b *
399                 10)}")
400                 ws.cell(row, 3, round(amount, 2)); row += 1
401
402         wb = openpyxl.load_workbook(tp1 / "result4-2.xlsx")
403         fill_plan(wb["计划购电量"], q42_b[31:], "q0")
404         fill_storage(wb["充放电量"], q42_b[31:])
405         fill_emergency(wb["紧急购电量"], q42_b[31:])
406         wb.save(out_dir / "result4-2.xlsx")
407
408         wb = openpyxl.load_workbook(tp1 / "result4-3.xlsx")
409         fill_plan(wb["计划购电量"], q43_deployed[31:], "q0")
410         fill_plan(wb["调整购电量"], q43_deployed[31:], "qc", include_adj=True)
411         fill_storage(wb["充放电量"], q43_deployed[31:])
412         fill_emergency(wb["紧急购电量"], q43_deployed[31:])
413         wb.save(out_dir / "result4-3.xlsx")

```

C13. 问题 4/p102c_q4_finish.py (96 行)

问题 4 结果合成与落盘（五格汇总、审计项登记）

```

1  # -*- coding: utf-8 -*-
2  """问题4 口径B: 单臂合成器（读取 paths/*.pkl, 执行部署策略并写出全部产物）。"""
3  import argparse, json, pickle, sys
4  from pathlib import Path
5  import numpy as np
6  import pandas as pd
7
8  HERE = Path(__file__).resolve().parent
9  sys.path.insert(0, str(HERE))
10 from p102_q4_core import (COMBOS, E_INIT, PROTOCOL, code_hashes, daily_frame, # noqa: E402
11                           deploy_from_shadow, input_hashes, key, load_all,
12                           load_loadpv_forecasts, save_pickle, summary, write_json,
13                           write_result_workbooks)
14 from p16_price_forecast import load_cache # noqa: E402
15
16
17 def main():
18     ap = argparse.ArgumentParser()
19     ap.add_argument("--case-dir", required=True)
20     ap.add_argument("--case-id", required=True)
21     ap.add_argument("--track", default="hist7")
22     ap.add_argument("--refresh-mode", choices=["none", "hourly"], default="none")

```

```

23 ap.add_argument("--v-mode", choices=["rolling", "zero"], default="rolling")
24 ap.add_argument("--price-source", choices=["forecast", "actual"], default="forecast")
25 ap.add_argument("--v-const", type=float, default=None)
26 a = ap.parse_args()
27 case = Path(a.case_dir)
28
29 def load_path(label):
30     p = case / "paths" / (label.replace("+", "plus") + ".pkl")
31     if not p.exists():
32         raise FileNotFoundError(p)
33     with open(p, "rb") as f:
34         obj = pickle.load(f)
35         assert obj["protocol"] == PROTOCOL and obj["label"] == label
36         return obj
37
38 q2_labels = ["A", "B", "C"]
39 q3_labels = [key(c) for c in COMBOS]
40 q2, q3, d2, d3 = {}, {}, {}, {}
41 for lb in q2_labels:
42     o = load_path(lb); q2[lb] = o["recs"]; d2[lb] = float(o["elapsed_seconds"])
43 for lb in q3_labels:
44     o = load_path(lb); q3[lb] = o["recs"]; d3[lb] = float(o["elapsed_seconds"])
45
46 cfg = dict(track=a.track, refresh_mode=a.refresh_mode, v_mode=a.v_mode,
47            price_source=a.price_source, v_const=a.v_const, S=10, seed=2025,
48            eps_soc=1e-3, tp=1e-6, lam=.3, method="hold")
49 data = load_all()
50 forecasts = load_loadpv_forecasts(case / "forecast" / "负荷光伏预测缓存.npz")
51 price_fc = load_cache(case / "forecast" / "电价预测缓存.npz")
52 deployed, cumulative = deploy_from_shadow(data, forecasts, price_fc, q3, cfg)
53
54 save_pickle(case / "q42_best.pkl", dict(protocol=PROTOCOL, best="B", recs=q2["B"],
55                                         q2_all=q2, config=cfg))
56 save_pickle(case / "q43_daystore.pkl", dict(protocol=PROTOCOL, best_key="历史选型执行策略",
57                                             store=deployed, all_stores=q3,
58                                             results={k: summary(v) for k, v in q3.items
59                                                         ()} |
60                                                         {"历史选型执行策略": summary(deployed
61                                                         config=cfg))
62 q2_df = pd.concat([daily_frame(q2[k], k) for k in q2_labels], ignore_index=True)
63 q2_df.to_csv(case / "q42_逐日费用.csv", index=False, encoding="utf-8-sig")
64 q2_df.groupby("模型")[["计划购电费", "调整费", "紧急购电费", "紧急购电量", "总费用"]].sum().to_csv(
65     case / "q42_汇总.csv", encoding="utf-8-sig")
66 q3_df = pd.concat([daily_frame(q3[k], k) for k in q3_labels] +
67                  [daily_frame(deployed, "历史选型执行策略")], ignore_index=True)
68 q3_df.to_csv(case / "q43_逐日费用.csv", index=False, encoding="utf-8-sig")
69 rows = [{"组合": k, **summary(q3[k])} for k in q3_labels]
70 rows.append({"组合": "历史选型执行策略", **summary(deployed)})

```

```

70 pd.DataFrame(rows).to_csv(case / "q43_组合对照.csv", index=False, encoding="utf-8-sig")
71 write_result_workbooks(case, q2["B"], deployed)
72
73 pmeta = json.loads((case / "forecast" / "电价预测缓存_meta.json").read_text(encoding="
74 utf-8"))
75 lmeta = json.loads((case / "forecast" / "负荷光伏预测缓存_meta.json").read_text(encoding=
76 "utf-8"))
77 metadata = dict(case_id=a.case_id, protocol=PROTOCOL, project_root=str(HERE.parent),
78 phase="main", 口径="B (0:00 不知道当日电价; 决策用预测价、结算用真实价)",
79 price_track=a.track, refresh_mode=a.refresh_mode, v_mode=a.v_mode,
80 price_source=a.price_source, v_const=a.v_const,
81 input_hashes=input_hashes(), code_hashes=code_hashes(),
82 price_forecast_meta=dict(chosen_track=pmeta.get("chosen_track"),
83 metrics=pmeta.get("metrics"),
84 audit_no_future=pmeta.get("audit_no_future")),
85 loadpv_forecast_meta=lmeta,
86 parameters=dict(S=10, seed=2025, eps_soc=1e-3, tp=1e-6, lam=.3, method=
87 "hold"))
88 write_json(case / "main_metadata.json", dict(metadata, config=cfg, q42_durations=d2,
89 q43_durations=d3,
90 q42_summary={k: summary(v) for k, v in q2.
91 items()},
92 q43_summary={k: summary(v) for k, v in q3.
93 items()},
94 deployed_summary=summary(deployed),
95 selection_cumulative=cumulative))
96 print(json.dumps(dict(status="PASS", case=str(case)), ensure_ascii=False), flush=True)
97
98 if __name__ == "__main__":
99     main()

```

C14. 问题 4/p105_q4_specified.py (170 行)

问题 4 四个指定日期表的生成 (表 1/2/3 与 xlsx 交叉核对)

```

1 # -*- coding: utf-8 -*-
2 """问题4 口径B: 四个指定日期的表1/表2/表3 生成器 (Q4-2 B 与 Q4-3 部署)。
3
4 来源: 口径B 正式格 (默认 重跑_Q4_B_20260912/B_v) 的 q42_best.pkl 与 q43_daystore.pkl。
5 口径: 决策用价格预测、**账单一律用附件4 真实价** (与 p103 审计一致)。
6 输出: results/四个指定日期_表123_q4B.json、检验日志/四个指定日期_表123_q4B.md,
7       并把 q42_* 条目合并进 results/指定日期表格.json (带 _口径 标注)。
8 内置: 与 result4-2.xlsx / result4-3.xlsx 逐格交叉校验。
9 """
10 import argparse, json, pickle, sys
11 from pathlib import Path
12 import numpy as np
13 import openpyxl
14
15 HERE = Path(__file__).resolve().parent
16 sys.path.insert(0, str(HERE))
17 from p10_core import fmt_min, load_all, merge_emergency_intervals # noqa: E402

```

```

18
19 DATES = ["2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"]
20 BUCKETS = [(0, 24), (24, 48), (48, 72), (72, 96), (96, 120), (120, 144)]
21 BLABEL = ["0:00-4:00", "4:00-8:00", "8:00-12:00", "12:00-16:00", "16:00-20:00", "
22         20:00-24:00"]
23 KOUJING = "q4_scenarioB_v1 (口径B: 0:00 不知当日电价; 决策用预测价、结算用真实价; 滚动跨日价值)"
24 # 2026-09-12 晚: 内核版本标注 (问题4 已按 eps 归一化修正后的内核重跑; 问题2/3 按用户裁定未重跑)
25 KERN_Q4 = ("eps归一化修正后: p10_core.py b3242ad925c82d63..."
26         " (末槽系数 (eps_soc+v_term)/S, 有效日末系数 0.482548) ")
27 KERN_Q23 = ("eps归一化修正前: p10_core.py 0a2cef2e2b4428bb..."
28         " (末槽系数 eps_soc+v_term/S, 有效日末系数 0.491548) ")
29
30 def slot_label(j):
31     return f"{fmt_min(j * 10)}-{fmt_min((j + 1) * 10)}"
32
33
34 def build(case: Path):
35     P = np.asarray(load_all()["P_var"], float)
36     with open(case / "q42_best.pkl", "rb") as f:
37         q42 = pickle.load(f)
38     with open(case / "q43_daystore.pkl", "rb") as f:
39         q43 = pickle.load(f)
40     assert q42["protocol"] == q43["protocol"] == "q4_scenarioB_v1", "协议不符, 拒绝生成"
41     out, old = {}, {}
42     for tag, recs, is_q3 in (("q42", q42["recs"], False), ("q43", q43["store"], True)):
43         for ds in DATES:
44             di = (np.datetime64(ds) - np.datetime64("2025-01-01")).astype(int)
45             r = recs[di]
46             assert str(r["日期"]) == ds, f"{tag} {ds} 索引错位: {r['日期']}"
47             p = P[di] # 结算价 = 真实价
48             q0 = np.asarray(r["q0"], float); qc = np.asarray(r.get("qc", r["q0"]), float)
49             c = np.asarray(r["c"], float); d = np.asarray(r["d"], float)
50             qEM = np.asarray(r["qEM"], float)
51             plan_fee = float(p @ q0); adj = float(r["调整费"]); emg = float(5 * p @ qEM)
52             out[f"{tag}_{ds}"] = {
53                 "日期": ds, "阶段": r["阶段"], "口径": KOUJING,
54                 "轨道": "问题4-2 B (两阶段随机, 价格未知)" if not is_q3 else "问题4-3 历史选型
部署",
55                 "当日真实电价": {"均值": round(float(p.mean()), 4), "最小": round(float(p.min
()), 4),
56                 "最大": round(float(p.max()), 4)},
57                 "价格预测MAE": round(float(r.get("price_mae", float("nan"))), 4),
58                 "滚动v_term": round(float(r.get("v_term_used", 0.0)), 6),
59                 "表1_计划购电量_kWh": {slot_label(j): round(float(q0[j]), 4) for j in range
(144)},
60                 "表2_充放电电量_kWh": {BLABEL[i]: [round(float(c[a:b].sum()), 4), round(float(
d[a:b].sum()), 4)]
61                     for i, (a, b) in enumerate(BUCKETS)},
62                 "表3_紧急购电量_kWh": [[f"{fmt_min(a)}-{fmt_min(b)}", round(float(v), 4)]
63                     for a, b, v in merge_emergency_intervals(qEM)],

```

```

64         "储电量": {"0:00": round(float(r["E0"]), 4), "24:00": round(float(r["Eend"])),
65         , 4)},
66         "汇总": {"计划购电量": round(float(q0.sum()), 4), "计划购电费": round(
67         plan_fee, 4),
68         "紧急购电量": round(float(qEM.sum()), 4), "紧急购电费": round(emg, 4)
69         ,
70         "调整费": round(adj, 4), "总费用": round(plan_fee + adj + emg, 4)},
71     }
72     if is_q3:
73         out[f"{tag}_{ds}"]["调整购电量_kWh"] = {slot_label(j): round(float(qc[j]),
74         4) for j in range(144)}
75         out[f"{tag}_{ds}"]["更新事件"] = [dict(tau=int(u["tau"]), hour=int(u["hour"]
76         )),
77         fee=round(float(u["fee"]), 4))
78         for u in r.get("updates", [])]
79     old[f"{tag}_{ds}"] = {
80         "_口径": KOUJING,
81         "全天购电量": round(float(q0.sum() + qEM.sum()), 1),
82         "计划购电费": round(plan_fee, 1), "调整费": round(adj, 1),
83         "紧急购电费": round(emg, 1), "总费用": round(plan_fee + adj + emg, 1),
84         "紧急购电区间": [[f"{fmt_min(a)}-{fmt_min(b)}", round(float(v), 1)]
85         for a, b, v in merge_emergency_intervals(qEM)],
86         "充放电六段": [[round(float(c[a:b].sum()), 1), round(float(d[a:b].sum()), 1)
87         ]
88         for a, b in BUCKETS],
89         "E0": round(float(r["E0"]), 1), "E24": round(float(r["Eend"]), 1),
90     }
91     return out, old
92
93 def cross_check(case: Path, out: dict):
94     res = {}
95     for fname, tag, adj_sheet in (("result4-2.xlsx", "q42", None), ("result4-3.xlsx", "q43",
96     "调整购电量")):
97         wb = openpyxl.load_workbook(case / fname, data_only=True)
98         worst = 0.0
99         for sheet, field in (("计划购电量", "表1_计划购电量_kWh"), (adj_sheet, "调整购电量_kWh")):
100             if sheet is None:
101                 continue
102             ws = wb[sheet]
103             for ds in DATES:
104                 i = (np.datetime64(ds) - np.datetime64("2025-02-01")).astype(int)
105                 vals = [ws.cell(i + 2, 2 + j).value for j in range(144)]
106                 exp = [out[f"{tag}_{ds}"][field][slot_label(j)] for j in range(144)]
107                 worst = max(worst, max(abs(float(a) - b) for a, b in zip(vals, exp)))
108             wb.close()
109             res[fname] = dict(max_abs_diff=worst, tol=0.0051, status="PASS" if worst <= 0.0051
110             else "FAIL")
111     return res

```

```

106
107 def to_md(out: dict) -> str:
108     L = ["# 问题4（口径B）四个指定日期表1/表2/表3", "", f"> 口径: {KOUJING}",
109         "> 指定日期 = 2025.3.20、2025.6.21、2025.9.23、2025.12.21；账单一律按当日**真实**附件4
110         电价结算。", ""]
111     for key, e in out.items():
112         s = e["汇总"]
113         L += [f"## {key} ({e['轨道']}, 阶段 {e['阶段']}) ", "",
114             f"- 当日真实电价: 均值 {e['当日真实电价']['均值']}, 范围 {e['当日真实电价']['最小'
115             ']] - {e['当日真实电价']['最大']] 元/kWh"
116             f"- 价格预测 MAE {e['价格预测MAE']}; 滚动 v_term {e['滚动v_term']}",
117             f"- **全天**: 计划购电量 {s['计划购电量']:, .4f} kWh / {s['计划购电费']:, .4f} 元"
118             f"- 紧急购电量 {s['紧急购电量']:, .4f} kWh / {s['紧急购电费']:, .4f} 元"
119             + (f"- 调整费 {s['调整费']:, .4f} 元" if s["调整费"] else "")
120             + f"- **总费用 {s['总费用']:, .4f} 元**",
121             f"- 储电量: 0:00 {e['储电量']['0:00']:, .4f} kWh → 24:00 {e['储电量'
122             ']['24:00']:, .4f} kWh", "",
123             "### 表1 计划购电量 (kWh) ", "", "| 时段 | 购电量 | 时段 | 购电量 |", "
124             |---|---|---|---|"]
125         items = list(e["表1_计划购电量_kWh"].items())
126         for i in range(0, 144, 2):
127             L.append(f"| {items[i][0]} | {items[i][1]:, .4f} | {items[i+1][0]} | {items[i
128             +1][1]:, .4f} |")
129         L += ["", "### 表2 充放电量（六段）与储电量", "", "| 时段 | 充电量 | 放电量 |", "
130             |---|---|---|"]
131         for k, (cc, dd) in e["表2_充放电量_kWh"].items():
132             L.append(f"| {k} | {cc:, .4f} | {dd:, .4f} |")
133         L += [f"| 0:00 储电量 | {e['储电量']['0:00']:, .4f} | 24:00 储电量 | {e['储电量'
134             ']['24:00']:, .4f} |", "",
135             "### 表3 紧急购电量（区间）", ""]
136         if e["表3_紧急购电量_kWh"]:
137             L += ["| 时段 | 紧急购电量 |", "|---|---|"]
138             for a, v in e["表3_紧急购电量_kWh"]:
139                 L.append(f"| {a} | {v:, .4f} |")
140         else:
141             L.append("（当日无紧急购电）")
142         L.append("")
143     return "\n".join(L)
144
145 def main():
146     ap = argparse.ArgumentParser()
147     ap.add_argument("--case-dir", default=str(HERE.parent / "04_结果与检验" /
148         "重跑_Q4_B_20260912" / "B_v"))
149     ap.add_argument("--res-dir", default=str(HERE.parent / "04_结果与检验" / "results"))
150     ap.add_argument("--log-dir", default=str(HERE.parent / "04_结果与检验" / "检验日志"))
151     a = ap.parse_args()
152     case, res, logd = Path(a.case_dir), Path(a.res_dir), Path(a.log_dir)
153     out, old = build(case)
154     checks = cross_check(case, out)
155     assert all(v["status"] == "PASS" for v in checks.values()), f"工作簿交叉校验失败: {checks

```

```

150     }"
151     (res / "四个指定日期_表123_q4B.json").write_text(
152         json.dumps(dict(protocol="q4_scenarioB_v1", 口径=KOUJING, 内核=KERN_Q4, dates=DATES,
153             xlsx_cross_check=checks, tables=out), ensure_ascii=False, indent=1)
154         ,
155         encoding="utf-8")
156     (logd / "四个指定日期_表123_q4B.md").write_text(to_md(out), encoding="utf-8")
157     # 合并进 指定日期表格.json (替换 q42_*, 保留 q2_*/q3_*/旧 q42 备份在 备份目录)
158     p = res / "指定日期表格.json"
159     merged = json.loads(p.read_text(encoding="utf-8")) if p.exists() else {}
160     for k, v in old.items():
161         merged[k] = v
162     for k, v in list(merged.items()):
163         if isinstance(v, dict):
164             v["_内核"] = KERN_Q4 if k.startswith("q4") else KERN_Q23
165     (res / "指定日期表格.json").write_text(json.dumps(merged, ensure_ascii=False, indent=1),
166         encoding="utf-8")
167     print(json.dumps(dict(status="PASS", entries=len(out), xlsx_cross_check=checks,
168         keys=sorted(merged.keys())), ensure_ascii=False))
169
170 if __name__ == "__main__":
171     main()

```

C15. 问题 4/p106_pi_q4.py (66 行)

问题 4 完全信息下界 (PI) 求解与缺口计算

```

1  # -*- coding: utf-8 -*-
2  """问题4 口径B: 按被比较路径**真实 2/1 初态**计算全期 PI (价格完全信息) 上界与缺口。
3
4  口径说明: PI 是"价格与负荷/光伏均完全预知"的理想化连续优化, **用附件4 真实价**；
5  口径B 的可执行策略用价格预测, 故 GAP 同时包含价格预测误差 —— 用于量化"信息价值 + 算法/滚动近似",
6  不是 MILP 求解 gap。终端自由、物理约束与滚动策略一致。
7  """
8  import argparse, json, pickle, sys
9  from pathlib import Path
10 import numpy as np
11
12 HERE = Path(__file__).resolve().parent
13 sys.path.insert(0, str(HERE))
14 from p10_core import load_all, solve_plan_lp # noqa: E402
15
16
17 def main():
18     ap = argparse.ArgumentParser()
19     ap.add_argument("--q4-dir", required=True)
20     ap.add_argument("--cells", default="A_v0,A_v,B_v0,B_v")
21     ap.add_argument("--out", required=True)
22     a = ap.parse_args()
23     q4 = Path(a.q4_dir)
24     data = load_all()
25     sl = slice(31, 365) # 2-01—12-31, 334 天

```

```

26 L = np.asarray(data["L"][s1], float).reshape(-1)
27 G = np.asarray(data["G"][s1], float).reshape(-1)
28 P = np.asarray(data["P_var"][s1], float).reshape(-1) # 真实波动电价（完全信息）
29 cache, rows = {}, []
30 for cell in a.cells.split(","):
31     c = q4 / cell
32     if not (c / "main_metadata.json").exists():
33         print("跳过未完成格:", cell); continue
34     with open(c / "q42_best.pkl", "rb") as f:
35         q42 = pickle.load(f)
36     with open(c / "q43_daystore.pkl", "rb") as f:
37         q43 = pickle.load(f)
38     tracks = [(("Q4-2_B", q42["recs"])] + [(f"Q4-3_{k}", v) for k, v in q43["all_stores"]
39                                     ].items()) \
40               + [(("Q4-3_部署", q43["store"])]
41     for name, recs in tracks:
42         E0 = float(recs[31]["E0"])
43         key = f"{E0:.10f}"
44         if key not in cache:
45             r = solve_plan_lp(P, L[None, :], G[None, :], E0, eps_soc=0.0, tp=0.0)
46             if r["status"] != 0:
47                 raise RuntimeError(f"{cell}/{name}: PI 求解失败 {r['msg']}")
48             cache[key] = float(r["obj"])
49             actual = float(sum(r["总费用"] for r in recs[31:]))
50             rows.append(dict(cell=cell, path=name, E0_feb1=E0, actual_eval_cost=actual,
51                             pi_cost=cache[key], gap=actual - cache[key],
52                             gap_pct=(actual - cache[key]) / actual * 100))
53 out = dict(protocol="q4_scenarioB_v1", scope="2025-02-01--12-31 (334 天)",
54             note=("PI = 价格与负荷/光伏完全预知的连续优化（真实附件4 电价、自由终端、相同物理
55                 约束）；"
56                 "口径B 策略用价格预测 => GAP 含价格预测误差，是「信息+算法+滚动近似」的综合差距"),
57             unique_pi_solves=len(cache), paths=rows)
58 Path(a.out).write_text(json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
59 print(f"{'格':<8}{'路径':<14}{'2/1初态':>10}{'实际(元)':>15}{'PI(元)':>15}{'缺口(元)':>14}{'缺口%':>8}")
60 for r in rows:
61     print(f"{'cell':<8}{'path':<14}{'E0_feb1':>10,.1f}{'actual_eval_cost':>15,.2f}"
62           f"{'pi_cost':>15,.2f}{'gap':>14,.2f}{'gap_pct':>7.2f}%")
63 print("\n唯一 PI 求解数:", len(cache))
64 if __name__ == "__main__":
65     main()

```

C16. 问题 4/p107_q4_stats.py (121 行)

问题 4 配对统计与跨格对照（7 日分块 *bootstrap*）

```

1 # -*- coding: utf-8 -*-
2 """问题4 口径B: 配对统计（7 日分块 bootstrap），补齐此前只报总额的缺口。
3

```

```

4 覆盖:
5 ① 各格组内: Q4-2 B-A、C-B; Q4-3 部署-none、各固定组合-none
6 ② 跨格: B_v-A_v (Q4-2 B 与 Q4-3 部署)、B_v-B_v0 (跨日价值)、A_v-A_v0
7 ③ 价格轨: hist7 (主轨) vs lag1 / fix / shape (Q4-2 B)
8 ④ 刷新: 可刷新B vs 严格B (Q4-3 部署)
9 全部在同日期配对 (2-01—12-31, 334 天)。
10 """
11 import argparse, json, pickle, sys
12 from pathlib import Path
13 import numpy as np
14 import pandas as pd
15
16 HERE = Path(__file__).resolve().parent
17
18
19 def block_bootstrap(x, reps=2000, block=7, seed=2025):
20     x = np.asarray(x, float); n = len(x); rng = np.random.default_rng(seed)
21     starts = rng.integers(0, n, size=(reps, int(np.ceil(n / block))))
22     idx = (starts[:, :, None] + np.arange(block)) % n
23     return np.mean(x[idx.reshape(reps, -1)[:, :n]], axis=1)
24
25
26 def stat(x):
27     x = np.asarray(x, float); b = block_bootstrap(x)
28     return dict(n=int(len(x)), mean=float(x.mean()), median=float(np.median(x)),
29               q025=float(np.quantile(x, .025)), q975=float(np.quantile(x, .975)),
30               bootstrap_q025=float(np.quantile(b, .025)),
31               bootstrap_q975=float(np.quantile(b, .975)),
32               positive_days=int((x > 0).sum()), negative_days=int((x < 0).sum()),
33               significant=bool(np.quantile(b, .025) > 0 or np.quantile(b, .975) < 0))
34
35
36 def load_daily(case: Path, which="q42"):
37     df = pd.read_csv(case / f"{which}_逐日费用.csv")
38     df["日期"] = df["日期"].astype(str)
39     df = df[~df["日期"].str.startswith("2025-01")]
40     return {k: v.sort_values("日期").reset_index(drop=True) for k, v in df.groupby("模型")}
41
42
43 def load_single(pk1: Path):
44     with open(pk1, "rb") as f:
45         recs = pickle.load(f)["recs"]
46     df = pd.DataFrame([dict(日期=r["日期"], 总费用=r["总费用"]) for r in recs])
47     return df[~df["日期"].str.startswith("2025-01")].sort_values("日期").reset_index(drop=True)
48
49
50 def diff(dfa, ka, dfb, kb, col="总费用"):
51     A = dfa[ka].sort_values("日期").reset_index(drop=True)
52     B = dfb[kb].sort_values("日期").reset_index(drop=True)
53     assert (A["日期"] == B["日期"]).all(), "日期未对齐"

```

```

54     return (B[col].to_numpy() - A[col].to_numpy())
55
56
57 def main():
58     ap = argparse.ArgumentParser()
59     ap.add_argument("--q4-dir", required=True)
60     ap.add_argument("--out", required=True)
61     a = ap.parse_args()
62     q4 = Path(a.q4_dir)
63     out = dict(protocol="q4_scenarioB_v1", scope="2025-02-01—12-31 (334 天)",
64               bootstrap=dict(reps=2000, block_days=7, seed=2025),
65               note="全部为同日期配对; 正 = 前者更贵; 账单一律真实价", comparisons={})
66     cells = {}
67     for c in ("A_v0", "A_v", "B_v0", "B_v", "sens_refresh"):
68         p = q4 / c
69         if (p / "q42_逐日费用.csv").exists():
70             cells[c] = (load_daily(p, "q42"), load_daily(p, "q43"))
71
72     # ① 组内
73     for c, (q2, q3) in cells.items():
74         out["comparisons"][f"{c}_Q4-2_B_minus_A"] = stat(diff(q2, "A", q2, "B"))
75         out["comparisons"][f"{c}_Q4-2_C_minus_B"] = stat(diff(q2, "B", q2, "C"))
76         out["comparisons"][f"{c}_Q4-3_部署_minus_none"] = stat(
77             diff(q3, "none", q3, "历史选型执行策略"))
78         for k in ("6", "12", "18", "6+12", "6+18", "12+18", "6+12+18"):
79             out["comparisons"][f"{c}_Q4-3_{k}_minus_none"] = stat(diff(q3, "none", q3, k))
80
81     # ② 跨格
82     if "A_v" in cells and "B_v" in cells:
83         out["comparisons"]["跨格_B_v_minus_A_v_Q4-2B"] = stat(diff(cells["A_v"][0], "B",
84             cells["B_v"][0], "B"))
85         out["comparisons"]["跨格_B_v_minus_A_v_Q4-3部署"] = stat(
86             diff(cells["A_v"][1], "历史选型执行策略", cells["B_v"][1], "历史选型执行策略"))
87     if "B_v0" in cells and "B_v" in cells:
88         out["comparisons"]["跨格_B_v_minus_B_v0_Q4-2B"] = stat(diff(cells["B_v0"][0], "B",
89             cells["B_v"][0], "B"))
90         out["comparisons"]["跨格_B_v_minus_B_v0_Q4-3部署"] = stat(
91             diff(cells["B_v0"][1], "历史选型执行策略", cells["B_v"][1], "历史选型执行策略"))
92     if "A_v0" in cells and "A_v" in cells:
93         out["comparisons"]["跨格_A_v_minus_A_v0_Q4-2B"] = stat(diff(cells["A_v0"][0], "B",
94             cells["A_v"][0], "B"))
95
96     # ③ 价格轨 (单路径格)
97     base = cells.get("B_v", (None, None))[0]
98     if base is not None:
99         bref = base["B"].sort_values("日期").reset_index(drop=True)
100         for name in ("sens_track_lag1", "sens_track_fix", "sens_track_shape"):
101             p = q4 / name / "paths" / "B.pk1"
102             if not p.exists():
103                 continue
104             s = load_single(p)

```

```

102         d = s["总费用"].to_numpy() - bref["总费用"].to_numpy()
103         out["comparisons"][f"价格轨_{name.replace('sens_track_', '')}_minus_hist7"] =
stat(d)
104
105     # ④ 刷新
106     if "sens_refresh" in cells and "B_v" in cells:
107         out["comparisons"]["刷新_可刷新B_minus_严格B_Q4-3部署"] = stat(
108             diff(cells["B_v"][1], "历史选型执行策略", cells["sens_refresh"][1], "历史选型执行
策略"))
109
110     Path(a.out).write_text(json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
111     sig = sum(1 for v in out["comparisons"].values() if v["significant"])
112     print(f"比较数 {len(out['comparisons'])}, 显著 {sig}, 非显著 {len(out['comparisons'])-sig}
\n")
113     for k, v in out["comparisons"].items():
114         mark = "显著" if v["significant"] else "不显著"
115         print(f"    {k:<46} 均值={v['mean']:>11,.2f} CI=[{v['bootstrap_q025']:>10,.1f},{v['
bootstrap_q975']:>10,.1f}] "
116             f"负/正={v['negative_days']}/{v['positive_days']} {mark}")
117
118
119 if __name__ == "__main__":
120     main()

```

C17. 问题 4/p108_q4_vstep_d4.py (132 行)

问题 4 的 v 台阶响应与 D4 收益拆分汇总

```

1  # -*- coding: utf-8 -*-
2  """问题4 补做批次: v 台阶探针与 D4 式拆分的**可复现汇总脚本** (补落盘)。
3
4  背景: 2026-09-12 晚首次生成 `v台阶_Q4B.json` / `d4_Q4B.json` 时用的是临时内联脚本,
5  未落盘 (复现性缺口)。本脚本依据保留的路径 pk1 重算并覆盖这两个 JSON,
6  用于验证既有数值与后续复现。
7
8  用法:
9      python p108_q4_vstep_d4.py [--q4-dir <重跑_Q4_B_20260912>] [--write]
10
11  口径: 结算价 = 附件4 真实价; 评估窗口 = 2025-02-01—12-31 (334 天, 即 recs[31:]);
12      v 台阶四轨与 d4_F 轨均为 365 天因果冷启动运行。
13  """
14  import argparse
15  import json
16  import pickle
17  import sys
18  from pathlib import Path
19
20  import numpy as np
21
22  HERE = Path(__file__).resolve().parent
23  sys.path.insert(0, str(HERE))
24  from p10_core import load_all # noqa: E402
25

```

```

26 ROOT = HERE.parent
27 Q4_DEFAULT = ROOT / "04_结果与检验" / "重跑_Q4_B_20260912"
28 E_MIN = 1200.0
29 TOL = 0.0          # 触底判定容差 (kWh) ; 0 = 严格「日初储能 ≤ 1200」,
30                    # 这与 2026-09-12 首次生成的 v台阶_Q4B.json 的 bottom 定义一致 (56
                        /26/5/4/1/0)
31
32
33 def daily(recs, P):
34     """逐日汇总 (recs 为 365 天记录, 取 [31:] 为 334 天评估窗口)。"""
35     ev = recs[31:]
36     cost = float(sum(r["总费用"] for r in ev))
37     emg = float(sum(5.0 * float(P[int(r["di"])]) @ np.asarray(r["qEM"], float)) for r in ev)
38     adj = float(sum(r["调整费"] for r in ev))
39     E0s = np.array([float(r["E0"]) for r in ev])
40     # 「触底天数」= 日初储能 ≤ E_MIN(+TOL) 的天数: 即当天的电是「昨天没留下电」开始的
41     bottom = int(sum(1 for e in E0s if e <= E_MIN + TOL))
42     return dict(days=len(ev), cost334=cost, emg334=emg, adj334=adj,
43                E0_mean=float(E0s.mean()), E0_med=float(np.median(E0s)),
44                bottom=bottom, eend=float(recs[-1]["Eend"]))
45
46
47 def load_pkl(p):
48     with open(p, "rb") as fh:
49         o = pickle.load(fh)
50     return o["recs"] if isinstance(o, dict) and "recs" in o else o
51
52
53 def main():
54     ap = argparse.ArgumentParser()
55     ap.add_argument("--q4-dir", default=str(Q4_DEFAULT))
56     ap.add_argument("--write", action="store_true", help="覆盖写回两个 JSON (默认只打印对比) ")
57     a = ap.parse_args()
58     Q4 = Path(a.q4_dir)
59     P = np.asarray(load_all()["P_var"], float)
60
61     # ---- v 台阶 (v=0 行取自 B_v0 格; 0.35-0.50 取自 probe_* 轨; 滚动行取自正式格 B_v)
62     steps = [("v=0 (对照)", 0.0, Q4 / "B_v0" / "paths" / "B.pkl"),
63              ("v=0.35", 0.35, Q4 / "probe_v035" / "paths" / "B.pkl"),
64              ("v=0.44", 0.44, Q4 / "probe_v044" / "paths" / "B.pkl"),
65              ("v=0.46", 0.46, Q4 / "probe_v046" / "paths" / "B.pkl"),
66              ("v=0.50", 0.50, Q4 / "probe_v050" / "paths" / "B.pkl"),
67              ("滚动 v (正式, 均值0.4800)", None, Q4 / "B_v" / "paths" / "B.pkl")]
68     rows = []
69     for label, v, p in steps:
70         if not p.exists():
71             print(f"[缺] {label}: {p}")
72             continue
73         recs = load_pkl(p)

```

```

74     d = daily(recs, P)
75     rows.append(dict(label=label, v=v, **d))
76 print("=== v 台阶 (334 天, 元) ===")
77 for r in rows:
78     print(f" {r['label']:<26} v={str(r['v']):>5} 总={r['cost334']:>15,.2f} "
79           f"紧急费={r['emg334']:>13,.2f} E0均值={r['E0_mean']:>8,.1f} "
80           f"触底天数={r['bottom']:>3} 年末储能={r['eend']:>10,.2f}")
81
82 # 校验: v=0 行应等于 B_v0 格 Q4-2 B 轨的 334 天总额; 滚动行应等于 B_v 格 Q4-2 B 轨
83 def q42B(cell):
84     import pandas as pd
85     d = pd.read_csv(Q4 / cell / "q42_逐日费用.csv")
86     d["日期"] = d["日期"].astype(str)
87     return float(d[~d["日期"].str.startswith("2025-01") & (d["模型"] == "B")]["总费用"].
88 sum())
89
90 try:
91     zc, rc = q42B("B_v0"), q42B("B_v")
92     print(f"\n 交叉核对: B_v0 格 Q4-2 B 334 = {zc:,.2f}; B_v 格 Q4-2 B 334 = {rc:,.2f}")
93     print(f"          v=0 轨 (本脚本)          = {rows[0]['cost334']:, .2f} 差 = {rows
94 ['cost334'] - zc:+, .2f}")
95     print(f"          滚动 v 轨 (本脚本)          = {rows[-1]['cost334']:, .2f} 差 = {rows
96 [-1]['cost334'] - rc:+, .2f}")
97 except Exception as e:
98     # noqa: BLE001
99     print(" 交叉核对跳过:", e)
100
101 # ---- D4 式拆分 (N 取 B_v 格 q43 none; F 取 d4_F 轨; FM 取 B_v 格 q43 6+12+18)
102 fm = Q4 / "B_v" / "paths" / "6plus12plus18.pkl"
103 ff = Q4 / "d4_F" / "paths" / "6plus12plus18.pkl"
104 d4 = {}
105 try:
106     import pandas as pd
107     d = pd.read_csv(Q4 / "B_v" / "q43_逐日费用.csv")
108     d["日期"] = d["日期"].astype(str)
109     d = d[~d["日期"].str.startswith("2025-01")]
110     d4["N"] = float(d[d["模型"] == "none"]["总费用"].sum())
111 except Exception as e:
112     # noqa: BLE001
113     print(" N 读取失败:", e)
114 if ff.exists():
115     d4["F"] = daily(load_pkl(ff), P)["cost334"]
116 if fm.exists():
117     d4["FM"] = daily(load_pkl(fm), P)["cost334"]
118 if {"N", "F", "FM"} <= set(d4):
119     d4["N_minus_F"] = d4["N"] - d4["F"]
120     d4["F_minus_FM"] = d4["F"] - d4["FM"]
121     d4["N_minus_FM"] = d4["N"] - d4["FM"]
122     print("\n=== D4 式拆分 (B_v, 334 天, 元) ===")
123     print(f" N 不更新          = {d4['N']:>15,.2f}")
124     print(f" F 仅刷新预报          = {d4['F']:>15,.2f} N-F = {d4['N_minus_F']:>+13,.2f
125 }")
126     print(f" FM 刷新+购电调整      = {d4['FM']:>15,.2f} F-FM = {d4['F_minus_FM']

```

```

121         ']:>+13,.2f}")
122         print(f"                                N-FM = {d4['N_minus_FM']:>+13,.2f}")
123     )
124     if a.write:
125         (Q4 / "v台阶_Q4B.json").write_text(json.dumps(rows, ensure_ascii=False, indent=1),
126                                             encoding="utf-8")
127         (Q4 / "d4_Q4B.json").write_text(json.dumps(d4, ensure_ascii=False, indent=1),
128                                             encoding="utf-8")
129         print("\n已写回 v台阶_Q4B.json / d4_Q4B.json")
130 if __name__ == "__main__":
131     main()

```

C18. 问题 4/p41c_figures_q4.py (156 行)

问题 4 图件绘制（四格对照、效应分解、电价预测与 *MAE* 带、原始价格、价格-负荷）

```

1  # -*- coding: utf-8 -*-
2  """figures_q4B: 问题4 口径B 图件（重出受影响的 result_q4_costs, 新增四格分解与价格预测质量图）。
3
4  未受影响的 Q4 数据描述图（raw_q4_prices、process_q4_price_load）按 SHA256 校验后逐字节复用。
5  """
6  import hashlib, json, os, shutil, sys
7  import os
8  from pathlib import Path
9  import numpy as np
10 import pandas as pd
11 import matplotlib
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14
15 ROOT = Path(__file__).resolve().parent.parent
16 sys.path.insert(0, str(ROOT / "utils"))
17 sys.path.insert(0, str(ROOT / "03_代码"))
18 from setup_style import setup_style          # noqa: E402
19 from export_figure import export_figure      # noqa: E402
20 from visual_qa import audit_layout           # noqa: E402
21 from p10_core import load_all                # noqa: E402
22 from p16_price_forecast import load_cache    # noqa: E402
23
24 R = ROOT / "04_结果与检验"
25 # 2026-09-12 晚：可用环境变量指向新树（默认仍为交付基线树，行为不变）
26 Q4 = Path(os.environ.get("Q4_TREE", str(R / "重跑_Q4_B_20260912")))
27 OLD_FIG = R / "figures_v2"
28 OUT = R / "figures_q4B"
29 REUSE = ["raw_q4_prices", "process_q4_price_load"]
30
31
32 def sha(p):
33     return hashlib.sha256(Path(p).read_bytes()).hexdigest()
34

```

```

35
36 def main():
37     setup_style(journal="general", lang="zh", use_sciplots=False)
38     plt.rcParams.update({"font.size": 10, "axes.titlesize": 11, "axes.labelsize": 10,
39                          "xtick.labelsize": 9, "ytick.labelsize": 9, "legend.fontsize": 9})
40     OUT.mkdir(parents=True, exist_ok=True)
41     contracts = []
42
43     def finish(fig, name, claim, source, reused=False):
44         fig.tight_layout()
45         issues = audit_layout(fig)
46         if any(lv in ("WARN", "FAIL") for lv, _ in issues):
47             raise RuntimeError((name, issues))
48         export_figure(fig, os.path.join(OUT, name), formats=["png", "svg"], dpi=300,
49                          grayscale_preview=True)
50         contracts.append(dict.figure=name, claim=claim, source=source, backend="matplotlib"
51 ,
52                          口径="q4_scenarioB_v1 (口径B, 决策用预测价、结算用真实价)",
53                          reused=reused, units="各轴明确标注",
54                          layout="单面板, 7.2x4.2 英寸", issues=issues))
55         plt.close(fig)
56
57     def panel(title, xlabel, ylabel, height=4.2):
58         f, a = plt.subplots(figsize=(7.2, height))
59         a.set(title=title, xlabel=xlabel, ylabel=ylabel)
60         a.spines[["top", "right"]].set_visible(False)
61         return f, a
62
63     def feb(df):
64         df = df.copy(); df["日期"] = df["日期"].astype(str)
65         return df[~df["日期"].str.startswith("2025-01")]
66
67     def cell_tot(name, track="B", where="q42"):
68         if where == "q42":
69             df = pd.read_csv(Q4 / name / "q42_逐日费用.csv")
70             return float(feb(df[df["模型"] == track])["总费用"].sum())
71         df = pd.read_csv(Q4 / name / "q43_逐日费用.csv")
72         return float(feb(df[df["模型"] == track])["总费用"].sum())
73
74     # ----- 1) 四格分解 (跨日价值 vs 电价未知代价)
75     cells = ["A_v0", "A_v", "B_v0", "B_v"]
76     lab = ["口径A.v=0\n(价格已知)", "口径A.滚动v", "口径B.v=0\n(价格未知)", "口径B.滚动v\n(正式)"]
77     q42b = [cell_tot(c, "B") for c in cells]
78     dep = [cell_tot(c, "历史选型执行策略", "q43") for c in cells]
79     f, a = panel("问题4: 口径与跨日库存价值的四格对照 (2—12月)", "组合", "总费用 / 万元",
80                  height=4.6)
81     x = np.arange(4); w = 0.38
82     a.bar(x - w / 2, np.array(q42b) / 1e4, w, color="#0072B2", label="Q4-2 B (两阶段随机)")
83     a.bar(x + w / 2, np.array(dep) / 1e4, w, color="#009E73", label="Q4-3 历史选型部署")
84     for i in range(4):

```

```

83     a.annotate(f"{q42b[i]/1e4:,.1f}", (i - w / 2, q42b[i] / 1e4), ha="center", va="
bottom", fontsize=8)
84     a.annotate(f"{dep[i]/1e4:,.1f}", (i + w / 2, dep[i] / 1e4), ha="center", va="bottom
", fontsize=8)
85 a.set_xticks(x, lab, fontsize=9); a.legend(loc="lower right")
86 finish(f, "result_q4_four_cells_q4B",
87         "跨日库存价值使费用下降 (Q4-2 B: B_v-B_v0 = -274,261.28 元; Q4-3 部署: -608,762.77
元); "
88         "Q4-2 中电价未知使其上升 (B_v-A_v = +112,302.23 元)、交互项为 +3,519.48 元; "
89         "***Q4-3 部署中口径B 反而比口径A 便宜 219,360.61 元**, 该反直觉差异来自"
90         "「知真实价 => 更激进套利 => 储能余量更薄 => 被 5 倍紧急罚通道放大」, "
91         "***不得解读为价格信息价值** (见 结论_Q4_补做.md §5)",
92         "重跑_Q4_B_20260912/{A_v0,A_v,B_v0,B_v}/q42_逐日费用.csv、q43_逐日费用.csv (334天)
")
93
94 # ----- 2) 效应分解瀑布 (Q4-2 B)
95 e = json.loads((Q4 / "q4_对照汇总.json").read_text(encoding="utf-8"))["effects_334"]["
Q2B"]
96 base = e["A_v0"]; d_v = e["跨日价值效应_Bv_minus_Bv0"]; d_p = e["电价信息效应_Bv_minus_Av
"]
97 f, a = panel("问题4-2 B: 两个变化的效应分解 (2—12月)", "步骤", "总费用 / 万元")
98 steps = [base, d_v, d_p, e["B_v"]]
99 names = ["口径A.v=0\n(基准)", "+跨日库存价值\n(滚动 v)", "+电价未知\n(口径B)", "口径B.滚动
v\n(正式)"]
100 bottoms = [0, base + d_v, base + d_v, 0]
101 heights = [base, d_v, d_p, e["B_v"]]
102 colors = ["#9ecae1", "#009E73", "#D55E00", "#0072B2"]
103 for i in range(4):
104     a.bar(i, heights[i] / 1e4, bottom=bottoms[i] / 1e4, color=colors[i])
105     a.annotate(f"{heights[i]/1e4:+,.1f}" if i in (1, 2) else f"{heights[i]/1e4:,.1f}",
106               (i, (bottoms[i] + heights[i]) / 1e4), ha="center", va="bottom", fontsize
=9)
107 a.set_xticks(range(4), names, fontsize=9)
108 finish(f, "result_q4_effect_decomp_q4B",
109         "跨日价值约 -27.4 万元 (口径B 基准 B_v-B_v0 = -274,261.28 元)、"
110         "电价未知代价约 +11.2 万元 (同基准 B_v-A_v = +112,302.23 元), 交互项 +3,519.48 元;
"
111         "口径A 基准的跨日价值为 -275,006.70 元.**注意: 口径A 同时改变了控制激进度, "
112         "故 +10.7 万元是价格预报价值的「下界」而非干净的 VOI** (见 结论_Q4_补做.md §5)",
113         "q4_对照汇总.json 的 effects_334.Q2B")
114
115 # ----- 3) 价格预测质量
116 data = load_all(); P = np.asarray(data["P_var"], float)
117 fc = load_cache(Q4 / "B_v" / "forecast" / "电价预测缓存.npz")
118 fp = np.asarray(fc["FP_hist7"], float)
119 di = 132 # 典型日 (曾触发退化 LP 的当日)
120 h = (np.arange(144) + .5) / 6
121 f, a = panel("电价预测与真实电价 (示例日 2025-05-13)", "时刻 / h", "电价 / 元每kWh")
122 a.plot(h, P[di], color="#0072B2", label="真实 (附件4)")
123 a.plot(h, fp[di], "--", color="#D55E00", label="预测 hist7 (0:00 可得)")
124 a.legend()

```

```

125 finish(f, "process_q4_price_forecast_q4B", "日前价格预测只用历史、可复现当日形状但水平有误差",
126         "附件4 + 电价预测缓存.npz (FP_hist7) ")
127
128 f, a = panel("价格预测误差的分时段分布", "时段", "绝对误差 / 元每kWh")
129 err = np.abs(fp[31:] - P[31:])
130 bands = [(0, 36, "0:00-6:00"), (36, 72, "6:00-12:00"), (72, 108, "12:00-18:00"), (108,
131         144, "18:00-24:00")]
132 a.boxplot([err[:, s:e].ravel() for s, e, _ in bands],
133           tick_labels=[n for _, _, n in bands], showfliers=False)
134 a.annotate(f"总体 MAE {err.mean():.4f} 元/kWh", (0.02, 0.92), xycoords="axes fraction",
135           fontsize=9)
136 finish(f, "process_q4_price_mae_bands_q4B", "预测误差随时段不同，凌晨最低、白天最高",
137         "附件4 与 FP_hist7 (334 天 × 144 槽)；箱体 IQR 中位数，须 1.5IQR，不显示离群点")
138
139 # ----- 4) 复用的数据描述图
140 for name in REUSE:
141     for ext in ("png", "svg"):
142         src = OLD_FIG / f"{name}.{ext}"
143         if src.exists():
144             shutil.copy2(src, OUT / src.name)
145             contracts.append(dict(figure=name, claim="问题4 数据描述类，未受口径切换影响",
146                                 source="figures_v2/ (逐字节复用)", backend="matplotlib",
147                                 口径="不涉及口径", reused=True, sha256=sha(OLD_FIG / f"{name}.
148                                 png"),
149                                 units="同原图", layout="同原图", issues=[]))
150             (OUT / "图表契约.json").write_text(json.dumps(contracts, ensure_ascii=False, indent=2),
151             encoding="utf-8")
152             print(json.dumps(dict(status="PASS", figures=len(contracts),
153                                 recomputed=[c["figure"] for c in contracts if not c["reused"]],
154                                 reused=[c["figure"] for c in contracts if c["reused"]]),
155             ensure_ascii=False))
156
157 if __name__ == "__main__":
158     main()

```

C19. 用户指定图源/用户指定最终图 _ 代码映射.md (一行)

5 张用户指定最终图与出图脚本、源数据的映射说明

```

1 # 用户指定最终图—代码映射
2
3 整理日期：2026-09-13
4
5 本目录中的代码按用户指定的 5 张最终图逐张对应。图件副本位于 `05_图件/用户指定最终图/`。
6
7 | 最终图 | 对应代码 | 来源性质与核验说明 |
8 |---|---|---|
9 | 问题1_六时段储电量变化瀑布图.png | `plot_q1_soc_waterfall.py` | 当前工作区直接存在的出图脚本；
   | 脚本输出文件名与图件一致。 |
10 | fig5-7_四季典型日合并图_统一纵轴.png | `plot_typical_day_combined.py` | 根据项目会话记录中的
   | 原始创建内容恢复；该脚本调用 `plot_q23_contour.py` 的 `load_data`，但该依赖文件当前工作区未

```

找到，因此本次只做代码归档和图件映射，未宣称重新运行。 |

11 | `fig2_全年光伏_修正版.png` | ``rebuild_q23_G.py`` | 当前工作区直接存在的出图脚本；脚本中的 ``draw(pv, 'fig2_全年光伏_修正版', ...)`` 与图件一致。 |

12 | `fig3_全年负载_修正版.png` | ``rebuild_q23_G.py`` | 当前工作区直接存在的出图脚本；脚本中的 ``draw(load, 'fig3_全年负载_修正版', ...)`` 与图件一致。 |

13 | `fig4_全年净负荷_修正版.png` | ``rebuild_q23_G.py`` | 当前工作区直接存在的出图脚本；脚本中的 ``draw(net, 'fig4_全年净负荷_修正版', ...)`` 与图件一致。 |

14

15 说明：本次保留的是“最终图 + 对应出图脚本”的最小支撑内容，没有把完整原始数据、重复图件或运行环境复制进来。脚本中的相对路径仍以原项目目录结构为准。

C20. 用户指定图源/`plot_q1_soc_waterfall.py` (21 行)

用户指定图：问题 1 六时段储电量变化瀑布图

```
1 from pathlib import Path
2 import openpyxl, numpy as np, matplotlib.pyplot as plt
3 R=Path(__file__).resolve().parent
4 plt.rcParams.update({'font.sans-serif':['Microsoft YaHei','Noto Sans SC','SimHei','DejaVu Sans'], 'axes.unicode_minus':False, 'font.size':10})
5 p=R/'项目'/'04_结果与检验'/'问题1_结果'/'result1.xlsx'; w=openpyxl.load_workbook(p, data_only=True, read_only=True)
6 rows=list(w.worksheets[1].values)[1:7]; c=np.array([float(r[1]) for r in rows]); d=np.array([float(r[2]) for r in rows]); eta=.9
7 delta=eta*c-d/eta; start=6000.; end=start+delta.sum(); cum=np.r_[start, start+np.cumsum(delta)]
8 labels=['0:00—4:00', '4:00—8:00', '8:00—12:00', '12:00—16:00', '16:00—20:00', '20:00—24:00']
9 fig, ax=plt.subplots(figsize=(9, 5.2), dpi=160); x=np.arange(8)
10 ax.bar(0, start, width=.62, color='#2F80ED', edgecolor='white')
11 for i, v in enumerate(delta):
12     bottom=cum[i] if v>=0 else cum[i]+v
13     ax.bar(i+1, abs(v), bottom=bottom, width=.62, color=( '#E74C3C' if v>=0 else '#2ECC71' ), edgecolor='white')
14     ax.text(i+1, cum[i+1]+(160 if v>=0 else -260), f'{v:+.1f}', ha='center', va='bottom' if v>=0 else 'top', fontsize=9, color='black')
15 ax.bar(7, end, width=.62, color='#2F80ED', edgecolor='white')
16 ax.text(0, start+180, f'{start:.0f}', ha='center', fontsize=9); ax.text(7, end+180, f'{end:.1f}', ha='center', fontsize=9, fontweight='bold')
17 ax.set_xticks(x, ['0:00\n初始']+labels+['24:00\n末端'], rotation=18, fontsize=8); ax.set_ylabel('储电量 (kWh)'); ax.set_ylim(0, 14000); ax.set_title('问题一六时段储电量变化瀑布图')
18 ax.axhline(1200, color='#777', ls='--', lw=.9, label='下限1200'); ax.axhline(10800, color='#777', ls='-.', lw=.9, label='上限10800'); ax.grid(axis='y', alpha=.22); ax.legend(frameon=False, ncol=1, loc='center left', bbox_to_anchor=(1.01, .5)); fig.subplots_adjust(bottom=.22, top=.88, right=.82)
19 out=R/'figures'/'问题1_六时段储电量变化瀑布图.png'; fig.savefig(out, dpi=300, bbox_inches='tight'); fig.savefig(out.with_suffix('.svg'), bbox_inches='tight'); plt.close(fig)
20 print('delta=', delta.tolist(), 'end=', end, 'output=', out)
```

C21. 用户指定图源/`plot_typical_day_combined.py` (116 行)

用户指定图：四季典型日合并图（含统一纵轴版）

```

1  """把 fig5/fig6/fig7 三张「四季典型日」折线图合并成一张三联图。
2
3  与直接粘贴三张 PNG 不同，这里是用原始数据重画：
4      - 三块面板共享 x 轴，只在最底部标一次「时间段（10 min）」；
5      - 四季图例只出一次，放在图顶；
6      - 输出矢量 PDF/SVG，可直接进 LaTeX。
7
8  出两个版本，差别只在纵轴：
9      fig5-7_四季典型日合并图          各面板纵轴独立（忠实于原来的 fig5/6/7）
10     fig5-7_四季典型日合并图_统一纵轴  光伏与负载共用（0, 9000）纵轴 —— 堆叠在一起时
11                                         读者必然横向比高度，纵轴不一致会误导对量级的判断
12
13  两版都做了两处必要的补充（原 fig5/6/7 没有）：
14      - 净负荷面板画一条 0 参考线：典型日里 3/4 天存在光伏过剩时段，
15        不标零线就无法判断正负；
16      - 纵轴单位写明 kW。
17  """
18  import sys
19  from pathlib import Path
20
21  import matplotlib
22  matplotlib.use('Agg')
23  import matplotlib.pyplot as plt
24  import numpy as np
25
26  ROOT = Path(__file__).resolve().parent
27  sys.path.insert(0, str(ROOT))
28  from plot_q23_contour import load_data          # noqa: E402 复用同一套数据读取
29
30  OUT = ROOT / 'figures' / '问题23_新增7图'
31  OUT.mkdir(parents=True, exist_ok=True)
32
33  plt.rcParams.update({
34      'font.sans-serif': ['SimHei', 'Microsoft YaHei', 'DejaVu Sans'],
35      'axes.unicode_minus': False,
36      'font.size': 9, 'figure.dpi': 150, 'savefig.dpi': 300,
37  })
38
39  # 四季典型日与配色（沿用原 fig5/6/7 的 Okabe-Ito 色盲友好配色）
40  SEASONS = [('2025-03-20', '3月20日'), ('2025-06-21', '6月21日'),
41            ('2025-09-23', '9月23日'), ('2025-12-21', '12月21日')]
42  COLORS = ['#0072B2', '#E69F00', '#009E73', '#CC79A7']
43  PANEL_TITLES = ['(a) 四季典型日光伏出力曲线',
44                 '(b) 四季典型日负载曲线',
45                 '(c) 四季典型日净负荷曲线']
46
47
48  def typical_day_rows(d1):
49      """返回四个典型日在 365 天矩阵中的行号。"""
50      rows = []
51      for ds, lab in SEASONS:

```

```

52     hit = np.flatnonzero(d1.values == np.datetime64(ds))
53     assert hit.size == 1, '典型日 %s 在数据中未唯一命中' % ds
54     rows.append(int(hit[0]))
55     return rows
56
57
58 def build(shared_y, filename, title):
59     d1, load, pv, times = load_data()
60     net = load - pv
61     rows = typical_day_rows(d1)
62     x = np.arange(1, 145)
63
64     fig, axes = plt.subplots(3, 1, figsize=(8.6, 9.8), sharex=True)
65     fig.subplots_adjust(top=0.885, bottom=0.072, left=0.105,
66                        right=0.975, hspace=0.20)
67
68     for ax, arr, ptitle in zip(axes, [pv, load, net], PANEL_TITLES):
69         for r, (_, lab), c in zip(rows, SEASONS, COLORS):
70             ax.plot(x, arr[r], lw=1.8, color=c, label=lab)
71             ax.set_ylabel('功率 / kW')
72             ax.set_title(ptitle, loc='left', pad=6)
73             ax.grid(alpha=.25)
74             ax.set_xlim(1, 144)
75             ax.set_xticks([24, 48, 72, 96, 120, 144])
76
77     if shared_y:
78         # 光伏与负载同一纵轴，两者的量级差才读得出来
79         for ax in axes[:2]:
80             ax.set_ylim(0, 9000)
81
82     # 净负荷必须标零线才能读出正负
83     axes[2].axhline(0, color='#555555', lw=0.9, ls=(0, (4, 3)), zorder=0)
84
85     axes[2].set_xlabel('时间段 (10 min) ')
86
87     handles = [plt.Line2D([0], [0], color=c, lw=2.0, label=lab)
88               for (_, lab), c in zip(SEASONS, COLORS)]
89     fig.legend(handles=handles, ncol=4, frameon=False,
90              loc='upper center', bbox_to_anchor=(0.5, 0.955))
91     fig.suptitle(title, y=0.982)
92
93     for ext in ('png', 'pdf', 'svg'):
94         fig.savefig(OUT / (filename + '.' + ext), bbox_inches='tight')
95     plt.close(fig)
96
97     spans = {k: (float(v.min()), float(v.max()))
98             for k, v in (('光伏', pv), ('负载', load), ('净负荷', net))}
99     print('->', filename)
100    for k, (a, b) in spans.items():
101        print('    %-4s 四季典型日范围 %9.1f ~ %9.1f kW' % (k, a, b))
102    if shared_y:

```

```

103         print('        纵轴：光伏/负载 统一 (0, 9000)；净负荷独立')
104
105
106 def main():
107     build(False, 'fig5-7_四季典型日合并图',
108             '四季典型日光伏、负载与净负荷曲线')
109     build(True, 'fig5-7_四季典型日合并图_统一纵轴',
110            '四季典型日光伏、负载与净负荷曲线（光伏与负载共用纵轴）')
111     print('输出目录:', OUT)
112
113
114 if __name__ == '__main__':
115     main()

```

C22. 用户指定图源/rebuild_q23_G.py (21 行)

用户指定图：问题 2/3 三维图修正版（3D+2D 双联）重建

```

1 from pathlib import Path
2 import openpyxl, numpy as np, matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import Axes3D
4 R=Path(__file__).resolve().parent
5 src=R/'项目'/'00_题目与原始数据'/'Q1_Q2_Q3_Q4_原始附件'/'附件2.xlsx'
6 w=openpyxl.load_workbook(src,data_only=True,read_only=True)
7 def read(ws):
8     rows=list(ws.values); return np.asarray([r[1:145] for r in rows[1:]],float)
9 load,pv=read(w.worksheets[0]),read(w.worksheets[1]); net=load-pv
10 out=R/'figures'/'问题23_新增7图'/'3D遮挡方案对比'/'G_3D加2D双联_修正版'; out.mkdir(parents=
    True,exist_ok=True)
11 plt.rcParams.update({'font.sans-serif':['Microsoft YaHei','Noto Sans SC','SimHei','DejaVu
    Sans'],'axes.unicode_minus':False,'font.size':9})
12 def draw(a,name,title,cmap):
13     x=np.arange(1,366); y=np.arange(1,145); X,Y=np.meshgrid(x,y,indexing='ij')
14     fig=plt.figure(figsize=(12,5.7)); gs=fig.add_gridspec(1,2,left=.045,right=.91,bottom
    =.13,top=.84,wspace=.22)
15     ax=fig.add_subplot(gs[0],projection='3d'); ax.plot_surface(X,Y,a,cmap=cmap,rstride=3,
    cstride=2,linewidth=0,antialiased=True)
16     ax.set_xlabel('日期',labelpad=5); ax.set_ylabel('时间段',labelpad=7); ax.set_zlabel('功
    率',labelpad=13); ax.zaxis.label.set_rotation(90); ax.view_init(elev=32,azim=-60); ax.
    set_title('(a) 3D曲面',pad=8); ax.tick_params(pad=1,labelsize=8)
17     ax2=fig.add_subplot(gs[1]); cf=ax2.contourf(X,Y,a,levels=18,cmap=cmap); ax2.set_xlabel(
    '日期'); ax2.set_ylabel('时间段'); ax2.set_title('(b) 同一场的2D等高线',pad=8); ax2.
    set_xticks([1,91,182,274,365],[ '1月','4月','7月','10月','12月']); ax2.set_yticks
    ([1,25,49,73,97,121,144],[ '00:00','04:00','08:00','12:00','16:00','20:00','24:00']);
    ax2.tick_params(labelsize=8); fig.colorbar(cf,ax=ax2,shrink=.82,pad=.03,label='功率 /
    kW')
18     fig.suptitle(title,y=.93); fig.savefig(out/(name+'.png'),dpi=300,bbox_inches='tight');
    fig.savefig(out/(name+'.pdf'),bbox_inches='tight'); plt.close(fig)
19 draw(pv,'fig2_全年光伏_修正版','全年光伏发电量时空分布','YlOrBr'); draw(load,'fig3_全年负载_修正
    版','全年负载时空分布','Blues'); draw(net,'fig4_全年净负荷_修正版','全年净负荷时空分布','
    RdYlBu_r')
20 print(out)

```

C23. 04_ 结果/问题 4_ 口径 B 正式/结论 _Q4.md (一行)

```
1 # 问题4 口径B 结论 (电价未知条件下的重算 • eps 归一化修正后 • 2026-09-12 晚)
2
3 > **本文件是新树的权威结论文件** (`重跑_Q4_B_eps修正/结论_Q4.md`), **全部数字为修后值**, 取自唯一数字源
4 > `重跑_Q4_B_eps修正/修后数字总表_eps修正.md` (该总表数字又取自本目录 `q4_对照汇总.json`、`q4_配对统计.json`、
5 > `pi_bounds_q4.json`、`v台阶_Q4B.json`、`d4_Q4B.json`、各格 `p2_arithmetic.json`)。
6 > 结构底稿为修前树 `重跑_Q4_B_20260912/结论_Q4.md`; 凡引用修前值处一律出现在明确的"修前"列/句中 (见 §12)。
7
8 - 口径: **B (0:00 不知道当日电价)** —— 决策用**价格预测**、账单一律用**附件4 真实价** (含紧急 5
  x真实价);
9 跨日库存续存价值 v **逐日滚动** (L1 规则, 用因果预测构造); 1 月因果冷启动、2-01—12-31 交付 (与问题2/3 同协议)。
10 - 触发一: 用户裁定问题4 应采用口径B (原交付为口径A); 本轮按 `02_模型方案/问题4_情形B_改造方案.md` §5 实施,
11 并额外加入跨日库存价值、修正 `fp[di+1]` 前视 (原实现用含当日实际价的窗口 → 改为只用 < di 天)。
12 - **触发二 (2026-09-12 晚, 第二批)**: 共享内核 `03_代码/p10_core.py` 做 **eps 归一化修正** (见 §0),
13 用户裁定**问题4 采纳该修正并全量重跑**、**问题2/3 不重跑**。本文件即修后权威结论。
14 - 为实现对照, 做 **2x2 四格**: 口径A/B x v=0/滚动v。四格同预测缓存、同场景、同初值、同结算规则,
15 **全部用同一新引擎**, 故口径效应与库存价值效应可干净分解。
16 - 产物: `重跑_Q4_B_eps修正/` (`A_v0`、`A_v`、`B_v0`、`B_v`、`sens_refresh`、`probe_v035 /044/046/050`、`d4_F`),
17 含 `q4_对照汇总.json`、`q4_配对统计.json`、`pi_bounds_q4.json`、`v台阶_Q4B.json`、`d4_Q4B.json`、
18 各格 `p2_arithmetic.json`、`result4-2/4-3.xlsx`; 交付件已刷新到 `results/` (哈希见 §8/§12)。
19 - **前置修正 (2026-09-12 晚, 第一批)**: `p102_q4_core.scenarios()` 生成更新情景时**未使用附件3 (`fc3_step_day`)**
20 的缺陷已修复, **Q4-3 家族全部按修复后引擎重跑**。本文件的 Q4-3 数字**同时含 fc3 修复与 eps 修正两项**。
21 **Q4-2 的 A/B/C 三轨在 fc3 修复下不受影响** (`combo=None` 时两引擎同用点预报), 但在 **eps 修正下会变**——
22 这是本轮与上一轮的关键差别, 逐项见 §12。缺陷指纹见 §2 方框。
23
24 > **预测层审计判据说明 (沿用修前结论, 证据文件在修前树)**: 价格预测因果性审计原用**逐位相等**判据,
25 > `FP_shape` 因此被记为 FAIL(116/365); 经交付包独立复核定位并由本侧复核确认, 其为 **1-2 ULP (最大 6.66e-16)**
26 > 浮点不可复现假警报** (`exact_mismatches=116` 而 `max_abs_diff=6.66e-16`, 窗口水平值逐日相同), **不是前视泄漏**。
27 > 判据已改为 `allclose(rtol=1e-12, atol=1e-15)`, 7 变体现在全部 PASS。**本轮新树未重出 `audit/` 报告目录**,
28 > 但两树的 `forecast/电价预测缓存.npz` **逐字节相同** (SHA256 `A28E46BD04C00E5A...`, 见 §1), 故该结论直接适用。
29
30 ---
31
```

```

32 ## 0. 内核版本 (eps 归一化修正)
33
34 ### 0.1 修正内容
35
36 共享内核 `03_代码/p10_core.py` 的末槽 (日末库存) 目标系数:
37
38 | | 末槽目标系数 |  $S=10$ 、 $v=0.481548$  时的**有效日末系数** |
39 |---|---|---|
40 | **修前** | ` $\text{eps\_soc} + v\_term / S` |  $0.491548$  (=  $v\_term + S \cdot \text{eps\_soc}$ , 即协议 §13 第 8 条
    **披露的 +2.08% 残留**) |
41 | **修后** | ` $(\text{eps\_soc} + v\_term) / S` |  $0.482548$  |
42
43 即把 ` $\text{eps\_soc}$ ` 也纳入等权期望归一 ( $1/S$ ), 使**总末端系数 = ` $\text{eps\_soc} + v\_term$ `, 不再随情景数  $S$  变
    化**,
44 **消除了协议 §13 第 8 条的残留, 恢复"S 复制不变性". (出处: `02_模型方案/修订协议.md` §15。)
45
46 ### 0.2 两版内核哈希 (SHA256 前 16 位)
47
48 | 用途 | ` $\text{p10\_core.py}$ ` 哈希 | 时间 | 留档位置 |
49 |---|---|---|---|
50 | **问题4 交付 (修后)** | b3242ad925c82d63... | 2026-09-12 18:44:13 | ` $\text{03\_代码/p10\_core}$ 
    .py` (当前; 本机现算全哈希 `
    B3242AD925C82D63DBB46071E7135CE2A7C2675D6A36A4F5F63319B573B70148`) |
51 | 问题2/3 交付 (修前) | 0a2cef2e2b4428bb... | 2026-09-12 16:44:57 | ` $\text{修订前备份\_eps修正}$ 
    _20260912_184405/p10_core.py` |
52
53 ### 0.3 跨问题可比性 (**引用任何跨问题数字时必须随附**)
54
55 1. **问题2/3 (`q23_janwarm_v4`) 按用户裁定不重跑**, 其交付 (`results/result2.xlsx`、`result3.
    xlsx`、
56 四日表 ` $\text{q2\_}^*$ ` / ` $\text{q3\_}^*$ `) **仍为修前内核 (`0a2cef2e2b4428bb...`) 结果**; 问题4 交付为修后内核 (`
    b3242ad925c82d63...`)。
57 2. => **跨问题比较 (Q2/Q3 ↔ Q4) 必须注明两问使用不同内核版本**, 不得把两问数字置于同一"同引擎"口径
    下直接相减或排序。
58 3. ` $\text{results/}$ 指定日期表格.json` 因此**逐条目混合两种内核**: ` $\text{q2\_}^*$ ` / ` $\text{q3\_}^*$ ` 8 条为**修前内核**
    (字段值写明
59 "eps归一化修正前: p10_core.py 0a2cef2e2b4428bb... (末槽系数  $\text{eps\_soc}+v\_term/S$ , 有效日末系数
     $0.491548$ )"),
60 ` $\text{q42\_}^*$ ` / ` $\text{q43\_}^*$ ` 8 条为**修后内核** ("eps归一化修正后: p10_core.py b3242ad925c82d63... (末
    槽系数
61  $(\text{eps\_soc}+v\_term)/S$ , 有效日末系数  $0.482548$ )"); 共 16 条, 每条均带 ` $\text{\_内核}$ ` 字段。
62 4. **同问题4 内部**的所有比较 (四格、组合、部署、 $v$  台阶、PI、D4) **均为同一修后内核, 不受影响**。
63 5. **修前交付件留档位置** (回滚与对照用):
64 - 修前工作簿与四日表: ` $\text{04\_结果与检验/备份\_正式交付\_20260912/eps修正前\_口径B\_交付件/}$ `
65 - 修前 Q4 附带数据: ` $\text{04\_结果与检验/重跑\_Q4\_B\_eps修正/修前eps\_交付附带件/}$ `
66 - 修前图件: ` $\text{04\_结果与检验/备份\_正式交付\_20260912/eps修正前\_图件/}$ `
67 - 修前完整跑批 (含 ` $\text{sens\_track\_}^*$ `、`reg_old_protocol`、`sens_none_nofc3`): ` $\text{04\_结果与检验/}$ 
    重跑_Q4_B_20260912/`
68 - 修前内核源码: ` $\text{修订前备份\_eps修正\_20260912\_184405/p10\_core.py}$ `
69
70 ---$$ 
```

```

71
72 ## 1. 验收
73
74 | 项 | 结果 |
75 |---|---|
76 | 独立算术审计 (p103) | **五格全部 PASS** (修后重跑审计): A_v0 **75,451** 项、A_v **75,831**、
    B_v0 **75,686**、B_v **75,566**、sens_refresh **75,576**；**账单残差 0.0**、SOC 残差
    ≤1.55e-11、供需 ≤2.3e-13、同槽互斥 0.0 (详见 §10)。修前 eps 版计数为 75,446 / 75,561 /
    75,671 / 75,561 / 75,576 |
77 | 决策价/结算价分离 | 审计对 A/B/C 三轨抽样重解计划 LP (用记录 v)，逐位复现 q0 (**18 次求解全
    PASS**)；账单一律按真实价独立重建 |
78 | 价格预测因果性 | 独立重实现 hist7 规则与缓存逐位一致 (**364 天 0 不一致**)；v 用 `fp[di]` 而非
    `fp[di+1]`；7 变体因果性 PASS (判据 `allclose(1e-12, 1e-15)`，见文首说明) |
79 | 预测缓存未受影响 | 新树 `B_v/forecast/电价预测缓存.npz` 与修前树**逐字节相同** (SHA256 `
    A28E46BD04C00E5A4AF96223AB760DC7B1795EE80F20088B6E09E13DEFD3ED34`)；价格 MAE、v 均值、
    chosen_track 全部不变 |
80 | 内核健壮性修正 | HiGHS 在退化算例 (储能恰为上限 10800、购电锁定) 偶发 `Status 15 Unknown/
    primal Feasible`；新增**求解阶梯** (highs → highs-ds → highs-ipm → 无 presolve)，仅首次失
    败时触发，**对首次成功的求解逐位无影响** |
81 | 交付件校验 | 四日表 ↔ 工作簿逐格校验：最大绝对差 **0.005000** (容差 0.0051) => **PASS**；四项
    交付哈希见 §12.3 |
82 | 旧协议回归位 | 口径A、v=0、旧初始化复现 **15,156,589.57** vs 历史 **15,221,458.20** (
    **-0.43%** ) ——**该回归轨 (`reg_old_protocol`) 只存在于修前树，是修前 eps 内核结果，本轮未
    重跑** |
83
84 ---
85
86 ## 2. 四格结果 (2025-02-01—12-31, 334 天, 元)
87
88 | 格 | Q4-2 B (主模型) | Q4-2 A | Q4-2 C | Q4-3 部署 | Q4-3 none | v 均值 |
89 |---|---|---|---|---|---|---|
90 | **A_v0** 口径A.v=0 | 15,160,985.57 | 15,446,506.79 | 15,502,572.52 | 16,255,264.10 |
    17,370,316.11 | 0 |
91 | **A_v** 口径A.滚动v | 14,883,204.81 | 15,403,090.82 | 15,179,804.11 | 15,935,449.22 |
    16,922,745.22 | 0.4800 |
92 | **B_v0** 口径B.v=0 | 15,269,768.32 | 15,602,004.90 | 15,583,334.90 | 16,324,851.37 |
    17,490,464.82 | 0 |
93 | **B_v** 口径B.滚动v (**正式**) | **14,995,507.04** | 15,550,200.15 | 15,276,342.25 |
    **15,716,088.61** | 16,506,164.40 | 0.4800 |
94
95 - **问题4 头条数字**：Q4-2 B 四格 = 15,160,985.57 | 14,883,204.81 | 15,269,768.32 |
    **14,995,507.04**；
96 Q4-3 部署四格 = 16,255,264.10 | 15,935,449.22 | 16,324,851.37 | **15,716,088.61**。
97 - **正式格 `B_v` (口径B × 滚动 v)**：Q4-2 主模型 B **14,995,507.04** 元、Q4-3 历史选型部署
    **15,716,088.61** 元。
98 - Q4-2 的 A/B/C 排序在四格中**一致**：B 最便宜、C 最贵、A 居中；同格内 B-A 分别为
99 -285,521.22 (A_v0)、-519,886.01 (A_v)、-332,236.58 (B_v0)、-554,693.12 (B_v) 元。
100 - **Q4-3 none (B_v) = 16,506,164.40**；**部署 - none = -790,075.79 元 (-4.79%)**。
101 365 天总额 (仅参考)：none **19,303,263.54**、部署 **18,496,064.62**。
102 - 分项 (334 天)：官方口径的数字全部由各格 `q42_逐日费用.csv` / `q43_逐日费用.csv` (`日期 ≥
    2025-02-01`)

```

103 ****独立求和复算****，与 `q4\_对照汇总.json` 的 `q42\_334` / `q43\_334` ****逐格一致****。

104 ****注意****：`B\_v/q43\_组合对照.csv` 是**365** 天汇总文件******（每组合仅 1 行、`天数=365`），

105 ****不能****按日期筛出 334 天；334 天组合口径必须从 `q43\_逐日费用.csv` 按 `模型` 分组求和（本文件即如此，逐格与 `q43\_334` 一致）。

106 - ****Q4-3 组合对照（B_v, 334 天，元）****（正式表见总表 §1b；本文件亦与之一致）：

107 修前 → 修后 = none 16,430,707.21 → ****16,506,164.40**** | 6（最优固定）15,670,846.79 → ****15,712,944.80**** |

108 12 16,356,233.30 → ****16,429,010.31**** | 18（最差）16,500,977.36 → ****16,583,022.61**** | 6+12 15,782,816.29 → ****15,825,198.58**** |

109 6+18 15,712,487.02 → ****15,760,850.14**** | 12+18 16,400,748.07 → ****16,478,897.70**** |

110 6+12+18 15,810,241.80 → ****15,858,374.49**** | 部署 15,675,523.45 → ****15,716,088.61****。

111 - ****部署路径比"永远用固定组合 6"贵 3,143.81 元****（总表 §1b；按未舍入总额相减为 3,143.80，差 0.01 系两位小数展示舍入；

112 修前为 4,676.66 元）=> ****选型规则不是事后最优****；最优固定组合仍是 `6`、最差仍是 `18`（与 Q2/Q3 同构）。

113

114 **### 2.1 fc3 缺陷（保留节）**

115

116 `p102\_q4\_core.scenarios()` 生成更新情景时****未使用附件3（`fc3\_step\_day`）****的缺陷（`use\_fc3` 分支缺失，

117 症状 `N-F = 0.00`）曾使 Q4-3 家族数字****恰好等于同格 Q4-2 点预报结果****。

118

119 - ****缺陷指纹（fc3 修复前的 eps 修前树 `重跑\_Q4\_B\_20260912/`）：`q43\_334/none` = `q42\_334/B` = 14,990,278.87 元****。

120 - 该等式****在本修后树中已不成立****（修后 `q42\_334/B` = 14,995,507.04、修后 `q43\_334/none` = 16,506,164.40），

121 正说明 Q4-3 已真正使用附件3（已发布预报，MAE 332 kW、更差）。

122 - ****"Q4-3 与 Q4-2 数值接近、结论一致"是该缺陷造成的假象****：修后 Q4-3 费用明显****高于**** Q4-2。

123 - 修前留档：`重跑\_Q4\_B\_20260912/q4\_对照汇总\_修前fc3缺陷.json`。

124

125 ---

126

127 **## 3. 效应分解与配对统计（334 天）**

128

129 **### 3.1 效应分解（元）**

130

效应	修前	**修后**
Q4-2 跨日价值 (B_v - B_v0)	-276,112.31	** -274,261.28** (-1.80%)
Q4-2 跨日价值 (A_v - A_v0)	-275,006.70	** -277,780.76**
Q4-2 跨口径差 (B_v - A_v)	+107,105.71	** +112,302.23** (+0.75%)
Q4-2 跨口径差 (B_v0 - A_v0)	+108,211.32	** +108,782.75**
Q4-2 交互项	-1,105.61	** +3,519.48** (仍=0, 近似可加)
Q4-3 部署 跨日价值 (B_v - B_v0)	-648,436.14	** -608,762.77** (-3.73%)
Q4-3 部署 跨口径差 (B_v - A_v)	-236,851.66	** -219,360.61** (-1.38%; **仍为负, 不得作 VOI**, 见 §8)

140

141 ****读法****：口径B 的代价 (Q4-2 主模型 B, ****+112,302.23 元 = +0.75%****) 与改造方案原型实测的 +0.74%~+1.07%

142 (`问题4\_情形B\_改造方案.md` §四，仅供对照) 一致，****远小于****跨日库存价值 (Q4-2 约 **** -27.4 万元/年 ****, -1.80%)；

143 两者**近似独立可加**（交互项 +3,519.48 $\approx$ 0）。

144 Q4-3 家族上跨日库存价值更大（**-60.9 万元，-3.73%**），因附件3 已发布预报误差更大、更需要“留电”；

145 而 Q4-3 的“口径A→B”为**负值（-21.9 万元）**——属**跨口径机制副作用**，**不得解读为价格信息价值**（§8）。

146 （修前对应值：+0.72%、-27.6 万元/-1.81%、-64.8 万元/-4.0%、-23.7 万元/-1.49%。）

147

148 **### 3.2 配对统计（`q4\_配对统计.json`）**

149

150 同日期配对、7 日分块 bootstrap×2000、seed 2025；**正 = 前者更贵**。

151 **共 56 组比较，51 组显著**（修前批次为 59 组 / 54 组显著）。关键项（元/日）：

152

比较	修后均值	95% CI	天数（正/负）	显著
--- --- --- --- ---				
B_v 部署 - none (Q4-3)	-2,365.50	[-2,911.42, -1,826.11]	89 / 245	是
B_v 6 - none (Q4-3)	-2,374.91	[-2,928.10, -1,832.31]	86 / 248	是
B_v 6+12 - none	-2,038.82	[-2,583.82, -1,471.33]	101 / 233	是
B_v 6+18 - none	-2,231.48	[-2,808.86, -1,664.29]	91 / 243	是
B_v 6+12+18 - none	-1,939.49	[-2,496.93, -1,379.51]	108 / 226	是
B_v 12 - none	-231.00	[-454.54, +10.82]	139 / 195	**否**
B_v 12+18 - none	-81.64	[-314.72, +167.21]	158 / 176	**否**
B_v 18 - none	+230.11	[+126.76, +345.46]	165 / 147	**是（有害）**
跨格 B_v - A_v (Q4-2 B)	+336.23	[+250.27, +421.99]	201 / 133	是
跨格 B_v - A_v (Q4-3 部署)	-656.77	[-939.59, -389.07]	126 / 208	是 (**不得作 VOI**)
跨格 B_v - A_v (Q4-3 none)	-1,247.25	[-1,722.02, -801.02]	125 / 209	是（同上）
跨格 B_v - B_v0 (Q4-2 B)	-821.14	[-993.90, -652.52]	75 / 259	是
跨格 B_v - B_v0 (Q4-3 部署)	-1,822.64	[-2,186.29, -1,485.20]	55 / 279	是
跨格 A_v - A_v0 (Q4-2 B)	-831.68	[-1,017.76, -645.43]	116 / 218	是
可刷新B - 严格B (Q4-3 部署)	+3.92	[-14.59, +21.71]	143 / 145	**否**
B_v Q4-2 B - A	-1,660.76	[-1,879.26, -1,432.48]	43 / 291	是
B_v Q4-2 C - B	+840.82	[+489.99, +1,195.98]	204 / 130	是
B_v0 部署 - none (Q4-3)	-3,489.86	[-3,992.15, -2,973.82]	58 / 276	是
B_v0 18 - none (Q4-3)	+58.57	[+29.35, +88.70]	177 / 156	是（有害）

174

175 - 口径A 与口径B 的跨日价值效应**几乎相同**（-831.68 vs -821.14 元/日）——因为 v 由 `hist7` 规则确定，

176 **与决策用价无关**。

177 - **Q4-3 家族上跨口径对比全部反号**（none / 组合 6 / 组合 6+12+18 / 部署 = **-1,247.25 / -637.54 / -572.50 / -656.77** 元/日，**B 更便宜**）——属**跨口径机制副作用**，**不得解读为价格信息价值**（§8）；同口径内比较不受影响。

178

179 - 更新组合的定性排序：**最佳 `6`、最差 `18`（显著有害）**，与 Q2/Q3 同构；`12` 与 `12+18` 不显著。

180

181 ---

182

183 **## 4. 价格轨与刷新敏感性**

184

185 **### 4.1 价格轨（Q4-2 主模型 B，334 天）**

186

187 > **重要标注**：价格轨敏感性（`sens\_track\_lag1/fix/shape`）**只在修前树存在，本轮 eps 修正未重跑

**,
 > 故下表**总费用列是修前 eps 内核值**（已明确标注）。**预测 MAE 列与内核无关**（预测缓存两树逐字节相同）。

变体	总费用（元，**修前 eps 内核**，未重跑）	相对主轨	预测 MAE（元/kWh）
--- --- --- ---			
hist7（主轨，按 MAE 预选）	**14,990,278.87**（修后为 **14,995,507.04**，见 §2）	—	**0.082669**
lag1	15,040,626.62	+50,347.75	0.084326
fix（附件1 曲线原样）	15,043,344.68	+53,065.81	0.096323
shape（附件1 形状×历史水平）	15,044,162.32	+53,883.45	0.092185

- **按预测精度预选的主轨同时是费用最低的**（四轨中 MAE 最小且费用最低）=> 选轨规则不是"按结果挑"。该结论**不受 eps 修正影响**：修正只改变末槽系数，不改变四轨之间的预测精度排序。
 - 修后正式主轨 `hist7` 的 Q4-2 B 费用为 **14,995,507.04** 元（见 §2）；**修后 lag1/fix/shape 的费用未重算，不得引用**。
 #### 4.2 可刷新B vs 严格B（334 天，元）

组合	严格B（元）	可刷新B（元）	差（元）
--- --- --- ---			
部署（历史选型）	**15,716,088.61**	15,717,398.34	**+1,309.73**
6+12+18	15,858,374.49	15,852,214.95	-6,159.54
none	16,506,164.40	16,506,164.40	0.00

配对日均 **+3.92 元/日，95% 区间 [-14.59, +21.71]（含 0，中位数 0.0，143/145 天）=> 不显著**。
 结论：**当日价格水平几乎不可预测，日内刷新无实质收益**，正式口径取**严格B**。
 （修前对应：+1,507.60 / -2,521.52 / 0.00，配对 **+4.51** [-11.95, +20.12]。）
 #### 4.3 误差传导形态（B_v vs A_v，同为主模型 B，334 天）

预测误差主要转成"多买/少买计划电"：计划购电费 **+102,090.13 元**、紧急购电量 **-11,508.64 kWh**（443,895.95 vs 455,404.60，持平量级）、紧急购电费 **+10,212.10 元**，即"紧急购电量基本持平"。
 - **该形态只成立于 Q4-2（预报较准）**：修后 Q4-3 部署上，口径 A 的紧急购电量反而高出 **513,670.64 vs 452,102.46 kWh（+13.6%）**，紧急费 A 高 **177,937.72 元**、计划费 A 反而便宜 **98,597.24 元**，
 但 A 的**调整费高 140,020.13 元**，合计 A 净贵 **219,360.61 元**——这是 §8 机制诊断的核心证据，
 Q4-3 家族上"紧急购电量基本持平"不成立。
 - 预测缓存的来源核对（MAE 与 v 均值**修前修后完全相同**）：`hist7` MAE **0.082669** 元/kWh；
 各时段 MAE（`start=31`，334 天 × 144 槽 = 48,096 点）0-6 时 0.046431、6-12 时 0.107469、12-18 时 0.114959、
 18-24 时 0.061819；RMSE 0.105265、bias -0.000328；滚动 v 均值 **0.479968**（365 天 0.480298）。

 ## 5. 与旧交付（口径A）的差异归因

旧交付 `result4-2/4-3.xlsx`（旧协议 334 天独立起跑、口径A、v=0）为历史参照。新引擎在等价配置下复

现

231 ****15,156,589.57** vs 历史 **15,221,458.20** (**-0.43%**) ——**该回归轨只存在于修前树，是修前**

232 **eps 内核结果**，归因：**

233 **1. **计划/更新 LP 的平局参数**：旧引擎调用 `solve\_plan\_lp` 未传 `eps\_soc`/`tp`（取 0/0），新引擎**

234 **按项目协议传**

235 **1e-3/1e-6；机制测试显示该参数确实改变计划解（抽样 5 天 $|\Delta q|$ 合计 580 kWh，经 5× 紧急价放大后与**

236 **-0.43% 同量级）。**

237 **2. **预测缓存重建**（同算法同数据的新构建）。**

238 **==> 四格内比较**不受**该差异影响（全部同一引擎）；旧交付仅作历史参照并已在交付目录标注。**

239 **---**

240 **## 6. v 台阶探针（Q4-2 B 主模型，334 天）**

241

242 | v | 总费用（元） | 紧急费（元） | E0 均值（kWh） | E0 中位（kWh） | 触底天数 | 年末储能（kWh）

243 | ---|---|---|---|---|---|

244 | 0 (B_v0 格对照) | 15,269,768.32 | 1,989,970.70 | 1,631.8 | 1,392.7 | 58 | 1,200.00 |

245 | 0.35 | 15,135,439.62 | 1,835,251.02 | 2,058.5 | 1,715.6 | 26 | 1,200.00 |

246 | 0.44 | 15,033,942.20 | 1,738,712.55 | 5,435.9 | 5,758.8 | 6 | 7,549.88 |

247 | 0.46 | 15,011,992.48 | 1,711,758.94 | 7,005.9 | 8,220.9 | 4 | 9,293.93 |

248 | 0.50 | 14,983,782.28 | 1,672,677.08 | 7,938.2 | 9,683.0 | 1 | 10,068.78 |

249 | ****滚动 v（正式，均值 0.4800）**** | ****14,995,507.04**** | 1,691,260.45 | 7,139.0 | 8,335.3 | 1

250 | 9,501.53 |

251 - ****响应是台阶不是平滑最优**：v 由 0 → 0.35 只改善 ****134,328.70**** 元，0.35 → 0.44 再改善**

252 ****101,497.42** 元，**

253 ****台阶区间仍约 [0.35, 0.44]**；再往上（0.46 / 0.50）每次只再改善 2-3 万元以内。固定价场景的台**

254 **阶为 0.455-0.470（Q2/Q3），**

255 **波动价场景下移到约 [0.35, 0.44]——同一机制，量级不同。**

256 - ****滚动 v（L1 规则，均值 0.4800）落在台阶之上**：比 v = 0.50 贵 ****11,724.76**** 元（+0.078%）、**

257 **比 v = 0.46 便宜**

258 ******16,485.44**** 元、比 v = 0.44 便宜 38,435.16 元 => 选值只在台阶上方 2-9% 区间内，其影响是**量**

259 **级切换**而非精细调优。**

260 ****引用 v 时必须随附"台阶式、规则指定而非最优"。****

261 **（取总表 §5 的 11,724.76，即由两位小数展示值相减（14,995,507.04 - 14,983,782.28）所得；**

262 **按未舍入总额相减为 ****11,724.75****，差 0.01 系舍入。本文件与全部交付文档**派生差值一律先取总表展示**

263 **值相减、再附未舍入精确值**。）**

264 - ****与 v=0 的对照才是结论主体**：14,995,507.04 - 15,269,768.32 = ****274,261.28** 元（-1.80%）****

265 **，且触底天数**

266 ******58 → 1****、日均 E0 由 1,631.8 → 7,139.0 kWh；跨日价值项消除的是"每天把储能挖空"的行为。**

267 - **触底天数定义 = **日初储能 ≤1,200 kWh（严格）的天数**。v **永不计入账单**（残差 ≤1.6e-11 元）。**

268 - ****修前对应表**（仅作参考）：滚动 v 14,990,278.87、v=0 15,266,391.18、0.35 15,131,351.13、0**

269 **.44 15,026,114.57、**

270 **0.46 15,006,151.32、0.50 14,985,332.32；触底 56 / 26 / 5 / 4 / 1 / 0。**

271 **---**

272 **## 7. 补做：PI（价格+负荷/光伏完全信息）上界与缺口**

269 ****定义****: PI 为"价格与负荷/光伏****均完全预知****"的全期连续优化 (LP), 用****真实附件4 电价****、自由终端、
270 与滚动臂相同的物理约束 (容量/功率/效率/末值区间), 并把****初始储能对齐到该路径真实的 2/1 初态****。
271 ****价格与负荷/光伏均完全信息, 且含全期规划, 故 PI 为综合上界, 不等于价格预报价值****
272 (其 GAP 同时含价格预报误差 + 负荷/光伏预报误差 + 全期规划 + 滚动近似)。40 条路径只需求解 ****22 个**
273 唯一 LP******
274 (同初态复用)。`pi\_bounds\_q4.json`。
275 | 格 | 路径 | 2/1 初态 (kWh) | 实际 (元) | PI (元) | 缺口 (元) | 缺口% |
276 |---|---|---|---|---|---|---|
277 | A_v0 | Q4-2 B | 1,645.7 | 15,160,985.57 | 12,782,476.66 | 2,378,508.91 | 15.69% |
278 | A_v | Q4-2 B | 10,798.3 | 14,883,204.81 | 12,779,221.79 | 2,103,983.01 | 14.14% |
279 | B_v0 | Q4-2 B | 1,615.1 | 15,269,768.32 | 12,782,488.02 | 2,487,280.30 | 16.29% |
280 | ****B_v**** | ****Q4-2 B (正式)**** | ****10,677.2**** | ****14,995,507.04**** | ****12,779,262.16**** |
****2,216,244.88**** | ****14.78%**** |
281 | A_v0 | Q4-3 部署 | 1,629.6 | 16,255,264.10 | 12,782,482.63 | 3,472,781.47 | 21.36% |
282 | A_v | Q4-3 部署 | 10,798.3 | 15,935,449.22 | 12,779,221.79 | 3,156,227.43 | 19.81% |
283 | B_v0 | Q4-3 部署 | 1,625.9 | 16,324,851.37 | 12,782,483.99 | 3,542,367.39 | 21.70% |
284 | ****B_v**** | ****Q4-3 部署 (正式)**** | ****10,620.8**** | ****15,716,088.61**** | ****12,779,280.98**** |
****2,936,807.62**** | ****18.69%**** |
285 | B_v | Q4-3 none | 10,574.9 | 16,506,164.40 | 12,779,296.28 | 3,726,868.12 | 22.58% |
286
287 - ****正式口径下的综合缺口 14.78% (Q4-2 B 格) / 18.69% (Q4-3 部署)****, 即约 ****221.6 万 / 293.7 万**
元******;
288 滚动 MPC 距"完全信息理想解"仍有 15% - 19% 的空间。(修前为 14.75% / 18.48%。)
289 - ****PI 对初态几乎不敏感****: 初始态 1,615.1 vs 10,677.2 kWh (差 9,062.1 kWh) 只使 PI 变动
****3,225.86 元****
290 (12,782,488.02 vs 12,779,262.16)——PI 可自由安排全年购电节奏, 初态被完全"吸收"。
291 - ****限制 (务必随数字引用)****: PI 含价格、负荷/光伏、全期规划****三重信息****, ****不是价格预报的价值****;
292 正式臂年末仍有库存未获计价 (Q4-2 B 9,501.53 kWh、Q4-3 部署 9,593.72 kWh; 按有效日末系数
0.482548 折算约
293 ****4,585 / 4,629 元****, 占缺口 0.21% / 0.16%, 可忽略)。
294 - Q4-3 族内"更新"把缺口从 ****22.58% 压到 18.69%**** (none → 部署), 与 §2 的更新价值一致。
295
296 ---
297
298 **## 8. 跨口径使用限制 (A/B 差异不得解读为价格信息价值)**
299
300 ****现象****: 在同一家族、同一初始态、同一负荷/光伏预报下, 仅把"决策价"从预测价 (口径B) 换成真实价 (口
径A),
301 ****方向相反****:
302
303 | 家族 | 比较 | A (元) | B (元) | B - A (元) |
304 |---|---|---|---|---|
305 | Q4-2 (q42, 无 fc3) | 主模型 B, 滚动 v | 14,883,204.81 | 14,995,507.04 | ****+112,302.23 (B**
更贵, 符合预期)****** |
306 | Q4-3 (q43, 附件3 预报) | none, 滚动 v | 16,922,745.22 | 16,506,164.40 | **** -416,580.82 (B**
更便宜, 反直觉)****** |
307 | Q4-3 (q43) | 组合 `6`, 滚动 v | 15,925,883.16 | 15,712,944.80 | -212,938.36 |
308 | Q4-3 (q43) | 部署, 滚动 v | 15,935,449.22 | 15,716,088.61 | **** -656.77 元/日 →**
-219,360.61****** |

309

310 ****机制证据 (Q4-3 部署, B_v vs A_v, 334 天, 逐项, 修后重算) **:**

311

312 1. ****计划费 A 反而更便宜**:** 13,525,246.02 (A) vs 13,623,843.26 (B), A 便宜 ****98,597.24 元** =>**
不是**"计划买少了"**。

313 2. ****紧急罚通道被占用更狠**:** 紧急购电量 ****513,670.64 (A) vs 452,102.46 (B), +13.6%**;**
314 紧急购电费 A 高 ****177,937.72 元**.** ****none 轨同一形态更极端**:** 965,120.82 (A) vs 807,553.84
(B), ****+19.5%****,
315 紧急费 A 高 521,982.78 元。

316 3. ****调整费 A 更高**:** 710,507.54 (A) vs 570,487.41 (B), A 高 ****140,020.13 元**.**

317 4. ****净结果**:** -98,597.24 (计划费) + 140,020.13 (调整费) + 177,937.72 (紧急费) = ****A 净贵**
219,360.61 元****.**

318 5. ****对照 Q4-2 (预报更准) **:** 同为主模型 B 的紧急量差仅 **** -11,508.64 kWh (-2.5%) ****, 因为 q42 用
在线选出的较准预报;
319 q43 用附件3 已发布预报 (MAE 332 kW), 误差更大。

320 6. ****修前内核诊断 (未按修后内核重算, 仅作机制佐证, 不得作为修后数字引用) **:** 槽级触底槽数 3,176 (A)
vs 2,640 (B)
321 (+20.3%)、紧急槽处平均储能 5,019 vs 5,594 kWh、日未储能 7,203 vs 7,796 kWh、有紧急购电的天
数 326 vs 317、
322 紧急量集中在清晨 4-10 时 (A-B: 4-6 时 +29,291、6-8 时 +75,528、8-10 时 +20,449 kWh)。

323

324 ****结论 (机制) **:** 知道真实价格使控制器做****更激进的价格套利**** (更深放电、更少留存电量),
325 从而在负荷/光伏预报误差出现时****更常处于无余量状态****, 只能以 5 倍价紧急购电。
326 该副作用在预报较差 (q43) 时足以****压过价格信息的正价值****, 使口径**B"反而更便宜"**。

327

328 ****因此 (使用限制, 引用处必须随附) **:**

329

330 1. ****禁止**把跨口径 A/B 的差异 (尤其 q43 家族的 **** -416,580.82 元** / ** -656.77 元/日****) 解读为"**
价格信息价值**"或 VOI.**

331 2. Q4-2 四格的 ****+112,302.23 元****应表述为**"**在该执行机制下、用预测价决策相对用真实价决策的成本差**
****"**,
332 并可视为价格预报价值的****下界**** (口径A 同时改变了控制激进度, 该副作用与口径B 不可分离)。

333 3. ****正式交付口径为 B****, 其全部交付数字 (四个交付表、结论正文) 不依赖跨口径比较, ****不受本诊断影响****
。

334 4. ****同口径内****的比较 (none / 组合 / 部署 / v) 均为同机制对照, ****不受影响****。

335 5. 若要干净的 VOI, 需另设实验: 把 5x 紧急通道从决策目标中剥离 (或固定控制激进度) 后再比较口径A/B。
336 ****已登记为待办, 未做** (§11)。**

337

338 ---

339

340 **## 9. 补做: Q4 版 D4 式拆分 (更新价值来源)**

341

342 对 Q4-3 家族 (B_v, 334 天) 做与 Q2/Q3 同构的三层拆分 (`d4\_Q4B.json`):

343

项	含义	修前 (元)	**修后 (元) **
N	不更新 (锁定计划购电)	16,430,707.21	**16,506,164.40**
F	仅刷新预报 (仍锁定购电安排)	16,428,212.81	**16,503,426.56**
FM	刷新预报 + 允许按新预报调整购电	15,810,241.80	**15,858,374.49**
N - F	只看不买	+2,494.40 (+0.02%)	**+2,737.85 (+0.017%) **
F - FM	用于市场购电再交易	+617,971.01 (+3.76%)	**+645,052.06 (+3.91%) **

350

351 | N - FM | 更新总收益 | +620,465.41 (+3.78%) | **+647,789.91 (+3.92%)** |

352

353 - **N - F = +2,737.85 元 (+0.017%)**：把新预报只用于"看"、不用于"买"，几乎一文不值

354 (**fc3 修复前该值为 0.00，正是缺陷症状**)。

355 - **F - FM = +645,052.06 元 (+3.91%)**：把新预报用于**市场购电再交易**，是收益的 **99.58%**。

356 - **与 Q2/Q3 同构**：Q2/Q3 的"再交易收益占比"为 **87.6%**，Q4 进一步升到 **99.58%**——在波动电价下

357 "预报用于储能调度"的边际价值**比固定价场景更低**，几乎全部价值来自现货市场的滚动再交易。

358 **这是对"更新机制为什么有用"的核心解释，而非模型缺陷**。

359

360 ---

361

362 ## 10. 审计 (修后, 各格 `p2\_arithmetic.json`)

363

364 | 格 | 检查项 | 账单残差 | SOC 残差 | 供需残差 | 同槽互斥 | 状态 |

365 |---|---|---|---|---|---|---|

366 | A_v0 | **75,451** | 0.0 | 1.46e-11 | 2.27e-13 | 0.0 | **PASS** |

367 | A_v | **75,831** | 0.0 | 1.27e-11 | 2.27e-13 | 0.0 | **PASS** |

368 | B_v0 | **75,686** | 0.0 | 1.18e-11 | 2.27e-13 | 0.0 | **PASS** |

369 | B_v (正式) | **75,566** | 0.0 | 1.55e-11 | 2.27e-13 | 0.0 | **PASS** |

370 | sens_refresh (可刷新B) | **75,576** | 0.0 | 1.55e-11 | 2.27e-13 | 0.0 | **PASS** |

371

372 审计内容：独立重建 SOC 递推、供需、功率、同槽互斥、**按真实价的全部账单**、部署选型累积；**账单残差 0.0**。

373 每格另含两项 PASS 发现：`hist7` 独立重建` (364 天 0 不一致)、`v` 因果性 (用 fp[di] 而非 fp[di+1])`；

374 以及 A/B/C 三轨抽样重解计划 LP (每轨 6 天，共 18 次求解) 全 PASS。

375 (修前 eps 版计数为 75,446 / 75,561 / 75,671 / 75,561 / 75,576。)

376

377 ---

378

379 ## 11. 未做与已补做

380

381 **已补做 (2026-09-12 补做批, 本轮已按修后内核重算并更新)**:

382

383 - **PI (价格+负荷/光伏完全信息) 上界与缺口** (§7)；

384 - **Q4 版 D4 式拆分** (§9)；

385 - **v 台阶探针** (§6)；

386 - **Q4-3 组合与跨格配对显著性检验** (§3.2；全 **56** 组比较中 **51** 组显著)；

387 - **价格轨与刷新轨的配对检验**：lag1 / fix / shape 相对主轨 (**修前内核值, 未重跑**)；可刷新B - 严格B

388 **+3.92** 元/日 [-14.59, +21.71]，**不显著** (§4.2)。

389

390 **仍未做 (不得声称已完成)**:

391

392 - **价格轨 `lag1` / `fix` / `shape` 在修后内核下的费用重算** (本轮只重跑了 hist7 主轨；§4.1 表为修前内核值)；

393 - **旧协议回归轨 (`reg\_old\_protocol`) 在修后内核下的重跑** (§1 的 -0.43% 为修前内核值)；

394 - `none` 轨信息集敏感性 (`Q4\_NONE\_NO\_FC3=1` 开关已就绪，**修后内核内对照未做**；修前树有 `sens\_none\_nofc3/`)；

395 - 干净的 VOI 实验 (剥离 5x 紧急通道或固定控制激进度后再比较口径A/B)——无此实验时 §8 的限定**必须

```

    随附**；
396 - 问题4 的 Sxseed 稳定性矩阵（Q2/Q3 已有 9 格，Q4 未做）；
397 - 口径B 下 linear（线性外推）轨重跑（Q2/Q3 已完成，Q4 未做）；
398 - 价格预报层（LightGBM/7 变体）的第三方脚本级独立复现；`v = 0.481548` 的经济学论证；
399 - 价格轨超参搜索（W=7 与刷新时点冻结）；hist7 与 lag1 的 MAE 差（0.001657）显著性未判；
400 - 9 格矩阵原始逐日数据归档（只存汇总）；打包目录标注为"证据归档、非端到端可复跑"
401   (`p102_q4_core.py` 硬编码 `ROOT = HERE.parent`)。
402
403 ---
404
405 ## 12. 修前 → 修后对照（eps 归一化修正）
406
407 > **修前** = `重跑_Q4_B_20260912/`（fc3 已修复、eps **未**修正，内核 `0a2cef2e2b4428bb...`）；
408 > **修后** = 本树（内核 `b3242ad925c82d63...`）。两树**预测缓存逐字节相同**，故全部差异只来自未槽
    系数 0.491548 → 0.482548。
409
410 ### 12.1 四格（334 天，元）
411
412 | 格 | Q4-2 B 修前 | Q4-2 B 修后 | Δ | Q4-3 部署 修前 | Q4-3 部署 修后 | Δ |
413 | ---|---|---|---|---|---|---|
414 | A_v0 | 15,158,179.86 | **15,160,985.57** | +2,805.71 (+0.019%) | 16,252,511.47 |
    **16,255,264.10** | +2,752.63 (+0.017%) |
415 | A_v | 14,883,173.16 | **14,883,204.81** | +31.65 (+0.000%) | 15,912,375.11 |
    **15,935,449.22** | +23,074.12 (+0.145%) |
416 | B_v0 | 15,266,391.18 | **15,269,768.32** | +3,377.14 (+0.022%) | 16,323,959.59 |
    **16,324,851.37** | +891.78 (+0.005%) |
417 | **B_v（正式）** | 14,990,278.87 | **14,995,507.04** | **+5,228.16 (+0.035%)** |
    15,675,523.45 | **15,716,088.61** | **+40,565.16 (+0.259%)** |
418
419 Q4-3 none (B_v)：修前 16,430,707.21 → 修后 **16,506,164.40**；**部署 - none**：修前 -755,184
    → 修后 **-790,075.79**。
```

420

```

421 > **365 天总额（仅参考，非交付窗口）**：修后 none **19,303,263.54**、修后部署 **18,496,064.62**
    ；
422 > 修前对应为 **19,227,982.20 / 18,455,634.33**。
```

423 > **注**：总表 §1 括注写作"部署 18,718,106.19"，该值**在两棵树的任何格、任何 365 天口径下都不存在

424 > （已用 `q4\_对照汇总.json` 的 `q43\_365` 与各格 `p2\_arithmetic.json` 逐格核对），视为**总表笔误

425 > 本文件按可复核的 **18,496,064.62** 引用。

426

```

427 ### 12.2 效应、PI、v 台阶、D4、审计
428
429 | 项 | 修前 | 修后 |
430 | ---|---|---|
431 | Q4-2 跨日价值 (B_v-B_v0) | -276,112.31 | **-274,261.28** |
432 | Q4-2 跨日价值 (A_v-A_v0) | -275,006.70 | **-277,780.76** |
433 | Q4-2 跨口径差 (B_v-A_v) | +107,105.71 | **+112,302.23** |
434 | Q4-2 交互项 | -1,105.61 | **+3,519.48** |
435 | Q4-3 部署 跨日价值 | -648,436.14 | **-608,762.77** |
436 | Q4-3 部署 跨口径差 | -236,851.66 | **-219,360.61**（仍为负） |
```

```

437 | PI 缺口 (Q4-2 B / Q4-3 部署) | 2,211,018 (14.75%) / 2,896,250 (18.48%) | **2,216,244.88 (
      14.78%) / 2,936,807.62 (18.69%) ** |
438 | 配对 部署 - none (Q4-3) | -2,261.03 [-2,804.33, -1,705.81] | ** -2,365.50 [-2,911.42,
      -1,826.11]** |
439 | 配对 18 - none (Q4-3) | +210.39 [+111.51, +329.09] | ** +230.11 [+126.76, +345.46]** |
440 | 配对 可刷新B - 严格B | +4.51 [-11.95, +20.12] | ** +3.92 [-14.59, +21.71]** |
441 | 比较组数 / 显著组数 | 59 / 54 | **56 / 51** |
442 | D4: N-F / F-FM / 再交易占比 | +2,494.40 / +617,971.01 / 99.6% | ** +2,737.85 / +645,052.06
      / 99.58%** |
443 | 审计项数 (A_v0/A_v/B_v0/B_v/sens) | 75,446 / 75,561 / 75,671 / 75,561 / 75,576 | **75,451
      / 75,831 / 75,686 / 75,566 / 75,576** |
444 | 滚动 v 年末储能 (Q4-2 B) | 10,068.78 kWh | **9,501.53 kWh** |
445 | 触底天数 (v=0 → 滚动) | 56 → 0 | **58 → 1** |
446 | **部署 - 固定组合 6 (Q4-3)** | +4,676.66 | ** +3,143.81** (未舍入 3,143.80) |
447 | Q4-3 组合修后值 (none/6/12/18/6+12/6+18/12+18/6+12+18/部署) | 16,430,707.21 /
      15,670,846.79 / 16,356,233.30 / 16,500,977.36 / 15,782,816.29 / 15,712,487.02 /
      16,400,748.07 / 15,810,241.80 / 15,675,523.45 | **16,506,164.40 / 15,712,944.80 /
      16,429,010.31 / 16,583,022.61 / 15,825,198.58 / 15,760,850.14 / 16,478,897.70 /
      15,858,374.49 / 15,716,088.61** (总表 $1b) |
448
449 ### 12.3 交付件哈希 (`results/`, 本机现算)
450
451 | 文件 | 修后 SHA256 | 大小 | 修前 (已被取代) |
452 | ---|---|---|---|
453 | `results/result4-2.xlsx` | `1
      BC81EF5059E99AA4C0DDF30F890905A8F2519A2866599FFFD41DCD20AD5C0F8` | 341,535 B | `51676
      B01FB52B359...` |
454 | `results/result4-3.xlsx` | `70719
      B92BF2FD3DEA081E5FBD2EC3B4AA6F22E5D682A628BFCFA3C29E62BB83D` | 636,693 B | `608
      E99270A609D40...` |
455 | `results/四个指定日期_表123_q4B.json` | `01
      C90A853B43F3661ACCBBA4C903CC34627EC408B82B104BFEF93C9C55D39442` | 60,432 B | `
      AF984CC9C922FCAC...` |
456 | `results/指定日期表格.json` | `6
      D4E2A6B67AA4FA861A2FEDFB76C8924213AD2C06402674180E497923D89AF8C` | 14,917 B (16 条舍 `
      内核`) | `5E0110DB2C82303F...` |
457
458 四日汇总 (元): Q4-2 **44,064.7109 / 16,984.4956 / 52,682.4934 / 71,456.3902**;
```

459 Q4-3 部署 **55,521.2593 / 18,727.8857 / 55,509.9190 / 78,827.6601** (日期 2025-03-20 / 06-21 / 09-23 / 12-21)。

460 工作簿 ↔ 四日表逐格校验: 最大绝对差 **0.005000** (容差 0.0051) **PASS**。

461 `sens\_refresh` 可刷新B 一并与 `B\_v` 并列审计 PASS。图件已按修后数据重出 (`figures\_q4B/`, 脚本 `p41c\_figures\_q4.py`,

462 支持 `Q4\_TREE` 环境变量指向新树); 修前图件留档 `备份\_正式交付\_20260912/eps修正前\_图件/`。

463

464 ---

465

466 ## 13. 必须随附的边界

467

468 1. 口径B 的价格预测 MAE 约 **0.083 元/kWh (≈10.9% 均价)**; 当日**水平**几乎不可预测 (与前一日的
相关约 0.48),

形状来自附件1 的结构（相关 0.97）。**不采用**"附件1 即日前公布曲线"这一额外假设，主轨用纯历史统计。

2. 跨日价值 v 逐日滚动、**因果**（只用 $<$ 当天），修掉了原实现的 ``fp[di+1]`` 前视；但与问题2/3 相同， v 是

规则指定而非最优值，且响应为台阶式； v 不进任何账单，年末库存不计入费用。

3. 可刷新B 与严格B 差异**不显著**（**+3.92** 元/日 $[-14.59, +21.71]$ ，CI 含 0），正式口径取**严格B**。

4. 账单一律真实价；决策价与实际价的差额体现在计划购电费的增加。**该形态只成立于 Q4-2**：修后 Q4-3 部署上口径A 的紧急购电量反而高出 **+13.6%**（\$4.3、\$8）。

5. **问题4 的修后结果只在本目录**（`重跑\_Q4\_B\_eps修正/`）；交付件已刷新到 `results/`，修前留档位置见 \$0.3。

6. **跨口径 A/B 差异不得解读为价格信息价值**（\$8）：修后 Q4-3 部署 $B-A = -219,360.61$ 元（ -656.77 元/日）**、none ** $-416,580.82$ 元**、组合 6 ** $-212,938.36$ 元**；Q4-2 的 ** $+112,302.23$ 元**只能作价格预报价值的**下界**。

正式交付口径为 B，其全部数字不受影响；同口径内比较不受影响。

7. **跨问题（Q2/Q3 $\leftrightarrow$ Q4）比较必须注明内核不同**（\$0.3）：Q2/Q3 为修前内核 `0a2cef2e2b4428bb...`，Q4 为修后内核 `b3242ad925c82d63...`。

8. **PID/PI 为综合信息上界，不等于价格预报价值**（\$7）。

9. **统计区间性质**：bootstrap 区间是对**本回测路径**的探索性时间相关分析，不证明所有未来年份的优势；单年（2025）单路径。

修订记录：2026-09-12: fc3 缺陷修复后 Q4-3 家族数字全量更正；新增跨口径使用限制（见 `重跑\_Q4\_B\_20260912/结论\_Q4\_补做.md` \$5）。

修订记录：2026-09-12 晚（第二批）：采纳 eps 归一化修正后全量重跑；数字按 `修后数字总表\_eps修正.md` 更新。

C24. 04_ 结果/问题 4_ 口径 B 正式/四个指定日期 _ 表 123_q4B.md（一行）

```
1 # 问题4（口径B）四个指定日期表1/表2/表3
2
3 > 口径: q4_scenarioB_v1（口径B: 0:00 不知当日电价；决策用预测价、结算用真实价；滚动跨日价值）
4 > 指定日期 = 2025.3.20、2025.6.21、2025.9.23、2025.12.21；账单一律按当日**真实**附件4 电价结算。
5
6 ## q42_2025-03-20（问题4-2 B（两阶段随机，价格未知），阶段 calibration）
7
8 - 当日真实电价：均值 0.7786，范围 0.2794 - 1.3228 元/kWh；价格预测 MAE 0.0568；滚动 v_term
  0.45226
9 - **全天**：计划购电量 65,350.5834 kWh / 40,610.6561 元；紧急购电量 1,029.3311 kWh /
  3,454.0547 元；**总费用 44,064.7109 元**
10 - 储电量: 0:00 10,303.7363 kWh  $\rightarrow$  24:00 9,879.9473 kWh
11
12 ### 表1 计划购电量（kWh）
13
14 | 时段 | 购电量 | 时段 | 购电量 |
15 | --- | --- | --- | --- |
16 | 0:00-0:10 | 582.2197 | 0:10-0:20 | 586.4570 |
17 | 0:20-0:30 | 581.9653 | 0:30-0:40 | 566.2932 |
```

18	0:40-0:50	592.7313	0:50-1:00	769.6006	
19	1:00-1:10	577.7413	1:10-1:20	561.7491	
20	1:20-1:30	576.7137	1:30-1:40	568.7713	
21	1:40-1:50	585.1578	1:50-2:00	579.3796	
22	2:00-2:10	629.4156	2:10-2:20	594.3311	
23	2:20-2:30	565.5146	2:30-2:40	597.6311	
24	2:40-2:50	621.1664	2:50-3:00	605.9821	
25	3:00-3:10	587.7834	3:10-3:20	593.7213	
26	3:20-3:30	605.9507	3:30-3:40	605.4271	
27	3:40-3:50	601.4279	3:50-4:00	599.5529	
28	4:00-4:10	595.8981	4:10-4:20	601.8763	
29	4:20-4:30	596.7047	4:30-4:40	628.9359	
30	4:40-4:50	613.3122	4:50-5:00	605.2659	
31	5:00-5:10	612.9952	5:10-5:20	622.9766	
32	5:20-5:30	598.7920	5:30-5:40	669.5304	
33	5:40-5:50	640.2655	5:50-6:00	0.0000	
34	6:00-6:10	0.0000	6:10-6:20	0.0000	
35	6:20-6:30	0.0000	6:30-6:40	0.0000	
36	6:40-6:50	0.0000	6:50-7:00	56.8245	
37	7:00-7:10	437.9527	7:10-7:20	438.0215	
38	7:20-7:30	385.3598	7:30-7:40	457.2308	
39	7:40-7:50	0.0000	7:50-8:00	0.0000	
40	8:00-8:10	0.0000	8:10-8:20	0.0000	
41	8:20-8:30	0.0000	8:30-8:40	0.0000	
42	8:40-8:50	0.0000	8:50-9:00	0.0000	
43	9:00-9:10	0.0000	9:10-9:20	0.0000	
44	9:20-9:30	0.0000	9:30-9:40	0.0000	
45	9:40-9:50	0.0000	9:50-10:00	0.0000	
46	10:00-10:10	0.0000	10:10-10:20	0.0000	
47	10:20-10:30	0.0000	10:30-10:40	0.0000	
48	10:40-10:50	0.0000	10:50-11:00	585.4892	
49	11:00-11:10	717.9761	11:10-11:20	0.0000	
50	11:20-11:30	510.0460	11:30-11:40	650.0810	
51	11:40-11:50	667.2061	11:50-12:00	605.0574	
52	12:00-12:10	601.7047	12:10-12:20	591.8338	
53	12:20-12:30	566.1449	12:30-12:40	612.6061	
54	12:40-12:50	0.0000	12:50-13:00	0.0000	
55	13:00-13:10	0.0000	13:10-13:20	0.0000	
56	13:20-13:30	0.0000	13:30-13:40	0.0000	
57	13:40-13:50	0.0000	13:50-14:00	0.0000	
58	14:00-14:10	0.0000	14:10-14:20	0.0000	
59	14:20-14:30	0.0000	14:30-14:40	0.0000	
60	14:40-14:50	0.0000	14:50-15:00	224.7346	
61	15:00-15:10	696.2370	15:10-15:20	254.6876	
62	15:20-15:30	275.6147	15:30-15:40	365.2342	
63	15:40-15:50	379.0195	15:50-16:00	421.7556	
64	16:00-16:10	490.7175	16:10-16:20	558.4173	
65	16:20-16:30	594.9742	16:30-16:40	650.2537	
66	16:40-16:50	709.5124	16:50-17:00	771.5558	
67	17:00-17:10	829.4438	17:10-17:20	871.6245	
68	17:20-17:30	929.6702	17:30-17:40	998.7345	

```

69 | 17:40-17:50 | 902.5083 | 17:50-18:00 | 0.0000 |
70 | 18:00-18:10 | 0.0000 | 18:10-18:20 | 696.8333 |
71 | 18:20-18:30 | 0.0000 | 18:30-18:40 | 0.0000 |
72 | 18:40-18:50 | 0.0000 | 18:50-19:00 | 0.0000 |
73 | 19:00-19:10 | 700.1832 | 19:10-19:20 | 698.1744 |
74 | 19:20-19:30 | 277.9315 | 19:30-19:40 | 0.0000 |
75 | 19:40-19:50 | 0.0000 | 19:50-20:00 | 0.0000 |
76 | 20:00-20:10 | 684.5515 | 20:10-20:20 | 700.9930 |
77 | 20:20-20:30 | 0.0000 | 20:30-20:40 | 0.0000 |
78 | 20:40-20:50 | 699.1815 | 20:50-21:00 | 724.9744 |
79 | 21:00-21:10 | 755.0983 | 21:10-21:20 | 758.1496 |
80 | 21:20-21:30 | 741.5229 | 21:30-21:40 | 752.6859 |
81 | 21:40-21:50 | 679.0276 | 21:50-22:00 | 1,337.2231 |
82 | 22:00-22:10 | 1,325.0180 | 22:10-22:20 | 1,355.9397 |
83 | 22:20-22:30 | 1,346.9690 | 22:30-22:40 | 1,335.9001 |
84 | 22:40-22:50 | 1,353.2664 | 22:50-23:00 | 1,342.2727 |
85 | 23:00-23:10 | 1,356.5556 | 23:10-23:20 | 1,372.9950 |
86 | 23:20-23:30 | 1,311.0383 | 23:30-23:40 | 1,329.2371 |
87 | 23:40-23:50 | 1,345.7058 | 23:50-24:00 | 437.6886 |
88
89 ### 表2 充放电量（六段）与储电量
90
91 | 时段 | 充电量 | 放电量 |
92 | ---|---|---|
93 | 0:00-4:00 | 376.5965 | 11.5966 |
94 | 4:00-8:00 | 418.2276 | 5,552.4356 |
95 | 8:00-12:00 | 5,558.6870 | 2,777.3772 |
96 | 12:00-16:00 | 4,802.5876 | 318.5877 |
97 | 16:00-20:00 | 80.0953 | 7,090.2588 |
98 | 20:00-24:00 | 9,661.1914 | 1,558.0363 |
99 | 0:00 储电量 | 10,303.7363 | 24:00 储电量 | 9,879.9473 |
100
101 ### 表3 紧急购电量（区间）
102
103 | 时段 | 紧急购电量 |
104 | ---|---|
105 | 0:20-0:40 | 45.0493 |
106 | 1:00-2:00 | 134.7156 |
107 | 3:00-4:00 | 75.7575 |
108 | 4:40-5:10 | 30.1949 |
109 | 5:20-5:40 | 31.3685 |
110 | 10:50-11:40 | 246.8311 |
111 | 12:30-12:50 | 65.0345 |
112 | 15:20-17:20 | 373.9894 |
113 | 20:10-20:20 | 26.3903 |
114
115 ## q42_2025-06-21（问题4-2 B（两阶段随机，价格未知），阶段 eval）
116
117 - 当日真实电价：均值 0.6095，范围 0.1828 - 1.2633 元/kWh；价格预测 MAE 0.1559；滚动 v_term
    0.488371
118 - **全天**：计划购电量 33,490.8237 kWh / 16,719.2780 元；紧急购电量 144.4817 kWh / 265.2176

```

元; **总费用 16,984.4956 元**

- 储电量: 0:00 2,517.7157 kWh → 24:00 3,150.1401 kWh

表1 计划购电量 (kWh)

时段	购电量	时段	购电量
0:00-0:10	498.7123	0:10-0:20	528.9011
0:20-0:30	519.6303	0:30-0:40	514.2310
0:40-0:50	521.3937	0:50-1:00	528.0365
1:00-1:10	500.4478	1:10-1:20	504.5354
1:20-1:30	513.2853	1:30-1:40	490.2387
1:40-1:50	469.3816	1:50-2:00	468.9144
2:00-2:10	480.5562	2:10-2:20	492.7320
2:20-2:30	471.3202	2:30-2:40	494.9588
2:40-2:50	476.7121	2:50-3:00	476.3213
3:00-3:10	496.5093	3:10-3:20	486.5087
3:20-3:30	455.0573	3:30-3:40	486.7547
3:40-3:50	474.6427	3:50-4:00	429.1665
4:00-4:10	440.9626	4:10-4:20	459.5486
4:20-4:30	454.0426	4:30-4:40	455.1832
4:40-4:50	437.7767	4:50-5:00	465.0176
5:00-5:10	447.7409	5:10-5:20	411.5639
5:20-5:30	421.2592	5:30-5:40	1,181.3699
5:40-5:50	0.0000	5:50-6:00	0.0000
6:00-6:10	0.0000	6:10-6:20	0.0000
6:20-6:30	0.0000	6:30-6:40	0.0000
6:40-6:50	0.0000	6:50-7:00	0.0000
7:00-7:10	0.0000	7:10-7:20	0.0000
7:20-7:30	0.0000	7:30-7:40	0.0000
7:40-7:50	0.0000	7:50-8:00	0.0000
8:00-8:10	0.0000	8:10-8:20	0.0000
8:20-8:30	0.0000	8:30-8:40	0.0000
8:40-8:50	0.0000	8:50-9:00	0.0000
9:00-9:10	0.0000	9:10-9:20	0.0000
9:20-9:30	0.0000	9:30-9:40	0.0000
9:40-9:50	0.0000	9:50-10:00	0.0000
10:00-10:10	0.0000	10:10-10:20	0.0000
10:20-10:30	0.0000	10:30-10:40	0.0000
10:40-10:50	0.0000	10:50-11:00	0.0000
11:00-11:10	0.0000	11:10-11:20	0.0000
11:20-11:30	0.0000	11:30-11:40	0.0000
11:40-11:50	0.0000	11:50-12:00	0.0000
12:00-12:10	0.0000	12:10-12:20	0.0000
12:20-12:30	0.0000	12:30-12:40	0.0000
12:40-12:50	0.0000	12:50-13:00	0.0000
13:00-13:10	0.0000	13:10-13:20	0.0000
13:20-13:30	0.0000	13:30-13:40	0.0000
13:40-13:50	0.0000	13:50-14:00	0.0000
14:00-14:10	0.0000	14:10-14:20	0.0000
14:20-14:30	0.0000	14:30-14:40	0.0000

169		14:40-14:50		0.0000		14:50-15:00		0.0000	
170		15:00-15:10		0.0000		15:10-15:20		0.0000	
171		15:20-15:30		0.0000		15:30-15:40		0.0000	
172		15:40-15:50		0.0000		15:50-16:00		32.3996	
173		16:00-16:10		90.1227		16:10-16:20		122.4124	
174		16:20-16:30		155.1327		16:30-16:40		228.2776	
175		16:40-16:50		254.3986		16:50-17:00		224.0570	
176		17:00-17:10		236.7421		17:10-17:20		281.2251	
177		17:20-17:30		306.6811		17:30-17:40		333.9399	
178		17:40-17:50		353.4617		17:50-18:00		397.2128	
179		18:00-18:10		415.8619		18:10-18:20		479.4704	
180		18:20-18:30		72.2073		18:30-18:40		0.0000	
181		18:40-18:50		0.0000		18:50-19:00		0.0000	
182		19:00-19:10		0.0000		19:10-19:20		0.0000	
183		19:20-19:30		0.0000		19:30-19:40		0.0000	
184		19:40-19:50		0.0000		19:50-20:00		0.0000	
185		20:00-20:10		0.0000		20:10-20:20		0.0000	
186		20:20-20:30		0.0000		20:30-20:40		0.0000	
187		20:40-20:50		672.8231		20:50-21:00		669.8516	
188		21:00-21:10		613.7829		21:10-21:20		600.1835	
189		21:20-21:30		597.0610		21:30-21:40		610.3926	
190		21:40-21:50		569.4429		21:50-22:00		551.1881	
191		22:00-22:10		1,288.0870		22:10-22:20		554.0321	
192		22:20-22:30		565.1836		22:30-22:40		614.0462	
193		22:40-22:50		622.4872		22:50-23:00		583.1994	
194		23:00-23:10		532.8616		23:10-23:20		541.3256	
195		23:20-23:30		548.3673		23:30-23:40		549.3434	
196		23:40-23:50		610.5134		23:50-24:00		659.6352	
197									
198		### 表2 充放电量（六段）与储电量							
199									
200		时段		充电量		放电量			
201		---		---		---			
202		0:00-4:00		57.9071		229.6933			
203		4:00-8:00		898.3801		1,708.9328			
204		8:00-12:00		3,764.0580		0.0000			
205		12:00-16:00		6,875.5585		0.0000			
206		16:00-20:00		59.7401		5,743.3545			
207		20:00-24:00		1,646.9469		2,523.9358			
208		0:00 储电量		2,517.7157		24:00 储电量		3,150.1401	
209									
210		### 表3 紧急购电量（区间）							
211									
212		时段		紧急购电量					
213		---		---					
214		2:50-3:00		6.5247					
215		3:20-3:30		21.5998					
216		3:50-4:10		81.1845					
217		5:30-5:50		35.1726					
218									
219		## q42_2025-09-23（问题4-2 B（两阶段随机，价格未知），阶段 eval）							

```

220
221 - 当日真实电价：均值 0.7818，范围 0.3271 - 1.3866 元/kWh；价格预测 MAE 0.0624；滚动 v_term
    0.456443
222 - **全天**：计划购电量 65,786.4877 kWh / 42,726.8113 元；紧急购电量 2,860.3234 kWh /
    9,955.6821 元；**总费用 52,682.4934 元**
223 - 储电量：0:00 3,630.1884 kWh → 24:00 5,314.8333 kWh
224
225 ### 表1 计划购电量 (kWh)
226
227 | 时段 | 购电量 | 时段 | 购电量 |
228 |---|---|---|---|
229 | 0:00-0:10 | 626.8977 | 0:10-0:20 | 613.6053 |
230 | 0:20-0:30 | 628.3686 | 0:30-0:40 | 603.6035 |
231 | 0:40-0:50 | 612.8777 | 0:50-1:00 | 616.8915 |
232 | 1:00-1:10 | 610.3532 | 1:10-1:20 | 626.1225 |
233 | 1:20-1:30 | 1,410.2342 | 1:30-1:40 | 1,425.4964 |
234 | 1:40-1:50 | 1,423.4664 | 1:50-2:00 | 1,405.1099 |
235 | 2:00-2:10 | 1,417.7395 | 2:10-2:20 | 1,412.6322 |
236 | 2:20-2:30 | 643.7894 | 2:30-2:40 | 620.5039 |
237 | 2:40-2:50 | 641.2968 | 2:50-3:00 | 622.3169 |
238 | 3:00-3:10 | 618.8645 | 3:10-3:20 | 614.0987 |
239 | 3:20-3:30 | 624.9325 | 3:30-3:40 | 597.0729 |
240 | 3:40-3:50 | 597.6842 | 3:50-4:00 | 638.4182 |
241 | 4:00-4:10 | 626.1526 | 4:10-4:20 | 631.6539 |
242 | 4:20-4:30 | 1,463.2740 | 4:30-4:40 | 1,413.2222 |
243 | 4:40-4:50 | 818.8385 | 4:50-5:00 | 630.0244 |
244 | 5:00-5:10 | 594.9995 | 5:10-5:20 | 599.1121 |
245 | 5:20-5:30 | 605.7952 | 5:30-5:40 | 1,392.6877 |
246 | 5:40-5:50 | 621.5759 | 5:50-6:00 | 0.0000 |
247 | 6:00-6:10 | 0.0000 | 6:10-6:20 | 0.0000 |
248 | 6:20-6:30 | 0.0000 | 6:30-6:40 | 0.0000 |
249 | 6:40-6:50 | 0.0000 | 6:50-7:00 | 0.0000 |
250 | 7:00-7:10 | 0.0000 | 7:10-7:20 | 343.1510 |
251 | 7:20-7:30 | 366.9208 | 7:30-7:40 | 316.0683 |
252 | 7:40-7:50 | 0.0000 | 7:50-8:00 | 0.0000 |
253 | 8:00-8:10 | 0.0000 | 8:10-8:20 | 0.0000 |
254 | 8:20-8:30 | 0.0000 | 8:30-8:40 | 0.0000 |
255 | 8:40-8:50 | 0.0000 | 8:50-9:00 | 0.0000 |
256 | 9:00-9:10 | 0.0000 | 9:10-9:20 | 0.0000 |
257 | 9:20-9:30 | 0.0000 | 9:30-9:40 | 0.0000 |
258 | 9:40-9:50 | 0.0000 | 9:50-10:00 | 0.0000 |
259 | 10:00-10:10 | 0.0000 | 10:10-10:20 | 0.0000 |
260 | 10:20-10:30 | 0.0000 | 10:30-10:40 | 0.0000 |
261 | 10:40-10:50 | 0.0000 | 10:50-11:00 | 0.0000 |
262 | 11:00-11:10 | 508.0720 | 11:10-11:20 | 0.0000 |
263 | 11:20-11:30 | 0.0000 | 11:30-11:40 | 0.0000 |
264 | 11:40-11:50 | 30.0798 | 11:50-12:00 | 498.2617 |
265 | 12:00-12:10 | 263.8788 | 12:10-12:20 | 247.3876 |
266 | 12:20-12:30 | 418.3643 | 12:30-12:40 | 492.4853 |
267 | 12:40-12:50 | 0.0000 | 12:50-13:00 | 0.0000 |
268 | 13:00-13:10 | 0.0000 | 13:10-13:20 | 0.0000 |

```

269		13:20-13:30		0.0000		13:30-13:40		0.0000	
270		13:40-13:50		0.0000		13:50-14:00		0.0000	
271		14:00-14:10		0.0000		14:10-14:20		0.0000	
272		14:20-14:30		0.0000		14:30-14:40		0.0000	
273		14:40-14:50		0.0000		14:50-15:00		91.2627	
274		15:00-15:10		136.1004		15:10-15:20		184.2923	
275		15:20-15:30		255.9789		15:30-15:40		348.2473	
276		15:40-15:50		413.8186		15:50-16:00		465.6435	
277		16:00-16:10		534.6904		16:10-16:20		517.7513	
278		16:20-16:30		654.8930		16:30-16:40		646.5811	
279		16:40-16:50		702.7607		16:50-17:00		781.8473	
280		17:00-17:10		827.6527		17:10-17:20		869.9496	
281		17:20-17:30		930.9008		17:30-17:40		1,009.8386	
282		17:40-17:50		927.5520		17:50-18:00		0.7775	
283		18:00-18:10		0.0000		18:10-18:20		3.7057	
284		18:20-18:30		770.3608		18:30-18:40		764.6498	
285		18:40-18:50		40.5740		18:50-19:00		0.0000	
286		19:00-19:10		0.0000		19:10-19:20		0.0000	
287		19:20-19:30		728.4201		19:30-19:40		525.6070	
288		19:40-19:50		765.5505		19:50-20:00		724.1415	
289		20:00-20:10		2.5871		20:10-20:20		1.1296	
290		20:20-20:30		786.0461		20:30-20:40		6.3682	
291		20:40-20:50		779.1895		20:50-21:00		807.8891	
292		21:00-21:10		853.0750		21:10-21:20		841.8359	
293		21:20-21:30		804.8068		21:30-21:40		810.5400	
294		21:40-21:50		790.0441		21:50-22:00		690.4543	
295		22:00-22:10		1,381.4456		22:10-22:20		619.8324	
296		22:20-22:30		635.3239		22:30-22:40		1,406.5557	
297		22:40-22:50		636.1343		22:50-23:00		650.1514	
298		23:00-23:10		666.6439		23:10-23:20		1,406.5750	
299		23:20-23:30		642.1951		23:30-23:40		1,397.4267	
300		23:40-23:50		1,373.2094		23:50-24:00		639.0989	
301									
302		### 表2 充放电量（六段）与储电量							
303									
304		时段		充电量		放电量			
305		--- --- ---							
306		0:00-4:00		5,368.4665		5.0692			
307		4:00-8:00		2,676.0217		6,269.2887			
308		8:00-12:00		4,461.7097		1,896.6157			
309		12:00-16:00		5,988.0536		403.6249			
310		16:00-20:00		0.0000		6,167.8719			
311		20:00-24:00		4,621.1564		2,464.8297			
312		0:00 储电量		3,630.1884		24:00 储电量		5,314.8333	
313									
314		### 表3 紧急购电量（区间）							
315									
316		时段		紧急购电量					
317		--- ---							
318		5:00-5:10		17.1483					
319		11:00-11:10		87.3196					

```

320 | 11:20-11:30 | 416.5953 |
321 | 11:40-11:50 | 455.9826 |
322 | 12:00-12:20 | 234.3575 |
323 | 14:50-15:30 | 981.4926 |
324 | 15:50-17:40 | 509.4522 |
325 | 20:10-20:30 | 96.4077 |
326 | 20:40-21:00 | 61.5677 |
327
328 ## q42_2025-12-21 (问题4-2 B (两阶段随机, 价格未知), 阶段 eval)
329
330 - 当日真实电价: 均值 0.9101, 范围 0.3905 - 1.5659 元/kWh; 价格预测 MAE 0.0543; 滚动 v_term
    0.490271
331 - **全天**: 计划购电量 92,693.6521 kWh / 71,456.3902 元; 紧急购电量 0.0000 kWh / 0.0000 元; **
    总费用 71,456.3902 元**
332 - 储电量: 0:00 10,568.5722 kWh → 24:00 8,930.9939 kWh
333
334 ### 表1 计划购电量 (kWh)
335
336 | 时段 | 购电量 | 时段 | 购电量 |
337 | --- | --- | --- | --- |
338 | 0:00-0:10 | 700.7656 | 0:10-0:20 | 600.4566 |
339 | 0:20-0:30 | 629.5252 | 0:30-0:40 | 615.5048 |
340 | 0:40-0:50 | 614.1259 | 0:50-1:00 | 600.8090 |
341 | 1:00-1:10 | 633.6402 | 1:10-1:20 | 623.6935 |
342 | 1:20-1:30 | 610.3597 | 1:30-1:40 | 621.7429 |
343 | 1:40-1:50 | 629.8676 | 1:50-2:00 | 618.2261 |
344 | 2:00-2:10 | 624.7421 | 2:10-2:20 | 635.2984 |
345 | 2:20-2:30 | 647.4008 | 2:30-2:40 | 614.4459 |
346 | 2:40-2:50 | 621.7268 | 2:50-3:00 | 648.4450 |
347 | 3:00-3:10 | 610.4154 | 3:10-3:20 | 597.2929 |
348 | 3:20-3:30 | 633.5959 | 3:30-3:40 | 646.0944 |
349 | 3:40-3:50 | 618.0753 | 3:50-4:00 | 629.9548 |
350 | 4:00-4:10 | 625.3730 | 4:10-4:20 | 662.8991 |
351 | 4:20-4:30 | 703.7322 | 4:30-4:40 | 657.4791 |
352 | 4:40-4:50 | 626.9410 | 4:50-5:00 | 642.4862 |
353 | 5:00-5:10 | 651.8525 | 5:10-5:20 | 664.0557 |
354 | 5:20-5:30 | 644.1463 | 5:30-5:40 | 691.5586 |
355 | 5:40-5:50 | 784.3196 | 5:50-6:00 | 883.0963 |
356 | 6:00-6:10 | 885.3851 | 6:10-6:20 | 866.6295 |
357 | 6:20-6:30 | 841.0182 | 6:30-6:40 | 793.5024 |
358 | 6:40-6:50 | 811.5996 | 6:50-7:00 | 892.9920 |
359 | 7:00-7:10 | 877.8405 | 7:10-7:20 | 844.5724 |
360 | 7:20-7:30 | 808.7238 | 7:30-7:40 | 797.9941 |
361 | 7:40-7:50 | 862.6064 | 7:50-8:00 | 240.4488 |
362 | 8:00-8:10 | 247.7434 | 8:10-8:20 | 146.3128 |
363 | 8:20-8:30 | 68.6118 | 8:30-8:40 | 79.8641 |
364 | 8:40-8:50 | 16.4240 | 8:50-9:00 | 0.0000 |
365 | 9:00-9:10 | 0.0000 | 9:10-9:20 | 605.2932 |
366 | 9:20-9:30 | 545.7141 | 9:30-9:40 | 474.5137 |
367 | 9:40-9:50 | 330.2571 | 9:50-10:00 | 0.0000 |
368 | 10:00-10:10 | 0.0000 | 10:10-10:20 | 0.0000 |

```

369		10:20-10:30		0.0000		10:30-10:40		0.0000	
370		10:40-10:50		199.9397		10:50-11:00		897.0647	
371		11:00-11:10		878.7871		11:10-11:20		944.6243	
372		11:20-11:30		1,002.2677		11:30-11:40		991.6898	
373		11:40-11:50		955.7424		11:50-12:00		963.9228	
374		12:00-12:10		1,035.6285		12:10-12:20		1,023.6496	
375		12:20-12:30		973.6174		12:30-12:40		987.1778	
376		12:40-12:50		952.0956		12:50-13:00		98.4255	
377		13:00-13:10		199.4720		13:10-13:20		252.9312	
378		13:20-13:30		244.2739		13:30-13:40		270.2533	
379		13:40-13:50		0.0000		13:50-14:00		0.0000	
380		14:00-14:10		0.0000		14:10-14:20		0.0000	
381		14:20-14:30		0.0000		14:30-14:40		0.0000	
382		14:40-14:50		0.0000		14:50-15:00		1,358.1327	
383		15:00-15:10		1,332.2822		15:10-15:20		1,402.3674	
384		15:20-15:30		1,472.4067		15:30-15:40		1,007.1892	
385		15:40-15:50		840.1156		15:50-16:00		889.2416	
386		16:00-16:10		831.4297		16:10-16:20		881.8913	
387		16:20-16:30		970.0144		16:30-16:40		1,027.3303	
388		16:40-16:50		1,024.9598		16:50-17:00		1,033.4085	
389		17:00-17:10		1,091.8951		17:10-17:20		1,072.4775	
390		17:20-17:30		1,025.9514		17:30-17:40		1,090.2823	
391		17:40-17:50		942.8188		17:50-18:00		743.3171	
392		18:00-18:10		722.9543		18:10-18:20		741.4683	
393		18:20-18:30		722.6122		18:30-18:40		716.6420	
394		18:40-18:50		732.7591		18:50-19:00		0.0000	
395		19:00-19:10		0.0000		19:10-19:20		4.1902	
396		19:20-19:30		26.7610		19:30-19:40		0.0000	
397		19:40-19:50		0.0000		19:50-20:00		351.9772	
398		20:00-20:10		3.1349		20:10-20:20		4.7850	
399		20:20-20:30		0.0000		20:30-20:40		0.0000	
400		20:40-20:50		656.4394		20:50-21:00		704.4154	
401		21:00-21:10		726.3550		21:10-21:20		716.0444	
402		21:20-21:30		686.8586		21:30-21:40		679.3406	
403		21:40-21:50		720.5130		21:50-22:00		1,364.0958	
404		22:00-22:10		1,336.3186		22:10-22:20		1,267.5683	
405		22:20-22:30		1,373.7806		22:30-22:40		1,383.1355	
406		22:40-22:50		1,383.3805		22:50-23:00		641.6293	
407		23:00-23:10		1,353.0919		23:10-23:20		1,405.3014	
408		23:20-23:30		1,373.2347		23:30-23:40		675.1772	
409		23:40-23:50		686.9595		23:50-24:00		1,321.4615	

410

411 ### 表2 充放电量（六段）与储电量

412

413 | 时段 | 充电量 | 放电量 |

414 | --- | --- | --- |

415 | 0:00-4:00 | 0.0000 | 507.9893 |

416 | 4:00-8:00 | 237.9995 | 1,347.8690 |

417 | 8:00-12:00 | 4,793.7529 | 6,580.8037 |

418 | 12:00-16:00 | 7,864.2650 | 2,681.1489 |

419 | 16:00-20:00 | 462.6050 | 4,868.6610 |

```

420 | 20:00-24:00 | 8,268.9817 | 3,005.7077 |
421 | 0:00 储电量 | 10,568.5722 | 24:00 储电量 | 8,930.9939 |
422
423 ### 表3 紧急购电量 (区间)
424
425 (当日无紧急购电)
426
427 ## q43_2025-03-20 (问题4-3 历史选型部署, 阶段 calibration)
428
429 - 当日真实电价: 均值 0.7786, 范围 0.2794 - 1.3228 元/kWh; 价格预测 MAE 0.0568; 滚动 v_term
    0.45226
430 - **全天**: 计划购电量 69,381.1068 kWh / 43,578.2875 元; 紧急购电量 3,290.2318 kWh /
    12,526.8283 元; 调整费 -583.8565 元; **总费用 55,521.2593 元**
431 - 储电量: 0:00 10,323.7499 kWh → 24:00 10,013.7782 kWh
432
433 ### 表1 计划购电量 (kWh)
434
435 | 时段 | 购电量 | 时段 | 购电量 |
436 |---|---|---|---|
437 | 0:00-0:10 | 571.7535 | 0:10-0:20 | 545.7343 |
438 | 0:20-0:30 | 505.8406 | 0:30-0:40 | 544.5919 |
439 | 0:40-0:50 | 584.5824 | 0:50-1:00 | 1,069.1240 |
440 | 1:00-1:10 | 555.6333 | 1:10-1:20 | 561.7491 |
441 | 1:20-1:30 | 586.6464 | 1:30-1:40 | 567.4314 |
442 | 1:40-1:50 | 589.8514 | 1:50-2:00 | 556.8527 |
443 | 2:00-2:10 | 577.0164 | 2:10-2:20 | 594.3311 |
444 | 2:20-2:30 | 565.5146 | 2:30-2:40 | 597.6311 |
445 | 2:40-2:50 | 611.3898 | 2:50-3:00 | 592.0445 |
446 | 3:00-3:10 | 611.3583 | 3:10-3:20 | 597.3948 |
447 | 3:20-3:30 | 605.9507 | 3:30-3:40 | 589.0655 |
448 | 3:40-3:50 | 570.8720 | 3:50-4:00 | 564.5537 |
449 | 4:00-4:10 | 595.8982 | 4:10-4:20 | 607.2237 |
450 | 4:20-4:30 | 596.7047 | 4:30-4:40 | 609.1410 |
451 | 4:40-4:50 | 593.0014 | 4:50-5:00 | 582.5527 |
452 | 5:00-5:10 | 582.8053 | 5:10-5:20 | 584.9074 |
453 | 5:20-5:30 | 598.7920 | 5:30-5:40 | 612.5097 |
454 | 5:40-5:50 | 681.2707 | 5:50-6:00 | 0.0000 |
455 | 6:00-6:10 | 0.0000 | 6:10-6:20 | 0.0000 |
456 | 6:20-6:30 | 0.0000 | 6:30-6:40 | 0.0000 |
457 | 6:40-6:50 | 0.0000 | 6:50-7:00 | 0.0000 |
458 | 7:00-7:10 | 0.0000 | 7:10-7:20 | 0.0000 |
459 | 7:20-7:30 | 46.0448 | 7:30-7:40 | 249.4588 |
460 | 7:40-7:50 | 0.0000 | 7:50-8:00 | 0.0000 |
461 | 8:00-8:10 | 0.0000 | 8:10-8:20 | 0.0000 |
462 | 8:20-8:30 | 0.0000 | 8:30-8:40 | 0.0000 |
463 | 8:40-8:50 | 0.0000 | 8:50-9:00 | 0.0000 |
464 | 9:00-9:10 | 0.0000 | 9:10-9:20 | 0.0000 |
465 | 9:20-9:30 | 0.0000 | 9:30-9:40 | 0.0000 |
466 | 9:40-9:50 | 0.0000 | 9:50-10:00 | 0.0000 |
467 | 10:00-10:10 | 0.0000 | 10:10-10:20 | 0.0000 |
468 | 10:20-10:30 | 0.0000 | 10:30-10:40 | 0.0000 |

```

469		10:40-10:50		0.0000		10:50-11:00		553.3934	
470		11:00-11:10		687.1508		11:10-11:20		276.2569	
471		11:20-11:30		604.1850		11:30-11:40		586.1483	
472		11:40-11:50		582.4539		11:50-12:00		577.6090	
473		12:00-12:10		882.6947		12:10-12:20		876.0747	
474		12:20-12:30		838.3747		12:30-12:40		824.8182	
475		12:40-12:50		193.2622		12:50-13:00		47.1688	
476		13:00-13:10		0.0000		13:10-13:20		0.0000	
477		13:20-13:30		0.0000		13:30-13:40		0.0000	
478		13:40-13:50		0.0000		13:50-14:00		0.0000	
479		14:00-14:10		0.0000		14:10-14:20		0.0000	
480		14:20-14:30		0.0000		14:30-14:40		0.0000	
481		14:40-14:50		0.0000		14:50-15:00		1,271.6670	
482		15:00-15:10		1,204.1064		15:10-15:20		1,150.7881	
483		15:20-15:30		543.8786		15:30-15:40		560.0786	
484		15:40-15:50		615.7739		15:50-16:00		644.2706	
485		16:00-16:10		546.4174		16:10-16:20		617.4980	
486		16:20-16:30		663.4749		16:30-16:40		712.5917	
487		16:40-16:50		758.4376		16:50-17:00		827.0029	
488		17:00-17:10		1,017.2845		17:10-17:20		1,012.3920	
489		17:20-17:30		1,024.5283		17:30-17:40		1,048.8114	
490		17:40-17:50		919.5787		17:50-18:00		0.0000	
491		18:00-18:10		0.0000		18:10-18:20		697.3564	
492		18:20-18:30		0.0000		18:30-18:40		0.0000	
493		18:40-18:50		0.0000		18:50-19:00		0.0000	
494		19:00-19:10		700.1832		19:10-19:20		698.1771	
495		19:20-19:30		530.4143		19:30-19:40		0.0000	
496		19:40-19:50		0.0000		19:50-20:00		0.0000	
497		20:00-20:10		684.5515		20:10-20:20		700.9930	
498		20:20-20:30		0.0000		20:30-20:40		0.0000	
499		20:40-20:50		708.8444		20:50-21:00		719.3315	
500		21:00-21:10		712.8766		21:10-21:20		714.6659	
501		21:20-21:30		723.8942		21:30-21:40		726.9080	
502		21:40-21:50		679.0276		21:50-22:00		1,337.2231	
503		22:00-22:10		1,325.0180		22:10-22:20		1,355.9397	
504		22:20-22:30		1,346.9690		22:30-22:40		1,335.9001	
505		22:40-22:50		1,261.9451		22:50-23:00		1,342.2727	
506		23:00-23:10		1,356.5556		23:10-23:20		1,372.9950	
507		23:20-23:30		1,311.0383		23:30-23:40		1,329.2371	
508		23:40-23:50		1,345.7058		23:50-24:00		437.6886	

509

510 ### 表2 充放电量（六段）与储电量

511

512		时段		充电量		放电量	
513		---		---		---	
514		0:00-4:00		0.0000		481.6890	
515		4:00-8:00		0.0000		6,066.6788	
516		8:00-12:00		5,101.6009		1,663.0071	
517		12:00-16:00		5,879.5383		254.7228	
518		16:00-20:00		100.5025		6,843.8880	
519		20:00-24:00		9,502.4745		1,642.1230	

```

520 | 0:00 储电量 | 10,323.7499 | 24:00 储电量 | 10,013.7782 |
521
522 ### 表3 紧急购电量 (区间)
523
524 | 时段 | 紧急购电量 |
525 |---|---|
526 | 8:30-10:00 | 1,114.3701 |
527 | 10:50-12:00 | 1,012.4614 |
528 | 12:20-12:50 | 391.1899 |
529 | 15:00-15:10 | 604.8589 |
530 | 15:30-16:10 | 167.3516 |
531
532 ## q43_2025-06-21 (问题4-3 历史选型部署, 阶段 eval)
533
534 - 当日真实电价: 均值 0.6095, 范围 0.1828 - 1.2633 元/kWh; 价格预测 MAE 0.1559; 滚动 v_term
    0.488371
535 - **全天**: 计划购电量 33,492.3944 kWh / 17,320.5056 元; 紧急购电量 260.5891 kWh / 751.2813
    元; 调整费 656.0987 元; **总费用 18,727.8857 元**
536 - 储电量: 0:00 2,962.0759 kWh → 24:00 3,588.8658 kWh
537
538 ### 表1 计划购电量 (kWh)
539
540 | 时段 | 购电量 | 时段 | 购电量 |
541 |---|---|---|---|
542 | 0:00-0:10 | 508.4136 | 0:10-0:20 | 536.9975 |
543 | 0:20-0:30 | 530.2988 | 0:30-0:40 | 516.3155 |
544 | 0:40-0:50 | 515.5759 | 0:50-1:00 | 522.0136 |
545 | 1:00-1:10 | 488.2235 | 1:10-1:20 | 458.2178 |
546 | 1:20-1:30 | 482.1579 | 1:30-1:40 | 495.5574 |
547 | 1:40-1:50 | 469.3816 | 1:50-2:00 | 468.9144 |
548 | 2:00-2:10 | 480.5562 | 2:10-2:20 | 492.1609 |
549 | 2:20-2:30 | 467.9893 | 2:30-2:40 | 460.1723 |
550 | 2:40-2:50 | 453.7179 | 2:50-3:00 | 457.9907 |
551 | 3:00-3:10 | 454.5886 | 3:10-3:20 | 445.0793 |
552 | 3:20-3:30 | 466.5535 | 3:30-3:40 | 462.8138 |
553 | 3:40-3:50 | 451.5651 | 3:50-4:00 | 428.9228 |
554 | 4:00-4:10 | 444.5625 | 4:10-4:20 | 431.3345 |
555 | 4:20-4:30 | 401.8811 | 4:30-4:40 | 444.9442 |
556 | 4:40-4:50 | 487.9379 | 4:50-5:00 | 476.9925 |
557 | 5:00-5:10 | 330.7847 | 5:10-5:20 | 323.3195 |
558 | 5:20-5:30 | 349.3801 | 5:30-5:40 | 1,104.7540 |
559 | 5:40-5:50 | 0.0000 | 5:50-6:00 | 0.0000 |
560 | 6:00-6:10 | 0.0000 | 6:10-6:20 | 0.0000 |
561 | 6:20-6:30 | 0.0000 | 6:30-6:40 | 0.0000 |
562 | 6:40-6:50 | 0.0000 | 6:50-7:00 | 0.0000 |
563 | 7:00-7:10 | 0.0000 | 7:10-7:20 | 0.0000 |
564 | 7:20-7:30 | 0.0000 | 7:30-7:40 | 0.0000 |
565 | 7:40-7:50 | 0.0000 | 7:50-8:00 | 0.0000 |
566 | 8:00-8:10 | 0.0000 | 8:10-8:20 | 0.0000 |
567 | 8:20-8:30 | 0.0000 | 8:30-8:40 | 0.0000 |
568 | 8:40-8:50 | 0.0000 | 8:50-9:00 | 0.0000 |

```

569		9:00-9:10		0.0000		9:10-9:20		0.0000	
570		9:20-9:30		0.0000		9:30-9:40		0.0000	
571		9:40-9:50		0.0000		9:50-10:00		0.0000	
572		10:00-10:10		0.0000		10:10-10:20		0.0000	
573		10:20-10:30		0.0000		10:30-10:40		0.0000	
574		10:40-10:50		0.0000		10:50-11:00		0.0000	
575		11:00-11:10		0.0000		11:10-11:20		0.0000	
576		11:20-11:30		0.0000		11:30-11:40		0.0000	
577		11:40-11:50		0.0000		11:50-12:00		0.0000	
578		12:00-12:10		0.0000		12:10-12:20		0.0000	
579		12:20-12:30		0.0000		12:30-12:40		0.0000	
580		12:40-12:50		0.0000		12:50-13:00		0.0000	
581		13:00-13:10		0.0000		13:10-13:20		0.0000	
582		13:20-13:30		0.0000		13:30-13:40		0.0000	
583		13:40-13:50		0.0000		13:50-14:00		0.0000	
584		14:00-14:10		0.0000		14:10-14:20		0.0000	
585		14:20-14:30		0.0000		14:30-14:40		0.0000	
586		14:40-14:50		0.0000		14:50-15:00		0.0000	
587		15:00-15:10		0.0000		15:10-15:20		0.0000	
588		15:20-15:30		0.0000		15:30-15:40		0.0000	
589		15:40-15:50		0.0000		15:50-16:00		0.0000	
590		16:00-16:10		0.0000		16:10-16:20		5.6588	
591		16:20-16:30		59.5357		16:30-16:40		159.3564	
592		16:40-16:50		199.1291		16:50-17:00		211.6126	
593		17:00-17:10		395.3914		17:10-17:20		366.5188	
594		17:20-17:30		379.3649		17:30-17:40		401.6212	
595		17:40-17:50		392.0361		17:50-18:00		412.7335	
596		18:00-18:10		517.9009		18:10-18:20		574.8658	
597		18:20-18:30		563.6381		18:30-18:40		0.0000	
598		18:40-18:50		0.0000		18:50-19:00		0.0000	
599		19:00-19:10		0.0000		19:10-19:20		0.0000	
600		19:20-19:30		0.0000		19:30-19:40		0.0000	
601		19:40-19:50		0.0000		19:50-20:00		0.0000	
602		20:00-20:10		0.0000		20:10-20:20		0.0000	
603		20:20-20:30		0.0000		20:30-20:40		0.0000	
604		20:40-20:50		682.2530		20:50-21:00		681.2006	
605		21:00-21:10		628.7112		21:10-21:20		584.3485	
606		21:20-21:30		578.6047		21:30-21:40		598.1312	
607		21:40-21:50		569.4429		21:50-22:00		551.1881	
608		22:00-22:10		1,288.0870		22:10-22:20		554.0321	
609		22:20-22:30		565.1836		22:30-22:40		614.0462	
610		22:40-22:50		622.4872		22:50-23:00		583.1994	
611		23:00-23:10		532.8616		23:10-23:20		541.3256	
612		23:20-23:30		548.3673		23:30-23:40		549.3434	
613		23:40-23:50		610.5134		23:50-24:00		659.6352	
614									
615		### 表2 充放电量（六段）与储电量							
616									
617		时段		充电量		放电量			
618		---		---		---			
619		0:00-4:00		0.0000		450.7122			

```

620 | 4:00-8:00 | 109.4497 | 1,223.8103 |
621 | 8:00-12:00 | 2,730.3703 | 0.0000 |
622 | 12:00-16:00 | 7,936.2963 | 0.0000 |
623 | 16:00-20:00 | 164.6405 | 5,418.8495 |
624 | 20:00-24:00 | 1,666.8191 | 2,554.6535 |
625 | 0:00 储电量 | 2,962.0759 | 24:00 储电量 | 3,588.8658 |
626
627 ### 表3 紧急购电量 (区间)
628
629 | 时段 | 紧急购电量 |
630 |---|---|
631 | 5:40-5:50 | 21.4730 |
632 | 6:10-6:30 | 58.8121 |
633 | 6:40-7:00 | 64.1689 |
634 | 15:50-16:10 | 105.5766 |
635 | 16:20-16:30 | 10.5586 |
636
637 ## q43_2025-09-23 (问题4-3 历史选型部署, 阶段 eval)
638
639 - 当日真实电价: 均值 0.7818, 范围 0.3271 - 1.3866 元/kWh; 价格预测 MAE 0.0624; 滚动 v_term
    0.456443
640 - **全天**: 计划购电量 69,961.9846 kWh / 46,580.7693 元; 紧急购电量 3,111.5525 kWh /
    8,413.4998 元; 调整费 515.6500 元; **总费用 55,509.9190 元**
641 - 储电量: 0:00 4,175.4920 kWh → 24:00 5,662.3912 kWh
642
643 ### 表1 计划购电量 (kWh)
644
645 | 时段 | 购电量 | 时段 | 购电量 |
646 |---|---|---|---|
647 | 0:00-0:10 | 620.6593 | 0:10-0:20 | 642.1278 |
648 | 0:20-0:30 | 646.5885 | 0:30-0:40 | 625.1895 |
649 | 0:40-0:50 | 610.1085 | 0:50-1:00 | 618.3404 |
650 | 1:00-1:10 | 619.8328 | 1:10-1:20 | 604.9539 |
651 | 1:20-1:30 | 959.4167 | 1:30-1:40 | 1,418.0206 |
652 | 1:40-1:50 | 1,404.2249 | 1:50-2:00 | 1,409.0233 |
653 | 2:00-2:10 | 1,417.7395 | 2:10-2:20 | 1,412.6322 |
654 | 2:20-2:30 | 600.8645 | 2:30-2:40 | 607.2364 |
655 | 2:40-2:50 | 604.3212 | 2:50-3:00 | 592.8224 |
656 | 3:00-3:10 | 609.4308 | 3:10-3:20 | 604.1043 |
657 | 3:20-3:30 | 631.3055 | 3:30-3:40 | 591.4354 |
658 | 3:40-3:50 | 610.9388 | 3:50-4:00 | 631.6785 |
659 | 4:00-4:10 | 614.3583 | 4:10-4:20 | 627.8575 |
660 | 4:20-4:30 | 1,432.7457 | 4:30-4:40 | 1,413.3835 |
661 | 4:40-4:50 | 619.1803 | 4:50-5:00 | 619.6651 |
662 | 5:00-5:10 | 624.7303 | 5:10-5:20 | 601.3979 |
663 | 5:20-5:30 | 646.9619 | 5:30-5:40 | 1,419.4477 |
664 | 5:40-5:50 | 620.6563 | 5:50-6:00 | 0.0000 |
665 | 6:00-6:10 | 0.0000 | 6:10-6:20 | 0.0000 |
666 | 6:20-6:30 | 0.0000 | 6:30-6:40 | 0.0000 |
667 | 6:40-6:50 | 0.0000 | 6:50-7:00 | 0.0000 |
668 | 7:00-7:10 | 291.1538 | 7:10-7:20 | 269.1852 |

```

669		7:20-7:30		301.3650		7:30-7:40		745.6082	
670		7:40-7:50		0.0000		7:50-8:00		0.0000	
671		8:00-8:10		0.0000		8:10-8:20		0.0000	
672		8:20-8:30		0.0000		8:30-8:40		0.0000	
673		8:40-8:50		0.0000		8:50-9:00		0.0000	
674		9:00-9:10		0.0000		9:10-9:20		0.0000	
675		9:20-9:30		0.0000		9:30-9:40		0.0000	
676		9:40-9:50		0.0000		9:50-10:00		0.0000	
677		10:00-10:10		0.0000		10:10-10:20		0.0000	
678		10:20-10:30		0.0000		10:30-10:40		0.0000	
679		10:40-10:50		0.0000		10:50-11:00		0.0000	
680		11:00-11:10		30.5448		11:10-11:20		0.0000	
681		11:20-11:30		0.0000		11:30-11:40		0.0000	
682		11:40-11:50		0.0000		11:50-12:00		358.3958	
683		12:00-12:10		479.4788		12:10-12:20		411.6512	
684		12:20-12:30		519.8503		12:30-12:40		491.9062	
685		12:40-12:50		0.0000		12:50-13:00		0.0000	
686		13:00-13:10		0.0000		13:10-13:20		0.0000	
687		13:20-13:30		0.0000		13:30-13:40		0.0000	
688		13:40-13:50		0.0000		13:50-14:00		0.0000	
689		14:00-14:10		8.9337		14:10-14:20		38.7967	
690		14:20-14:30		0.0000		14:30-14:40		0.0000	
691		14:40-14:50		150.0775		14:50-15:00		1,045.4293	
692		15:00-15:10		984.7939		15:10-15:20		1,022.1214	
693		15:20-15:30		510.0316		15:30-15:40		552.7528	
694		15:40-15:50		455.5170		15:50-16:00		498.9926	
695		16:00-16:10		633.5383		16:10-16:20		632.7851	
696		16:20-16:30		704.5607		16:30-16:40		754.4183	
697		16:40-16:50		781.2941		16:50-17:00		851.4843	
698		17:00-17:10		946.2010		17:10-17:20		934.5477	
699		17:20-17:30		1,007.1509		17:30-17:40		1,051.4875	
700		17:40-17:50		936.5937		17:50-18:00		64.9519	
701		18:00-18:10		0.0000		18:10-18:20		0.0000	
702		18:20-18:30		713.3317		18:30-18:40		768.5030	
703		18:40-18:50		17.6000		18:50-19:00		0.0000	
704		19:00-19:10		0.0000		19:10-19:20		0.0000	
705		19:20-19:30		728.4201		19:30-19:40		764.0340	
706		19:40-19:50		765.5507		19:50-20:00		780.9620	
707		20:00-20:10		2.5871		20:10-20:20		1.1296	
708		20:20-20:30		790.1156		20:30-20:40		6.3682	
709		20:40-20:50		779.1893		20:50-21:00		808.6854	
710		21:00-21:10		844.3783		21:10-21:20		842.4049	
711		21:20-21:30		790.5189		21:30-21:40		828.4930	
712		21:40-21:50		785.6865		21:50-22:00		690.4543	
713		22:00-22:10		1,381.4456		22:10-22:20		619.8324	
714		22:20-22:30		635.3239		22:30-22:40		1,406.5557	
715		22:40-22:50		636.1343		22:50-23:00		650.1514	
716		23:00-23:10		666.6439		23:10-23:20		1,406.5750	
717		23:20-23:30		642.1951		23:30-23:40		1,397.4267	
718		23:40-23:50		1,373.2094		23:50-24:00		639.0989	
719									

```

720 ### 表2 充放电量（六段）与储电量
721
722 | 时段 | 充电量 | 放电量 |
723 |---|---|---|
724 | 0:00-4:00 | 3,070.7765 | 0.0000 |
725 | 4:00-8:00 | 2,905.5179 | 6,098.1073 |
726 | 8:00-12:00 | 5,581.9907 | 1,424.0229 |
727 | 12:00-16:00 | 5,401.8813 | 253.5474 |
728 | 16:00-20:00 | 0.0000 | 5,837.9227 |
729 | 20:00-24:00 | 4,646.1845 | 2,549.3347 |
730 | 0:00 储电量 | 4,175.4920 | 24:00 储电量 | 5,662.3912 |
731
732 ### 表3 紧急购电量（区间）
733
734 | 时段 | 紧急购电量 |
735 |---|---|
736 | 8:40-9:50 | 472.5929 |
737 | 11:00-12:00 | 2,609.4109 |
738 | 12:10-12:20 | 26.5201 |
739 | 16:10-16:20 | 3.0286 |
740
741 ## q43_2025-12-21（问题4-3 历史选型部署，阶段 eval）
742
743 - 当日真实电价：均值 0.9101，范围 0.3905 - 1.5659 元/kWh；价格预测 MAE 0.0543；滚动 v_term
    0.490271
744 - **全天**：计划购电量 94,530.8371 kWh / 73,902.0994 元；紧急购电量 1,471.3998 kWh /
    5,573.8844 元；调整费 -648.3236 元；**总费用 78,827.6601 元**
745 - 储电量：0:00 9,906.7384 kWh → 24:00 9,390.9306 kWh
746
747 ### 表1 计划购电量（kWh）
748
749 | 时段 | 购电量 | 时段 | 购电量 |
750 |---|---|---|---|
751 | 0:00-0:10 | 1,335.1532 | 0:10-0:20 | 596.5815 |
752 | 0:20-0:30 | 630.0283 | 0:30-0:40 | 641.8274 |
753 | 0:40-0:50 | 615.0696 | 0:50-1:00 | 585.9753 |
754 | 1:00-1:10 | 633.6402 | 1:10-1:20 | 647.1142 |
755 | 1:20-1:30 | 610.3597 | 1:30-1:40 | 621.7429 |
756 | 1:40-1:50 | 629.8676 | 1:50-2:00 | 603.9432 |
757 | 2:00-2:10 | 631.8982 | 2:10-2:20 | 630.8298 |
758 | 2:20-2:30 | 620.0637 | 2:30-2:40 | 633.2464 |
759 | 2:40-2:50 | 627.8746 | 2:50-3:00 | 629.3189 |
760 | 3:00-3:10 | 625.5023 | 3:10-3:20 | 647.2805 |
761 | 3:20-3:30 | 643.0762 | 3:30-3:40 | 684.6690 |
762 | 3:40-3:50 | 627.3498 | 3:50-4:00 | 658.8254 |
763 | 4:00-4:10 | 670.1371 | 4:10-4:20 | 672.1752 |
764 | 4:20-4:30 | 704.7564 | 4:30-4:40 | 683.2901 |
765 | 4:40-4:50 | 675.1906 | 4:50-5:00 | 657.9735 |
766 | 5:00-5:10 | 636.4673 | 5:10-5:20 | 643.1310 |
767 | 5:20-5:30 | 681.3989 | 5:30-5:40 | 699.9735 |
768 | 5:40-5:50 | 809.0121 | 5:50-6:00 | 872.2231 |

```

769	6:00-6:10	855.4603	6:10-6:20	873.0916	
770	6:20-6:30	885.3937	6:30-6:40	870.3557	
771	6:40-6:50	823.7918	6:50-7:00	840.8943	
772	7:00-7:10	807.1434	7:10-7:20	801.0555	
773	7:20-7:30	786.4770	7:30-7:40	797.9534	
774	7:40-7:50	743.9883	7:50-8:00	161.6142	
775	8:00-8:10	246.2623	8:10-8:20	173.7575	
776	8:20-8:30	53.6606	8:30-8:40	8.3239	
777	8:40-8:50	0.0000	8:50-9:00	0.0000	
778	9:00-9:10	509.4056	9:10-9:20	624.0838	
779	9:20-9:30	589.8496	9:30-9:40	548.3911	
780	9:40-9:50	466.5517	9:50-10:00	0.0000	
781	10:00-10:10	0.0000	10:10-10:20	0.0000	
782	10:20-10:30	0.0000	10:30-10:40	0.0000	
783	10:40-10:50	467.0680	10:50-11:00	1,214.7192	
784	11:00-11:10	1,127.3122	11:10-11:20	1,148.1592	
785	11:20-11:30	1,194.7081	11:30-11:40	1,199.4397	
786	11:40-11:50	1,124.6366	11:50-12:00	1,022.4405	
787	12:00-12:10	1,141.5696	12:10-12:20	1,106.3055	
788	12:20-12:30	674.3755	12:30-12:40	997.8002	
789	12:40-12:50	490.2358	12:50-13:00	219.8033	
790	13:00-13:10	87.2494	13:10-13:20	137.9492	
791	13:20-13:30	117.3070	13:30-13:40	388.5467	
792	13:40-13:50	0.0000	13:50-14:00	0.0000	
793	14:00-14:10	0.0000	14:10-14:20	0.0000	
794	14:20-14:30	0.0000	14:30-14:40	0.0000	
795	14:40-14:50	0.0000	14:50-15:00	1,338.9641	
796	15:00-15:10	1,452.8069	15:10-15:20	809.6539	
797	15:20-15:30	1,449.6899	15:30-15:40	837.3805	
798	15:40-15:50	853.2879	15:50-16:00	911.4217	
799	16:00-16:10	949.8051	16:10-16:20	1,018.5250	
800	16:20-16:30	1,092.0264	16:30-16:40	1,124.4216	
801	16:40-16:50	1,094.5552	16:50-17:00	1,102.6634	
802	17:00-17:10	1,002.5722	17:10-17:20	1,013.8882	
803	17:20-17:30	1,025.9514	17:30-17:40	1,099.7717	
804	17:40-17:50	939.4723	17:50-18:00	720.0232	
805	18:00-18:10	629.6499	18:10-18:20	681.5360	
806	18:20-18:30	662.1699	18:30-18:40	704.1344	
807	18:40-18:50	687.9808	18:50-19:00	647.8006	
808	19:00-19:10	0.0000	19:10-19:20	3.7587	
809	19:20-19:30	26.7610	19:30-19:40	0.0000	
810	19:40-19:50	0.0000	19:50-20:00	641.7435	
811	20:00-20:10	3.1349	20:10-20:20	4.7828	
812	20:20-20:30	0.0000	20:30-20:40	0.0000	
813	20:40-20:50	715.0522	20:50-21:00	767.1147	
814	21:00-21:10	739.2836	21:10-21:20	725.4596	
815	21:20-21:30	751.3915	21:30-21:40	728.4356	
816	21:40-21:50	702.8440	21:50-22:00	1,364.0958	
817	22:00-22:10	1,336.3186	22:10-22:20	676.8061	
818	22:20-22:30	1,373.7806	22:30-22:40	1,383.1355	
819	22:40-22:50	1,383.3805	22:50-23:00	641.6293	

```

820 | 23:00-23:10 | 1,353.0919 | 23:10-23:20 | 1,405.3014 |
821 | 23:20-23:30 | 1,373.2347 | 23:30-23:40 | 675.1772 |
822 | 23:40-23:50 | 686.9595 | 23:50-24:00 | 673.9145 |

```

824 **### 表2 充放电量（六段）与储电量**

```

826 | 时段 | 充电量 | 放电量 |
827 | --- | --- | --- |
828 | 0:00-4:00 | 0.0000 | 330.7031 |
829 | 4:00-8:00 | 0.0000 | 1,334.9657 |
830 | 8:00-12:00 | 6,144.9765 | 5,873.9403 |
831 | 12:00-16:00 | 7,106.3424 | 2,753.9508 |
832 | 16:00-20:00 | 521.5096 | 4,222.9365 |
833 | 20:00-24:00 | 7,098.4168 | 2,853.4394 |
834 | 0:00 储电量 | 9,906.7384 | 24:00 储电量 | 9,390.9306 |

```

836 **### 表3 紧急购电量（区间）**

```

838 | 时段 | 紧急购电量 |
839 | --- | --- |
840 | 10:50-12:00 | 764.7471 |
841 | 12:20-12:30 | 227.0595 |
842 | 12:40-13:00 | 435.5958 |
843 | 13:10-13:20 | 43.9974 |

```

C25. 04_ 结果/问题 4_ 口径 B 正式/问题 4_ 已切换为口径 B.md（一行）

```

1 # 问题4 交付已切换为口径B（2026-09-12）
2
3 - `result4-2.xlsx`: 旧 `4667FEDDD32A63D6...`（口径A）→ **新 `1BC81EF5059E99AA...`**（口径B）
4 - `result4-3.xlsx`: 旧 `A63C101B0E577A5E...`（口径A）→ **新 `70719B92BF2FD3DE...`**（口径B）
5 - 旧件处置: `results/` 下 **6 个 Q4 附带数据文件**已改名加 `_旧口径A_causal_v2` 后缀（q42_best.
  pk1、q42_汇总.csv、q42_逐日费用.csv、q43_daystore.pk1、q43_逐日费用.csv、q43_组合对照.csv
  ）；**两个旧工作簿未在 `results/` 下留改副本**，而是以原名保存在 `04_结果与检验/备份_正式交
  付_20260912/results_旧/`（`4667FEDD...` / `A63C101B...`，备份保持交付原样以便逐字节回滚）；
6 - 四个指定日期表 `q42_*/q43_*` 已按口径B 重建并与工作簿逐格校验（最大差 0.005000）；
7 - **2026-09-12 18:24—18:32 再次刷新（fc3 缺陷修复后）**: `result4-2.xlsx` 16:54 版 `81
  FC90AEF32E4626...` → 现版 `1BC81EF5059E99AA...`；`result4-3.xlsx` 16:54 版 `81
  FE7217E8F1F08F...` → 现版 `70719B92BF2FD3DE...`（Q4-3 家族数字全量更正：部署 B_v
  14,935,075.29 → 15,675,523.45）；`results/` 下 6 个 Q4 附带数据文件亦同步刷新，修前版留档于
  `重跑_Q4_B_20260912/修前fc3缺陷_交付附带件/`；
8 - **跨口径（A/B）使用限制**：Q4-3 上口径B 反而比口径A 便宜 236,851.66 元，该差异**不得**解读为价
  格信息价值（机制为控制激进度 × 5 倍紧急罚通道），详见 `07_问题4_口径B/结论_Q4_补做.md` §5；
9 - 问题1（`result1.xlsx`）仍为 2026-09-11 口径、未切换；详见 `07_问题4_口径B/结论_Q4.md` 与协议
  §14。

```

C26. 04_ 结果/问题 4_ 口径 B 正式/修后数字总表 _eps 修正.md（一行）

```

1 # 问题4 • eps 归一化修正后 • 数字总表（2026-09-12 晚）

```

2

3 > 唯一数字源。所有文档更新一律引用本表；本表数字取自 `重跑\_Q4\_B\_eps修正/` 下的

4 > `q4\_对照汇总.json`、`q4\_配对统计.json`、`pi\_bounds\_q4.json`、`v台阶\_Q4B.json`、`d4\_Q4B.json`、
各格 `p2\_arithmetic.json`。

5

6 **## 0. 内核版本（必须随数字引用）**

7

8 | 项 | 值 |

9 | ---|---|

10 | 问题4 交付所用内核 | `03\_代码/p10\_core.py` = `**`b3242ad925c82d63...`**`（eps 归一化修正后，2026
-09-12 18:44:13） |

11 | 问题2/3 交付所用内核 | 同一文件的 `**`0a2cef2e2b4428bb...`**`（16:44:57 版；**按用户裁定未重跑**
） |

12 | 末槽目标系数 | 修后 ``(eps_soc + v_term)/S``；修前 ``eps_soc + v_term/S`` |

13 | 有效日末系数（ $v=0.481548$, $S=10$ ） | 修后 `**0.482548**`；修前 `**0.491548**`（+2.08%，即协议 §13
第 8 条披露的残留） |

14 | 修前版本留档 | `修订前备份\_eps修正\_20260912\_184405/p10\_core.py`；修前交付件见 `备份_正式交付
_20260912/eps修正前_口径B_交付件/` |

15 | 一致性要求 | **跨问题比较（Q2/Q3 ↔ Q4）必须注明两问内核不同**；`results/指定日期表格.json` 逐条
目带 `\_内核` 字段 |

16

17 **## 1. 四格对照（334 天，元）：修前 → 修后**

18

19 | 格 | Q4-2 B 修前 | Q4-2 B 修后 | Δ | Q4-3 部署 修前 | Q4-3 部署 修后 | Δ |

20 | ---|---|---|---|---|---|---|

21 | A_v0 | 15,158,179.86 | `**15,160,985.57**` | +2,805.71 (+0.019%) | 16,252,511.47 |
`**16,255,264.10**` | +2,752.63 (+0.017%) |

22 | A_v | 14,883,173.16 | `**14,883,204.81**` | +31.65 (+0.000%) | 15,912,375.11 |
`**15,935,449.22**` | +23,074.12 (+0.145%) |

23 | B_v0 | 15,266,391.18 | `**15,269,768.32**` | +3,377.14 (+0.022%) | 16,323,959.59 |
`**16,324,851.37**` | +891.78 (+0.005%) |

24 | `**B_v（正式）**` | 14,990,278.87 | `**14,995,507.04**` | `**+5,228.16 (+0.035%）**` |
15,675,523.45 | `**15,716,088.61**` | `**+40,565.16 (+0.259%）**` |

25

26 Q4-3 none (B_v)：修前 16,430,707.21 → 修后 `**16,506,164.40**`；**部署 - none = -790,075.79**
（修前 -755,184）

27 （注：365 天总额 none 修后 `**19,303,263.54**`、部署修后 `**18,496,064.62**`；修前部署 365 天为
18,455,634.33；其余格修后部署 365 天：A_v0 19,108,850.64、A_v 18,757,034.96、B_v0
19,183,966.22。本文档早期版本此处误写 18,718,106.19，已更正。**)）

28

29 **### 1b. Q4-3 组合对照（B_v, 334 天，元；由 `B\_v/q43\_逐日费用.csv` 求和，与 `q4\_对照汇总.json` 的
`q43\_334` 逐格一致）**

30

31 | 组合 | 修前 | 修后 | Δ |

32 | ---|---|---|---|

33 | none | 16,430,707.21 | `**16,506,164.40**` | +75,457.19 |

34 | `**6（最优固定组合）**` | 15,670,846.79 | `**15,712,944.80**` | +42,098.01 |

35 | 12 | 16,356,233.30 | `**16,429,010.31**` | +72,777.01 |

36 | 18（最差） | 16,500,977.36 | `**16,583,022.61**` | +82,045.25 |

37 | 6+12 | 15,782,816.29 | `**15,825,198.58**` | +42,382.29 |

38 | 6+18 | 15,712,487.02 | `**15,760,850.14**` | +48,363.12 |

39 | 12+18 | 16,400,748.07 | **16,478,897.70** | +78,149.63 |
40 | 6+12+18 | 15,810,241.80 | **15,858,374.49** | +48,132.69 |
41 | **历史选型部署（正式）** | 15,675,523.45 | **15,716,088.61** | +40,565.16 |
42
43 正式交付的部署路径比"永远用固定组合 6"贵 **3,143.81** 元（修前为 4,676.66 元；未舍入精确值
3,143.8039）——选型规则**非事后最优**。
44
45 > **派生差值的取数口径（本表统一）**：一律先用表中两位小数展示值相减，再在括注中给出未舍入精确值；因
此个别处会出现 0.01 元级差异（如 3,143.81 vs 3,143.8039、11,724.76 vs 11,724.758）。
46
47 ## 2. 效应分解（334 天，元）
48
49 | 效应 | 修前 | 修后 |
50 | ---|---|---|
51 | Q4-2 跨日价值 (B_v-B_v0) | -276,112.31 | ** -274,261.28** |
52 | Q4-2 跨日价值 (A_v-A_v0) | -275,006.70 | ** -277,780.76** |
53 | Q4-2 跨口径差 (B_v-A_v) | +107,105.71 | ** +112,302.23** |
54 | Q4-2 跨口径差 (B_v0-A_v0) | +108,211.32 | ** +108,782.75** |
55 | Q4-2 交互项 | -1,105.61 | ** +3,519.48** |
56 | Q4-3 部署 跨日价值 (B_v-B_v0) | -648,436.14 | ** -608,762.77** |
57 | Q4-3 部署 跨口径差 (B_v-A_v) | -236,851.66 | ** -219,360.61** (**仍为负值，不得作 VOI**) |
58
59 ## 3. 配对统计（`q4\_配对统计.json`，同日期配对、7 日分块 bootstrap×2000、seed 2025；正 = 前者更
贵）
60
61 **56 组比较，51 组显著。** 关键项（元/日）：
62
63 | 比较 | 修后均值 | 95% CI | 显著 |
64 | ---|---|---|---|
65 | B_v 部署 - none | ** -2,365.50** | [-2,911.42, -1,826.11] | 是 |
66 | B_v 6 - none | -2,374.91 | [-2,928.10, -1,832.31] | 是 |
67 | B_v 6+12 - none | -2,038.82 | [-2,583.82, -1,471.33] | 是 |
68 | B_v 6+18 - none | -2,231.48 | [-2,808.86, -1,664.29] | 是 |
69 | B_v 6+12+18 - none | -1,939.49 | [-2,496.93, -1,379.51] | 是 |
70 | B_v 12 - none | -231.00 | [-454.54, +10.82] | 否 |
71 | B_v 12+18 - none | -81.64 | [-314.72, +167.21] | 否 |
72 | **B_v 18 - none** | ** +230.11** | [+126.76, +345.46] | **是（有害）** |
73 | 跨格 B_v-A_v (Q4-2 B) | +336.23 | [+250.27, +421.99] | 是 |
74 | 跨格 B_v-A_v (Q4-3 部署) | -656.77 | [-939.59, -389.07] | 是 (**不得作 VOI**) |
75 | 跨格 B_v-B_v0 (Q4-2 B) | -821.14 | [-993.90, -652.52] | 是 |
76 | 跨格 B_v-B_v0 (Q4-3 部署) | -1,822.64 | [-2,186.29, -1,485.20] | 是 |
77 | 可刷新B - 严格B (Q4-3 部署) | +3.92 | [-14.59, +21.71] | 否 |
78 | B_v Q4-2 B - A | -1,660.76 | [-1,879.26, -1,432.48] | 是 |
79 | B_v Q4-2 C - B | +840.82 | [+489.99, +1,195.98] | 是 |
80
81 ## 4. PI（综合信息上界；价格+负荷/光伏+全期规划，故**不是**价格预报的价值）
82
83 | 格 | 路径 | 实际（元） | PI（元） | 缺口（元） | 缺口% |
84 | ---|---|---|---|---|---|
85 | A_v0 | Q4-2 B | 15,160,985.57 | 12,782,476.66 | 2,378,508.91 | 15.69% |
86 | A_v | Q4-2 B | 14,883,204.81 | 12,779,221.79 | 2,103,983.01 | 14.14% |

87 | B_v0 | Q4-2 B | 15,269,768.32 | 12,782,488.02 | 2,487,280.30 | 16.29% |
88 | **B_v** | **Q4-2 B** | **14,995,507.04** | **12,779,262.16** | **2,216,244.88** |
| **14.78%** |
89 | A_v0 | Q4-3 部署 | 16,255,264.10 | 12,782,482.63 | 3,472,781.47 | 21.36% |
90 | A_v | Q4-3 部署 | 15,935,449.22 | 12,779,221.79 | 3,156,227.43 | 19.81% |
91 | B_v0 | Q4-3 部署 | 16,324,851.37 | 12,782,483.99 | 3,542,367.39 | 21.70% |
92 | **B_v** | **Q4-3 部署** | **15,716,088.61** | **12,779,280.98** | **2,936,807.62** |
| **18.69%** |
93 | B_v | Q4-3 none | 16,506,164.40 | 12,779,296.28 | 3,726,868.12 | 22.58% |
94
95 唯一 PI 求解数 22。 **2/1 初态** (kWh) : A_v0 Q4-2 B 1,645.7 | A_v 10,798.3 | B_v0 1,615.1 | **B_v
10,677.2** ; Q4-3 部署: A_v0 1,629.6 | A_v 10,798.3 | B_v0 1,625.9 | **B_v 10,620.8** ; Q4-3
none (B_v) 10,574.9。
96 **PI 对初态几乎不敏感** : B_v0 (初态 1,615.1 kWh) PI = 12,782,488.02, B_v (初态 10,677.2 kWh)
PI = 12,779,262.16, 初态差 9,062.1 kWh 只使 PI 变动 **3,225.86 元**。
97
98 ## 5. v 台阶探针 (Q4-2 B 主模型, 334 天)
99
100 | v | 总费用 (元) | 紧急费 (元) | E0 均值 | 触底天数 | 年末储能 |
101 | ---|---|---|---|---|---|
102 | 0 (B_v0 格对照) | 15,269,768.32 | 1,989,970.70 | 1,631.8 | 58 | 1,200.00 |
103 | 0.35 | 15,135,439.62 | 1,835,251.02 | 2,058.5 | 26 | 1,200.00 |
104 | 0.44 | 15,033,942.20 | 1,738,712.55 | 5,435.9 | 6 | 7,549.88 |
105 | 0.46 | 15,011,992.48 | 1,711,758.94 | 7,005.9 | 4 | 9,293.93 |
106 | 0.50 | 14,983,782.28 | 1,672,677.08 | 7,938.2 | 1 | 10,068.78 |
107 | **滚动 v (正式, 均值 0.4800) ** | **14,995,507.04** | 1,691,260.45 | 7,139.0 | 1 | 9,501.53
|
108
109 要点: 台阶区间仍约 **[0.35, 0.44]**; 滚动 v 比 v=0.50 贵 **11,724.76** 元 (未舍入 11,724.758
)、比 v=0.46 便宜 **16,485.44** 元;
110 与 v=0 对照 = ** -274,261.28 元** (-1.80%), 日初储能贴底天数 58 → 1。触底天数定义 = 日初储能
≤1200 kWh 的天数。
111 (修前对应表: 滚动 v 14,990,278.87、v=0 15,266,391.18、0.35 15,131,351.13、0.44 15,026,114.57
、0.46 15,006,151.32、0.50 14,985,332.32; 触底 56/26/5/4/1/0)
112
113 ## 6. Q4 版 D4 式拆分 (Q4-3 B_v, 334 天, 元)
114
115 | 项 | 修前 | 修后 |
116 | ---|---|---|
117 | N 不更新 | 16,430,707.21 | **16,506,164.40** |
118 | F 仅刷新预报 | 16,428,212.81 | **16,503,426.56** |
119 | FM 刷新+购电调整 | 15,810,241.80 | **15,858,374.49** |
120 | **N - F** | +2,494.40 (+0.02%) | **+2,737.85 (+0.017%)** |
121 | **F - FM** | +617,971.01 (+3.76%) | **+645,052.06 (+3.91%)** |
122 | N - FM | +620,465.41 (+3.78%) | **+647,789.91 (+3.92%)** |
123
124 => 更新收益的 **99.58%** 仍来自市场购电再交易 (Q2/Q3 为 87.6%, 同构且更极端)。
125
126 ## 7. 审计 (修后, `p2\_arithmetic.json`)
127
128 | 格 | 检查项 | 账单残差 | SOC 残差 | 供需残差 | 同槽互斥 | 状态 |

```

129 | ---|---|---|---|---|---|
130 | A_v0 | 75,451 | 0.0 | 1.46e-11 | 2.27e-13 | 0.0 | PASS |
131 | A_v | 75,831 | 0.0 | 1.27e-11 | 2.27e-13 | 0.0 | PASS |
132 | B_v0 | 75,686 | 0.0 | 1.18e-11 | 2.27e-13 | 0.0 | PASS |
133 | B_v | 75,566 | 0.0 | 1.55e-11 | 2.27e-13 | 0.0 | PASS |
134 | sens_refresh | 75,576 | 0.0 | 1.55e-11 | 2.27e-13 | 0.0 | PASS |
135
136 ## 8. 交付件（修后；已装入 `results/`，均与 `重跑_Q4_B_eps修正/B_v/` 同名文件逐字节相同）
137
138 | 文件 | SHA256 | 大小 | 修前（已被取代） |
139 | ---|---|---|---|
140 | `results/result4-2.xlsx` | `1
      BC81EF5059E99AA4C0DDF30F890905A8F2519A2866599FFFD41DCD20AD5C0F8` | 341,535 B | `51676
      B01FB52B359...` |
141 | `results/result4-3.xlsx` | `70719
      B92BF2FD3DEA081E5FBD2EC3B4AA6F22E5D682A628BFCFA3C29E62BB83D` | 636,693 B | `608
      E99270A609D40...` |
142 | `results/四个指定日期_表123_q4B.json` | `01
      C90A853B43F3661ACCBBA4C903CC34627EC408B82B104BFEF93C9C55D39442` | 60,432 B | `
      AF984CC9C922FCAC...` |
143 | `results/指定日期表格.json` | `6
      D4E2A6B67AA4FA861A2FEDFB76C8924213AD2C06402674180E497923D89AF8C` | 14,917 B | `5
      E0110DB2C82303F...` |
144
145 四日汇总（元）：Q4-2 44,064.7 / 16,984.5 / 52,682.5 / 71,456.4；Q4-3 部署 55,521.3 / 18,727.9
      / 55,509.9 / 78,827.7。
146 工作簿 ⇄ 四日表逐格校验：最大差 0.005000（容差 0.0051）**PASS**。
147 修前的 Q4 附带数据文件留档于 `重跑_Q4_B_eps修正/修前eps_交付附带件/`；修前工作簿与四日表留档于
148 `备份_正式交付_20260912/eps修正前_口径B_交付件/`；修前图件留档于 `备份_正式交付_20260912/eps修正
      前_图件/`。
149
150 ## 9. 跨口径使用限制（修后内核下复算，仍强制）
151
152 同一家族、同一初态、同一负荷/光伏预报下，仅把决策价从预测价（口径B）换成真实价（口径A），Q4-3 **反
      而更贵**：
153
154 | Q4-3 轨 | A_v（口径A） | B_v（口径B） | A-B | 净差（= 总费用差） |
155 | ---|---|---|---|---|
156 | **none**（334 天） | 16,922,745.22 | 16,506,164.40 | **+416,580.82** | +416,580.82 |
157 | 组合 6 | — | — | **+212,938.36** | +212,938.36 |
158 | 组合 6+12+18 | — | — | **+191,215.44** | +191,215.44 |
159 | **部署（正式）** | 15,935,449.22 | 15,716,088.61 | **+219,360.61（-656.77 元/日）** |
      +219,360.61 |
160
161 **机制证据（2026-09-12 修后内核下由主控复算，两条轨分别给出）**：
162
163 | 指标（334 天） | none 轨：A_v | none 轨：B_v | 差 | 部署轨：A_v | 部署轨：B_v | 差 |
164 | ---|---|---|---|---|---|---|
165 | 计划购电量（kWh） | 20,787,743.11 | 20,733,434.25 | +0.26% | 20,790,206.89 | 20,721,633.10
      | +0.33% |
166 | 紧急购电量（kWh） | 965,120.82 | 807,553.84 | **+19.51%** | 513,670.64 | 452,102.46 |

```

****+13.62%**** |
 167 | 紧急购电费 (元) | 3,398,944.76 | 2,876,961.98 | ****+521,982.78**** | 1,699,695.66 |
 1,521,757.94 | ****+177,937.72**** |
 168 | 计划购电费 (元) | 13,523,800.47 | 13,629,202.42 | ****−105,401.96**** (A 更便宜) |
 13,525,246.02 | 13,623,843.26 | −98,597.24 |
 169 | 调整费 (元) | 0 | 0 | — | 710,507.54 | 570,487.41 | +140,020.13 |
 170 | 储能触及下限的****槽数**** ($E \leq 1201$ kWh) | 3,217 | 2,782 | ****+15.64%**** | 2,139 | 1,705 |
****+25.45%**** |
 171 | 有紧急购电的天数 | 329 | 319 | +10 天 | 328 | 321 | +7 天 |
 172 | 日末储能均值 (kWh) | 6,776.2 | 7,332.7 | −556.5 | 6,768.5 | 7,363.3 | −594.8 |
 173
 174 ****机制结论****: 知道真实价格使控制器做****更激进的价格套利**** (更深放电、更少留存电量), 在负荷/光伏预报
 误差出现时****更常处于无余量状态****, 被 ****5 倍紧急罚通道****放大, 可****压过价格信息的正价值****。
 175 (说明: 槽级触底数、日末储能、紧急天数、清晨时段分布等均取自修后树 `q43\_daystore.pkl` 的 `E/qEM/
 Eend` 序列; 修前内核下的同类诊断见 `重跑_Q4_B_20260912/结论_Q4_补做.md` §5.2, 两套数字不可混
 用。)
 176
 177 ****使用限制 (不变)****:
 178 1. ****禁止****把跨口径 A/B 差异 (尤其 q43 家族) 解读为"价格信息价值"或 ****VOI****。
 179 2. Q4-2 的 ****+112,302.23 元**** ($v=0$ 时 +108,782.75) 只是"在该执行机制下、用预测价决策相对用真实价
 决策的成本差", 可视为价格预报价值的****下界**** (口径A 同时改变了控制激进度)。
 180 3. 正式交付为口径B, 其数字不受影响; 同口径内比较 (none / 组合 / 部署 / v) 不受影响。
 181 4. 干净的 VOI 实验 (剥离 5x 紧急通道或固定控制激进度) ****未做****, 见 §10。
 182
 183 **## 10. 待办与未重跑项 (截至 2026-09-12 晚第二批)**
 184
 185 ****本批已在修后内核内补做****: 四格 (4 格 × 11 轨)、`sens\_refresh` (8 轨)、v 台阶 4 条常数 v 轨、
 d4_F、PI、配对统计、四日表、图件、五格审计。
 186
 187 ****本批未在修后内核重跑 (引用时须注明"修前内核")****:
 188 1. ****价格轨敏感性 `lag1` / `fix` / `shape`****: 本批只跑 `hist7`, 故"MAE 最小的主轨同时费用最低"在
 新内核下****未复算**** (文档中该三行保留修前值 15,040,626.62 / 15,043,344.68 / 15,044,162.32,
 且其"相对主轨"基准是****修前**** hist7 14,990,278.87, 不得与修后 14,995,507.04 相减)。
 189 2. ****旧协议回归位 `reg\_old\_protocol`****: 修前测得 15,156,589.57 vs 历史 15,221,458.20 (−0.43%
), 未重算。
 190 3. ****`none` 轨信息集敏感性****: 已用 `Q4\_NONE\_NO\_FC3=1` 在修后内核内重跑 (`sens\_none\_nofc3/`, 驱
 动脚本 `run\_none\_sens\_epsfix.ps1`), 结果见 `none信息集敏感性\_Q4.md`; ****若该文件不存在, 说明
 尚未完成, 引用时须注明"未量化"****。
 191 4. ****`audit/` 目录**** (p16 预测层自检报告) 只存在于修前树; 本批以"两树 `电价预测缓存.npz` 逐字节相
 同 (SHA256 `A28E46BD04C00E5A...`)"作为结论可继承的依据。另: 两树缓存 `\_meta.json` 中
 FP_shape 仍保留旧判据下的 `violations=116 / status=FAIL` (`allclose` 改判只记录在 audit
 的 .md 中, 未回写 meta)。
 192 5. ****审计检查项计数的格间差异**** (A_v 由修前 75,561 变为修后 75,831, 而其余格 +5/+15/+5/±0): ****
 只登记、未归因****。
 193
 194 ****仍未做 (能力/范围外)****: 干净的跨口径 VOI 实验、Q4 版 Sxseed 矩阵、口径B 下 linear 轨、价格预报
 层第三方独立复现、`v\_term` 取值经济学论证、9 格矩阵原始逐日归档、打包声明为"证据归档、非端到端
 可复跑" (例外见 §15.6 的环境变量覆盖机制)。
 195
 196 ---
 197

198 *本文档为 2026-09-12 晚第二批（eps 归一化修正）后的问题4 唯一数字源；修订记录：补 §1b 组合对照、修正 §1 的 365 天部署总额、§9 换为修后内核复算的两条轨机制表、§4 补初态与 PI 不敏感值、§10 补未重跑清单。*

C27. AI 工具使用说明.md（一行）

```
1 # AI 工具使用说明
2
3 适用材料：当前微网购电与储能优化调度论文及配套项目。本文依据论文正文（`2026C.tex`）、项目说明与项目
  状态记录、结果订正记录、图件来源记录、结果选型说明及团队确认的披露信息起草。**已核实部分**为团
  队确认的工具名称、版本、使用时间、工作分工归属及问题三的预报分解口径，以上内容已写入正文；**尚未
  完成的部分**（问题二表格的结果批次统一、问题三记录的 hold/linear 同步、研究期间交互记录、姓名
  披露要求与团队终审）集中列于文末，本稿不将其写成已完成。
4
5 ## 一、所用 AI 工具名称和版本
6
7 **论文研究期间使用的工具：**DeepSeek Harness，使用时间为**累计 4 日**（2026 年 9 月 10 日、9 月
  11 日、9 月 12 日、9 月 13 日，间断使用，非连续占用）。该工具用于论文行文框架整理、科研配图生
  成、结果文件与检查器的订正，以及数据分析表述整理。
8
9 该期间经该工具**实际调用的模型**（依会话记录中的路由登记）：
10
11 | 模型 | 路由 | 用途与范围 |
12 | --- | --- | --- |
13 | `deepseek-v4-flash` | `deepseek-official/deepseek-v4-flash` | 研究期间主要模型（团队对外称 `
  deepseek-flashv4.1`），承担建模核验、结果重出、文档同步与配图 |
14 | `glm-5.3-flash` | `zai-coding-cn/glm-5.3-flash` | 2026-09-12 前后若干会话使用，用于问题 2/3
  重跑结果的独立复核与文档核查 |
15 | `deepseek-v4-flash-vision-exp` | `deepseek-official/deepseek-v4-flash-vision-exp` | 仅用于
  图件视觉质检 |
16
17 **本说明的起草会话：**本说明初稿在本次起草会话中生成，该会话读取指定文件夹中的生成要求并结合现有材料
  形成初稿；该会话不参与论文研究期间的建模、求解与结果计算，其输出不作为研究期间工具使用记录的替
  代。按团队决定的披露范围，本次起草会话的工具名称与版本不列入本节工具清单。
18
19 本节披露范围为团队确认的论文研究期间使用工具；如提交前补充其他工具或环节，须一并登记并同步修订本说
  明。
20
21 现有材料中的历史任务摘要记录了 AI 参与论文语言审阅、修改建议整理、数据分析文字润色、行文框架整理和绘
  图源代码修改等工作；项目状态记录另记载了 AI 执行的建模方案核验与结果文件订正。仅凭项目文件中存
  在模型说明、求解代码和审查材料，不能确定其作者及 AI 参与程度，因此不能声明“AI 仅参与语言润
  色”，也不能声明“全部核心建模工作均由团队独立完成”。团队确认的实际参与范围为：**论文配图生成与
  论文行文框架整理、模型方案中的公式与结算代数核验，以及求解编程中的工具性订正**。
```

```
22
23 ## 二、具体使用目的和应用环节
24
25 **分工一览（按团队确认）：**
26
27 | 环节 | 归属 | 说明 |
28 | --- | --- | --- |
29 | 问题分析、模型假设、算法设计 | 团队自主完成 | 由团队确认，不依据论文正文推定 |
```

30 | 公式推导 | AI 参与（核验与订正） | AI 对模型方案阶段的公式与结算代数作独立核验并提出订正，采用与
 31 | 求解编程 | AI 参与（工具性订正） | 按团队裁定重出结果文件、重写失效检查器；模型与求解方案由团队确
 32 | 参数设置 | 团队自主完成 | 参数取值不在 AI 决定范围内 |
 33 | 结论形成 | 团队自主完成 | 结果解释与主要结论由团队作出 |
 34 | 数据分析表述整理、配图与行文框架 | AI 辅助 | 表述性建议、绘图源代码修改与配图生成 |
 35
 36
 37 **### 1. 问题分析与建模方案（含公式推导）**
 38
 39 ****团队自主完成的核心工作：****论文涉及微网购电计划、储能调度、预测不确定性和实时电价条件下的策略研究。
 问题分析、模型假设、算法设计由团队自主完成；参数设置由团队自主完成。上述归属不依据论文正文推定，
 40 而由团队确认。
 41 ****AI 辅助完成的非核心工作：****公式推导环节有 AI 参与，形式为对模型方案阶段的公式与结算代数作独立核
 验，包括字典引用逐条比对、文献 DOI 复核、等价式与净分解抽查，并就被发现的量纲与表述问题提出订正
 意见（项目状态记录 2026-09-10：核验字典引用 3/3 逐字属实、3 条文献 DOI 经 Crossref 复核真实，
 提出并落实 3 项 P1 与 6 项 P2 修正）。核验意见供团队裁定，不构成对模型合理性的独立证明，也不代
 42 表 AI 参与模型假设或算法的原创决定。
 43 ****人工复核与最终责任：****建模方案的选择与最终认定由团队作出。
 44
 45 **### 2. 模型求解与编程**
 46
 47 ****团队自主完成的核心工作：****求解模型的目标函数、约束与总体求解方案由团队确定；对 AI 提出的修改，由
 团队裁定是否执行并作出验收（如 2026-09-11 裁定“先修正再打包”，要求整体订正后重新打包）。参数
 48 设置与结论形成由团队自主完成，不由 AI 决定。
 49 ****AI 辅助完成的非核心工作：****按团队裁定，AI 执行了问题 1 表 2 时钟口径的整体订正：将 5 个正式结果
 文件的「充放电量」工作表改为真实时钟区间口径（`0:00-4:00` 取自前一交易日第 144 槽加本日槽 1—
 23，其余各段顺推）并重出结果文件；重写 `最小运行\_全部检查.py` 中 C4/C5 两处失效检查器；新增首
 日 `0:00—0:10` 子槽的边界状态恢复、SOC 链闭合判据与内容级指纹。可核对结果：订正前后逐格比对变
 动 ****7,086 格****且全部位于「充放电量」表；`result1.xlsx` 表 2 的 12 个汇总数中 ****8 个****发生变
 化，最大偏差 ****1,251.7547 kWh****；其余工作表逐格未变；`最小运行\_全部检查.py` 重跑 ****20/20**
 50 **PASS****，独立终检 ****12/12 PASS****。上述工作属于按已定口径的重新落盘与检查器修复。
 51 ****人工复核与最终责任：****结果口径与落盘规则由团队裁定，AI 执行的重出与检查器修改不构成对调度结果最优
 性的独立验证。因检查器文件已被修改，原 P1 门禁验收哈希失效，需重新独立验收；本说明不代作 P1
 52 **PASS 声明。**
 53 **### 3. 数据分析表述与结果核对**
 54
 55 ****团队自主完成的核心工作：****数据来源确认、时间口径确定、统计方法选择、统计结果解释及结论形成由团队完
 成（人工确定数据来源、时间口径与统计结果解释）。现有论文和图件记录涉及负荷、光伏、电价、净负荷及
 56 储能状态，相关材料能够说明研究内容，但不能单独证明由谁完成。
 57 ****AI 辅助完成的非核心工作：****历史任务摘要记录了 AI 对数据分析表述的整理，包括提示区分相关关系与因果
 关系、区分不同分组下的波动，以及记录表中统计量与附件复算结果的差异；上述工作属于辅助检查。
 58
 59 ****采纳与修正的具体内容（分别说明）：****

60

61 - 采纳的表述：团队采纳 AI 提出的表述性建议，即在正文中区分相关关系与因果关系、区分不同分组下的波动，
并如实登记记录表统计量与附件复算结果的差异。

62 - 修正的数值：表 2 时钟口径订正涉及的数值已改写，共 ****7,086 格****（全部位于「充放电量」表，其中 `result1.xlsx` 表 2 的 12 个数中 8 个变化，最大偏差 1,251.7547 kWh）。数值是否被改动的判定以
内容级指纹（`content\_sha256`）为准，字节级哈希仅作参考。

63 - 口径的人工对齐对象（问题 1 窗口与问题 2—4 不同，按以下两项分别对齐，不是笼统“对齐”）：
64 - ****时段编号与时钟顺序****：问题 1 的时段编号按真实时钟区间重排，表 2 六段改为覆盖 `0:00-24:00`
的时钟区间（`0:00-4:00` 段由前一交易日第 144 槽加本日槽 1—23 组成），储电量时间变化图按真实时
钟顺序（槽序 144、1、…、143）排列，而不是原报的自然分组槽号。

65 - ****评价日期口径****：问题 2—4 的评价窗口统一为 ****2025-02-01—2025-12-31 共 334 天****，2025 年
1 月作为冷启动与状态传递阶段，不计入正式费用比较；问题 1 为单日 144 槽模型窗口。两问窗口不同属
题目与协议的客观差异，处理方式是各自写明口径、分别统计，不把两个窗口的数据合并或互相换算；365
天连续轨迹不得与 334 天正式费用混称为同一统计量。

66

67 ****数值差异的核查结果（已定位到具体文件，非舍入差异）：****

68

69 - ****正文问题二表格取自 eps_soc 修正前的批次。****正文表内五个数（计划购电费 1266.18 万元、紧急购电量
442623.47 kWh、紧急购电费 158.24 万元、总费用 1424.42 万元、日均费用 42647.32 元/日）与 `重跑\_Q23\_v4/main\_vterm/q2\_逐日费用.csv`（该批次的 `main\_linear` 文件同值）中 B 路径
2025-02-01—12-31 共 334 天的合计逐项一致：计划 12,661,829.49 元、紧急购电费 1,582,374.60
元、紧急购电量 442,623.47 kWh、总费用 ****14,244,204.085 元****（日均 42,647.32 元）。该批次生
成于 2026-09-12 14:20—16:22，属 `eps\_soc` 修正前。

70 - ****图件来源记录登记的是 eps_soc 修正后的交付批次。****记录登记的 14,249,287.607 元取自 `项目/04_结
果与检验/问题2-3_正式交付/q2_逐日费用.csv`，该文件与交付件 `result2.xlsx` 同源（`q2_逐日费用
.csv` 的 SHA-256 为 `190867EA…6C34E`，与 `q23_linear_epsfix_20260912` 批次逐字节相同），
334 天合计为计划 12,658,339.36 元、紧急购电费 1,590,948.25 元、紧急购电量 445,178.91 kWh、
日均 42,662.54 元。

71 - ****两批差额为 5,083.52 元****（334 天、B 路径；365 天口径差 4,100.49 元）。差异不是舍入造成，而是
`eps\_soc` 修正改变了该路径的解；同一批次内 A 路径两版逐位相同（14,717,580.864595544 元）。

72 - ****此前记作“5,087.60 元”或“5,087.61 元”属口径误算****：5,087.61 元只在把正文展示值“1424.42
万元”直接当作 14,244,200 元、而不使用来源文件的未舍入原值 14,244,204.085 元时才出现；两者都
不是批次差本身。

73 - ****团队裁定（2026-09-13）：统一到 `eps\_soc` 修正后批次。****正文问题二表格五个数已更新为计划
****1265.83 万元****、紧急购电量 ****445178.91 kWh****、紧急购电费 ****159.09 万元****、总费用
****1424.93 万元****、日均 ****42662.54 元/日****（`2026C.tex` §5.3.8 正文与表内五处），与交付件 `result2.xlsx`、
图件来源记录一致；Q2/Q3 与问题 4 因而同为修后内核。该批次机读元数据登记 `case\_id = q23\_linear\_epsfix\_20260912`、
`method = linear`、`p10\_core.py = b3242ad925c82d63…`。裁定留痕见 `项目/02\_模型方案/问题4\_模型与口径方案/修订协议.md` 第 16 节
与同目录 `项目状态.md`（`项目/01\_任务与数据说明/问题4\_任务与协议/` 下的同源副本同步追加）。

74 - ****随之产生的新待办****：`项目/04\_结果与检验/results/result2.xlsx`（SHA-256 前缀 `63906
DD93EFE8A36…`，2026-09-12 14:20 的 vterm 修前版）未被本次裁定刷新。若最终提交目录仍为 `results/`，
须先把 `result2.xlsx`、`result3.xlsx` 及相关 `q2\_\*`/`q3\_\*` 附带数据替换为 19:40
批次（`result2.xlsx` = `9AE1D21554850B7E…`、`q2\_逐日费用.csv` = `190867EA64271EC9…`），
否则提交工作簿与论文数值不一致。

75

76 ****人工复核与最终责任****：团队对照原始附件核对单位、时段顺序、聚合方法和统计量，并区分全年数据范围与正
式费用评价窗口；上述批次差异由上列文件复算定位，最终采用哪一批次由团队裁定。自动检查结果不等于人
工复核完成，也不构成数据真实性或建模语义正确性的保证。

77

78 **### 4. 可视化与绘图代码**

****团队自主完成的核心工作：****图件的论证目标、结果选取、图表解释和正式采用版本的决策由团队作出，；问题三的预报分解口径采用线性插值亦由团队裁定。本项决策不来源于任何附件内容，附件只提供数据与结果依据。

****AI 辅助完成的非核心工作：****历史任务摘要记录了 AI 修改绘图源代码、根据原始附件或已有结果重新生成图件、按团队要求生成论文配图，并调整字体、配色、坐标标注和布局。当前图件来源记录保存有数据路径、文件哈希与聚合规则，可用于追溯图件制作依据。

****结果口径与版本（问题三）：****

- 正式口径（团队裁定）：附件 3 每个整点发布的光伏预报对相邻整点值作****线性插值****，展开到 10 分钟尺度；协议标识 `q23\_janwarm\_v4`，正式选型组合为 `6+12+18`。`项目/02\_模型方案/问题2-3\_模型方案/问题3正式选型说明.json` 标记的 `FORMALLY\_SELECTED | method: linear` 与该裁定一致。
- 正式结果版本：正文与图件均以此为准。`2026C.tex` 第 725、787、791 行按线性插值表述（基于 2025-02-01 至 12-31 共 334 天，三次更新节省 63.37 万元，即不更新 14,492,338.58 元 - 三次更新 13,858,598.93 元 = 633,739.65 元）；图件来源记录中 `q3\_update\_savings` 的来源为 `项目/04\_结果与检验/问题2-3\_正式交付/q3\_逐日费用.csv`（内容哈希 `9f7c5d03...`）。
- 对照口径：`hold`（整点值保持覆盖其后六个 10 分钟槽）为对照口径，其事后最优固定组合为 `6`（334 天 14,624,695.29 元），与 linear 口径的 `6+12+18` 排序相反。按协议，涉及“哪个时刻值得更新”的排序结论须双口径并列，并限定为“2025 年单年 334 天回测的样本内结果”，不称未来全局最优。
- 数值口径说明：linear 的 334 天结果存在 `eps\_soc` 修正前后两批——修正前 `6+12+18` 为 13,855,616.43 元（见 `重跑\_Q23\_v4/main\_linear` 及 `问题2-3\_验证/tables\_and\_stats.json`），修正后为 13,858,598.93 元（交付批次，正文与图件采用），两者相差 2,982.50 元，属批次差异而非记录矛盾。`问题3正式选型说明.json` 登记的 `6+12+18` 合计 ****16,177,847.99 元**** 为 ****365 天****口径（字段名 `selected\_total\_cost\_365d`，取自修正后批次），与 334 天口径不是同一统计量。
- 记录同步（2026-09-13 已完成）：`项目/02\_模型方案/问题4\_模型与口径方案/修订协议.md` 第 3 条与同目录 `项目状态.md` 原把 `hold` 登记为主口径、把 `linear` 列为“不替换主口径”的对照，与团队裁定不一致。已按裁定在两份记录及其 `项目/01\_任务与数据说明/问题4\_任务与协议/` 同源副本中****追加带日期的裁定留痕****（`修订协议.md` 第 16 节 / `项目状态.md` “团队裁定留痕”），不删改原条款；同一留痕同时登记了问题二/三改采 `eps\_soc` 修正后批次并取代 `results/result2.xlsx`（14:20 vterm 修前版）的结论。

****人工复核与最终责任：****团队核对数据来源、单位、坐标范围、图例及图注，确认图件使用的是线性插值口径下 `eps\_soc` 修正后批次的正式结果版本。根据已有购电计划恢复展示曲线不等于重新求解优化模型；重新绘图也不等于独立验证调度结果。

5. 论文撰写与排版

****团队自主完成的核心工作：****论文的研究内容、模型、结果解释与主要结论由团队负责；涉及模型语义或结论边界的修改由团队逐项判定。

****AI 辅助完成的非核心工作：****依据历史任务摘要，AI 曾审阅论文行文，提出术语统一、长句拆分和段落衔接建议；按要求以括号批注方式整理修改意见，并辅助调整数据分析文字和 LaTeX 排版，同时整理论文的行文框架。

****人工复核与最终责任：****团队逐项比较修改前后文本，确认公式、数据、约束和结论含义未被无依据改变，并核对论文中的模板占位内容。；最终提交内容由团队负责。

6. AI 使用说明整理

104 ****团队自主完成的核心工作：****确认实际使用工具、使用环节、核心工作分工及最终披露内容；上述信息已由团队
确认，并据以修订本稿（含工具版本与使用时间、各环节分工、问题三口径裁定、复核人员与采纳情况）。

105

106 ****AI 辅助完成的非核心工作：****读取指定文件夹中的生成要求，结合现有材料整理本说明，区分材料支持的事项
与缺失事实，按团队核验意见修订，并按各批次逐日费用文件复算合计以核对数值差异。

107

108 ****人工复核与最终责任：****团队需核对本说明与真实研究过程是否一致后决定最终文本。本稿已按团队核验意见修
订；团队终审与签署仍属团队责任。

109

110 **## 三、关键交互记录**

111

112 **### 记录 1：起草本次 AI 工具使用说明**

113

114 - ****提示词：****“根据 ai-use-report-prompt 中的内容生成一份ai使用说明”

115 - ****模型回复摘要：****读取模板与生成要求，结合当前论文、项目材料、图件来源记录及历史任务摘要，形成 AI
使用说明初稿；对缺失的工具版本、研究分工、人工复核和采纳情况作待确认标注。

116 - ****团队处理方式：****部分采纳。上轮由**本会话 AI 提出核验意见**（工具归属、分工缺项、填答对应关系、
结果口径的确认依据、过时表述、差额计算精度），**团队据此补充信息、作出问题三口径裁定并要求修订
**，本稿已逐项落实；其中模板结构、材料归纳方式被采纳，初稿中工具归属与分工的写法未被采纳，问题三
口径改以团队裁定为依据而非以正文与图件相互一致为据。

117

118 **### 论文研究期间的交互记录（2026-09-13 自工具会话存储中恢复）**

119

120 研究期间的原始会话记录于 2026-09-13 从工具会话存储（`~/dsh/sessions/`，53 个会话，2026-09-10
21:23 至 2026-09-13 08:07）中恢复，并解码提取真实提示词与回复摘要。提示词为原文，摘要为原意摘
录。

121

122 ****记录 2 | 问题 1 表 2 时钟口径订正与交付打包 | 2026-09-11 | `deepseek-v4-flash`****

123

124 - ****提示词：****“检查问题1还存在什么问题不,没有问题帮我将数据预处理和问题1相关的内容和可视化图表打
包”

125 - ****回复摘要：****以自建模型独立复核后确认最优值可靠，但查明现行 `result1.xlsx` 表 2 的 12 个汇总数
中 8 个与题目时钟区间不符（最大偏差 1,251.75 kWh）、检查器 C4/C5 失效；随后整体改为真实时钟区
间口径、重出结果文件、修正检查器并重建交付包（135 个文件逐字节校验）。

126 - ****团队处理方式：****采纳。团队先前已裁定“先修正再打包”，修正后的独立验收与订正记录由团队执行。

127

128 ****记录 3 | 问题 2/3 日末库存系数（`eps\_soc`）核查 | 2026-09-11 | `deepseek-v4-flash`****

129

130 - ****提示词：****“我在检查2,3两问的过程中发现求解时将日末电价 eps_soc 设置成了 0.001 元,这是不是不合
理”

131 - ****回复摘要：****该系数并非“日末电价”，而是消除 LP 退化的数值平局项；第 2 问结论不受影响，第 3 问
须按组合排序核对；并指出末槽奖励未按情景数归一，使 A 与 B/C 的末端权重口径不一致。

132 - ****团队处理方式：****采纳其发现。团队据此裁定问题 4 做 `eps` 归一化修正并全量重跑；问题 2/3 当时维
持修前内核，2026-09-13 重新裁定统一到修正后批次。

133

134 ****记录 4 | 问题 2/3 重跑结果的独立复核 | 2026-09-13 | `glm-5.3-flash`****

135

136 - ****提示词：****“我重跑了23问,你看一下结果有问题没”

137 - ****回复摘要：****独立核对运行元数据、汇总表、审计项、输入与代码哈希及统计溯源，结论为本轮重跑未发现硬
伤、旧正式结果未被改动；并提示标定轨与主轨的 `eps` 差异、linear 口径与 Sxseed 矩阵尚未完成。

138 - ****团队处理方式：****采纳为项目状态记录的依据；其列出的未做项按团队后续裁定分别处置。

139

140 ****记录 5 | 论文配图重制 | 2026-09-13 | `deepseek-v4-flash`****

141

142 - ****提示词:**** “帮我将 **fig2_全年光伏3D图、fig3_全年负载3D图、fig4_全年净负荷3D图** 中的内容改为一个 2 维的等高线图我看看效果,这几张图的绘制都是python得到的源码应该就在文件夹了”

143 - ****回复摘要:****新增独立绘图脚本把三张三维曲面图改为二维等高线图,输出 PNG/PDF/SVG 共 27 个文件; 并声明所用模型不支持读图、未能亲自查看成品,视觉效果需团队确认,坐标、色阶、字体与缺字告警由程序逐项验证。

144 - ****团队处理方式:****部分采纳。团队在多个方案中选定采用版本后重出正式图件;本说明只登记该次交互,不代表图件已通过人工视觉验收。

145

146 ****恢复边界说明:****上述记录覆盖研究期间的建模核验、结果订正、重跑复核与配图四个环节;其余会话为只读质检子代理、文档同步子代理与短问短答,未逐条列入。记录 1 (起草本说明) 来自起草会话,与研究期间工具分列。未列入的会话材料可作为检索线索,本稿不改写为逐字对话,也不生成示例对话。

147

148 **## 四、人工复核清单**

149

150 - [x] 工具名称、具体版本和使用时间已核验(研究期间:DeepSeek Harness, `deepseek-v4-flash` 为主、另有 `glm-5.3-flash` 与 `deepseek-v4-flash-vision-exp`, 2026-09-10 至 09-13 累计 4 日;起草会话按团队披露范围不列工具名)。

151 - [x] 每项 AI 使用均有材料依据(项目状态记录、表 2 时钟口径订正记录、图件来源与检查记录、结果文件),历史任务摘要已与项目记录交叉核对,仅作参考使用。

152 - [x] 没有把 AI 建议写成团队已执行的事实。

153 - [x] 核心建模方案、模型、实验、结果和结论的责任边界准确(问题分析、模型假设、算法设计、参数设置、结论形成由团队自主完成;公式核验与求解编程的工具性订正由 AI 执行、团队裁定与验收)。

154 - [x] 求解代码、绘图代码与文字修改的实际 AI 参与程度已如实披露。

155 - [x] 问题三正式版本已由团队裁定为线性插值口径,正文与图件均按该口径的交付批次表述,`hold` 列为对照口径。

156 - [x] 问题二数值差异已定位并裁定(正文原引 `eps\_soc` 修正前批次 14,244,204.085 元,图件来源记录与交付件为修正后批次 14,249,287.607 元,差额 5,083.52 元,非舍入差异)。

157 - [x] 问题二/三已统一为 `eps\_soc` 修正后批次,`2026C.tex` §5.3.8 表内五数已同步为 1265.83 / 445178.91 / 159.09 / 1424.93 / 42662.54,与交付件 `result2.xlsx` 一致。

158 - [x] 问题三的 `hold`/`linear` 记录同步已完成(`修订协议.md` 第 16 节与 `问题4\_模型与口径方案/项目状态.md` 及其同源副本已追加 2026-09-13 裁定留痕)。

159

160 - [x] 提交工作簿与论文数值一致(团队 2026-09-13 决定以 `项目/04\_结果与检验/问题2-3\_正式交付/` 为提交目录,其 `result2.xlsx` = `9AE1D21554850B7E...` 与论文同批次;不再以 `results/` 目录提交)。

161 - [x] 关键交互的提示词、回复摘要与采纳情况可追溯(2026-09-13 自工具会话存储恢复,见记录 1—5)。

162

163

164 **## 仍需人工确认的事项**

165

166 1. ****其他披露范围与格式要求**:**是否还有其他工具调用、文献整理或其他研究环节需要披露,以及适用的学校、竞赛或期刊格式要求(如页数上限、是否要求匿名化)。

167 2. ****记录恢复范围**:**本次恢复的是工具会话存储中留存的记录;如团队另有会话副本(其他工具、其他机器),应据实比对应后补入。

168

169 ---

170

171 ****本次裁定执行记录(2026-09-13)****

172 | 事项 | 裁定结果 | 执行动作 |

173 | --- | --- | --- |

174 | 问题二表格批次 | 统一到 `eps\_soc` 修正后批次 | 已改 `2026C.tex` §5.3.8 正文与表内五数 |

175 | 问题三 `hold`/`linear` 记录同步 | 追加裁定留痕, 不删原条款 | 已在 `修订协议.md` §16、`问题4\_模型与口径方案/项目状态.md` 及 `01\_任务与数据说明/问题4\_任务与协议/` 同源副本共 4 处追加 |

176 | 研究期间交互记录 | **改为补入真实记录** (记录 2—5) | 自 `~/dsh/sessions/` 恢复并解码 53 个会话, 提取提示词原文与回复摘要 |

177

178 | 提交工作簿 | 以 `04\_结果与检验/问题2-3\_正式交付/` 为提交目录 | 无需替换文件, 该目录与论文同批次; `results/` 目录不再作提交用 |

179 | GLM 披露 | 补入 `glm-5.3-flash` 与 `deepseek-v4-flash-vision-exp` 的调用说明 | 已写入第一节模型表与 PDF |

180

181

182

183 ---

184

185 ## 人工复核与终审声明 (2026-09-13 团队确认)

186

187 经复核, 本说明所载工具、模型、使用时间、工作分工、采纳与修正情况、结果口径与交付批次与项目记录、论文正文及交付件一致。团队确认如下结论:

188

189 1. **工具与模型**: 论文研究期间使用 DeepSeek Harness; 实际调用模型为 `deepseek-v4-flash` (主)、`glm-5.3-flash` (2026-09-12 前后若干会话, 用于问题 2/3 重跑结果的独立复核与文档核查)、`deepseek-v4-flash-vision-exp` (图件视觉质检); 使用时间累计 4 日 (2026-09-10 至 09-13, 间断使用)。

190 2. **分工**: 问题分析、模型假设、算法设计、参数设置、结论形成由团队自主完成; 公式与结算代数核验、求解编程中的工具性订正 (表 2 时钟口径重出、检查器重写) 由 AI 执行、团队裁定与验收; 配图与行文框架由 AI 辅助、团队选定采用版本。

191 3. **口径与数值**: 问题三正式口径为线性插值 (`hold` 为对照口径); 问题二/三与问题四同为 `eps` 修正后内核; `2026C.tex` §5.3.8 表内五数 (1265.83 / 445178.91 / 159.09 / 1424.93 / 42662.54) 与提交工作簿 `result2.xlsx` 一致; 提交目录为 `04\_结果与检验/问题2-3\_正式交付/`。

192 4. **记录真实性**: 第三节记录 1—5 的提示词与回复摘要均自工具会话存储恢复, 无虚构内容; 未能核实的事项已如实标注。

193 5. **责任**: 最终提交内容由团队负责。

194

195

196 ---

197

198 编写依据 (供核验, 非交互记录): `ai-use-report-prompt/ai-use-report-prompt.md`、`ai-use-report-prompt/README.md`、`2026C.tex`、`项目/README.md`、`项目/01\_任务与数据说明/问题2-3\_任务与解题协议/问题2-3\_审查与解题思路.md`、`项目/01\_任务与数据说明/问题2-3\_任务与解题协议/术语表格.md`、`项目/02\_模型方案/问题2-3\_模型方案/问题3正式选型说明.json`、`项目/02\_模型方案/问题4\_模型与口径方案/修订协议.md`、`项目/02\_模型方案/问题4\_模型与口径方案/项目状态.md`、`项目/02\_模型方案/问题4\_模型与口径方案/实验总结报告.md`、`项目/04\_结果与检验/问题2-3\_正式交付/q2\_逐日费用.csv`、`项目/04\_结果与检验/问题2-3\_正式交付/q3\_逐日费用.csv`、`项目/04\_结果与检验/问题2-3\_正式交付/result2.xlsx`、`项目/04\_结果与检验/问题4\_验证/矩阵汇总.json`、`项目/04\_结果与检验/问题2-3\_验证/tables\_and\_stats.json`、`项目/05\_论文/问题1-4\_综合解题与结果报告.md`、`项目/05\_论文/问题1\_论文相关文档/项目状态\_全项目进度记录.md`、`项目/05\_论文/问题1\_论文相关文档/订正记录\_表2时钟口径\_20260911.md`、`figures/图件来源与检查.json`、`figures/数据分析复核.json` 及 2026 年 9 月 12 日论文行文润色与科研配图美化历史任务摘要; 问题二/三各批次数值另按原始项目目录 (`重跑\_Q23\_v4/`

main_vterm`、`重跑_Q23_v4/main_linear`、`q23_linear_epsfix_20260912`）的逐日费用文件复算核对。模板中的示例工具、示例数据集和示例对话均未作为事实使用。本次按团队 2026-09-13 裁定执行四项动作（论文问题二表五个数与正文同步、在 4 处项目记录追加裁定留痕、交互记录降级为材料线索、确认姓名披露方式），未重新运行调度求解，也未对整篇论文进行全面验收；问题三的口径归属依据团队裁定，未由本次会话重新选型。

附录 D AI 工具使用说明

完整说明见支撑材料根目录 AI 工具使用说明.md / .pdf。要点：论文研究期间使用 DeepSeek Harness, 累计 4 日 (2026-09-10 至 09-13), 实际调用模型为 deepseek-v4-flash (主)、glm-5.3-flash (问题 2/3 重跑结果的独立复核与文档核查)、deepseek-v4-flash-vision-exp (图件视觉质检)；问题分析、模型假设、算法设计、参数设置与结论形成由团队自主完成，公式核验与求解编程中的工具性订正由 AI 执行、团队裁定与验收；问题三正式口径为线性插值，问题二/三与问题四同为 eps 修正后内核；记录 1—5 由工具会话存储恢复；本说明于 2026-09-13 经团队终审确认。